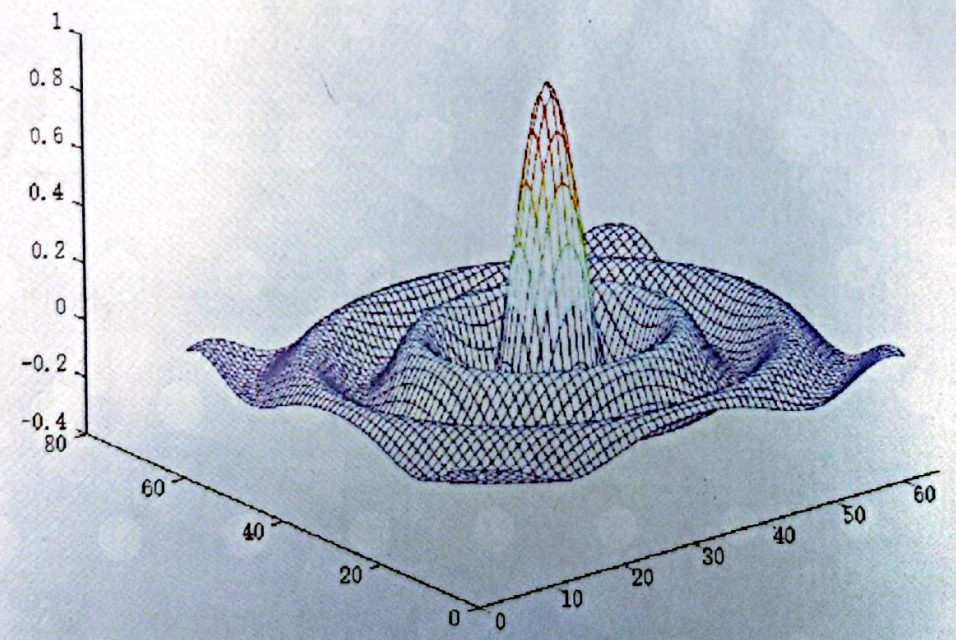
高等学校教材

数值计算方法

第二版

北京理工大学
姜海燕　季　霞
丁丽娟　程杞元　编著



中国教育出版传媒集团
高等教育出版社

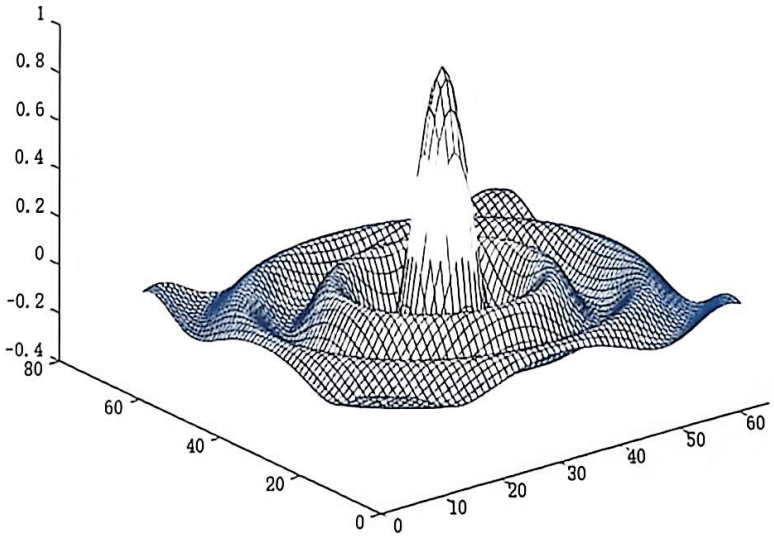
扫描全能王 创建

高 等 学 校 教 材

数值计算方法

第二版

北京理工大学
姜海燕　季　霞
丁丽娟　程杞元　编著



中国教育出版传媒集团
高等教育出版社·北京

扫描全能王 创建

内容提要

本书根据理工科数值计算方法课程要求，结合作者多年的教学实践经验和成果编写而成。在编写过程中注重体现数值计算方法的原理和实用性，并介绍了各类数值算法的新发展，以北太天元数值计算软件为平台，展示数值计算方法的计算效果。

全书内容包括数值代数、数值逼近与常微分方程数值解法，各章均配备了丰富的例题与应用实例，给出了各种基本算法的计算机实现过程。书末附有北太天元数值计算软件介绍，便于读者编程实现。

本书可作为理科各专业本科生和工科专业研究生数值计算方法课程的教材，也可供相关工程技术人员参考。

图书在版编目（CIP）数据

数值计算方法 / 北京理工大学等编著 . --2 版 .
北京：高等教育出版社，2024.9（2025.11重印）.
ISBN 978-7-04-062722-0
Ⅰ. O241
中国国家版本馆 CIP 数据核字第 2024DX2409 号

SHUZHI JISUAN FANGFA

策划编辑 李 茜	责任编辑 李 茜	封面设计 李小璐	版式设计 马 云
责任绘图 杨伟露	责任校对 胡美萍	责任印制 刘弘远	

出版发行	高等教育出版社	网 址	http://www.hep.edu.cn
社 址	北京市西城区德外大街 4 号		http://www.hep.com.cn
邮政编码	100120	网上订购	http://www.hepmall.com.cn
印 刷	天津鑫丰华印务有限公司		http://www.hepmall.com
开 本	787mm × 1092mm 1/16		http://www.hepmall.cn
印 张	21.75	版 次	2011 年 8 月第 1 版
字 数	520 千字		2024 年 9 月第 2 版
购书热线	010-58581118	印 次	2025 年 11 月第 2 次印刷
咨询电话	400-810-0598	定 价	45.90 元

本书如有缺页、倒页、脱页等质量问题，请到所购图书销售部门联系调换
版权所有 侵权必究
物 料 号 62722-00

扫描全能王 创建

/第二版前言/

本书第一版于 2011 年 8 月出版, 十几年来被多所高校用作教材和参考书, 为科学计算方面的人才培养起到了积极的作用。本次修订秉承上一版的优点, 兼顾算法理论的系统性与完整性, 重视算法的实践, 订正了一些印刷错误与表述不太完整之处, 补充了科学计算方法的一些新结果, 特别是以下内容:

(1) 对幂法的介绍补充了特征值为复数情形。

(2) 对正交多项式系的证明由数学归纳法验证修改为构造法, 使读者加深对 Schmidt 正交化方法的理解。

(3) 对 FFT 算法的介绍采用了更加清晰的分割、组合思路介绍。

(4) 对所有的数值实践均采用北太天元数值计算软件作为实践平台, 做到让熟悉 MATLAB 的用户无障碍迁移。

(5) 每个章节均增加了与其相关的算法历史和相关数学家的介绍, 增加读者的兴趣和对算法发展历史的了解。

本次修订主要由姜海燕和季霞负责, 姜海燕负责第一、二、四、五、七章及附录的编写, 季霞负责第三、六、八、九章的编写。感谢北京大学的卢朓老师对本教材提出很多宝贵的修改意见, 感谢高兴明、郭梅莉、王一丹、韩瑜、张伟彤、喻星纯同学帮助校对书稿。在出版过程中得到了高等教育出版社和北京理工大学的大力支持, 在此一致表示衷心的感谢。

编　者

2024 年 3 月

扫描全能王 创建

/第一版前言/

随着计算机的广泛使用与科学技术的迅速发展, 科学计算已成为科学研究和工程应用领域中的一种重要研究工具, 它是与理论分析、科学试验并驾齐驱的一种科学研究方法。科学计算能力也成为理工科大学生必须具备的基本素质之一。数值计算方法课程是计算科学的重要基础课程, 是理工科院校各专业本科生和工科专业硕士研究生的学位课, 主要内容是引导科学计算的入门、介绍数值计算的基本方法以及如何利用现代计算工具高效率解决数值计算问题。

本书是作者团队根据数值计算方法课程的基本要求, 在多年的教学实践和原有教材基础上编写而成的, 包含了数值代数、数值逼近和常微分方程数值解法的基本内容。力求全面、系统地介绍求解各类数学问题近似解的最基本、最常用的方法, 并且着重阐明构造算法的基本思想与原理。根据“重概念、重方法、重应用、重能力”的培养原则, 我们在编写中强调以下几点:

1. 注重数值计算基本方法的介绍, 简明介绍数值计算中的“病态”问题和工程计算应用中常见问题的处理方法。如病态线性方程组和稀疏分块方程组的求解、二维数据插值拟合、重积分的数值计算、刚性微分方程 (组) 的求解等。

2. 注重方法的实用性。精编了一些例题, 各章配备了应用实例, 以各章的数值方法为工具, 比较各类数值方法的优劣性并帮助读者如何选择数值方法求解问题, 更好地体会应用数值计算方法有效地解决工程技术问题。

3. 注重提高计算效率, 以 MATLAB 为平台, 培养学生的编程能力。教材附录中简单介绍了 MATLAB 软件的基本功能, 同时各章中介绍实现各种数值方法的函数命令和程序, 使学生在了解数值计算方法的原理之后, 快速实现算法并能编程求解工程计算问题。

4. 注重各类数值方法的发展。本书对每一章的数值计算方法作了简要的评述, 介绍近几年发展起来的计算效率高的各种数值方法, 并给出参考书目, 以便读者自学。

全书讲授约需 60 学时, 也可根据具体学时选择部分章节讲授。在本书的编

扫描全能王 创建

写过程中，我们参考了许多国内外相关教材和专著，并将它们列在参考文献中，在此表示衷心的感谢。由于编者水平有限，书中难免有不当之处，恳请读者批评指正。

编　者

2011 年 3 月

扫描全能王 创建

/目 录/

第一章 绪论 1

1.1 科学计算导论 …… 1
1.2 误差 …… 2
1.3 误差的传播 …… 6
1.4 数值计算中应注意的问题 …… 10
评注一 …… 13
算法历史及背后的数学家 …… 13
习题一 …… 14
数值实验一 …… 15

第二章 解线性方程组的直接方法 / 16

2.1 消去法 …… 17
2.2 直接三角分解法 …… 23
2.3 特殊矩阵的三角分解法 …… 33
2.4 误差分析 …… 40
2.5 超定线性方程组的最小二乘解 …… 49
2.6 算法实现与应用实例 …… 52
评注二 …… 57
算法历史及背后的数学家 …… 57
习题二 …… 57
数值实验二 …… 60

扫描全能王 创建

第三章　解线性方程组的迭代法　/ 62

　　迭代法概述 …… 62
　　几种基本的迭代法 …… 64
3.3　迭代法的收敛条件 …… 70
3.4　最速下降法与共轭梯度法 …… 79
3.5　算法实现与应用实例 …… 83
评注三 …… 85
算法历史及背后的数学家 …… 85
习题三 …… 85
数值实验三 …… 88

第四章　矩阵特征值与特征向量的计算　/ 89

　　幂法和反幂法 …… 89
　　Jacobi 方法 …… 99
4.3　QR 方法 …… 104
4.4　算法实现与应用实例 …… 115
评注四 …… 118
算法历史及背后的数学家 …… 118
习题四 …… 118
数值实验四 …… 120

第五章　插值法　/121

5.1　Lagrange 插值 …… 121
5.2　Newton 插值 …… 128
5.3　分段线性插值 …… 138
5.4　Hermite 插值 …… 141
5.5　样条插值 …… 148
5.6　二维插值 …… 154
5.7　快速 Fourier 变换 …… 156

扫描全能王 创建

5.8 算法实现与应用实例 ……161
评注五 ……164
算法历史及背后的数学家 ……164
习题五 ……164
数值实验五 ……167

第六章 函数逼近 /169

6.1 数据拟合的最小二乘法 ……170
6.2 正交多项式 ……178
6.3 函数的最佳平方逼近 ……184
6.4 算法实现与应用实例 ……188
评注六 ……191
算法历史及背后的数学家 ……192
习题六 ……192
数值实验六 ……194

第七章 数值微分与数值积分 /195

7.1 数值微分 ……195
7.2 Newton-Cotes 求 ……199
7.3 复化求积公式 ……206
7.4 Romberg 求积公 ……212
7.5 Gauss 型求积公 ……217
7.6 振荡函数的积分 ……228
7.7 重积分的数值计算 ……231
7.8 算法实现与应用实例 ……235
评注七 ……242
算法历史及背后的数学家 ……243
习题七 ……243
数值实验七 ……247

扫描全能王 创建

第八章　非线性方程及方程组的解法 /248

8.1　二分法 ……248
8.2　简单迭代法 ……250
　　Newton 法与弦截法 ……258
　　抛物线法 ……263
8.5　非线性方程组的解法 ……264
8.6　算法实现与应用实例 ……268
评注八 ……271
算法历史及背后的数学家 ……272
习题八 ……272
数值实验八 ……274

第九章　常微分方程数值解法 /276

9.1　Euler 方法及其改进方法 ……277
9.2　Runge-Kutta 法 ……283
9.3　线性多步法 ……288
9.4　相容性、收敛性与稳定性 ……296
9.5　微分方程组的数值解法 ……303
9.6　边值问题的数值解法 ……308
9.7　Hamilton 系统保结构算法简介 ……310
9.8　算法实现与应用实例 ……311
评注九 ……314
算法历史及背后的数学家 ……314
习题九 ……314
数值实验九 ……317

附录　北太天元及其应用简介 /319

参考文献 /334

扫描全能王 创建

/第一章/

绪论

1.1 科学计算导论

科学计算、理论分析与科学实验并称为当今世界科学活动的三大主要方式. 数值计算方法, 也被称为“数值分析”或“计算方法”, 是数学研究的一个重要分支, 是横跨计算数学与计算机科学的交叉学科, 是科学计算的基础. 数值计算方法课程作为信息与计算科学专业的核心课程, 也是其他应用类专业的重要课程.

数值计算方法是研究求解数学问题 (数学模型) 近似解的方法、过程及其理论的一个数学分支, 由于它所研究的数学问题往往来源于科学研究与工程计算问题, 故数值计算方法也称为科学计算方法. 随着计算机技术的发展, 使用计算机通过数值计算或数值模拟的手段解决工程实际问题和科学研究中关键问题已成为越来越不可缺少的重要环节, 在某些领域内代替甚至超过了工程实验所起的作用. 目前, 掌握和运用科学计算的基本方法已是各学科科学研究者和工程技术人员的必要技能. 从科学与工程计算中抽象出的数学问题常常涉及求解微分方程、线性与非线性代数方程组、微分与积分计算、特征值计算、数据拟合与函数逼近问题以及数据处理问题, 数值计算方法利用离散、迭代、插值与拟合等最基础的方法, 介绍针对各个问题的基本数值算法, 再编程计算, 得到数学问题的近似解. 进一步分析数值算法的误差、精确度、稳定性和收敛性, 建立相关的计算数学理论. 数值计算方法不同于纯数学, 它不仅具有数学的抽象性和严密性, 又具有广泛的应用性和实验技术性. 它是一门与计算机紧密结合的、实用性很强的、有着自身研究方法与理论体系的数学课程. 学习数值计算方法时, 首先应掌握各种数值计算方法的基本原理和思想, 学会运用各种方法的处理技巧, 了解各种方法的优缺点, 并会利用数学计算软件编程计算, 求得数学问题的数值解.

随着计算机科学技术的快速发展, 计算机已经成为各个领域科学研究不可缺少的工具. 在自然科学与工程技术的发展过程中, 先后产生了计算数学、计算物理、计算化学、计算材料学、计算生物学等一系列数值计算应用分支. 数值计算方法也是如今快速发展的信息科学和数据科学的基础. 可见, 数值计算在科学研究、工程技术、信息科学、数据科学中几乎无处不在, 对科技发展起着重要作用.IEEE(电气与电子工程师协会) 主办的杂志 *Computing in Science & Engineering*(《科学与工程计算》) 开展了一次评选, 选出在 20 世纪对科学和工程的发展与实践影响最大的 10 个算法, 它们按时间顺序依次如下:

(1) 1946 年, 美国 Los Alamos(洛斯 · 阿拉莫斯) 国家实验室的 John von Neumann (冯 · 诺依曼)、S. Ulam (乌拉姆) 和 N. Metropolis(梅特罗波利斯) 提出的 Metropolis 算法, 属于 Monte Carlo(蒙特卡罗) 方法.

扫描全能王 创建

(2) 1947 年, 美国 RAND(兰德) 公司的 G. Dantzig(丹齐格) 创造的解线性规划问题的 Simplex(单纯型) 算法.

(3) 1950 年, 美国国家标准局数值分析研究所的 M. Hestenes(赫斯特内斯)、E. Stiefel(斯蒂费尔) 和 C. Lanczos(兰佐斯) 开创的 Krylov(克雷洛夫) 子空间迭代法.

(4) 1951 年, 美国 Oak Ridge(橡树岭) 国家实验室的 A. Householder(豪斯霍尔德) 形式化的矩阵计算的分解方法.

(5) 1957 年, 美国 IBM(国际商业机器公司) 的 J. Backus(巴克斯) 领导开发的 FORTRAN 最优编译器算法.

(6) 1959—1961 年, 英国伦敦 Ferranti(弗兰蒂) 公司的 J. Francis(弗兰西斯) 发明的计算矩阵特征值的稳定算法, 即 QR 算法.

(7) 1962 年, 英国伦敦 Elliott Brothers(伊里亚德兄弟) 公司的 T. Hoare(霍尔) 提出快速排序算法, 即 Quicksort 算法.

(8) 1965 年, 美国 IBM Watson(沃森) 研究中心的 J. Cooley(库利) 和普林斯顿大学及 AT & T(美国国际电话电报公司)Bell(贝尔) 实验室的 J. Tukey(图基) 共同提出的快速傅里叶变换算法, 即 FFT 算法.

(9) 1977 年, 美国 Brigham Young(杨百翰) 大学的 H. Ferguson(弗格森) 和 R. Forcade(福卡德) 提出的整数关系检测 (integer relation detection) 算法.

(10) 1987 年, 美国 Yale(耶鲁) 大学的 L. Greengard(格林加德) 和 V. Rokhlin(洛克林) 发明的快速多极 (fast multipole) 算法.

实际上, 除第 (5)(7)(9) 个算法外, 其余算法都与数值计算关系密切. 由此可见数值算法的重要性和普遍性. 本书将要讲解的一些最基础、应用最广的数值算法, 部分内容与这十大算法密切相关. 这十大算法没有涉及中国人的名字, 但随着我国经济、科技、军事的大力发展, 很多华人数学家已经在国际崭露头角, 特别是计算数学家, 很多都被邀在国际数学家大会上做邀请报告.

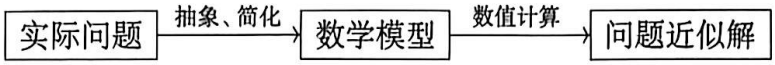
在数值计算软件方面, 由于之前对国外软件的依赖性, 有自主产权的软件发展比较缓慢. 北太天元数值计算通用软件 (Baltamatic), 是在北京大学、北京大学大数据分析与应用技术国家工程实验室、北京大学重庆大数据研究院共同支持下, 由北京大学重庆大数据研究院基础软件科学研究中心数值计算实验室, 突破关键核心技术, 独立自主研发的国产通用型科学计算软件 (以下简称“北太天元”). 本书中的计算实例均用北太天元编程实现. 关于北太天元的具体介绍请参见附录.

1.2 误 差

1.2.1 误差的基本概念

计算中误差是不可避免的. 由于用数学方法解决实际问题时, 常按以下过程进行:

扫描全能王 创建



从而引起误差的因素很多, 主要有以下几种:

1. 模型误差

实际问题的解与数学模型的解之差称为“模型误差”.

2. 观测误差

数学问题中所出现的一些参量, 其值往往由观测得到, 而观测不可能绝对准确, 由此产生的误差称为“观测误差”.

3. 截断误差

一般数学问题常常难以求出精确解, 需要简化为较易求解的问题, 以简化问题的解作为原问题解的近似. 如求一个收敛的无穷级数之和, 总是用它的部分和作为近似值, 也就是截去该级数后面的无穷多项. 这样由于简化问题所引起的解的误差称为“截断误差”或“方法误差”. 例如

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots + \frac{(-1)^n x^{2n}}{(2n)!} + \cdots$$

当 $|x|$ 很小时, 可以用 $1 - \dfrac{x^2}{2}$ 作为 $\cos x$ 近似值. 由交错级数判敛的 Leibniz(莱布尼茨) 准则, 它的截断误差的绝对值不超过 $\dfrac{x^4}{24}$.

4. 舍入误差

在计算过程中往往要对数字进行舍入. 如受机器字长的限制, 无穷小数和位数很多的数必须舍入成一定的位数. 这样产生的误差称为“舍入误差”.

1.2.2 绝对误差和相对误差

定义 1.1 *设 x^* 为准确值 x 的一个近似值, 称*

$$e(x^*) = x - x^* \tag{1.1}$$

为近似值 x^* 的绝对误差, 简称误差.

一般情况下准确值 x 难以求出, 从而也不能算出绝对误差 $e(x^*)$ 的准确值, 但可以根据测量工具或计算的情况估计出它的取值范围, 即估计出误差绝对值的一个上界 ε:

$$|e(x^*)| = |x - x^*| \leqslant \varepsilon$$

通常称 ε 为近似值 x^* 的绝对误差限, 简称误差限. 显然, 误差限不是唯一的, 越小的误差限越有意义.

有了误差限及近似值, 就可以得到准确值的范围

$$x^* - \varepsilon \leqslant x \leqslant x^* + \varepsilon$$

即准确值 x 必定在区间 $[x^* - \varepsilon, x^* + \varepsilon]$ 上, 也常记作

$$x = x^* \pm \varepsilon$$

扫描全能王 创建

容易看出, 经过四舍五入得到的数, 其误差必定不超过被保留的最后数位上的半个单位, 即最后数位上的半个单位为其误差限. 例如, 若取 π 的近似值为 3.14, 则

$$|\pi - 3.14| \leqslant 0.001\ 6 \leqslant \frac{1}{2} \times 10^{-2}$$

若取 $\pi \approx 3.142$, 则

$$|\pi - 3.142| \leqslant 0.000\ 41 \leqslant \frac{1}{2} \times 10^{-3}$$

误差限的大小不能完全反映近似值的精确程度. 要刻画近似值的精确程度, 不仅要看绝对误差的大小, 还必须考虑准确值本身的大小, 由此引出了相对误差的概念.

定义 1.2　设 x^* 为准确值 x 的近似值, 称绝对误差与准确值之比为近似值 x^* 的相对误差, 记为 $e_{\mathrm{r}}(x^*)$, 即

$$e_{\mathrm{r}}(x^*) = \frac{e(x^*)}{x} = \frac{x - x^*}{x}$$

由于在计算过程中准确值 x 总是未知的, 故一般取相对误差为

$$e_{\mathrm{r}}(x^*) = \frac{e(x^*)}{x^*} \tag{1.2}$$

可以证明, 当 $|e_{\mathrm{r}}(x^*)|$ 很小时, $\dfrac{e(x^*)}{x} - \dfrac{e(x^*)}{x^*}$ 是 $e_{\mathrm{r}}(x^*)$ 的高阶无穷小, 可以忽略不计. 所以, 取绝对误差与近似值之比为相对误差是合理的.

同样, 相对误差也只能估计其上限. 如果存在正数 ε_{r}, 使得

$$|e_{\mathrm{r}}(x^*)| = \left|\frac{e(x^*)}{x^*}\right| \leqslant \varepsilon_{\mathrm{r}}$$

则称 ε_{r} 为 x^* 的相对误差限. 显然, 误差限与近似值绝对值之比 $\dfrac{\varepsilon}{|x^*|}$ 为 x^* 的一个相对误差限.

例如, 由实验测得光速近似值为 $c^* = 2.997\ 925 \times 10^5$ km/s, 其误差限为 0.1 km/s, 于是

$$\frac{\varepsilon}{|c^*|} = \frac{0.1}{2.997\ 925 \times 10^5} < 4 \times 10^{-7}$$

所以 4×10^{-7} 是 c^* 的一个相对误差限.

1.2.3　有效数字

有效数字是近似值的一种表示法. 它既能表示近似值的大小, 又能表示其精确程度.

在计算过程中, 常常按四舍五入的原则取数 x 的前几位数 x^* 为其近似值. 例如, $x = \sqrt{2} = 1.414\ 213\ 562\cdots$, 取前四位数得近似值 $x^* = 1.414$, 取前八位数得近似值 $x^* = 1.414\ 213\ 6$, 它们的绝对误差

扫描全能王 创建

$$|\sqrt{2}-1.414| \leqslant \frac{1}{2} \times 10^{-3}$$

$$|\sqrt{2}-1.414\ 213\ 6| \leqslant \frac{1}{2} \times 10^{-7}$$

定义 1.3 如果近似值 x^* 的误差限是 $\frac{1}{2} \times 10^{-n}$, 则称 x^* 准确到小数点后第 n 位, 并从第一个非零数字到这一位的所有数字均称为有效数字.

例如, $\sqrt{2}$ 的近似值 1.414 准确到小数点后第 3 位, 它具有 4 位有效数字. 1.414 213 6 作为 $\sqrt{2}$ 的近似值准确到小数点后第 7 位, 有 8 位有效数字. 一般地, 如果近似值 x^* 的规格化形式为

$$x^* = \pm 0.a_1 a_2 \cdots a_n \cdots \times 10^m \tag{1.3}$$

其中 m 为整数, $a_1 \neq 0, a_i\ (i=1,2,\cdots)$ 为 0 到 9 之间的整数, 且

$$|x-x^*| \leqslant \frac{1}{2} \times 10^{m-n} \tag{1.4}$$

则称近似值 x^* 有 n 位有效数字.

例如, $x = 0.003\ 400 \pm \frac{1}{2} \times 10^{-5}$ 表示近似值 0.003 400 准确到小数点后第 5 位, 有 3 位有效数字. 也可以写为 $x^* = 0.34 \times 10^{-2}, x = 0.003\ 400 \pm \frac{1}{2} \times 10^{-2-3}$, 有 3 位有效数字.

上面的讨论表明, 可以用有效数字位数来刻画误差限. 形如式(1.3)的数, 当 m 一定时, 其有效数字位数 n 越大, 则误差限越小. 例如, 若 $x^* = 1\ 452.046$ 是具有 7 位有效数字的近似值, 则它的误差限为

$$|x-x^*| \leqslant \frac{1}{2} \times 10^{-3}$$

又若 $x^* = 1\ 452.0$ 是具有 5 位有效数字的近似值, 则其误差限为 $\frac{1}{2} \times 10^{-1}$.

下面的定理给出了相对误差限与有效数字的关系.

定理 1.1 若 x 的近似值 $x^* = \pm 0.a_1 a_2 \cdots a_n \times 10^m\ (a_1 \neq 0)$ 有 n 位有效数字, 则 $\frac{1}{2a_1} \times 10^{-n+1}$ 为其相对误差限. 反之, 若 x^* 的相对误差限 ε_{r} 满足

$$\varepsilon_{\mathrm{r}} \leqslant \frac{1}{2} \times 10^{-n}$$

则 x^* 至少具有 n 位有效数字.

证明 由式 (1.4)

$$|e(x^*)| = |x-x^*| \leqslant \frac{1}{2} \times 10^{m-n}$$

从而有

$$|e_{\mathrm{r}}(x^*)| = \left|\frac{e(x^*)}{x^*}\right| \leqslant \frac{\frac{1}{2} \times 10^{m-n}}{0.a_1 a_2 \cdots a_n \times 10^m} \leqslant \frac{1}{2a_1} \times 10^{-n+1}$$

扫描全能王 创建

所以 $\frac{1}{2a_1}\times 10^{-n+1}$ 是 x^* 的相对误差限.

若 $\varepsilon_{\mathrm{r}}\leqslant\frac{1}{2}\times 10^{-n}$, 则

$$\begin{aligned}|e(x^*)|=|x^*e_{\mathrm{r}}(x^*)|&\leqslant 0.a_1\cdots a_n\cdots\times 10^m\varepsilon_{\mathrm{r}}\\&\leqslant 1\times 10^m\times\frac{1}{2}\times 10^{-n}=\frac{1}{2}\times 10^{m-n}\end{aligned}$$

由式(1.4), x^* 至少有 n 位有效数字.

注 1.1 定理 1.1 表明, 有效数字位数越多相对误差越小, 相对误差越小有效数字位数越多, 所以有效数字位数和相对误差一样, 可以从本质上反映近似值的准确程度. 因此, 在衡量计算结果的准确程度时, 通常采用相对误差和有效数字. 这个定理只是帮我们理解相对误差和有效数字的关系, 计算时并不需要, 实际上在已知近似值的前提下, 绝对误差限、相对误差限、有效数字的位数, 三者只要知道一个就可以根据定义计算另外两个.

1.3 误差的传播

1.3.1 基本运算中的误差估计

本节中所讨论的基本运算是指四则运算与一些常用函数的计算.

由微分学, 当自变量的改变量 (误差) 很小时, 函数的微分作为函数的改变量的主要线性部分可以近似函数的改变量, 故利用微分运算公式可导出误差运算公式.

设数值计算中求得的解与参量 (原始数据) $x_1,x_2,\cdots,x_n$ 有关, 记为

$$y=f(x_1,x_2,\cdots,x_n)$$

参量的误差必定引起解的误差. 设 $x_1,x_2,\cdots,x_n$ 的近似值分别为 $x_1^*,x_2^*,\cdots,x_n^*$, 相应的解为

$$y^*=f(x_1^*,x_2^*,\cdots,x_n^*)$$

假定 f 在点 $(x_1^*,x_2^*,\cdots,x_n^*)$ 处可微, 则当参量误差较小时, 解的绝对误差为

$$\begin{aligned}e(y^*)=y-y^*&=f(x_1,x_2,\cdots,x_n)-f(x_1^*,x_2^*,\cdots,x_n^*)\\&\approx \mathrm{d}f(x_1^*,x_2^*,\cdots,x_n^*)\\&=\sum_{i=1}^{n}\frac{\partial f(x_1^*,x_2^*,\cdots,x_n^*)}{\partial x_i}(x_i-x_i^*)\\&=\sum_{i=1}^{n}\frac{\partial f(x_1^*,x_2^*,\cdots,x_n^*)}{\partial x_i}e(x_i^*)\end{aligned}\tag{1.5}$$

其相对误差为

扫描全能王 创建

$$
\begin{aligned}
e_{\mathrm{r}}\left(y^{*}\right) &= \frac{e\left(y^{*}\right)}{y^{*}} \approx \mathrm{d}(\ln f) \\
&= \sum_{i=1}^{n} \frac{\partial f\left(x_1^{*}, x_2^{*}, \cdots, x_n^{*}\right)}{\partial x_i} \frac{e\left(x_i^{*}\right)}{f\left(x_1^{*}, x_2^{*}, \cdots, x_n^{*}\right)} \\
&= \sum_{i=1}^{n} \frac{\partial f\left(x_1^{*}, x_2^{*}, \cdots, x_n^{*}\right)}{\partial x_i} \frac{x_i^{*}}{f\left(x_1^{*}, x_2^{*}, \cdots, x_n^{*}\right)} e_{\mathrm{r}}\left(x_i^{*}\right)
\end{aligned} \tag{1.6}
$$

特别地, 由式 (1.5) 和式 (1.6)可得和、差、积、商之误差公式

$$
\begin{cases}
e\left(x_1 \pm x_2\right) = e\left(x_1\right) \pm e\left(x_2\right) \\
e_{\mathrm{r}}\left(x_1 \pm x_2\right) = \dfrac{x_1}{x_1 \pm x_2} e_{\mathrm{r}}\left(x_1\right) \pm \dfrac{x_2}{x_1 \pm x_2} e_{\mathrm{r}}\left(x_2\right)
\end{cases}
$$

$$
\begin{cases}
e\left(x_1 x_2\right) \approx x_2 e\left(x_1\right) + x_1 e\left(x_2\right) \\
e_{\mathrm{r}}\left(x_1 x_2\right) \approx e_{\mathrm{r}}\left(x_1\right) + e_{\mathrm{r}}\left(x_2\right)
\end{cases} \tag{1.7}
$$

$$
\begin{cases}
e\left(\dfrac{x_1}{x_2}\right) \approx \dfrac{1}{x_2} e\left(x_1\right) - \dfrac{x_1}{x_2^2} e\left(x_2\right) \\
e_{\mathrm{r}}\left(\dfrac{x_1}{x_2}\right) \approx e_{\mathrm{r}}\left(x_1\right) - e_{\mathrm{r}}\left(x_2\right)
\end{cases}
$$

分析表明, 和、差之误差为误差之和、差, 积、商之相对误差为相对误差之和、差. 由以上各式还可得出

$$
\left|e\left(x_1 \pm x_2\right)\right| = \left|e\left(x_1\right) \pm e\left(x_2\right)\right| \leqslant \left|e\left(x_1\right)\right| + \left|e\left(x_2\right)\right|
$$

$$
\left|e_{\mathrm{r}}\left(x_1 x_2\right)\right| \approx \left|e_{\mathrm{r}}\left(x_1\right) + e_{\mathrm{r}}\left(x_2\right)\right| \leqslant \left|e_{\mathrm{r}}\left(x_1\right)\right| + \left|e_{\mathrm{r}}\left(x_2\right)\right|
$$

$$
\left|e_{\mathrm{r}}\left(\frac{x_1}{x_2}\right)\right| \approx \left|e_{\mathrm{r}}\left(x_1\right) - e_{\mathrm{r}}\left(x_2\right)\right| \leqslant \left|e_{\mathrm{r}}\left(x_1\right)\right| + \left|e_{\mathrm{r}}\left(x_2\right)\right|
$$

因此, 和、差的误差限不超过各数的误差限之和, 积、商的相对误差限不超过各数的相对误差限之和.

例 1.1　设 $y = x^n$, 求 y 的相对误差与 x 的相对误差之间的关系.

解　由式 (1.6)得

$$
e_{\mathrm{r}}(y) \approx \mathrm{d}\left(\ln x^n\right) = n\mathrm{d}(\ln x) \approx n e_{\mathrm{r}}(x)
$$

所以 x^n 的相对误差是 x 的相对误差的 n 倍. 特别地, $\sqrt{x}$ 的相对误差是 x 的相对误差的一半.

例 1.2　假定运算中数据都准确到两位小数, 试求 $x^{*} = 1.21 \times 3.65 - 9.81$ 的绝对误差限和相对误差限, 计算结果有几位有效数字?

解　由式 (1.7)中前两组结论

$$
e\left(x^{*}\right) \approx 3.65 \times e(1.21) + 1.21 \times e(3.65) - e(9.81)
$$

因为式中数据都准确到两位小数, 即其误差限均为 $\dfrac{1}{2} \times 10^{-2}$, 所以

扫描全能王 创建

$$|e(x^*)| \leqslant 3.65 \times |e(1.21)| + 1.21 \times |e(3.65)| + |e(9.81)|$$

$$\leqslant (3.65 + 1.21 + 1) \times \frac{1}{2} \times 10^{-2} = 0.029\ 3$$

$$|e_{\mathrm{r}}(x^*)| = \frac{|e(x^*)|}{|x^*|} \leqslant \frac{0.029\ 3}{5.393\ 5} \approx 0.005\ 4$$

故 x^* 的绝对误差限为 0.029 3, 相对误差限为 0.005 4, 计算结果有两位有效数字.

1.3.2　算法的数值稳定性

计算一个数学问题, 需要预先设计好由已知数据计算问题结果的运算顺序, 这就是算法. 例如, 计算积分值

$$I_n = \int_0^1 \frac{x^n}{x+5}\,\mathrm{d}x \quad (n = 0, 1, 2, \cdots)$$

由关系式

$$I_n + 5I_{n-1} = \int_0^1 \frac{x^n}{x+5}\,\mathrm{d}x + \int_0^1 \frac{5x^{n-1}}{x+5}\,\mathrm{d}x = \int_0^1 x^{n-1}\,\mathrm{d}x = \frac{1}{n}$$

且

$$\frac{1}{6(n+1)} < I_n < \frac{1}{5(n+1)}$$

可设计如下两种算法:

算法 I　取 $I_0 = \displaystyle\int_0^1 \frac{1}{x+5}\,\mathrm{d}x = \ln 1.2$, 按公式

$$I_n = \frac{1}{n} - 5I_{n-1} \quad (n = 1, 2, \cdots) \tag{1.8}$$

依次计算 $I_1, I_2, \cdots$ 的近似值.

算法 II　取 $I_n^* \approx \dfrac{1}{2}\left[\dfrac{1}{6(n+1)} + \dfrac{1}{5(n+1)}\right]$, 按公式

$$I_{k-1} = \frac{1}{5}\left(\frac{1}{k} - I_k\right) \quad (k = n, n-1, \cdots, 1) \tag{1.9}$$

依次计算 $I_{n-1}, I_{n-2}, \cdots, I_0$ 的近似值.

分别取

$$I_0^* = 0.182\ 321\ 55$$

$$I_{14}^* = \frac{1}{2}\left(\frac{1}{6 \times 15} + \frac{1}{5 \times 15}\right) \approx 0.012\ 222\ 22$$

算法 I , II 的计算结果见表 1.1.

由表 1.1 中结果可见, 按算法 I 得到 $I_{10}^* < 0$, 这显然是错的, 因为对任意 $n \geqslant 0$, 均有

$$I_n > \frac{1}{6(n+1)} > 0$$

扫描全能王 创建

表 1.1 两种迭代法的计算结果

n	I_n(按算法 I 计算)	I_n(按算法 II 计算)
0	0.182 321 55	0.182 321 55
1	0.088 392 25	0.088 392 22
2	0.058 038 75	0.058 038 92
3	0.043 139 58	0.043 138 73
4	0.034 302 08	0.034 306 33
5	0.028 489 58	0.028 468 35
6	0.024 218 75	0.024 324 91
7	0.021 763 39	0.021 232 60
8	0.016 183 05	0.018 836 99
9	0.030 195 88	0.016 926 17
10	−0.050 979 41	0.015 369 14
11	0.345 806 12	0.014 063 39
12	−0.645 697 26	0.013 016 36
13	8.305 409 38	0.011 841 27
14	−41.455 618 31	0.012 222 22

而按算法Ⅱ计算, 尽管 I_{14}^* 取值精度不高, 其误差限

$$\varepsilon = \frac{1}{2}\left(\frac{1}{75} - \frac{1}{90}\right) \approx 0.001\ 1$$

但递推计算得到的 I_0^* 却有 8 位有效数字. 为什么会出现这样的现象? 下面的分析说明, 这是舍入误差在计算过程中传播所引起的后果.

设 I_0^* 有误差 e_0. 假设计算过程中不产生新的舍入误差, 则由式 (1.8) 得

$$e_n = I_n - I_n^* = -5I_{n-1} + 5I_{n-1}^* = -5e_{n-1} \quad (n = 1, 2, \cdots)$$

从而有

$$e_n = (-5)^n e_0 \tag{1.10}$$

即原始数据 I_0^* 的误差 e_0 经公式 (1.8) 计算一次, 误差绝对值就扩大到 5 倍, 因而由算法 I 计算出的 I_n^* 的误差绝对值应是 e_0 绝对值的 5^n 倍. 在上例中, I_0^* 有 8 位有效数字, 故

$$\varepsilon_0 = \frac{1}{2} \times 10^{-8}$$

由式 (1.10) 得

$$\varepsilon_9 = 5^9\varepsilon_0 = \frac{10^9 \times 10^{-8}}{2^{10}} = \frac{10}{1\ 024} > \frac{1}{2} \times 10^{-2}$$

而 $I_9^* = 0.030\ 195\ 88$, 这说明 I_9^* 已无一位有效数字.

完全类似地, 按算法算法Ⅱ从 I_k 计算 I_{k-1}, 应有

$$e_{k-1} = \frac{-1}{5}e_k$$

扫描全能王 创建

从而有

$$e_0 = \left(-\frac{1}{5}\right)^n e_n$$

因此, 从 I_{14}^* 出发计算到 I_0^* 时, 其误差 e_0 绝对值已缩小到 $\frac{e_{14}}{5^{14}}$.

上述事实说明, 对于同一数学问题, 使用的算法不同, 效果也大不相同. 称计算过程中舍入误差不增长的算法具有数值稳定性, 否则就是数值不稳定的. 在上例中, 算法Ⅱ具有数值稳定性, 而算法Ⅰ则是数值不稳定的. 显然, 只有选用数值稳定性好的算法, 才能求得较准确的结果.

1.4　数值计算中应注意的问题

由于误差的影响, 计算过程中可能出现一些坏现象, 应尽量注意以下几个方面.

1. 避免两个相近的数相减

由式(1.7), 两数之差 $u = x - y$ 的相对误差为

$$e_{\mathrm{r}}(u) = e_{\mathrm{r}}(x - y) = \frac{e(x) - e(y)}{x - y}$$

当 x 与 y 很接近时, u 的相对误差很大, 有效数字位数将严重丢失. 例如, 求 $x = \sqrt{1 + 10^{-7}} - 1$ 的近似值时, 如果运算过程中取 8 位有效数字, 则得 $x \approx 0.5 \times 10^{-7}$, 只有一位有效数字. 为了避免这种情形出现, 常常改变计算公式. 当然也可以通过在计算过程中多保留几位有效数字来改善计算效果, 但不如前者有效. 如把 $x = \sqrt{1 + 10^{-7}} - 1$ 变换成以下公式计算

$$x = \frac{10^{-7}}{\sqrt{1 + 10^{-7}} + 1}$$

计算过程仍保留 8 位有效数字, 则得 $x \approx 0.499\ 999\ 99 \times 10^{-7}$, 结果仍有 8 位有效数字.

例 1.3　已知 $\cos 2^\circ \approx 0.999\ 4, \sin 1^\circ \approx 0.017\ 5$(均精确到小数点后第四位), 计算 $x = 1 - \cos 2^\circ$ 的近似值.

解　由 $\cos 2^\circ \approx 0.999\ 4$ 得

$$x = 1 - \cos 2^\circ \approx 1 - 0.999\ 4 = 0.000\ 6 = x^*$$

但

$$|x - x^*| = |\cos 2^\circ - 0.999\ 4| \leqslant \frac{1}{2} \times 10^{-4}$$

故近似值 $x^* = 0.000\ 6$ 有一位有效数字.

若改用公式

$$1 - \cos 2^\circ = 2\sin^2 1^\circ$$

计算, 由 $\sin 1^\circ \approx 0.017\ 5$ 得

$$x = 2\sin^2 1^\circ \approx 2 \times (0.017\ 5)^2 = 0.612\ 5 \times 10^{-3} = x^\circ$$

扫描全能王 创建

此时

$$|e(x^\circ)| = |4\sin 1^\circ e(\sin 1^\circ)| \leqslant 4 \times 0.017\ 5 \times \frac{1}{2} \times 10^{-4} < \frac{1}{2} \times 10^{-5}$$

故 $x^* = 0.612\ 5 \times 10^{-3}$ 至少有 2 位有效数字.

一般地, 当 x 接近于零时, 应作变换

$$1 - \cos x = 2\sin^2 \frac{x}{2}, \quad \frac{1-\cos x}{\sin x} = \frac{\sin x}{1+\cos x}$$

当 x 充分大时, 应作变换

$$\sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1}+\sqrt{x}}$$

$$\frac{1}{x} - \frac{1}{x+1} = \frac{1}{x(x+1)}$$

等.

2. 避免大数“吃”小数的现象

计算机在进行运算时, 首先要把参加运算的数对阶, 即把两数都写成绝对值小于 1 而阶码相同的数. 如 $a = 10^9 + 1$, 必须改写成

$$a = 0.1 \times 10^{10} + 0.000\ 000\ 000\ 1 \times 10^{10}$$

如果计算机只能表示 8 位小数, 则算出 $a = 0.1 \times 10^{10}$, 大数“吃”了小数. 这种情形有时是允许的, 有时则可能得出错误的结果.

例 1.4 *求二次方程*

$$x^2 - \left(10^9 + 1\right) x + 10^9 = 0$$

的根.

解 利用因式分解容易求出, 此方程的两个根为 $x_1 = 10^9, x_2 = 1$, 但若用求根公式, 则得

$$x = \frac{10^9 + 1 \pm \sqrt{\left(10^9+1\right)^2 - 4 \times 10^9}}{2}$$

若用 8 位小数的计算机运算, 由于对阶, 有

$$10^9 + 1 \approx 10^9$$

$$\sqrt{\left(10^9+1\right)^2 - 4 \times 10^9} \approx 10^9$$

这样求得 $x_1 = 10^9, x_2 = 0$, 结果显然是错的. 为避免这种情形出现, 也可采用改变计算公式的方法. 如将式

$$x_2 = \frac{10^9 + 1 - \sqrt{\left(10^9+1\right)^2 - 4 \times 10^9}}{2}$$

改变成

扫描全能王 创建

$$x_2=\frac{2\times 10^9}{10^9+1+\sqrt{(10^9+1)^2-4\times 10^9}}$$

则有

$$x_2\approx\frac{2\times 10^9}{10^9+10^9}=1$$

此结果是准确的.

3. 避免除数的绝对值远小于被除数的绝对值

由式 (1.7)有

$$e\left(\frac{x}{y}\right)=\frac{ye(x)-xe(y)}{y^2}$$

故当 $|y|\ll|x|$ 时, 舍入误差可能增大很多.

4. 要简化计算, 减少运算次数, 提高效率

求一个问题的数值解往往有多种算法, 不同的算法需要不同的计算量, 而计算量的大小会影响误差的积累. 例如, 若选用级数

$$\ln 2=1-\frac{1}{2}+\frac{1}{3}-\frac{1}{4}+\cdots+\frac{(-1)^{n-1}}{n}+\cdots$$

的前 n 项部分和来计算 $\ln 2$ 的近似值, 截断误差限为 $\dfrac{1}{n+1}$. 如果要求误差小于 10^{-5}, 则 $n\geqslant 10^5$, 即要取前十万项求和. 这样做不仅计算量大, 而且舍入误差的积累将使有效数字丢失严重. 如果利用级数

$$\ln\frac{1+x}{1-x}=2x\left(1+\frac{1}{3}x^2+\frac{1}{5}x^4+\cdots+\frac{x^{2n}}{2n+1}+\cdots\right)$$

来计算, 当 $x=\dfrac{1}{3}$ 时, 有

$$\ln 2=\frac{2}{3}\left[1+\frac{1}{3\times 9}+\frac{1}{5\times 9^2}+\cdots+\frac{1}{(2n+1)\times 9^n}+\cdots\right]$$

取前 5 项之和作为近似值, 产生的截断误差为

$$\begin{aligned}e&=\frac{2}{3}\left(\frac{1}{11\times 9^5}+\frac{1}{13\times 9^6}+\frac{1}{15\times 9^7}+\cdots\right)\\&<\frac{2}{3}\times\frac{1}{11\times 9^5}\left(1+\frac{1}{9}+\frac{1}{9^2}+\cdots\right)\\&=\frac{2}{3\times 11\times 9^5}\times\frac{1}{1-\frac{1}{9}}\\&=\frac{1}{12\times 11\times 9^4}<\frac{1}{10^5}\end{aligned}$$

显然, 第二种算法比第一种算法有效.

扫描全能王 创建

又如计算多项式

$$P_n(x)=a_nx^n+a_{n-1}x^{n-1}+\cdots+a_1x+a_0 \tag{1.11}$$

的值. 若直接按式(1.11)计算, 共需作 $\dfrac{n(n+1)}{2}$ 次乘法与 n 次加法. 若按秦九韶算法

$$\begin{cases} u_n=a_n \\ u_k=xu_{k+1}+a_k \quad (k=n-1,n-2,\cdots,1,0) \\ P_n(x)=u_0 \end{cases} \tag{1.12}$$

计算, 即将式(1.11) 改写成如下形式

$$P_n(x)=a_0+x\{a_1+x[\cdots x(a_{n-2}+x(a_{n-1}+a_nx))\cdots]\}$$

则只需作 n 次乘法和 n 次加法.

由以上例子可以看出选择合适的公式, 简化计算十分重要.

5. 选用数值稳定性好的算法

评注一

本章介绍了数值计算方法和误差的基本概念, 以及误差在近似计算中的传播规律. 误差在数值计算中的危害是十分严重的, 若未能控制误差的传播与积累, 误差会将精确值完全淹没, 得到完全错误的结果. 因此, 误差的分析及其控制是一个非常重要的问题, 本章讨论了选用和设计算法应遵循的若干原则, 这对防止误差的传播和积累、确定方法的稳定性以及判别计算结果的可靠性有很大的帮助. 实际计算中, 误差分析是一个困难而复杂的问题, 尤其对大型科学计算的误差估计仍无定量的估计方法. 在数值计算方法中, 一般注重误差的定性分析, 即只讨论各种算法的数值稳定性, 只要是数值稳定的, 就不再进行舍入误差估计; 截断误差将针对各种问题结合具体算法讨论.

附录简单介绍了本课程所需的通用计算软件北太天元的部分命令, 如何更加深入和熟练地运用北太天元进行数值计算, 以及开发利用北太天元工具箱解决大型科学计算与仿真实验问题, 有待于读者自己的摸索与实验, 在以后各章中也会有更详细的计算实例. 读者也可参阅有关北太天元方面的书籍或上网搜寻相关知识, 观看北京大学的卢朓老师等录制的讲解视频.

本章知识要点 误差的基本概念; 误差的传播; 数值稳定性.

算法历史及背后的数学家

扫描全能王 创建

习题一

1. 按四舍五入原则, 求下列各数的具有 4 位有效数字的近似值:

$$168.957, 3.000\ 45, 73.225\ 0, 0.001\ 526\ 32$$

2. 设下列各数均为经过四舍五入后得到的近似值, 试求各数的绝对误差限和相对误差限:

$$a = 3\ 580, b = 0.004\ 76, c = 2\ 958 \times 10^{-2}, d = 0.143\ 0 \times 10^{8}$$

3. 已知 $a = 1.203\ 1, b = 0.978$ 是经过四舍五入后得到的近似值, 问 $a+b$, $a \times b$ 有几位有效数字?

4. 设 $x > 0, x$ 的相对误差为 δ, 求 $\ln x$ 的绝对误差.

5. 求 $\sqrt{2}$ 的近似值 x^*, 使其相对误差不超过 0.1%.

6. 要使 $\sqrt{20}$ 的近似值有小于 0.1% 的相对误差, 要取几位有效数字?

7. 正方形的边长约 100 cm, 问测量边长时误差应多大, 才能保证面积的误差不超过 1 cm^2 ?

8. 计算球的体积, 为使其相对误差限为 1%, 问测量半径 R 时, 相对误差最大为多少?

9. 设 $s = \frac{1}{2}gt^2$, 假定 g 是准确的, 而对 t 的测量有 ± 0.1 s 的误差, 试证当 t 增大时, s 的绝对误差增大而相对误差却减小.

10. 已知 $\sqrt{783} \approx 27.983$ 有 5 位有效数字, 试求方程

$$x^2 - 56x + 1 = 0$$

的两个根, 使它们至少有 4 位有效数字.

11. 设 $I_n = \mathrm{e}^{-1} \int_0^1 x^n \mathrm{e}^x \, \mathrm{d}x$, 求证:

(1) $I_n = 1 - nI_{n-1} \quad (n = 0, 1, 2, \cdots)$;

(2) 利用 (1) 中的公式正向递推计算时误差逐步增大, 反向递推计算时误差逐步减小.

12. 试改变下列表达式, 使计算结果比较精确:

(1) $\dfrac{1}{1+2x} - \dfrac{1-x}{1+x}, |x| \ll 1$;

(2) $\sqrt{x + \dfrac{1}{x}} - \sqrt{x - \dfrac{1}{x}}, |x| \gg 1$;

(3) $\dfrac{\tan x - \sin x}{x^2}, |x| \ll 1$ 且 $x \neq 0$.

扫描全能王 创建

数值实验一

1. 利用循环语句, 计算数列 $\sqrt{5}, \sqrt{\sqrt{5}}, \sqrt{\sqrt{\sqrt{5}}}, \cdots$ 的极限, 要求误差小于 10^{-8}.

2. 利用级数

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots$$

计算无理数 π 的近似值. 由于交错级数的部分和数列 S_N 围绕级数的和上下摆动, 截断误差将小于第一个被舍弃的项 a_{n+1} 的绝对值. 试分析: 若要使截断误差小于 10^{-4} 或 10^{-8}, 应取多少项求和? 并分别计算 π 的近似值.

3. 在同一坐标系下, 利用 plot 函数画出函数

$$y = \sin x, \quad y_n = \sum_{i=0}^{n} (-1)^i \frac{x^{2i+1}}{(2i+1)!} \quad (n = 2, 5, 10)$$

的图形, 并加标注说明各条曲线的含义.

4. 分别将区间 $[-10, 10]$ 分为 $100, 200, 400$ 等份, 利用 mesh 或 surf 命令画出二元函数

$$z = \mathrm{e}^{-|x|} + \cos(x + y) + \frac{1}{x^2 + y^2 + 1}$$

的三维图形.

扫描全能王 创建

/第二章/

解线性方程组的直接方法

在科学研究和工程计算中, 经常会遇到求解线性方程组的问题. 这类问题一方面来源于广泛的实际应用, 如电网络分析、电磁场数值计算、结构设计问题等; 另一方面来源于数学问题本身, 如数据的插值与拟合、用差分方法或有限元法求解微分方程与积分方程等. 因此, 线性方程组的数值解法是数值计算的重要内容.

设 n 阶线性方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \cdots\cdots\cdots\cdots \\ a_{n1}x_1 + a_{n2}x_2 + \cdots + a_{nn}x_n = b_n \end{cases} \tag{2.1}$$

其矩阵形式为

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

其中

$$\boldsymbol{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}, \quad \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

如果线性方程组(2.1) 的系数矩阵 $\boldsymbol{A}$ 的行列式不为零, 即 $\det \boldsymbol{A} \neq 0$, 则该方程组有唯一解. 由 Cramer(克拉默) 法则, 其解为

$$x_i = \frac{\det \boldsymbol{A}_i}{\det \boldsymbol{A}} \quad (i = 1, 2, \cdots, n)$$

其中 $\boldsymbol{A}_i$ 为用方程组(2.1)的右端向量 $\boldsymbol{b}$ 代替 $\boldsymbol{A}$ 中第 i 列向量所得的矩阵. 这种方法需要计算 $n+1$ 个 n 阶行列式并作 n 次除法, 而每个 n 阶行列式计算需作 $(n-1) \times n!$ 次乘法, 计算量十分惊人. 如 $n = 30$ 时, 就需作约 2.38×10^{35} 次乘法. 可见 Cramer 法则在理论上是绝对正确的, 但当 n 较大时, 在实际计算中却是不可行的.

解线性方程组的方法大致可分为两类: 直接方法和迭代法. 直接方法是指假设计算过程中不产生舍入误差, 经过有限次运算可求得方程组的精确解的方法. 迭代法是从解的某个近似值出发, 通过构造一个无穷序列去逼近精确解的方法. 一般地, 有限步内得不到精确解.

本章介绍几种经典的解线性方程组的直接方法及其有关问题. 在以下文中, 如不作特别说明, 均假设方程组(2.1)的系数矩阵非奇异, 方程组存在唯一解.

扫描全能王 创建

2.1 消 去 法

2.1.1 Gauss 消去法

如果线性方程组的系数矩阵是三角形矩阵, 则方程组极易求解. 解线性方程组的直接方法的基本思想是将方程组(2.1) 变形为等价的三角形方程组, 然后求解. Gauss (高斯) 消去法是最基本和最简单的直接方法, 它由消元过程和回代过程构成. Gauss 消去法的基本思想和过程是: 将方程组逐列逐行消去变量, 转化为等价的上三角形方程组, 此过程称为消元过程. 然后按着方程组相反顺序求解上三角形方程组, 得原方程组的解, 此过程称为回代过程.

为统一起见, 将方程组(2.1)改写成

$$\begin{cases} a_{11}^{(1)}x_1 + a_{12}^{(1)}x_2 + \cdots + a_{1n}^{(1)}x_n = b_1^{(1)} \\ a_{21}^{(1)}x_1 + a_{22}^{(1)}x_2 + \cdots + a_{2n}^{(1)}x_n = b_2^{(1)} \\ \cdots\cdots\cdots\cdots \\ a_{n1}^{(1)}x_1 + a_{n2}^{(1)}x_2 + \cdots + a_{nn}^{(1)}x_n = b_n^{(1)} \end{cases} \tag{2.2}$$

简记为 $\boldsymbol{A}^{(1)}\boldsymbol{x} = \boldsymbol{b}^{(1)}$, 其中 $\boldsymbol{A}^{(1)} = \boldsymbol{A}, \boldsymbol{b}^{(1)} = \boldsymbol{b}$.

一般地, 求解 n 阶方程组(2.1)的 Gauss 消去法的步骤如下:

消元过程

第一步: 设 $a_{11}^{(1)} \neq 0$, 记 $l_{i1} = a_{i1}^{(1)}/a_{11}^{(1)} (i = 2, 3, \cdots, n)$, 将方程组(2.2)中第 i 个方程减去第 1 个方程乘 l_{i1} $(i = 2, 3, \cdots, n)$, 完成第一次消元, 得方程组(2.2) 的同解方程组

$$\begin{cases} a_{11}^{(1)}x_1 + a_{12}^{(1)}x_2 + \cdots + a_{1n}^{(1)}x_n = b_1^{(1)} \\ \qquad\quad a_{22}^{(2)}x_2 + \cdots + a_{2n}^{(2)}x_n = b_2^{(2)} \\ \qquad\quad \cdots\cdots\cdots\cdots \\ \qquad\quad a_{n2}^{(2)}x_2 + \cdots + a_{nn}^{(2)}x_n = b_n^{(2)} \end{cases} \tag{2.3}$$

其中 $a_{ij}^{(2)} = a_{ij}^{(1)} - l_{i1}a_{1j}^{(1)}, b_i^{(2)} = b_i^{(1)} - l_{i1}b_1^{(1)}$ $(i, j = 2, 3, \cdots, n)$. 方程组(2.3)简记为 $\boldsymbol{A}^{(2)}\boldsymbol{x} = \boldsymbol{b}^{(2)}$.

第二步: 设 $a_{22}^{(2)} \neq 0$, 记 $l_{i2} = a_{i2}^{(2)}/a_{22}^{(2)}$ $(i = 3, 4, \cdots, n)$. 将方程组(2.3)中第 i 个方程减去第 2 个方程乘 l_{i2} $(i = 3, 4, \cdots, n)$, 完成第二次消元.

第 k 步: 设第 $k-1$ 次消元完成后得方程组(2.2)的同解方程组为

扫描全能王 创建

$$\begin{cases} a_{11}^{(1)}x_1+a_{12}^{(1)}x_2+\cdots+a_{1k}^{(1)}x_k+\cdots+a_{1n}^{(1)}x_n=b_1^{(1)} \\ a_{22}^{(2)}x_2+\cdots+a_{2k}^{(2)}x_k+\cdots+a_{2n}^{(2)}x_n=b_2^{(2)} \\ \cdots\cdots\cdots\cdots \\ a_{kk}^{(k)}x_k+\cdots+a_{kn}^{(k)}x_n=b_k^{(k)} \\ \cdots\cdots\cdots\cdots \\ a_{nk}^{(k)}x_k+\cdots+a_{nn}^{(k)}x_n=b_n^{(k)} \end{cases} \tag{2.4}$$

简记为 $A^{(k)}x=b^{(k)}$. 设 $a_{kk}^{(k)}\neq 0$, 记 $l_{ik}=a_{ik}^{(k)}/a_{kk}^{(k)}$ $(i=k+1,k+2,\cdots,n)$. 将式(2.4)中第 i 个方程减去第 k 个方程乘 l_{ik} $(i=k+1,k+2,\cdots,n)$, 完成第 k 次消元, 得同解方程组

$$\begin{cases} a_{11}^{(1)}x_1+a_{12}^{(1)}x_2+\cdots+a_{1k}^{(1)}x_k+a_{1k+1}^{(1)}x_{k+1}+\cdots+a_{1n}^{(1)}x_n &=b_1^{(1)} \\ a_{22}^{(2)}x_2+a_{23}^{(2)}x_3+\cdots+a_{2k}^{(2)}x_k++\cdots+a_{2n}^{(2)}x_n &=b_2^{(2)} \\ \cdots\cdots\cdots\cdots & \\ a_{kk}^{(k)}x_k+a_{k\ k+1}^{(k)}x_{k+1}+\cdots+a_{kn}^{(k)}x_n &=b_k^{(k)} \\ a_{k+1\ k+1}^{(k+1)}x_{k+1}+\cdots a_{k+1\ n}^{(k+1)}x_n &=b_{k+1}^{(k+1)} \\ \cdots\cdots\cdots\cdots & \\ a_{n\ k+1}^{(k+1)}x_{k+1}+\cdots a_{nn}^{(k+1)}x_n &=b_n^{(k+1)} \end{cases}$$

其中 $a_{ij}^{(k+1)}=a_{ij}^{(k)}-l_{ik}a_{kj}^{(k)}, b_i^{(k+1)}=b_i^{(k)}-l_{ik}b_k^{(k)}$ $(i,j=k+1,k+2,\cdots,n)$. 按上述作法, 完成 $n-1$ 次消元后, 方程组 (2.2) 化成同解的上三角形方程组

$$\begin{cases} a_{11}^{(1)}x_1+a_{12}^{(1)}x_2+a_{13}^{(1)}x_3+\cdots+a_{1n}^{(1)}x_n=b_1^{(1)} \\ a_{22}^{(2)}x_2+a_{23}^{(2)}x_3+\cdots+a_{2n}^{(2)}x_n=b_2^{(2)} \\ a_{33}^{(3)}x_3+\cdots+a_{3n}^{(2)}x_n=b_3^{(2)} \\ \cdots\cdots\cdots\cdots \\ a_{nn}^{(n)}x_n=b_n^{(n)} \end{cases}$$

简记为 $A^{(n)}x=b^{(n)}$.

回代过程

按变量的逆序逐步回代得方程组 (2.2) 的解.

$$\begin{cases} x_n=b_n^{(n)}/a_{nn}^{(n)} \\ x_k=\left(b_k^{(k)}-\sum\limits_{l=k+1}^{n}a_{kl}^{(k)}x_l\right)/a_{kk}^{(k)} \quad (k=n-1,n-2,\cdots,1) \end{cases} \tag{2.5}$$

扫描全能王 创建

算法 2.1 Gauss 消去法

1. 输入 $\boldsymbol{A}=(a_{ij})$, $\boldsymbol{b}=(b_1,b_2,\cdots,b_n)^{\mathrm{T}}$, n, 置 $k=1$.
2. 若 $a_{kk}\neq 0$, 则转 3 ; 否则输出失败信息, 停机.
3. 对 $i=k+1,k+2,\cdots,n$, 置 $a_{ik}/a_{kk}\Rightarrow a_{ik}$, $b_i-a_{ik}b_k\Rightarrow b_i$;
   对 $j=k+1,k+2,\cdots,n$, 置 $a_{ij}-a_{ik}a_{kj}\Rightarrow a_{ij}$.
4. 若 $k=n-1$, 则转 5 ; 否则 $k+1\Rightarrow k$, 转 2 .
5. 若 $a_{nn}=0$, 则输出失败信息, 停机; 否则, 置 $b_n/a_{nn}\Rightarrow b_n$.
6. 对 $k=n-1,n-2,\cdots,1$, 置 $\left(b_k-\sum_{l=k+1}^{n}a_{kl}b_l\right)/a_{kk}\Rightarrow b_k$.
7. 输出 $\boldsymbol{x}=\boldsymbol{b}$, 停机.

Gauss 消去法的计算量

由于计算机作乘除运算所需时间远大于作加减运算所需时间, 故我们只讨论乘除运算量.

由消去法步骤知, 在进行第 k 次消元时, 需作除法 $n-k$ 次, 乘法 $(n-k)\,(n-k+1)$ 次, 故消元过程中乘除运算总量为

$$\text{乘法次数}\sum_{k=1}^{n-1}(n-k)(n-k+1)=\frac{n}{3}\left(n^2-1\right)$$

$$\text{除法次数}\sum_{k=1}^{n-1}(n-k)=\frac{n}{2}(n-1)$$

由式(2.5), 在回代过程中, 计算 x_k 需要 $n-k+1$ 次乘除法, 整个回代过程需要乘除运算的总量为

$$\sum_{k=1}^{n}(n-k+1)=\frac{n}{2}(n+1)$$

所以, Gauss 消去法的乘除总运算量为

$$\begin{aligned}N&=\frac{n}{3}\left(n^2-1\right)+\frac{n}{2}(n-1)+\frac{n}{2}(n+1)\\&=\frac{n^3}{3}+n^2-\frac{n}{3}\end{aligned}$$

由上式容易求出, 用 Gauss 消去法求解 30 阶线性方程组时, 共需乘除运算的次数为

$$\frac{30^3}{3}+30^2-\frac{30}{3}=9\ 890$$

远少于用 Cramer 法则求解所需的乘除运算量.

扫描全能王 创建

2.1.2　主元素消去法

Gauss 消去法虽然简单易行, 但计算过程要求 $a_{kk}^{(k)}$ (称为主元素) 均不为零, 否则 Gauss 消去法不能进行. 另一方面, 有时主元素 $a_{kk}^{(k)} \neq 0$, 但 $\left|a_{kk}^{(k)}\right|$ 很小, 用其作除数, 会使

$$\left|l_{ik}^{(k)}\right| = \left|\frac{a_{ik}^{(k)}}{a_{kk}^{(k)}}\right|$$

很大, 导致舍入误差的严重扩大以及计算结果的不正确.

例 2.1　求解方程组

$$\begin{cases} 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \\ 2.0x_1 + 4.5x_2 + 0.36x_3 = 0.020 \\ 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \end{cases}$$

其中所有系数均有 2 位有效数字. 为减小误差, 计算过程中保留 3 位有效数字.

解　按 Gauss 消去法求解, 第一次消元得同解方程组

$$\begin{cases} 0.50x_1 + \quad 1.1x_2 + \ 3.1x_3 = 6.0 \\ \qquad\quad 0.100x_2 - 12.0x_3 = -24.0 \\ \qquad\quad -10.0x_2 - 24.5x_3 = -59.0 \end{cases}$$

第二次消元得

$$\begin{cases} 0.50x_1 + \quad 1.1x_2 + 3.1x_3 = 6.0 \\ \qquad\quad 0.100x_2 - 12.0x_3 = -24.0 \\ \qquad\qquad\quad -1\ 220x_3 = -2\ 460 \end{cases}$$

回代得解

$$x_3 = 2.02, \quad x_2 = 2.40, \quad x_1 = -5.80$$

容易验证, 该方程组的精确解为

$$x_1 = -2.60, \quad x_2 = 1.00, \quad x_3 = 2.00$$

显然两者相差很大. 但若在解方程组前, 先把方程的次序交换一下, 将方程组改写成

$$\begin{cases} 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \\ 2.0x_1 + 4.5x_2 + 0.36x_3 = 0.020 \\ 0.50x_1 + 1.1x_2 + 3.1x_3 = 6.0 \end{cases}$$

再用 Gauss 消去法求解, 消元后得同解方程组

扫描全能王 创建

$$\begin{cases} 5.0x_1 + 0.96x_2 + 6.5x_3 = 0.96 \\ \qquad 4.12x_2 - 2.24x_3 = -0.364 \\ \qquad\qquad 2.99x_3 = 5.99 \end{cases}$$

回代得解

$$x_3 = 2.00, \quad x_2 = 1.00, \quad x_1 = -2.60$$

与精确解相同.

产生上述现象的原因在于舍入误差. 因为按原方程组的方程顺序进行消元时, 主元 $a_{11}^{(1)} = 0.50, a_{22}^{(2)} = 0.100$ 都比较小, 以它们为除数就增长了舍入误差, 从而导致计算结果不准确. 为了在计算过程中抑制舍入误差的增长, 应尽量避免小主元的出现. 如例 2.1 中第二种解法, 通过交换方程次序, 选取绝对值大的元素作主元. 基于这种想法导出了主元素消去法.

(一) 列主元素法

为简便起见, 用方程组(2.1)的增广矩阵

$$[\boldsymbol{A}, b] = \begin{bmatrix} a_{11} & \cdots & a_{1n} & b_1 \\ \vdots & & \vdots & \vdots \\ a_{n1} & \cdots & a_{nn} & b_n \end{bmatrix} \tag{2.6}$$

表示方程组, 并直接在增广矩阵上运算.

列主元素法的基本思想是在每次消元前, 在要消去未知数的系数中找到绝对值最大的系数作为主元, 通过方程对换将其换到对角线上, 然后进行消元. 其具体步骤如下:

第一步: 首先在矩阵(2.6) 的第 1 列中选取绝对值最大的元, 比如为 $a_{i_1 1}$, 则 $|a_{i_1 1}| = \max\limits_{1 \leqslant i \leqslant n} |a_{i1}|$. 将 (2.6) 中第 1 行与第 i_1 行互换. 为方便起见, 记行互换后的增广矩阵为 $\left[\boldsymbol{A}^{(1)}, b^{(1)}\right]$, 然后进行第一次消元, 得矩阵

$$\left[\boldsymbol{A}^{(2)}, b^{(2)}\right] = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & \vdots & & \vdots & \vdots \\ & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} & b_n^{(2)} \end{bmatrix}$$

第二步: 在矩阵 $\left[\boldsymbol{A}^{(2)}, b^{(2)}\right]$ 的第 2 列中选主元, 比如 $a_{i_2 2}^{(2)}$, 使 $\left|a_{i_2 2}^{(2)}\right| = \max\limits_{2 \leqslant i \leqslant n} \left|a_{i2}^{(2)}\right|$. 将矩阵 $\left[\boldsymbol{A}^{(2)}, b^{(2)}\right]$ 的第 2 行与第 i_2 行互换, 再进行第二次消元, 得矩阵 $\left[\boldsymbol{A}^{(3)}, b^{(3)}\right]$.

第 k 步: 在矩阵 $\left[\boldsymbol{A}^{(k)}, b^{(k)}\right]$ 的第 k 列中选主元, 如 $a_{i_k k}^{(k)}$, 使 $\left|a_{i_k k}^{(k)}\right| = \max\limits_{k \leqslant i \leqslant n} \left|a_{ik}^{(k)}\right|$. 将 $\left[\boldsymbol{A}^{(k)}, b^{(k)}\right]$ 的第 k 行与第 i_k 行互换, 进行第 k 次消元.

如此经过 $n-1$ 步, 增广矩阵(2.6)被化成上三角形矩阵, 最后由回代过程求解.

在上述过程中, 主元是按列选取的, 列主元素法由此得名. 例 2.1中的第二种解法就是

扫描全能王 创建

按列主元素法进行的.

(二) 全主元素法

如果不是按列选主元, 而是在全体待选系数 $a_{ij}^{(k)}$ $(i,j=k,k+1,\cdots,n)$ 中选取主元, 则得到全主元素法, 其计算过程如下:

第一步: 在全体系数 a_{ij} $(i,j=1,2,\cdots,n)$ 中选取绝对值最大的元作为主元, 并通过行与列的互换把它换到 $a_{11}^{(1)}$ 的位置, 然后进行第一次消元, 得到矩阵 $\left[A^{(2)},\ b^{(2)}\right]$.

第 k 步: 在矩阵 $A^{(k)}$ 的右下方 $n-k+1$ 阶子矩阵的所有元素 $a_{ij}^{(k)}$ $(i,j=k,k+1,\cdots,n)$ 中, 选取绝对值最大的元作为主元, 并通过行与列的互换将它换到 $a_{kk}^{(k)}$ 的位置, 然后进行第 k 次消元.

经过 $n-1$ 次消元后, 得到与方程组 (2.1)同解的上三角形方程组, 再由回代过程求解.

例 2.2　用主元素法求解线性方程组

$$\begin{cases} x_1+\ x_2+\ x_3=6 \\ 12x_1-3x_2+3x_3=15 \\ -18x_1+3x_2-\ x_3=-15 \end{cases}$$

计算过程保留 3 位小数.

解　按列主元素法, 求解过程如下:

$$\begin{bmatrix} 1 & 1 & 1 & 6 \\ 12 & -3 & 3 & 15 \\ \boxed{-18} & 3 & -1 & -15 \end{bmatrix} \xrightarrow{\text{第一、三行互换}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 12 & -3 & 3 & 15 \\ 1 & 1 & 1 & 6 \end{bmatrix}$$

$$\xrightarrow{\text{第一次消元}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 0 & -1 & 2.333 & 5 \\ 0 & \boxed{1.167} & 0.944 & 5.167 \end{bmatrix}$$

$$\xrightarrow{\text{第二、三行互换}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 0 & \boxed{1.167} & 0.944 & 5.167 \\ 0 & -1 & 2.333 & 5 \end{bmatrix}$$

$$\xrightarrow{\text{第二次消元}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 0 & \boxed{1.167} & 0.944 & 5.167 \\ 0 & 0 & 3.142 & 9.428 \end{bmatrix}$$

由回代过程得解

$$x_3=3.001,\quad x_2=2.000,\quad x_1=1.000$$

按全主元素法, 求解过程如下:

扫描全能王 创建

$$\begin{bmatrix} 1 & 1 & 1 & 6 \\ 12 & -3 & 3 & 15 \\ \boxed{-18} & 3 & -1 & -15 \end{bmatrix} \xrightarrow{\text{第一、三行互换}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 12 & -3 & 3 & 15 \\ 1 & 1 & 1 & 6 \end{bmatrix}$$

$$\xrightarrow{\text{第一次消元}} \begin{bmatrix} \boxed{-18} & 3 & -1 & -15 \\ 0 & -1 & \boxed{2.333} & 5 \\ 0 & 1.167 & 0.944 & 5.167 \end{bmatrix}$$

$$\xrightarrow{\text{第二、三列互换}} \begin{bmatrix} \boxed{-18} & -1 & 3 & -15 \\ 0 & \boxed{2.333} & -1 & 5 \\ 0 & 0.944 & 1.167 & 5.167 \end{bmatrix}$$

$$\xrightarrow{\text{第二次消元}} \begin{bmatrix} \boxed{-18} & -1 & 3 & -15 \\ 0 & \boxed{2.333} & -1 & 5 \\ 0 & 0 & 1.572 & 3.144 \end{bmatrix}$$

由回代过程得解

$$x_2 = 2.000, \quad x_3 = 3.000, \quad x_1 = 1.000$$

例 2.2的计算结果表明, 全主元素法的精度优于列主元素法, 这是由于全主元素是在全体系数中选主元, 故它对控制舍入误差十分有效. 但全主元素法在计算过程中, 需同时作行与列的互换, 因而程序比较复杂, 计算时间较长. 列主元素法的精度虽稍低于全主元素法, 但其计算简单, 工作量大为减少, 且计算经验与理论分析均表明, 它与全主元素法同样具有良好的数值稳定性, 故列主元素法是求解中小型稠密线性方程组的最好方法之一.

2.2 直接三角分解法

2.2.1 Gauss 消去法的矩阵形式

如果用增广矩阵 $[\boldsymbol{A}, \boldsymbol{b}]$ 表示线性方程组(2.1), Gauss 消去法的消元过程可以通过用一串初等矩阵左乘增广矩阵表示. 例如, 第一次消元等价于用初等矩阵

$$\boldsymbol{L}_1 = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ -l_{21} & 1 & 0 & \cdots & 0 \\ -l_{31} & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & & \vdots \\ -l_{n1} & 0 & 0 & \cdots & 1 \end{bmatrix}$$

左乘矩阵 $\left[\boldsymbol{A}^{(1)}, \boldsymbol{b}^{(1)}\right] = [\boldsymbol{A}, \boldsymbol{b}]$, 其中 $l_{i1} = a_{i1}^{(1)} / a_{11}^{(1)}$ $(i = 2, 3, \cdots, n)$, 即

扫描全能王 创建

$$\left[A^{(2)}, b^{(2)}\right] = L_1 \left[A^{(1)}, b^{(1)}\right]$$

一般地, 第 k 次消元等价于用初等矩阵

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1\ k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{bmatrix}$$

左乘矩阵 $\left[A^{(k)}, b^{(k)}\right]$, 其中 $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$ $(i = k+1, k+2, \cdots, n)$, 经过 $n-1$ 次消元后得到

$$\begin{aligned} \left[A^{(n)}, b^{(n)}\right] &= \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & \cdots & a_{1n}^{(1)} & b_1^{(1)} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} & b_2^{(2)} \\ & & \ddots & \vdots & \vdots \\ & & & a_{nn}^{(n)} & b_n^{(n)} \end{bmatrix} \\ &= L_{n-1} \left[A^{(n-1)}, b^{(n-1)}\right] \\ &= L_{n-1} L_{n-2} \cdots L_1 \left[A^{(1)}, b^{(1)}\right] \end{aligned}$$

因为 L_k $(k = 1, 2, \cdots, n-1)$ 均为非奇异矩阵, 故它们的逆矩阵存在. 容易求出

$$L_k^{-1} = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{k+1\ k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & 1 \end{bmatrix}$$

令

$$L = L_1^{-1} L_2^{-1} \cdots L_{n-1}^{-1} = \begin{bmatrix} 1 & & & & & \\ l_{21} & 1 & & & & \\ l_{31} & l_{32} & 1 & & & \\ \vdots & \vdots & \vdots & \ddots & & \\ & & & & 1 & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & l_{n\ n-1} & 1 \end{bmatrix} \tag{2.7}$$

于是有

扫描全能王 创建

$$
\begin{aligned}
[\boldsymbol{A}, \boldsymbol{b}] &= \left[\boldsymbol{A}^{(1)}, \boldsymbol{b}^{(1)}\right] \\
&= \boldsymbol{L}_1^{-1}\boldsymbol{L}_2^{-1}\cdots\boldsymbol{L}_{n-1}^{-1}\left[\boldsymbol{A}^{(n)}, \boldsymbol{b}^{(n)}\right]
\end{aligned}
$$

即

$$
[\boldsymbol{A}, \boldsymbol{b}] = \left[\boldsymbol{L}\boldsymbol{A}^{(n)}, \boldsymbol{L}\boldsymbol{b}^{(n)}\right] \tag{2.8}
$$

这说明, 消元过程实际上是把系数矩阵 $\boldsymbol{A}$ 分解成单位下三角形矩阵与上三角形矩阵的乘积的过程, 其中 L 为单位下三角形矩阵, $\boldsymbol{U}$ 为上三角形矩阵

$$
\boldsymbol{L} = \begin{bmatrix}
1 & & & & & \\
l_{21} & 1 & & & & \\
l_{31} & l_{32} & 1 & & & \\
\vdots & \vdots & \vdots & \ddots & & \\
l_{n-1,1} & l_{n-1,2} & l_{n-1,3} & \cdots & 1 & \\
l_{n1} & l_{n2} & l_{n3} & \cdots & l_{n,n-1} & 1
\end{bmatrix}
$$

$$
\boldsymbol{U} = \begin{bmatrix}
u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\
 & u_{22} & u_{23} & \cdots & u_{2n} \\
 & & \ddots & & \vdots \\
 & & & & u_{nn}
\end{bmatrix}
$$

上述分解称为 Doolittle (杜利特尔) 分解, 也称为 LU 分解.

当系数矩阵进行三角分解后, 求解方程组 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 的问题就变得十分容易, 它等价于求解两个三角形方程组 $\boldsymbol{L}\boldsymbol{y} = \boldsymbol{b}$ 和 $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{y}$. 因此, 解线性方程组问题可转化为矩阵的三角分解问题.

2.2.2 矩阵的直接三角分解

Gauss 消去法要求主元素 $a_{kk}^{(k)} \neq 0$ $(k = 1, 2, \cdots, n-1)$, 将这一条件与矩阵 $\boldsymbol{A}$ 自身的性质联系起来, 有

定理 2.1 *设 $\boldsymbol{A}$ 为 n 阶方阵, 若 $\boldsymbol{A}$ 的顺序主子式 $\det\ \boldsymbol{A}_i$ $(i = 1, 2, \cdots, n-1)$ 均不为零, 则矩阵 $\boldsymbol{A}$ 存在唯一的 Doolittle 分解.*

证明 当 $j = 1$ 时, 因为 $\det\ \boldsymbol{A}_1 = a_{11} \neq 0$, 由 2.2.1 中的讨论, 存在矩阵

$$
\boldsymbol{L}_1 = \begin{bmatrix}
1 & & \\
-l_{21} & 1 & \\
\vdots & & \ddots \\
-l_{n1} & \cdots & 1
\end{bmatrix}
$$

其中 $l_{i1} = a_{i1}/a_{11}$ $(i = 2, 3, \cdots, n)$, 使得

扫描全能王 创建

$$A^{(2)} = L_1 A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ & a_{22}^{(2)} & \cdots & a_{2n}^{(2)} \\ & \vdots & & \vdots \\ & a_{n2}^{(2)} & \cdots & a_{nn}^{(2)} \end{bmatrix}$$

由行列式的性质易得, $A^{(2)}$ 的各阶顺序主子式 $\det A_i^{(2)}$ $(i = 1, 2, \cdots, n-1)$ 满足

$$\det A_i^{(2)} = \det A_i \neq 0$$

假定结论对 $j = k-1$ 成立, 即

$$A^{(k)} = L_{k-1} \cdots L_1 A = \begin{bmatrix} a_{11} & a_{12} & & \cdots & a_{1n} \\ & a_{22}^{(2)} & & \cdots & a_{2n}^{(2)} \\ & & \ddots & & \vdots \\ & & a_{kk}^{(k)} & \cdots & a_{kn}^{(k)} \\ & & \vdots & & \vdots \\ & & a_{nk}^{(k)} & \cdots & a_{nn}^{(k)} \end{bmatrix}$$

且 $\det A_i^{(k)} \neq 0$ $(i = 1, 2, \cdots, n-1)$, 因为

$$\det A_k^{(k)} = \det A_{k-1}^{(k)} \times a_{kk}^{(k)} \neq 0$$

于是有 $a_{kk}^{(k)} \neq 0$, 故存在矩阵

$$L_k = \begin{bmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{k+1,\,k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{bmatrix}$$

其中 $l_{ik} = a_{ik}^{(k)} / a_{kk}^{(k)}$ $(i = k+1, k+2, \cdots, n)$, 使得

$$A^{(k+1)} = L_k A^{(k)} = \begin{bmatrix} a_{11} & a_{12} & & & \cdots & a_{1n} \\ & a_{22}^{(2)} & & & \cdots & a_{2n}^{(2)} \\ & & \ddots & & & \vdots \\ & & & a_{k+1,\,k+1}^{(k+1)} & \cdots & a_{k+1,\,n}^{(k+1)} \\ & & & \vdots & & \vdots \\ & & & a_{n,\,k+1}^{(k+1)} & \cdots & a_{nn}^{(k+1)} \end{bmatrix}$$

且 $\det A_i^{(k+1)} \neq 0$ $(i = 1, 2, \cdots, n-1)$.

按归纳法原理, 存在初等矩阵 $L_1, L_2, \cdots, L_{n-1}$, 使得

扫描全能王 创建

$$\boldsymbol{A}^{(n)}=\boldsymbol{L}_{n-1}\cdots\boldsymbol{L}_2\boldsymbol{L}_1\boldsymbol{A}=\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ & a_{22}^{(2)} & a_{23}^{(2)} & \cdots & a_{2n}^{(2)} \\ & & a_{33}^{(3)} & \cdots & a_{3n}^{(3)} \\ & & & \ddots & \vdots \\ & & & & a_{nn}^{(n)} \end{bmatrix}$$

由式(2.8) 得

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{A}^{(n)}=\boldsymbol{L}\boldsymbol{U}$$

其中 $\boldsymbol{L}=\boldsymbol{L}_1^{-1}\boldsymbol{L}_2^{-1}\cdots\boldsymbol{L}_{n-1}^{-1}$ 为单位下三角形矩阵, $\boldsymbol{U}=\boldsymbol{A}^{(n)}$ 为上三角形矩阵. 存在性得证.

下证唯一性. 设矩阵 $\boldsymbol{A}$ 有两种 Doolittle 分解

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{U}=\tilde{\boldsymbol{L}}\tilde{\boldsymbol{U}} \tag{2.9}$$

当 $\boldsymbol{A}$ 非奇异时, $\boldsymbol{L},\tilde{\boldsymbol{L}},\boldsymbol{U},\tilde{\boldsymbol{U}}$ 均为非奇异矩阵. 于是由式(2.9), 有

$$\tilde{\boldsymbol{L}}^{-1}\boldsymbol{L}=\tilde{\boldsymbol{U}}\boldsymbol{U}^{-1}$$

由于单位下 (上) 三角形矩阵的逆仍是单位下 (上) 三角形矩阵, 上 (下) 三角形矩阵的乘积仍为上 (下) 三角形矩阵, 故有

$$\tilde{\boldsymbol{L}}^{-1}\boldsymbol{L}=\tilde{\boldsymbol{U}}\boldsymbol{U}^{-1}=\boldsymbol{I}$$

所以 $\boldsymbol{L}=\tilde{\boldsymbol{L}},\boldsymbol{U}=\tilde{\boldsymbol{U}}$.

注 2.1　从以上存在性证明过程可以看出, Gauss 消去法的消去过程就是系数矩阵三角分解的一种算法, 消去过程完成, 即完成系数矩阵三角分解.

还可以通过待定系数法直接计算矩阵的三角分解, 下面讨论如何对 $\boldsymbol{A}$ 进行直接 LU 分解. 两个矩阵相等就是它们的对应元素都相等, 比较 $\boldsymbol{A}$ 与 $\boldsymbol{LU}$ 的对应元素, 即可得出 $\boldsymbol{L},\boldsymbol{U}$ 中元素的计算公式. $\boldsymbol{A}=\boldsymbol{LU}$, 即

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}=\begin{bmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & l_{n3} & \cdots & 1 \end{bmatrix}\begin{bmatrix} u_{11} & u_{12} & u_{13} & \cdots & u_{1n} \\ & u_{22} & u_{23} & \cdots & u_{2n} \\ & & u_{33} & \cdots & u_{3n} \\ & & & \ddots & \vdots \\ & & & & u_{nn} \end{bmatrix}$$

由矩阵的乘法规则, 得

$$a_{1j}=u_{1j}\quad(j=1,2,\cdots,n)$$

$$a_{ij}=\begin{cases} \displaystyle\sum_{k=1}^{i-1}l_{ik}u_{kj}+u_{ij}, & j\geqslant i \\ \displaystyle\sum_{k=1}^{j}l_{ik}u_{kj}, & j<i \end{cases}\qquad(i=2,3,\cdots,n)$$

由此可得计算 l_{ij} 和 u_{ij} 的公式

扫描全能王 创建

$$\begin{cases} u_{1j} = a_{1j} & (j = 1, 2, \cdots, n) \\ u_{ij} = a_{ij} - \sum\limits_{k=1}^{i-1} l_{ik} u_{kj} & (i = 2, 3, \cdots, n; j = i, i+1, \cdots, n) \\ l_{ij} = \left(a_{ij} - \sum\limits_{k=1}^{j-1} l_{ik} u_{kj} \right) / u_{jj} & (j = 1, 2, \cdots, n-1; i = j+1, j+2, \cdots, n) \end{cases} \tag{2.10}$$

注 2.2　计算过程应按第 1 行, 第 1 列, 第 2 行, 第 2 列, …… 的顺序进行, 这样才能保证计算新的元素时用到的其他元素已知.

具体计算步骤如下:

1. 计算 $\boldsymbol{U}$ 的第 1 行, $\boldsymbol{L}$ 的第 1 列

$$\begin{aligned} u_{1j} &= a_{1j} \quad (j = 1, 2, \cdots, n) \\ l_{i1} &= a_{i1} / u_{11} \quad (i = 2, 3, \cdots, n) \end{aligned}$$

2. 计算 $\boldsymbol{U}$ 的第 r 行, $\boldsymbol{L}$ 的第 r 列 $(r = 2, 3, \cdots, n)$

$$u_{rj} = a_{rj} - \sum_{k=1}^{r-1} l_{rk} u_{kj} \quad (j = r, r+1, \cdots, n)$$

$$l_{ir} = \left(a_{ir} - \sum_{k=1}^{r-1} l_{ik} u_{kr} \right) / u_{rr} \quad (i = r+1, r+2, \cdots, n)$$

实际计算时, 常将 u_{rj} 存放在 a_{rj} 的位置上, 将 l_{ir} 存放在 a_{ir} 的位置上, 按表 2.1所示的紧凑格式及顺序进行.

表 2.1　紧凑格式示意图

$(a_{11})\, u_{11}$	$(a_{12})\, u_{12}$	$(a_{13})\, u_{13}$	$\cdots$	$(a_{1n})\, u_{1n}$
$(a_{21})\, l_{21}$	$(a_{22})\, u_{22}$	$(a_{23})\, u_{23}$	$\cdots$	$(a_{2n})\, u_{2n}$
$(a_{31})\, l_{31}$	$(a_{32})\, l_{32}$	$(a_{33})\, u_{33}$	$\cdots$	$(a_{3n})\, u_{3n}$
$\vdots$	$\vdots$	$\vdots$		$\vdots$
$(a_{n1})\, l_{n1}$	$(a_{n2})\, l_{n2}$	$(a_{n3})\, l_{n3}$	$\cdots$	$(a_{nn})\, u_{nn}$

例 2.3　求矩阵

$$\boldsymbol{A} = \begin{bmatrix} 2 & 2 & 3 \\ 4 & 7 & 7 \\ -2 & 4 & 5 \end{bmatrix}$$

的三角分解.

解　按式(2.10) 及表 2.1, 得表 2.2:

表 2.2　紧凑格式直接三角分解

(2) 2	(2) 2	(3) 3
(4) $\frac{4}{2} = 2$	(7) $7 - 2 \times 2 = 3$	(7) $7 - 2 \times 3 = 1$
$(-2)\ \frac{-2}{2} = -1$	(4) $[4 - (-1) \times 2]/3 = 2$	(5) $5 - 2 \times 1 - (-1) \times 3 = 6$

扫描全能王 创建

所以

$$\boldsymbol{L}=\begin{bmatrix}1&0&0\\2&1&0\\-1&2&1\end{bmatrix}\quad \boldsymbol{U}=\begin{bmatrix}2&2&3\\0&3&1\\0&0&6\end{bmatrix}$$

$$\boldsymbol{A}=\boldsymbol{LU}$$

2.2.3 直接三角分解法

如果线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 的系数矩阵已进行三角分解, $\boldsymbol{A}=\boldsymbol{LU}$, 则解方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 等价于求解两个三角形方程组 $\boldsymbol{Ly}=\boldsymbol{b},\boldsymbol{Ux}=\boldsymbol{y}$, 即由

$$\boldsymbol{Ly}=\begin{bmatrix}1&&&&\\l_{21}&1&&&\\l_{31}&l_{32}&1&&\\\vdots&&&\ddots&\\l_{n1}&l_{n2}&l_{n3}&\cdots&1\end{bmatrix}\begin{bmatrix}y_1\\y_2\\y_3\\\vdots\\y_n\end{bmatrix}=\begin{bmatrix}b_1\\b_2\\b_3\\\vdots\\b_n\end{bmatrix}$$

可求出

$$y_k=\begin{cases}b_1 & (k=1)\\ b_k-\sum\limits_{j=1}^{k-1}l_{kj}y_j & (k=2,3,\cdots,n)\end{cases}\tag{2.11}$$

再由

$$\boldsymbol{Ux}=\begin{bmatrix}u_{11}&u_{12}&\cdots&u_{1n}\\&u_{22}&\cdots&u_{2n}\\&&\ddots&\vdots\\&&&u_{nn}\end{bmatrix}\begin{bmatrix}x_1\\x_2\\\vdots\\x_n\end{bmatrix}=\begin{bmatrix}y_1\\y_2\\\vdots\\y_n\end{bmatrix}$$

解得

$$x_k=\begin{cases}y_n/u_{nn} & (k=n)\\ \left(y_k-\sum\limits_{j=k+1}^{n}u_{kj}x_j\right)/u_{kk} & (k=n-1,n-2,\cdots,1)\end{cases}\tag{2.12}$$

容易看出, 式(2.11)和式(2.10)的运算规律相同, 故在利用三角分解求解方程组时, 只需把右端向量 $\boldsymbol{b}$ 列在表 2.1 的最后一列, 按 u_{kj} 的计算方法即可求出 y_k.

表 2.3 是求解线性方程组的紧凑格式, 其计算顺序与计算方法与三角分解相同. 按表 2.3 计算后, 只需按式 (2.12) 即可求出方程组的解.

扫描全能王 创建

表 2.3　紧凑格式解线性方程组

$(a_{11})\,u_{11}$	$(a_{12})\,u_{12}$	$(a_{13})\,u_{13}$	$\cdots$	$(a_{1n})\,u_{1n}$	$(b_1)\,y_1$
$(a_{21})\,l_{21}$	$(a_{22})\,u_{22}$	$(a_{23})\,u_{23}$	$\cdots$	$(a_{2n})\,u_{2n}$	$(b_2)\,y_2$
$(a_{31})\,l_{31}$	$(a_{32})\,l_{32}$	$(a_{33})\,u_{33}$	$\cdots$	$(a_{3n})\,u_{3n}$	$(b_3)\,y_3$
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
$(a_{n1})\,l_{n1}$	$(a_{n2})\,l_{n2}$	$(a_{n3})\,l_{n3}$	$\cdots$	$(a_{nn})\,u_{nn}$	$(b_n)\,y_n$

例 2.4　用紧凑格式解线性方程组

$$\begin{cases} 3x_1+2x_2+5x_3=6 \\ -x_1+4x_2+3x_3=5 \\ x_1-x_2+3x_3=1 \end{cases}$$

解　按表 2.3计算:

(3) 3	(2) 2	(5) 5	(6) 6
$(-1)\,-\frac{1}{3}$	(4) $4+\frac{2}{3}=\frac{14}{3}$	(3) $3+\frac{5}{3}=\frac{14}{3}$	(5) $5+2=7$
(1) $\frac{1}{3}$	(-1) $\frac{-1-2/3}{14/3}=-\frac{5}{14}$	(3) $3+\frac{5}{3}-\frac{5}{3}=3$	(1) $1+\frac{5}{2}-2=\frac{3}{2}$

所以

$$\boldsymbol{L}=\begin{bmatrix} 1 & 0 & 0 \\ -\frac{1}{3} & 1 & 0 \\ \frac{1}{3} & -\frac{5}{14} & 1 \end{bmatrix} \quad \boldsymbol{y}=\begin{bmatrix} 6 \\ 7 \\ \frac{3}{2} \end{bmatrix} \quad \boldsymbol{U}=\begin{bmatrix} 3 & 2 & 5 \\ 0 & \frac{14}{3} & \frac{14}{3} \\ 0 & 0 & 3 \end{bmatrix}$$

解方程组

$$\boldsymbol{U}\boldsymbol{x}=\begin{bmatrix} 3 & 2 & 5 \\ 0 & \frac{14}{3} & \frac{14}{3} \\ 0 & 0 & 3 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}=\begin{bmatrix} 6 \\ 7 \\ \frac{3}{2} \end{bmatrix}$$

得原方程组的解

$$x_3=\frac{1}{2},\quad x_2=1,\quad x_1=\frac{1}{2}$$

2.2.4　列主元素的三角分解法

从直接三角分解公式(2.10)知, 当 $u_{jj}=0$ $(j=1,2,\cdots,n-1)$ 时, 计算将中断; 或当 $|u_{jj}|$ $(j=1,2,\cdots,n-1)$ 很小时, 按公式 (2.10) 计算会使舍入误差扩散. 但若矩阵 $\boldsymbol{A}$ 非奇异, 可增加列选主元过程, 即先进行一系列的行交换, 保证 LU 分解能够实现. 从而有如下定理.

扫描全能王 创建

定理 2.2 设矩阵 $\boldsymbol{A}$ 非奇异, 则存在置换矩阵 $\boldsymbol{P}$, 使得 $\boldsymbol{PA}$ 有唯一的 Doolittle 分解, 其中 $\boldsymbol{L}$ 是单位下三角形矩阵, $\boldsymbol{U}$ 是上三角形矩阵, 且 $|l_{ij}| \leqslant 1$.

证明从略.

在实际计算中, 行交换与分解计算总是交替进行的. 假设按表 2.3的分解计算已完成了 $k-1$ 步, 即有表 2.4:

表 2.4 选主元三角分解法

$(a_{11})\,u_{11}$	$(a_{12})\,u_{12}$		$\cdots$		$(a_{1n})\,u_{1n}$		$(b_1)\,y_1$
$(a_{21})\,l_{21}$	$(a_{22})\,u_{22}$		$\cdots$		$(a_{2n})\,u_{2n}$		$(b_2)\,y_2$
$\vdots$	$\vdots$				$\vdots$		$\vdots$
			$(a_{k-1,k-1})\,u_{k-1,k-1}$	$\cdots$	$(a_{k-1,n})\,u_{k-1,n}$		$(b_{k-1})\,y_{k-1}$
$(a_{k1})\,l_{k1}$	$(a_{k2})\,l_{k2}$	$\cdots$	$(a_{k,k-1})\,l_{k,k-1}$	(a_{kk})	$\cdots$ (a_{kn})		(b_k)
$\vdots$	$\vdots$		$\vdots$	$\vdots$	$\vdots$		$\vdots$
$(a_{n1})\,l_{n1}$	$(a_{n2})\,l_{n2}$	$\cdots$	$(a_{n,k-1})\,l_{n,k-1}$	(a_{nk})	$\cdots$ (a_{nn})		(b_n)

下面开始第 k 步分解, 为避免用零或绝对值很小的数作除数, 由公式(2.10)引入变量

$$s_i = a_{ik} - \sum_{j=1}^{k-1} l_{ij}u_{jk} \quad (i = k, k+1, \cdots, n)$$

于是 s_k 即为公式(2.10) 中的 u_{kk}. 比较 $|s_i|\,(i = k, k+1, \cdots, n)$ 的大小, 设

$$|s_{i_k}| = \max_{k \leqslant i \leqslant n} |s_i|$$

取 s_{i_k} 作为第 k 次的主元素 u_{kk}, 交换表 2.4 中的第 k 行与第 i_k 行, 然后再作第 k 步分解. 结合紧凑格式表 2.4 将 u_{ij}, l_{ij} 的值存入原 a_{ij} 的存储单元, 将 y_k, x_k 的值先后存入 b_k 的存储单元, 得具体算法如下:

算法 2.2 主元消去法

1. 输入 $\boldsymbol{A} = (a_{ij})_{n\times n}, \boldsymbol{b} = (b_i)_{n\times 1}$.
2. 对于 $k = 1, 2, \cdots, n-1$ 进行消元.

   对于$i = k, k+1, \cdots, n$, 计算 $s_i = a_{ik} - \sum_{j=1}^{k-1} l_{ij}u_{jk}$, 若 $|s_{i_k}| = \max_{k \leqslant i \leqslant n} |s_i|$, 则

   对于$j = 1, 2, \cdots, n$, 交换 $a_{kj} \leftrightarrow a_{i_k j}, b_k \leftrightarrow b_{i_k}$;

   对于$j = k, k+1, \cdots, n$, 置 $a_{kj} - \sum_{r=1}^{k-1} l_{kr}u_{rj} = u_{kj} \Rightarrow a_{kj}, b_k - \sum_{r=1}^{k-1} l_{kr}b_r = y_k \Rightarrow b_k$;

   对于$i = k+1, k+2, \cdots, n$, 置 $\left(a_{ik} - \sum_{r=1}^{k-1} l_{ir}u_{rk}\right) / u_{kk} = l_{ik} \Rightarrow a_{ik}$.
3. 解方程组 $\boldsymbol{Ux} = \boldsymbol{y}$. 置 $b_n/u_{nn} = x_n \Rightarrow b_n$,

   对于$k = n-1, n-2, \cdots, 1$, 置 $\left(y_k - \sum_{r=k+1}^{n} l_{kr}y_r\right) / u_{kk} = x_k \Rightarrow b_k$.
4. 输出 $\boldsymbol{x} = \boldsymbol{b}$, 停机.

扫描全能王 创建

注 2.3　1. 算法 2.2 中仍保留 u_{ij}, l_{ij}, 仅为了体现算法结构的清晰, 编程时一般直接用 a_{ij}, 且当 $r_2 < r_1$ 时, $\sum_{r=r_1}^{r_2}(\quad) = 0$.

2. 对于主元素法可以进一步减少计算量, 方法是用两个数组专门记录行列序号, 需要互换时, 只需互换行列序号, 计算时直接引用行列序号, 无须真正去交换行列的数据.

例 2.5　用列主元素三角分解法求解方程组

$$\begin{cases} 2x_2 + x_4 = 0 \\ 2x_1 + 2x_2 + 3x_3 + 2x_4 = -2 \\ 4x_1 - 3x_2 + x_4 = -7 \\ 6x_1 + x_2 - 6x_3 - 5x_4 = 6 \end{cases}$$

解　对增广矩阵 $[A, b]$ 进行分解计算

$$[A,b] = \begin{bmatrix} 0 & 2 & 0 & 1 & 0 \\ 2 & 2 & 3 & 2 & -2 \\ 4 & -3 & 0 & 1 & -7 \\ 6 & 1 & -6 & -5 & 6 \end{bmatrix} \begin{matrix} s_1 = 0 \\ s_2 = 2 \\ s_3 = 4 \\ s_4 = 6 \end{matrix}$$

$$\xrightarrow[\text{主元素}s_4=6]{\text{交换第 1, 4 行}} \begin{bmatrix} 6 & 1 & -6 & -5 & 6 \\ 2 & 2 & 3 & 2 & -2 \\ 4 & -3 & 0 & 1 & -7 \\ 0 & 2 & 0 & 1 & 0 \end{bmatrix}$$

$$\xrightarrow{\text{计算}u_{1j},l_{i1}} \begin{bmatrix} 6 & 1 & -6 & -5 & 6 \\ \frac{1}{3} & 2 & 3 & 2 & -2 \\ \frac{2}{3} & -3 & 0 & 1 & -7 \\ 0 & 2 & 0 & 1 & 0 \end{bmatrix} \begin{matrix} \\ s_2 = 2 - \frac{1}{3} = \frac{5}{3} \\ s_3 = -3 - \frac{2}{3} = -\frac{11}{3} \\ s_4 = 2 - 0 = 2 \end{matrix}$$

$$\xrightarrow[\text{主元素}s_3=-\frac{11}{3}]{\text{交换第 2, 3 行}} \begin{bmatrix} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -3 & 0 & 1 & -7 \\ \frac{1}{3} & 2 & 3 & 2 & -2 \\ 0 & 2 & 0 & 1 & 0 \end{bmatrix}$$

$$\xrightarrow{\text{计算}u_{2j},l_{i2},y_2} \begin{bmatrix} 6 & 1 & -6 & -5 & 6 \\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11 \\ \frac{1}{3} & -\frac{5}{11} & 3 & 2 & -2 \\ 0 & -\frac{6}{11} & 0 & 1 & 0 \end{bmatrix} \begin{matrix} \\ \\ s_3 = 3 - \left(-2 - \frac{20}{11}\right) = \frac{75}{11} \\ s_4 = 0 - \left(0 - \frac{24}{11}\right) = \frac{24}{11} \end{matrix}$$

扫描全能王 创建

$$\xrightarrow[\text{主元素}s_3=\frac{75}{11}]{\text{计算}u_{3j},l_{i3},y_3}\begin{bmatrix}6 & 1 & -6 & -5 & 6\\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11\\ \frac{1}{3} & -\frac{5}{11} & \frac{75}{11} & \frac{62}{11} & -9\\ 0 & -\frac{6}{11} & \frac{8}{25} & 1 & 0\end{bmatrix}$$

$$\xrightarrow{\text{计算}u_{44},y_4}\begin{bmatrix}6 & 1 & -6 & -5 & 6\\ \frac{2}{3} & -\frac{11}{3} & 4 & \frac{13}{3} & -11\\ \frac{1}{3} & -\frac{5}{11} & \frac{75}{11} & \frac{62}{11} & -9\\ 0 & -\frac{6}{11} & \frac{8}{25} & \frac{39}{25} & -\frac{78}{25}\end{bmatrix}$$

解上三角形方程组

$$\begin{cases}6x_1+x_2-6x_3 & -5x_4=6\\ -\frac{11}{3}x_2+4x_3+\frac{13}{3}x_4=-11\\ \frac{75}{11}x_3+\frac{62}{11}x_4=-9\\ \frac{39}{25}x_4=-\frac{78}{25}\end{cases}$$

回代得原方程组的解

$$x_4=-2,\quad x_3=\frac{1}{3},\quad x_2=1,\quad x_1=-\frac{1}{2}$$

直接三角分解法计算量也是 $n^3/3$ 数量级. 由于计算出 u_{ij},l_{ij} 和 y_k 之后, a_{ij},b_i 就不再有用, 故在编程计算时, 可将 $\boldsymbol{L},\boldsymbol{U}$ 和 $\boldsymbol{y}$ 存放在 $\boldsymbol{A},\boldsymbol{b}$ 的存储单元中, 回代时 $\boldsymbol{x}$ 取代 $\boldsymbol{y}$, 整个计算过程无须增加新的存储单元.

由于系数矩阵的三角分解与右端项无关. 因此, 直接三角分解法尤其适合求解系数矩阵相同, 而右端项不同的一系列方程组, 如求矩阵的逆矩阵、用反幂法求矩阵的特征值与特征向量等.

也可将矩阵 $\boldsymbol{A}$ 分解成一个下三角形矩阵 $\boldsymbol{L}$ 与一个单位上三角形矩阵 $\boldsymbol{U}$ 的乘积, 矩阵的这种分解称为 Crout(克劳特) 分解. 由此引出的解线性方程组的方法也称为直接三角分解法.

2.3 特殊矩阵的三角分解法

在工程实际计算中, 如三次样条插值或用差分法求解常微分方程边值问题, 导出的线性方程组的系数矩阵 $\boldsymbol{A}$ 常常是稀疏的三对角形矩阵或 $\boldsymbol{A}$ 是对称正定矩阵, 这使得 $\boldsymbol{A}$ 的三角分解也具有更简洁的形式, 从而导出一些特殊的算法.

扫描全能王 创建

2.3.1　解三对角方程组的追赶法

定义 2.1　方程组

$$\begin{bmatrix} b_1 & c_1 & & & \\ a_2 & b_2 & c_2 & & \\ & \ddots & \ddots & \ddots & \\ & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ \vdots \\ d_{n-1} \\ d_n \end{bmatrix} \tag{2.13}$$

称为三对角方程组, 其中系数矩阵 $\boldsymbol{A}$ 是一种特殊的系数矩阵, 它的非零元素集中分布在主对角线及其相邻的两条次对角线上, 称为三对角矩阵.

如果 $\boldsymbol{A}$ 满足三角分解的条件, 可将 $\boldsymbol{A}$ 分解成 $\boldsymbol{LU}$, 按 Doolittle 分解法不难证明如下定理.

定理 2.3　设方程组(2.13)中的系数矩阵 $\boldsymbol{A}$ 满足条件

$$\begin{cases} |b_1| > |c_1| > 0 \\ |b_i| \geqslant |a_i| + |c_i| \quad (a_i c_i \neq 0, \text{ 且} i = 2, 3, \cdots, n-1) \\ |b_n| > |a_n| > 0 \end{cases}$$

则 $\boldsymbol{A}$ 可唯一分解为

$$\boldsymbol{A} = \boldsymbol{LU} = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & 1 & & \\ & & \ddots & \ddots & \\ & & & l_n & 1 \end{bmatrix} \begin{bmatrix} u_1 & c_1 & & & \\ & u_2 & c_2 & & \\ & & \ddots & \ddots & \\ & & & & c_{n-1} \\ & & & & u_n \end{bmatrix}$$

其中 c_i $(i = 1, 2, \cdots, n-1)$ 由原矩阵给出, 且

$$\begin{cases} u_1 = b_1 \\ l_i = a_i / u_{i-1} \qquad (i = 2, 3, \cdots, n) \\ u_i = b_i - c_{i-1} l_i \end{cases}$$

当矩阵 $\boldsymbol{A}$ 三角分解后, 解线性方程组(2.13)化为求解方程组 $\boldsymbol{Ly} = \boldsymbol{d}$ 和 $\boldsymbol{Ux} = \boldsymbol{y}$. 解 $\boldsymbol{Ly} = \boldsymbol{d}$, 得

$$\begin{cases} y_1 = d_1 \\ y_k = d_k - l_k y_{k-1} \quad (k = 2, 3, \cdots, n) \end{cases}$$

再解 $\boldsymbol{Ux} = \boldsymbol{y}$, 得三对角方程组 (2.13)的解

$$\begin{cases} x_n = y_n / u_n \\ x_k = (y_k - c_k x_{k+1}) / u_k \quad (k = n-1, n-2, \cdots, 1) \end{cases}$$

称上述求解三对角方程组的方法为追赶法.

扫描全能王 创建

算法 2.3 追赶法

1. 输入 $a=(a_2,a_3,\cdots,a_n),b=(b_1,b_2,\cdots,b_n),c=(c_1,c_2,\cdots,c_{n-1}),d=(d_1,d_2,\cdots,d_n),n.$
2. 对于 $i=2,3,\cdots,n$, 置 $a_i/b_{i-1}=l_i\Rightarrow a_i,b_i-c_{i-1}a_i=u_i\Rightarrow b_i,d_i-a_id_{i-1}=y_i\Rightarrow d_i.$
3. 置 $d_n/b_n=x_n\Rightarrow d_n.$
4. 对于 $i=n-1,n-2,\cdots,1$, 置 $(d_i-c_id_{i+1})/b_i=x_i\Rightarrow d_i.$
5. 输出 $\boldsymbol{d}=\boldsymbol{x}$, 停机.

追赶法是由于矩阵中出现了大量零元素, 从而使得计算公式简化, 计算量大为减少, 其乘除法的运算量为 $5n-4$.

2.3.2 平方根法 (Cholesky 分解法)

定理 2.4 设 $\boldsymbol{A}$ 是对称正定矩阵, 则存在唯一的非奇异下三角形矩阵 $\boldsymbol{L}$, 使得

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$$

且 $\boldsymbol{L}$ 的对角元素皆为正数.

证明 存在性. 因为 $\boldsymbol{A}$ 对称正定, 其各阶顺序主子式均大于零, 由定理 2.1, 矩阵 $\boldsymbol{A}$ 存在 Doolittle 分解, 即

$$\boldsymbol{A}=\tilde{\boldsymbol{L}}\boldsymbol{U}$$

其中 $\tilde{\boldsymbol{L}}$ 为单位下三角形矩阵, $\boldsymbol{U}$ 为上三角形矩阵. 令 $\boldsymbol{D}=\operatorname{diag}(u_{11},u_{22},\cdots,u_{nn})$, $\boldsymbol{P}=\boldsymbol{D}^{-1}\boldsymbol{U}$, 则 $\boldsymbol{P}$ 为单位上三角形矩阵, 且 $\boldsymbol{A}=\tilde{\boldsymbol{L}}\boldsymbol{D}\boldsymbol{P}$. 因为 $\boldsymbol{A}$ 对称, 故有

$$\boldsymbol{A}^{\mathrm{T}}=\boldsymbol{P}^{\mathrm{T}}\boldsymbol{D}\tilde{\boldsymbol{L}}^{\mathrm{T}}=\boldsymbol{A}=\tilde{\boldsymbol{L}}\boldsymbol{D}\boldsymbol{P}$$

由 Doolittle 分解的唯一性, 得 $\boldsymbol{P}^{\tau}=\tilde{\boldsymbol{L}}$, 即

$$\boldsymbol{A}=\boldsymbol{P}^{\mathrm{T}}\boldsymbol{D}\boldsymbol{P}$$

对任意非零向量 $\boldsymbol{x},\boldsymbol{y}=\boldsymbol{P}^{-1}\boldsymbol{x}\neq\boldsymbol{0}$, 于是, 由 $\boldsymbol{A}$ 正定得

$$\begin{aligned}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{D}\boldsymbol{x}&=\boldsymbol{x}^{\mathrm{T}}\left(\boldsymbol{P}^{\mathrm{T}}\right)^{-1}\boldsymbol{A}\boldsymbol{P}^{-1}\boldsymbol{x}\\&=\left(\boldsymbol{P}^{-1}\boldsymbol{x}\right)^{\mathrm{T}}\boldsymbol{A}\boldsymbol{P}^{-1}\boldsymbol{x}=\boldsymbol{y}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{y}>0\end{aligned}$$

即 $\boldsymbol{D}$ 正定, 所以 $\boldsymbol{D}$ 的对角元素均为正数. 令

$$\tilde{\boldsymbol{D}}=\operatorname{diag}(\sqrt{u_{11}},\sqrt{u_{22}},\cdots,\sqrt{u_{nn}})$$

则

$$\boldsymbol{A}=\boldsymbol{P}^{\mathrm{T}}\tilde{\boldsymbol{D}}^{\mathrm{T}}\tilde{\boldsymbol{D}}\boldsymbol{P}=(\tilde{\boldsymbol{D}}\boldsymbol{P})^{\mathrm{T}}\tilde{\boldsymbol{D}}\boldsymbol{P}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$$

其中 $\boldsymbol{L}=(\tilde{\boldsymbol{D}}\boldsymbol{P})^{\mathrm{T}}$ 为非奇异下三角形矩阵, 且对角元素皆为正数.

唯一性. 假设存在非奇异下三角形矩阵 $\boldsymbol{G}\neq\boldsymbol{L}$, 其对角元素皆为正数, 且使得

扫描全能王 创建

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}=\boldsymbol{G}\boldsymbol{G}^{\mathrm{T}}$$

于是有

$$\boldsymbol{L}^{\mathrm{T}}\left(\boldsymbol{G}^{\mathrm{T}}\right)^{-1}=\boldsymbol{L}^{-1}\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}\left(\boldsymbol{G}^{\mathrm{T}}\right)^{-1}=\boldsymbol{L}^{-1}\boldsymbol{G}\boldsymbol{G}^{\mathrm{T}}\left(\boldsymbol{G}^{\mathrm{T}}\right)^{-1}=\boldsymbol{L}^{-1}\boldsymbol{G}$$

因为 $\boldsymbol{L}^{\mathrm{T}}\left(\boldsymbol{G}^{\mathrm{T}}\right)^{-1}$ 是上三角形矩阵, $\boldsymbol{L}^{-1}\boldsymbol{G}$ 是下三角形矩阵, 故由上式必得

$$\boldsymbol{L}^{\mathrm{T}}\left(\boldsymbol{G}^{\mathrm{T}}\right)^{-1}=\boldsymbol{L}^{-1}\boldsymbol{G}=\boldsymbol{I}$$

即 $\boldsymbol{G}=\boldsymbol{L}$, 与假设矛盾. 矩阵的这种分解称为 Cholesky(楚列斯基) 分解. 用比较法可以导出 $\boldsymbol{L}$ 的计算公式. 设

$$\boldsymbol{L}=\begin{bmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & l_{nn} \end{bmatrix}$$

比较 $\boldsymbol{A}$ 与 $\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$ 的相应元素, 可得

$$\begin{cases} l_{kk}=\left(a_{kk}-\displaystyle\sum_{j=1}^{k-1}l_{kj}^{2}\right)^{1/2} & (k=1,2,\cdots,n) \\ l_{ik}=\left(a_{ik}-\displaystyle\sum_{j=1}^{k-1}l_{ij}l_{kj}\right)/l_{kk} & (i=k+1,k+2,\cdots,n) \end{cases}$$

计算顺序是按列进行, 即

$$l_{11}\to l_{i1}\ (i=2,3,\cdots,n)\to l_{22}\to l_{i2}\ (i=3,4,\cdots,n)\to\cdots$$

当矩阵 $\boldsymbol{A}$ 完成 Cholesky 分解后, 求解方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 就转化为依次求解方程组

$$\boldsymbol{L}\boldsymbol{y}=\boldsymbol{b},\quad \boldsymbol{L}^{\mathrm{T}}\boldsymbol{x}=\boldsymbol{y}$$

它们的解分别为

$$y_k=\left(b_k-\sum_{j=1}^{k-1}l_{kj}y_j\right)\Big/l_{kk}\quad(k=1,2,\cdots,n)$$

$$x_k=\left(y_k-\sum_{j=k+1}^{n}l_{jk}x_j\right)\Big/l_{kk}\quad(k=n,n-1,\cdots,1)$$

求解线性方程组的上述方法称为平方根法, 也称为 Cholesky 分解法. 这种方法无须选主元, 计算过程也是稳定的. 由于 $\boldsymbol{A}$ 的对称性, 平方根法的乘除运算量为 $n^3/6$ 数量级, 约是 Gauss 消去法的一半. 但这种方法在求 $\boldsymbol{L}$ 时需作 n 次开方运算, 这样又增加了计算量.

2.3.3　改进的平方根法 ($\mathrm{LDL}^{\mathrm{T}}$ 法)

定理 2.4的证明过程表明, 对称正定矩阵 $\boldsymbol{A}$ 又可以作如下分解:

$$\boldsymbol{A}=\boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^{\mathrm{T}}$$

扫描全能王 创建

其中 $\boldsymbol{L}$ 为单位下三角形矩阵, $\boldsymbol{D}$ 为对角矩阵. 记

$$\boldsymbol{L}=\begin{bmatrix} 1 & & & \\ l_{21} & 1 & & \\ \vdots & \vdots & \ddots & \\ l_{n1} & l_{n2} & \cdots & 1 \end{bmatrix}$$

$$\boldsymbol{D}=\operatorname{diag}(d_1,d_2,\cdots,d_n)$$

由比较法可以导出 $\mathrm{LDL}^{\mathrm{T}}$ 分解的计算公式:

对 $k=1,2,\cdots,n$,

$$\begin{cases} d_k=a_{kk}-\sum\limits_{j=1}^{k-1}l_{kj}^2d_j \quad (k=1,2,\cdots,n) \\ l_{ik}=\left(a_{ik}-\sum\limits_{j=1}^{k-1}l_{ij}d_jl_{kj}\right)\Big/d_k \quad (i=k+1,k+2,\cdots,n) \end{cases} \tag{2.14}$$

计算顺序如下

$$d_1\to l_{i1}\ (i=2,3,\cdots,n)\to d_2\to l_{i2}\ (i=3,4,\cdots,n)\to d_3\to\cdots$$

为避免重复计算, 引进辅助量 $u_{ik}=l_{ik}d_k\ (k=1,2,\cdots,n;i=k+1,k+2,\cdots,n)$, 则式 (2.14) 可改写成

$$\begin{cases} d_k=a_{kk}-\sum\limits_{j=1}^{k-1}u_{kj}l_{kj} \quad (k=1,2,\cdots,n) \\ u_{ik}=a_{ik}-\sum\limits_{j=1}^{k-1}u_{ij}l_{kj} \quad (i=k+1,k+2,\cdots,n) \\ l_{ik}=u_{ik}/d_k \end{cases} \tag{2.15}$$

按式(2.15)进行 $\mathrm{LDL}^{\mathrm{T}}$ 分解, 乘除运算量与 Cholesky 分解相当, 且避免了开方运算.

矩阵 $\boldsymbol{A}$ 作 $\mathrm{LDL}^{\mathrm{T}}$ 分解后, 解方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 可分两步进行: 先解方程组 $\boldsymbol{Ly}=\boldsymbol{b}$, 再由 $\boldsymbol{L}^{\mathrm{T}}\boldsymbol{x}=\boldsymbol{D}^{-1}\boldsymbol{y}$ 求 $\boldsymbol{x}$. 具体计算公式为

$$\begin{cases} y_1=b_1 \\ y_k=b_k-\sum\limits_{j=1}^{k-1}l_{kj}y_j \quad (k=2,3,\cdots,n) \\ x_n=y_n/d_n \\ x_k=y_k/d_k-\sum\limits_{j=k+1}^{n}l_{jk}x_j \quad (k=n-1,n-2,\cdots,1) \end{cases} \tag{2.16}$$

求解线性方程组的这一方法称为改进平方根法, 也叫 $\mathrm{LDL}^{\mathrm{T}}$ 法.

例 2.6 用改进平方根法求解方程组

扫描全能王 创建

$$\begin{cases} x_1 + 2x_2 + x_3 = 4 \\ 2x_1 + 5x_2 = 7 \\ x_1 + 14x_3 = 15 \end{cases}$$

解　容易验证, 系数矩阵

$$A = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 5 & 0 \\ 1 & 0 & 14 \end{bmatrix}$$

为对称正定矩阵. 按式(2.15) 计算分解式, 得

$$d_1 = 1$$

$$u_{21} = 2, \quad u_{31} = 1$$

$$l_{21} = 2, \quad l_{31} = 1$$

$$d_2 = 5 - 2 \times 2 = 1$$

$$u_{32} = 0 - 1 \times 2 = -2, \quad l_{32} = -2$$

$$d_3 = 14 - 1 \times 1 - (-2) \times (-2) = 9$$

按式(2.16) 计算, 得

$$y_1 = 4, \quad y_2 = 7 - 2 \times 4 = -1$$

$$y_3 = 15 - 1 \times 4 - (-2) \times (-1) = 9$$

$$x_3 = 1, \quad x_2 = -1 - (-2) \times 1 = 1$$

$$x_1 = 4 - 2 \times 1 - 1 \times 1 = 1$$

所以方程组的解为

$$x_1 = 1, \quad x_2 = 1, \quad x_3 = 1$$

平方根法与改进的平方根法不仅计算量仅是 Gauss 消去法的一半, 其数值稳定性良好, 是求解中小型稠密对称正定线性方程组的好方法.

2.3.4　分块三角分解法

在求解微分方程数值解时, 如果用差分法或有限元法离散微分方程, 导出的线性方程组往往是稀疏的, 即系数矩阵中绝大多数元素是零, 而且具有分块结构. 这些稀疏性和分块性, 对于我们提高求解方程组的效率是很有帮助的. 作为一个例子, 我们讨论如下的分块三对角方程组

$$\begin{bmatrix} \boldsymbol{B}_1 & \boldsymbol{C}_1 & & & \\ \boldsymbol{A}_2 & \boldsymbol{B}_2 & \boldsymbol{C}_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \boldsymbol{A}_{n-1} & \boldsymbol{B}_{n-1} & \boldsymbol{C}_{n-1} \\ & & & \boldsymbol{A}_n & \boldsymbol{B}_n \end{bmatrix} \begin{bmatrix} \boldsymbol{x}_1 \\ \boldsymbol{x}_2 \\ \vdots \\ \boldsymbol{x}_{n-1} \\ \boldsymbol{x}_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{b}_1 \\ \boldsymbol{b}_2 \\ \vdots \\ \boldsymbol{b}_{n-1} \\ \boldsymbol{b}_n \end{bmatrix}$$

扫描全能王 创建

在此假设所有的块矩阵 $\boldsymbol{A}_i, \boldsymbol{B}_i, \boldsymbol{C}_i$ 都是 $q \times q$ 矩阵, $\boldsymbol{x}_i$ 和 $\boldsymbol{b}_i$ 都属于 $\mathbf{R}^q$, 方程组为 nq 阶的.

首先考虑系数矩阵的 $\boldsymbol{LU}$ 分解. 设

$$\boldsymbol{A}^{(k)} = \begin{bmatrix} \boldsymbol{B}_1 & \boldsymbol{C}_1 & & & \\ \boldsymbol{A}_2 & \boldsymbol{B}_2 & \boldsymbol{C}_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \boldsymbol{A}_{k-1} & \boldsymbol{B}_{k-1} & \boldsymbol{C}_{k-1} \\ & & & \boldsymbol{A}_k & \boldsymbol{B}_k \end{bmatrix}$$

那么当 $\boldsymbol{A}^{(1)}, \boldsymbol{A}^{(2)}, \cdots, \boldsymbol{A}^{(n)}$ 非奇异时, 有

$$\begin{aligned} \boldsymbol{A} &= \begin{bmatrix} \boldsymbol{B}_1 & \boldsymbol{C}_1 & & & \\ \boldsymbol{A}_2 & \boldsymbol{B}_2 & \boldsymbol{C}_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \boldsymbol{A}_{n-1} & \boldsymbol{B}_{n-1} & \boldsymbol{C}_{n-1} \\ & & & \boldsymbol{A}_n & \boldsymbol{B}_n \end{bmatrix} \\ &= \begin{bmatrix} \boldsymbol{I} & & & & \\ \boldsymbol{L}_2 & \boldsymbol{I} & & & \\ & \ddots & \ddots & & \\ & & \boldsymbol{L}_{n-1} & \boldsymbol{I} & \\ & & & \boldsymbol{L}_n & \boldsymbol{I} \end{bmatrix} \begin{bmatrix} \boldsymbol{U}_1 & \boldsymbol{C}_1 & & & \\ & \boldsymbol{U}_2 & \boldsymbol{C}_2 & & \\ & & \ddots & \ddots & \\ & & & \boldsymbol{U}_{n-1} & \boldsymbol{C}_{n-1} \\ & & & & \boldsymbol{U}_n \end{bmatrix} \end{aligned}$$

其中 $\boldsymbol{L}_i, \boldsymbol{U}_i$ 按如下计算过程得到

$$\boldsymbol{U}_1 = \boldsymbol{B}_1$$

对于 $i = 2, 3, \cdots, n$, 由 $\boldsymbol{L}_i \boldsymbol{U}_{i-1} = \boldsymbol{A}_i$, 解得

$$\boldsymbol{L}_i = \boldsymbol{A}_i \boldsymbol{U}_{i-1}^{-1}$$

$$\boldsymbol{U}_i = \boldsymbol{B}_i - \boldsymbol{L}_i \boldsymbol{C}_{i-1}$$

$\boldsymbol{A}^{(1)}, \boldsymbol{A}^{(2)}, \cdots, \boldsymbol{A}^{(n)}$ 非奇异保证了 $\boldsymbol{U}_i$ 的可逆性, 使得 $\boldsymbol{A}$ 的 LU 分解可以进行. 随后未知向量 $\boldsymbol{x}$ 可以通过一追一赶得到

$$\begin{cases} \boldsymbol{y}_1 = \boldsymbol{b}_1 \\ \boldsymbol{y}_i = \boldsymbol{b}_i - \boldsymbol{L}_{i-1} \boldsymbol{y}_{i-1} \quad (i = 2, 3, \cdots, n) \\ \text{由} \boldsymbol{U}_n \boldsymbol{x}_n = \boldsymbol{y}_n \text{解得} \boldsymbol{x}_n \\ \text{再由} \boldsymbol{U}_i \boldsymbol{x}_i = \boldsymbol{y}_i - \boldsymbol{C}_i \boldsymbol{x}_{i+1} \text{解得} \boldsymbol{x}_i \quad (i = n-1, n-2, \cdots, 1) \end{cases}$$

上述的追赶过程需要求解以块矩阵 $\boldsymbol{U}_i$ 为系数的方程组, 可以通过消元法或对 $\boldsymbol{U}_i$ 进行分解求解方程组.

稀疏线性方程组的求解是一个复杂而困难的问题, 随着未知变量排序的不同, 矩阵的分块结构也不同, 具体的分解过程也会不一样. 有需要的读者可参阅相关文献. 利用稀疏性和分块结构直接分解求解方程组只是一种选择, 更多的时候我们将采用第三章介绍的迭代法.

扫描全能王 创建

2.4 误 差 分 析

由于不同的算法计算效果不同, 因此必须进行误差分析. 向量和矩阵的范数及矩阵的条件数在研究数值方法所求得的线性方程组解的误差分析中起着十分重要的作用.

2.4.1 向量和矩阵的范数

向量的范数

向量的范数是衡量向量大小的度量概念, 它可以用来刻画 $\mathbf{R}^n$ 中向量序列的收敛性. 一般地, 通过把距离满足的性质推广到向量空间定义范数.

定义 2.2 设对任意向量 $\boldsymbol{x} \in \mathbf{R}^n$, 按一定的规则有一实数与之对应, 记为 $\|\boldsymbol{x}\|$, 若 $\|\boldsymbol{x}\|$ 满足

(1) $\|\boldsymbol{x}\| \geqslant 0$, 且 $\|\boldsymbol{x}\| = \mathbf{0}$ 当且仅当 $\boldsymbol{x} = \mathbf{0}$;

(2) 对任意实数 α, 都有 $\|\alpha\boldsymbol{x}\| = |\alpha|\|\boldsymbol{x}\|$;

(3) 对任意 $\boldsymbol{x}, \boldsymbol{y} \in \mathbf{R}^n$, 都有

$$\|\boldsymbol{x} + \boldsymbol{y}\| \leqslant \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$$

则称 $\|\boldsymbol{x}\|$ 为向量 $\boldsymbol{x}$ 的范数.

容易验证, 向量的模符合以上三个条件, 因而它是 $\mathbf{R}^n$ 中的一种范数. 设向量 $\boldsymbol{x} = (x_1, x_2, \cdots, x_n)^{\mathrm{T}}$, 按定义 2.2, 向量 $\boldsymbol{x}$ 的范数的具体形式可以有多种, 常用的有以下三种:

2– 范数 (向量的模)

$$\|\boldsymbol{x}\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} \tag{2.17}$$

1– 范数

$$\|\boldsymbol{x}\|_1 = \sum_{i=1}^{n} |x_i| \tag{2.18}$$

∞– 范数

$$\|\boldsymbol{x}\|_\infty = \max_{1 \leqslant i \leqslant n} |x_i| \tag{2.19}$$

下面以 $\infty-$ 范数为例进行验证. 满足条件 (1), (2) 是显然的. 由实数绝对值的性质, 对任意实数 x, y, 都有 $|x+y| \leqslant |x|+|y|$. 故对任意向量 $\boldsymbol{x}, \boldsymbol{y} \in \mathbf{R}^n$, $\boldsymbol{x} = (x_1, x_2, \cdots, x_n)^{\mathrm{T}}$, $\boldsymbol{y} = (y_1, y_2, \cdots, y_n)^{\mathrm{T}}$, 有

$$\begin{aligned} \|\boldsymbol{x} + \boldsymbol{y}\|_\infty &= \max_{1 \leqslant i \leqslant n} |x_i + y_i| \leqslant \max_{1 \leqslant i \leqslant n} \{|x_i| + |y_i|\} \\ &\leqslant \max_{1 \leqslant i \leqslant n} |x_i| + \max_{1 \leqslant i \leqslant n} |y_i| = \|\boldsymbol{x}\|_\infty + \|\boldsymbol{y}\|_\infty \end{aligned}$$

所以条件 (3) 成立.

如果 $\mathbf{R}^n$ 中有两个范数 $\|\cdot\|$ 和 $\|\cdot\|'$, 存在实数 $m, M > 0$, 使得对任意 n 维向量 $\boldsymbol{x}$, 都有

$$m\|\boldsymbol{x}\| \leqslant \|\boldsymbol{x}\|' \leqslant M\|\boldsymbol{x}\|$$

扫描全能王 创建

则称这两种范数是等价的. 不难证明, 1– 范数, 2– 范数和 ∞– 范数是等价的. 例如, 由式(2.17)与式 (2.19), 对任意 $\boldsymbol{x} \in \mathbf{R}^n$, 有

$$\|\boldsymbol{x}\|_\infty = \max_{1 \leqslant i \leqslant n} |x_i| \leqslant \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \|\boldsymbol{x}\|_2$$

设 $\|\boldsymbol{x}\|_\infty = |x_j| = \max\limits_{1 \leqslant i \leqslant n} \{|x_i|\}$, 则

$$\|\boldsymbol{x}\|_\infty = |x_j| \geqslant \sqrt{\frac{x_1^2 + x_2^2 + \cdots + x_n^2}{n}} = \frac{\|\boldsymbol{x}\|_2}{\sqrt{n}}$$

所以, 2– 范数与 ∞– 范数是等价的. 事实上, $\mathbf{R}^n$ 中任意两种范数都是等价的.

今后如果不作说明, $\|\cdot\|$ 是指任意一种向量范数.

矩阵的范数

类似于向量范数的定义, 可以定义 $n \times n$ 矩阵的范数.

定义 2.3　对任意 n 阶方阵 $\boldsymbol{A}$, 按一定的规则有一实数与之对应, 记为 $\|\boldsymbol{A}\|$. 若 $\|\boldsymbol{A}\|$ 满足

(1) $\|\boldsymbol{A}\| \geqslant 0$, 且 $\|\boldsymbol{A}\| = 0$ 当且仅当 $\boldsymbol{A} = \boldsymbol{O}$;

(2) 对任意实数 α, 都有 $\|\alpha \boldsymbol{A}\| = |\alpha| \|\boldsymbol{A}\|$;

(3) 对任意两个 n 阶方阵 $\boldsymbol{A}, \boldsymbol{B}$, 都有

$$\|\boldsymbol{A} + \boldsymbol{B}\| \leqslant \|\boldsymbol{A}\| + \|\boldsymbol{B}\|$$

(4) $\|\boldsymbol{A}\boldsymbol{B}\| \leqslant \|\boldsymbol{A}\| \|\boldsymbol{B}\|$ (相容性条件),

则称 $\|\boldsymbol{A}\|$ 为矩阵 $\boldsymbol{A}$ 的范数.

这里(1)—(3)是向量范数定义的直接推广, 条件 (4) 则使矩阵范数在数值计算中使用更为方便.

例 2.7　若对任意 n 阶方阵 $\boldsymbol{A} = (a_{ij})$, 定义

$$\|\boldsymbol{A}\|_1 = \max_{1 \leqslant j \leqslant n} \sum_{i=1}^{n} |a_{ij}|$$

试证 $\|\boldsymbol{A}\|_1$ 为矩阵 $\boldsymbol{A}$ 的范数.

证明　(1) 显然 $\|\boldsymbol{A}\|_1 \geqslant 0$, 且 $\|\boldsymbol{A}\|_1 = 0$, 即

$$\max_{1 \leqslant j \leqslant n} \sum_{i=1}^{n} |a_{ij}| = 0$$

当且仅当对所有 $j = 1, 2, \cdots, n$, 有

$$\sum_{i=1}^{n} |a_{ij}| = 0$$

于是, 对任意 $1 \leqslant i, j \leqslant n$, 有

$$|a_{ij}| = 0$$

所以 $\boldsymbol{A} = \boldsymbol{O}$.

(2) 对任意实数 $\alpha, \alpha \boldsymbol{A} = (\alpha a_{ij})$, 于是

扫描全能王 创建

$$\|\alpha \boldsymbol{A}\|_1 = \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |\alpha a_{ij}| = \max_{1\leqslant j\leqslant n} |\alpha| \sum_{i=1}^{n} |a_{ij}|$$

$$= |\alpha| \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |a_{ij}| = |\alpha| \|\boldsymbol{A}\|_1$$

(3) 对任意两个 n 阶方阵 $\boldsymbol{A} = (a_{ij}), \boldsymbol{B} = (b_{ij})$

$$\|\boldsymbol{A} + \boldsymbol{B}\|_1 = \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |a_{ij} + b_{ij}| \leqslant \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} (|a_{ij}| + |b_{ij}|)$$

$$\leqslant \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |a_{ij}| + \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |b_{ij}| = \|\boldsymbol{A}\|_1 + \|\boldsymbol{B}\|_1$$

$$(4)\ \|\boldsymbol{AB}\|_1 = \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} \left| \sum_{k=1}^{n} a_{ik} b_{kj} \right| \leqslant \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} \left(\sum_{k=1}^{n} |a_{ik}|\, |b_{kj}| \right)$$

$$= \max_{1\leqslant j\leqslant n} \sum_{k=1}^{n} \left(\sum_{i=1}^{n} |a_{ik}| \right) |b_{kj}| \leqslant \max_{1\leqslant j\leqslant n} \sum_{k=1}^{n} \left(\max_{1\leqslant k\leqslant n} \sum_{i=1}^{n} |a_{ik}| \right) |b_{kj}|$$

$$= \left(\max_{1\leqslant k\leqslant n} \sum_{i=1}^{n} |a_{ik}| \right) \max_{1\leqslant j\leqslant n} \sum_{k=1}^{n} |b_{kj}| = \|\boldsymbol{A}\|_1 \|\boldsymbol{B}\|_1$$

所以 $\|\boldsymbol{A}\|_1$ 是 $\boldsymbol{A}$ 的矩阵范数.

定理 2.5　设 $\boldsymbol{A}$ 为 n 阶方阵, $\|\cdot\|$ 是 $\mathbf{R}^n$ 中的向量范数, 则

$$\|\boldsymbol{A}\| = \max_{\boldsymbol{x}\neq \boldsymbol{0}} \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|}$$

是一种矩阵范数, 称其为由向量范数 $\|\cdot\|$ 诱导出的矩阵范数, 也称算子范数.

证明　设 $\boldsymbol{A} = (a_{ij})$ 为任意 n 阶方阵, $\boldsymbol{x}$ 为任意 n 维非零向量. 因为

$$\frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} = \left\| \boldsymbol{A} \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|} \right\|$$

$\dfrac{\boldsymbol{x}}{\|\boldsymbol{x}\|}$ 为单位向量, 故

$$\|\boldsymbol{A}\| = \max_{\boldsymbol{x}\neq \boldsymbol{0}} \frac{\|\boldsymbol{Ax}\|}{\|\boldsymbol{x}\|} = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{Ax}\| \tag{2.20}$$

(1) 显然 $\|\boldsymbol{A}\| \geqslant 0$, 若 $\boldsymbol{A} = \boldsymbol{O}$, 则

$$\|\boldsymbol{A}\| = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{Ax}\| = 0$$

反之, 若 $\|\boldsymbol{A}\| = 0$, 即对任意非零向量, 有 $\|\boldsymbol{Ax}\| = 0$, 由定义 2.2 之 (1), 得 $\boldsymbol{Ax} = \boldsymbol{0}$, 所以 $\boldsymbol{A} = \boldsymbol{O}$.

(2) 对任意实数 α, 由式(2.20)及定义 2.2 有

扫描全能王 创建

$$\|\alpha \boldsymbol{A}\| = \max_{\|\boldsymbol{x}\|=1} \|\alpha \boldsymbol{A}\boldsymbol{x}\| = \max_{\|\boldsymbol{x}\|=1} |\alpha| \|\boldsymbol{A}\boldsymbol{x}\|$$

$$= |\alpha| \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\boldsymbol{x}\| = |\alpha| \|\boldsymbol{A}\|$$

(3) 对任意两个 n 阶方阵 $\boldsymbol{A}$ 和 $\boldsymbol{B}$

$$\|\boldsymbol{A}+\boldsymbol{B}\| = \max_{\|\boldsymbol{x}\|=1} \|(\boldsymbol{A}+\boldsymbol{B})\boldsymbol{x}\| = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\boldsymbol{x}+\boldsymbol{B}\boldsymbol{x}\|$$

$$\leqslant \max_{\|\boldsymbol{x}\|=1} (\|\boldsymbol{A}\boldsymbol{x}\| + \|\boldsymbol{B}\boldsymbol{x}\|)$$

$$\leqslant \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\boldsymbol{x}\| + \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{B}\boldsymbol{x}\|$$

$$= \|\boldsymbol{A}\| + \|\boldsymbol{B}\|$$

(4) 由式 (2.20), 对任意 n 维非零向量 $\boldsymbol{x}$, 有

$$\frac{\|\boldsymbol{A}\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leqslant \|\boldsymbol{A}\|$$

即

$$\|\boldsymbol{A}\boldsymbol{x}\| \leqslant \|\boldsymbol{A}\| \|\boldsymbol{x}\|$$

于是

$$\|\boldsymbol{A}\boldsymbol{B}\| = \max_{\|\boldsymbol{x}\|=1} \|(\boldsymbol{A}\boldsymbol{B})\boldsymbol{x}\| = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}(\boldsymbol{B}\boldsymbol{x})\|$$

$$\leqslant \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\| \|\boldsymbol{B}\boldsymbol{x}\| \leqslant \max_{\|\boldsymbol{x}|=1} \|\boldsymbol{A}\| \|\boldsymbol{B}\| \|\boldsymbol{x}\|$$

$$= \|\boldsymbol{A}\| \|\boldsymbol{B}\|$$

所以式(2.20)是矩阵 $\boldsymbol{A}$ 的范数.

由定理 2.5的证明知, 由向量范数诱导出的矩阵范数除满足定义 2.3 中的(1)—(4)外, 还满足性质

(5) 对任意 n 维向量 $\boldsymbol{x}$, 都有

$$\|\boldsymbol{A}\boldsymbol{x}\| \leqslant \|\boldsymbol{A}\| \|\boldsymbol{x}\|$$

这一性质称为矩阵范数与向量范数的相容性, 在误差分析中经常会用到.

常用的矩阵范数有四种, 前三种是由向量的 1− 范数, 2− 范数, ∞− 范数诱导出的矩阵范数. 设 $\boldsymbol{A} = (a_{ij})$ 为 n 阶方阵:

(1) 1− 范数 (列范数)

$$\|\boldsymbol{A}\|_1 = \max_{\|\boldsymbol{x}\|_1=1} \|\boldsymbol{A}\boldsymbol{x}\|_1 = \max_{1\leqslant j\leqslant n} \sum_{i=1}^{n} |a_{ij}|$$

(2) ∞− 范数 (行范数)

$$\|\boldsymbol{A}\|_\infty = \max_{\|\boldsymbol{x}\|_\infty=1} \|\boldsymbol{A}\boldsymbol{x}\|_\infty = \max_{1\leqslant i\leqslant n} \sum_{j=1}^{n} |a_{ij}|$$

扫描全能王 创建

(3) 2– 范数

$$\|\boldsymbol{A}\|_2 = \max_{\|\boldsymbol{x}\|_2=1} \|\boldsymbol{A}\boldsymbol{x}\|_2 = \sqrt{\lambda_1}$$

其中 λ_1 是矩阵 $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}$ 的最大特征值. 证明可参看文献 [3].

(4) $F-$ 范数

$$\|\boldsymbol{A}\|_F = \left(\sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij}^2\right)^{1/2}$$

矩阵的 $F-$ 范数可以认为是向量 2–范数的直接推广, 即将 n 阶方阵看作 n^2 维向量所得到. 可以证明, 矩阵的 $F-$范数与向量的 2–范数相容, 但 $F-$范数不是由向量范数诱导出的矩阵范数. 因为 $\|\boldsymbol{I}\|_F = \sqrt{n}$, 而对任意一种由向量范数诱导出的矩阵范数 $\|\cdot\|$, 都有

$$\|\boldsymbol{I}\| = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{I}\boldsymbol{x}\| = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{x}\| = 1$$

如果将矩阵范数看作 $\mathbf{R}^{n^2}$ 空间上的向量范数, 则由向量范数的等价性可得矩阵范数的等价性.

矩阵的误差可用矩阵范数表示. 设 $\boldsymbol{A}^*$ 是 $\boldsymbol{A}$ 的近似矩阵, $\|\boldsymbol{A}-\boldsymbol{A}^*\|, \|\boldsymbol{A}-\boldsymbol{A}^*\|/\|\boldsymbol{A}\|$ 分别称为 $\boldsymbol{A}^*$ 的关于范数 $\|\cdot\|$ 的绝对误差与相对误差.

2.4.2　方程组的状态与条件数

一个实际问题化为数学问题, 初始数据往往会有误差, 即有扰动, 从而使计算结果产生误差, 因此需要研究扰动对解的影响.

例 2.8　容易看出, 方程组

$$\begin{cases} x_1 + x_2 = 2 \\ x_1 + 1.000\ 01x_2 = 2 \end{cases}$$

的解为 $x_1 = 2, x_2 = 0$. 而方程组

$$\begin{cases} x_1 + x_2 = 2 \\ x_1 + 1.000\ 01x_2 = 2.000\ 01 \end{cases}$$

的解为 $x_1 = 1, x_2 = 1$.

比较这两个方程组可以看出, 它们只是右端项有微小的差别, 最大相对误差为 $\frac{1}{2}\times 10^{-5}$, 但它们的解却大不相同, 解分量的相对误差至少为 $\frac{1}{2}$.

当一个方程组, 由于系数矩阵或右端项的微小扰动, 而引起解发生巨大变化时, 称该方程组是“病态”的. 为了定量刻画方程组“病态”的程度, 下面对方程组 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 就系数矩阵或右端项分别有扰动的两种情形进行讨论.

首先考察右端项 $\boldsymbol{b}$ 的扰动对解的影响. 设 $\boldsymbol{b}$ 有扰动 $\delta\boldsymbol{b}$, 相应的解 $\boldsymbol{x}$ 的扰动记为 $\delta\boldsymbol{x}$, 即

$$\boldsymbol{A}(\boldsymbol{x} + \delta\boldsymbol{x}) = \boldsymbol{b} + \delta\boldsymbol{b}$$

由 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, 得 $\boldsymbol{A}\delta\boldsymbol{x} = \delta\boldsymbol{b}$, 从而有

扫描全能王 创建

$$\delta \boldsymbol{x} = \boldsymbol{A}^{-1}\delta \boldsymbol{b}$$

两边取范数, 得

$$\|\delta \boldsymbol{x}\| = \left\|\boldsymbol{A}^{-1}\delta \boldsymbol{b}\right\| \leqslant \left\|\boldsymbol{A}^{-1}\right\| \|\delta \boldsymbol{b}\|$$

又因为

$$\|\boldsymbol{x}\| \geqslant \frac{\|\boldsymbol{A}\boldsymbol{x}\|}{\|\boldsymbol{A}\|} = \frac{\|\boldsymbol{b}\|}{\|\boldsymbol{A}\|}$$

所以

$$\frac{\|\delta \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leqslant \|\boldsymbol{A}\|\|\boldsymbol{A}^{-1}\|\frac{\|\delta \boldsymbol{b}\|}{\|\boldsymbol{b}\|} \tag{2.21}$$

式 (2.21) 表明, 当右端项有扰动时, 解的相对误差不超过右端项的相对误差的 $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 倍.

如果右端项无扰动, 系数矩阵 $\boldsymbol{A}$ 有扰动 $\delta \boldsymbol{A}$, 相应的解 $\boldsymbol{x}$ 的扰动仍记为 $\delta \boldsymbol{x}$, 则

$$(\boldsymbol{A} + \delta \boldsymbol{A})(\boldsymbol{x} + \delta \boldsymbol{x}) = \boldsymbol{b}$$

消去 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, 得

$$\boldsymbol{A}\delta \boldsymbol{x} + \delta \boldsymbol{A}(\boldsymbol{x} + \delta \boldsymbol{x}) = 0$$

因而有

$$\begin{aligned}\|\delta \boldsymbol{x}\| &= \left\|\boldsymbol{A}^{-1}\delta \boldsymbol{A}(\boldsymbol{x} + \delta \boldsymbol{x})\right\| \\ &\leqslant \left\|\boldsymbol{A}^{-1}\right\| \|\delta \boldsymbol{A}\|(\|\boldsymbol{x} + \delta \boldsymbol{x}\|)\end{aligned}$$

则

$$\frac{\|\delta \boldsymbol{x}\|}{\|\boldsymbol{x} + \delta \boldsymbol{x}\|} \leqslant \|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\| \frac{\|\delta \boldsymbol{A}\|}{\|\boldsymbol{A}\|} \tag{2.22}$$

式(2.22)表明, 当系数矩阵有扰动时, 解的扰动仍与 $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 有关. $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 越大, 解的扰动也越大.

一般地, 若系数矩阵 $\boldsymbol{A}$ 有扰动 $\delta \boldsymbol{A}$, 右端项有扰动 $\delta \boldsymbol{b}$, 相应的解 $\boldsymbol{x}$ 有扰动 $\delta \boldsymbol{x}$, 且 $\left\|\boldsymbol{A}^{-1}\right\| \|\delta \boldsymbol{A}\| < 1$, 则有

$$\frac{\|\delta \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leqslant \frac{\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|}{1 - \|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\| \frac{\|\delta \boldsymbol{A}\|}{\|\boldsymbol{A}\|}} \left(\frac{\|\delta \boldsymbol{A}\|}{\|\boldsymbol{A}\|} + \frac{\|\delta \boldsymbol{b}\|}{\|\boldsymbol{b}\|}\right)$$

综合上述各式可以得出, 当系数矩阵或右端项有扰动时, 数 $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 控制了解的扰动程度. 也就是说, 数 $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 可以反映方程组的状态.

定义 2.4　对非奇异矩阵 $\boldsymbol{A}$, 称数 $\|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$ 为矩阵 $\boldsymbol{A}$ 的条件数, 记为

$$\operatorname{cond}(\boldsymbol{A}) = \|\boldsymbol{A}\| \left\|\boldsymbol{A}^{-1}\right\|$$

引入条件数后, 式 (2.21)和式(2.22)可改写为

扫描全能王 创建

$$\frac{\|\delta x\|}{\|x\|} \leqslant \operatorname{cond}(A)\frac{\|\delta b\|}{\|b\|}$$

$$\frac{\|\delta x\|}{\|x+\delta x\|} \leqslant \operatorname{cond}(A)\frac{\|\delta A\|}{\|A\|}$$

因此, 系数矩阵 A 的条件数刻画了方程组 $Ax=b$ 的“病态”程度, 条件数越大, “病态”越严重.

矩阵的条件数与所取范数有关, 最常用的条件数有

$$\operatorname{cond}(A)_\infty = \|A\|_\infty \left\|A^{-1}\right\|_\infty$$

$$\operatorname{cond}(A)_2 = \|A\|_2 \left\|A^{-1}\right\|_2 = \sqrt{\frac{\lambda_1}{\lambda_n}}$$

其中 λ_1, λ_n 分别为矩阵的 $A^{\mathrm{T}}A$ 的最大特征值和最小特征值, 故 $\operatorname{cond}(A)_2$ 又称为谱条件数. 特别地, 如果 A 为实对称矩阵, $\lambda_i\ (i=1,2,\cdots,n)$ 为 A 的特征值, 且 $|\lambda_1| \geqslant |\lambda_2| \geqslant \cdots \geqslant |\lambda_n|$, 则

$$\operatorname{cond}(A)_2 = \frac{|\lambda_1|}{|\lambda_n|}$$

例 2.8 中, 方程组的系数矩阵为

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1+10^{-5} \end{bmatrix}$$

其逆矩阵为

$$A^{-1} = \begin{bmatrix} 1+10^5 & -10^5 \\ -10^5 & 10^5 \end{bmatrix}$$

于是有

$$\operatorname{cond}(A)_\infty = \|A\|_\infty \left\|A^{-1}\right\|_\infty = \left(2+10^{-5}\right)\left(2\times 10^5+1\right) > 4\times 10^5$$

条件数很大, 解的相对误差至少是右端项相对误差的 4×10^5 倍, 所以方程组是“病态”的.

由条件数的定义及矩阵范数的性质, 易得

(1) 对任意 n 阶方阵 A, 都有 $\operatorname{cond}(A) \geqslant 1$.

(2) 对任意非零实数 α, 有 $\operatorname{cond}(\alpha A) = \operatorname{cond}(A)$.

(3) 若 A 为正交矩阵, 则 $\operatorname{cond}(A)_2 = 1$.

例 2.9　已知方程组

$$\begin{cases} x_1 + \dfrac{1}{2}x_2 + \dfrac{1}{3}x_3 = 1 \\ \dfrac{1}{2}x_1 + \dfrac{1}{3}x_2 + \dfrac{1}{4}x_3 = 0 \\ \dfrac{1}{3}x_1 + \dfrac{1}{4}x_2 + \dfrac{1}{5}x_3 = 0 \end{cases}$$

的解为 $x_1=9, x_2=-36, x_3=30$. 若把系数取成 2 位有效数字的小数, 得扰动方程组

扫描全能王 创建

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.50x_1 + 0.33x_2 + 0.25x_3 = 0 \\ 0.33x_1 + 0.25x_2 + 0.20x_3 = 0 \end{cases}$$

求解扰动方程组, 并分析所得结果.

解　用 Gauss 消去法求解, 第一次消元得

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.08x_2 + 0.085x_3 = -0.50 \\ 0.085x_2 + 0.091\ 1x_3 = -0.33 \end{cases}$$

第二次消元得

$$\begin{cases} 1.00x_1 + 0.50x_2 + 0.33x_3 = 1 \\ 0.08x_2 + 0.085x_3 = -0.50 \\ 7.875 \times 10^{-4}x_3 = 0.201\ 25 \end{cases}$$

回代得解

$$x_3 = 255.555\ 56, x_2 = -277.777\ 78, x_1 = 55.555\ 56$$

比较两个方程组可以看出, 它们的右端项相同, 系数矩阵中元的最大相对误差仅为 0.01, 而得到的解却严重失真. 这表明原方程组“病态”严重. 事实上, 原方程组的系数矩阵为

$$\boldsymbol{A} = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}$$

其逆矩阵为

$$\boldsymbol{A}^{-1} = \begin{bmatrix} 9 & -36 & 30 \\ -36 & 192 & -180 \\ 30 & -180 & 180 \end{bmatrix}$$

于是有

$$\operatorname{cond}(\boldsymbol{A})_\infty = \|\boldsymbol{A}\|_\infty \left\|\boldsymbol{A}^{-1}\right\|_\infty = \frac{11}{6} \times 408 = 748$$

条件数很大, 所以方程组“病态”严重.

原方程组的一般形式为

$$\begin{cases} x_1 + \frac{1}{2}x_2 + \frac{1}{3}x_3 + \cdots + \frac{1}{n}x_n = b_1 \\ \frac{1}{2}x_1 + \frac{1}{3}x_2 + \frac{1}{4}x_3 + \cdots + \frac{1}{n+1}x_n = b_2 \\ \cdots\cdots\cdots\cdots \\ \frac{1}{n}x_1 + \frac{1}{n+1}x_2 + \frac{1}{n+2}x_3 + \cdots + \frac{1}{2n-1}x_n = b_n \end{cases}$$

扫描全能王 创建

是典型的“病态”方程组, 称为 Hilbert (希尔伯特) 方程组, n 越大, “病态”越严重. 如 $n=6$ 时, $\operatorname{cond}(\boldsymbol{A})_\infty = 29\times 10^6$; $n=10$ 时, $\operatorname{cond}(\boldsymbol{A})_\infty = 35\times 10^{12}$. 严重“病态”的方程组, 即使用主元素法求解也是数值不稳定的.

由于条件数需要求矩阵的逆矩阵, 因而计算有一定的困难. 但在下列情况下, 方程组往往是“病态”的:

(1) 在用主元素法时出现小主元;

(2) 系数矩阵中有行 (或列) 近似线性相关;

(3) 系数矩阵的元素的值数量级相差很大.

对于病态方程组, 通常采用双精度计算, 可适当减轻病态矩阵的影响. 还可对方程组进行平衡预处理, 即选择非奇异对角矩阵 $\boldsymbol{D},\boldsymbol{C}$, 使得求解线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 的问题转化为求解等价线性方程组 $\boldsymbol{DAC}\left(\boldsymbol{C}^{-1}\boldsymbol{x}\right)=\boldsymbol{Db}$, 而矩阵 $\boldsymbol{DAC}$ 的条件数得以改善. 例如, 取 $\boldsymbol{C}=\boldsymbol{I}$ 为单位矩阵, $\boldsymbol{D}$ 的对角元素 d_i 满足

$$\frac{1}{d_i}=\max_{1\leqslant j\leqslant n}\left|a_{ij}\right|,\quad i=1,2,\cdots,n$$

令 $\tilde{\boldsymbol{A}}=\boldsymbol{DA},\tilde{\boldsymbol{b}}=\boldsymbol{Db}$, 线性方程组 $\tilde{\boldsymbol{A}}\boldsymbol{x}=\tilde{\boldsymbol{b}}$ 的条件数往往会得到改善, 解的精度能提高.

例 2.10　设方程 $\boldsymbol{Ax}=\boldsymbol{b}$ 为

$$\begin{bmatrix}1 & 10^6\\ 1 & 1\end{bmatrix}\begin{bmatrix}x_1\\ x_2\end{bmatrix}=\begin{bmatrix}10^6\\ 2\end{bmatrix}$$

计算条件数 $\operatorname{cond}(\boldsymbol{A})_\infty$, 并进行平衡预处理, 使等价方程 $\tilde{\boldsymbol{A}}\boldsymbol{x}=\tilde{\boldsymbol{b}}$ 的条件数 $\operatorname{cond}(\tilde{\boldsymbol{A}})_\infty$ 得到改善.

解　由于

$$\boldsymbol{A}=\begin{bmatrix}1 & 10^6\\ 1 & 1\end{bmatrix},\quad \boldsymbol{A}^{-1}=\frac{1}{10^6-1}\begin{bmatrix}-1 & 10^6\\ 1 & -1\end{bmatrix}$$

于是

$$\operatorname{cond}(\boldsymbol{A})_\infty=\frac{\left(1+10^6\right)^2}{10^6-1}\approx 10^6$$

方程组 $\boldsymbol{Ax}=\boldsymbol{b}$ 是“病态”的.

取 $\boldsymbol{D}=\begin{bmatrix}10^{-6} & 0\\ 0 & 1\end{bmatrix}$, 原方程组的等价方程组 $\boldsymbol{DAx}=\boldsymbol{Db}$ 的系数矩阵 $\tilde{\boldsymbol{A}}$ 及 $\tilde{\boldsymbol{A}}^{-1}$ 为

$$\tilde{\boldsymbol{A}}=\begin{bmatrix}10^{-6} & 1\\ 1 & 1\end{bmatrix},\quad \tilde{\boldsymbol{A}}^{-1}=\frac{1}{1-10^{-4}}\begin{bmatrix}-1 & 1\\ 1 & -10^{-6}\end{bmatrix}$$

则

$$\operatorname{cond}(\tilde{\boldsymbol{A}})_\infty=\frac{2^2}{1-10^{-6}}\approx 4$$

条件数得到极大改善, 等价方程组 $\tilde{\boldsymbol{A}}\boldsymbol{x}=\tilde{\boldsymbol{b}}$ 是“良态”的. 对 $\tilde{\boldsymbol{A}}\boldsymbol{x}=\tilde{\boldsymbol{b}}$ 用列主元素法得近似解

$$x_1=1,\quad x_2=1$$

扫描全能王 创建

有些病态方程组, 选用合适的迭代法求解, 也能得到高精度的解. 参阅文献 [3], [4], [18].

2.4.3 误差分析

在求得方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的一个近似解 $\boldsymbol{x}^*$ 后, 通常用残量 (余量) $\boldsymbol{r}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^*$ 或其模 $\|\boldsymbol{r}\|$ 的大小来检验近似解 $\boldsymbol{x}^*$ 精确程度.

定理 2.6 设 $\boldsymbol{x}$ 和 $\boldsymbol{x}^*$ 分别是方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的精确解和近似解, $\boldsymbol{r}$ 为 $\boldsymbol{x}^*$ 的残量, 则

$$\frac{\|\boldsymbol{x}-\boldsymbol{x}^*\|}{\|\boldsymbol{x}\|}\leqslant \operatorname{cond}(\boldsymbol{A})\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}$$

证明 因为

$$\|\boldsymbol{b}\|=\|\boldsymbol{A}\boldsymbol{x}\|\leqslant\|\boldsymbol{A}\|\|\boldsymbol{x}\|$$

$$\|\boldsymbol{x}-\boldsymbol{x}^*\|=\left\|\boldsymbol{A}^{-1}\left(\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^*\right)\right\|=\left\|\boldsymbol{A}^{-1}\boldsymbol{r}\right\|\leqslant\left\|\boldsymbol{A}^{-1}\right\|\|\boldsymbol{r}\|$$

所以

$$\frac{\|\boldsymbol{x}-\boldsymbol{x}^*\|}{\|\boldsymbol{x}\|}\leqslant\frac{\left\|\boldsymbol{A}^{-1}\right\|\|\boldsymbol{r}\|}{\frac{\|\boldsymbol{b}\|}{\|\boldsymbol{A}\|}}=\|\boldsymbol{A}\|\left\|\boldsymbol{A}^{-1}\right\|\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}=\operatorname{cond}(\boldsymbol{A})\frac{\|\boldsymbol{r}\|}{\|\boldsymbol{b}\|}$$

由于残量 $\boldsymbol{r}$ 的计算很容易, 当方程组是“良态”时, $\boldsymbol{r}$ 是检验近似解 $\boldsymbol{x}^*$ 精确程度的一个简单实用的指标. 但对于“病态”严重的方程组, 即使 $\|\boldsymbol{r}\|$ 很小, 近似解 $\boldsymbol{x}^*$ 与精确解 $\boldsymbol{x}$ 仍可能有很大的差别. 如例 2.8 中, 方程组

$$\begin{cases}x_1+x_2=2\\ x_1+1.000\ 01x_2=2\end{cases}$$

的精确解为 $x_1=2,x_2=0$. 若以 $x_1=1,x_2=1$ 作为它的近似解, 其残量

$$\boldsymbol{r}=(2,2)^{\mathrm{T}}-(2,2.000\ 01)^{\mathrm{T}}=\left(0,-10^{-5}\right)^{\mathrm{T}}$$

很小, 但解的误差

$$\boldsymbol{x}-\boldsymbol{x}^*=(2,0)^{\mathrm{T}}-(1,1)^{\mathrm{T}}=(1,-1)^{\mathrm{T}}$$

却不小.

由此可见, 利用残量 $\boldsymbol{r}$ 的大小来检验近似解 $\boldsymbol{x}^*$ 的精确程度仅适用于“良态”方程组, 对于“病态”方程组是不可靠的.

2.5 超定线性方程组的最小二乘解

如果方程组 (2.1) 中方程个数多于变量的个数, 称之为超定线性方程组. 这种方程组一般无精确解, 只能在各种规定的意义下求它的广义解.

设有超定线性方程组

扫描全能王 创建

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \cdots\cdots\cdots\cdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases} \tag{2.23}$$

其矩阵形式仍记为 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, 其中 $\boldsymbol{A}$ 为 $m \times n$ 矩阵, $\boldsymbol{x} \in \mathbf{R}^n, \boldsymbol{b} \in \mathbf{R}^m (m > n)$.

考虑方程组 (2.23) 在下述意义下求解: 求 $\boldsymbol{x}$, 使得由它产生的每个方程的偏差 $\left(b_i - \sum_{j=1}^{n} a_{ij}x_j\right)$ 的平方和最小, 即求问题 $\min F(\boldsymbol{x})$

$$F(\boldsymbol{x}) = \sum_{i=1}^{m} \left(b_i - \sum_{j=1}^{n} a_{ij}x_j\right)^2 = \|\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}\|_2^2 \tag{2.24}$$

的解. 问题 (2.24) 称为线性最小二乘问题, 其解称为问题 (2.23) 的最小二乘解.

由多元微分学, 问题 (2.24)的解必定是多元函数 $F(\boldsymbol{x})$ 的驻点. 因为

$$\begin{aligned} \frac{\partial F}{\partial x_k} &= \sum_{i=1}^{m} 2\left(b_i - \sum_{j=1}^{n} a_{ij}x_j\right)(-a_{ik}) \\ &= -2\sum_{i=1}^{m} a_{ik}\left(b_i - \sum_{j=1}^{n} a_{ij}x_j\right) \quad (k = 1, 2, \cdots, n) \end{aligned}$$

因此, 求解问题 (2.24) 可转化为求解方程组

$$\sum_{i=1}^{m} a_{ik}\left(b_i - \sum_{j=1}^{n} a_{ij}x_j\right) = 0 \quad (k = 1, 2, \cdots, n)$$

即

$$\begin{cases} \sum\limits_{i=1}^{m} a_{i1}\left(\sum\limits_{j=1}^{n} a_{ij}x_j\right) = \sum\limits_{i=1}^{m} a_{i1}b_i \\ \sum\limits_{i=1}^{m} a_{i2}\left(\sum\limits_{j=1}^{n} a_{ij}x_j\right) = \sum\limits_{i=1}^{m} a_{i2}b_i \\ \cdots\cdots\cdots\cdots \\ \sum\limits_{i=1}^{m} a_{in}\left(\sum\limits_{j=1}^{n} a_{ij}x_j\right) = \sum\limits_{i=1}^{m} a_{in}b_i \end{cases} \tag{2.25}$$

为简便起见, 记 $\boldsymbol{a}_k = (a_{1k}, a_{2k}, \cdots, a_{mk})^{\mathrm{T}}$, 则

$$\sum_{i=1}^{m} a_{ik}\left(\sum_{j=1}^{n} a_{ij}x_j\right) = \sum_{j=1}^{n}\left(\sum_{i=1}^{m} a_{ik}a_{ij}\right)x_j = \sum_{j=1}^{n} \boldsymbol{a}_k^{\mathrm{T}}\boldsymbol{a}_j x_j \quad (k = 1, 2, \cdots, n)$$

$$\sum_{i=1}^{m} a_{ik}b_i = \boldsymbol{a}_k^{\mathrm{T}}\boldsymbol{b}$$

于是方程组(2.25)可简记为

扫描全能王 创建

$$\begin{cases} \sum\limits_{j=1}^{n} \boldsymbol{a}_1^{\mathrm{T}} \boldsymbol{a}_j x_j = \boldsymbol{a}_1^{\mathrm{T}} \boldsymbol{b} \\ \sum\limits_{j=1}^{n} \boldsymbol{a}_2^{\mathrm{T}} \boldsymbol{a}_j x_j = \boldsymbol{a}_2^{\mathrm{T}} \boldsymbol{b} \\ \cdots\cdots\cdots\cdots \\ \sum\limits_{j=1}^{n} \boldsymbol{a}_n^{\mathrm{T}} \boldsymbol{a}_j x_j = \boldsymbol{a}_n^{\mathrm{T}} \boldsymbol{b} \end{cases}$$

用方程组 (2.23) 的系数矩阵表示, 则为

$$\boldsymbol{A}^{\mathrm{T}} \boldsymbol{A} \boldsymbol{x} = \boldsymbol{A}^{\mathrm{T}} \boldsymbol{b} \tag{2.26}$$

这是一个 n 阶线性方程组, 称为超定问题 (2.23) 的正则方程组或正规方程组. 如果 $\boldsymbol{A}$ 为列满秩, 即 $\mathrm{rank}(\boldsymbol{A}) = n$, 则 $\boldsymbol{A}^{\mathrm{T}} \boldsymbol{A}$ 是一个对称正定矩阵, 因而正则方程组 (2.23) 存在唯一解, 其解即为超定线性方程组 (2.23) 的最小二乘解. 求解使用的最小二乘法我们将在第六章做更加详细的介绍.

例 2.11 *求方程组*

$$\begin{cases} x_1 - x_2 = 5 \\ -x_1 + 2x_2 = -4 \\ 2x_1 - 3x_2 = 10 \end{cases}$$

的最小二乘解.

解 因为系数矩阵为

$$\boldsymbol{A} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \\ 2 & -3 \end{bmatrix}$$

于是有

$$\boldsymbol{A}^{\mathrm{T}} \boldsymbol{A} = \begin{bmatrix} 1 & -1 & 2 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ -1 & 2 \\ 2 & -3 \end{bmatrix} = \begin{bmatrix} 6 & -9 \\ -9 & 14 \end{bmatrix}$$

$$\boldsymbol{A}^{\mathrm{T}} \boldsymbol{b} = \begin{bmatrix} 1 & -1 & 2 \\ -1 & 2 & -3 \end{bmatrix} \begin{bmatrix} 5 \\ -4 \\ 10 \end{bmatrix} = \begin{bmatrix} 29 \\ -43 \end{bmatrix}$$

正则方程组为

$$\begin{cases} 6x_1 - 9x_2 = 29 \\ -9x_1 + 14x_2 = -43 \end{cases}$$

其解为 $x_1 = \dfrac{19}{3}, x_2 = 1$. 这就是所求超定线性方程组的最小二乘解.

一般地, 求解方程组 (2.26) 可用平方根法或 $\mathrm{LDL}^{\mathrm{T}}$ 方法. 因此, 求解超定线性方程组的最小二乘解的步骤为:

扫描全能王 创建

(1) 计算 $\boldsymbol{M}=\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}, \boldsymbol{f}=\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}$;

(2) 对 $\boldsymbol{M}$ 作 Cholesky 分解 $\boldsymbol{M}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$ (或 $\mathrm{LDL}^{\mathrm{T}}$ 分解 $\boldsymbol{M}=\boldsymbol{L}\boldsymbol{D}\boldsymbol{L}^{\mathrm{T}}$);

(3) 求解方程组 $\boldsymbol{L}\boldsymbol{y}=\boldsymbol{f}, \boldsymbol{L}^{\mathrm{T}}\boldsymbol{x}=\boldsymbol{y}$ (或 $\boldsymbol{L}\boldsymbol{z}=\boldsymbol{f}, \boldsymbol{y}=\boldsymbol{D}^{-1}\boldsymbol{z}, \boldsymbol{L}^{\mathrm{T}}\boldsymbol{x}=\boldsymbol{y}$).

这一方法简单易行, 计算量约为 $\frac{1}{2}n^2\left(m+\frac{1}{3}n\right)$, 特别当 $m\gg n$ 时, 问题(2.26)的规模远小于问题(2.23). 虽然其数值稳定性较差, 现在仍经常使用.

利用矩阵的 2-范数的正交不变性及矩阵 $\boldsymbol{M}$ 的正交三角分解 (也称 QR 分解) 提出的正交化方法, 具有良好的数值稳定性, 计算结果比正规化方法精确. 因此, 这一方法应用范围较广. 但它的运算量比较大, 约为 $n^2(m-n/3)$, 详细内容可参阅文献 [5].

2.6　算法实现与应用实例

常用命令 北太天元软件与 MATLAB 软件在函数命令和脚本文件使用规则方面高度相容, 如果之前熟悉 MATLAB 用法, 则所有的 MATLAB 命令和脚本文件均可以直接在北太天元软件中直接使用. 下面列出一些常用的求解线性方程组有关的命令和函数.

(1) 反斜杠运算 "\"(左除命令, 同 mldivide()), 实现线性方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 求解, 命令行直接输入

```
≫ x = A\b
% 其中 b 可以是矩阵, 运算符 "\" 将 b 的各列作为右端项依次求解.
```

如例 2.5 中先输入系数矩阵和右端项, 再用左除命令得方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的解:

```
≫ A = [0, 2, 0, 1; 2, 2, 3, 2; 4, −3, 0, 1; 6, 1, −6, −5];
≫ b = [0; −2; −7; 6];
≫ x = (A\b)'
% 其中'' 实现矩阵的转置运算
```

运行结果为

```
≫ x =
−0.5000   1.0000   0.3333   − 2.0000
```

(2) 斜杠运算 "/"(右除命令, 同 mrdivide()), 实现线性方程组 $\boldsymbol{x}\boldsymbol{A}=\boldsymbol{b}$ 求解, 以例 2.5 为例, 方程为 $\boldsymbol{x}\boldsymbol{A}'=\boldsymbol{b}'$, 命令行中直接输入

```
≫ x = b'/A'
```

结果为

```
x =
  −0.5000    1.000    0.3333    −2.000
```

(3) 库函数 lu( ), 执行矩阵的列主元素分解, 命令行中输入

```
≫ [L, U, P] = lu( A)
% 对矩阵A 进行列主元素分解, P 为置换矩阵, 即PA = LU, 则A = P^{-1}LU
≫ [L1, U] = lu(A)
```


扫描全能王 创建

% 对矩阵$\boldsymbol{A}$进行列主元素分解, 即$\boldsymbol{PA}=\boldsymbol{LU}$, 则$\boldsymbol{A}=\left(\boldsymbol{P}^{-1}\ \boldsymbol{L}\right)\boldsymbol{U}=\boldsymbol{L_1U}$

对例 2.5 中系数矩阵 $\boldsymbol{A}$ 做三角分解得

≫ [L, U, P] = lu(A)

L =

1.0000	0.0000	0.0000	0.0000
0.6667	1.0000	0.0000	0.0000
0.3333	−0.4545	1.0000	0.0000
0.0000	−0.5455	0.3200	1.0000

U =

6.0000	1.0000	−6.0000	−5.0000
0.0000	−3.6667	4.0000	4.3333
0.0000	0.0000	6.8182	5.6364
0.0000	0.0000	0.0000	1.5600

P =

0	0	0	1
0	0	1	0
0	1	0	0
1	0	0	0

矩阵 $\boldsymbol{P}$ 左乘 $\boldsymbol{A}$ 实现第 1, 4 行置换, 第 2, 3 行置换, 与例 2.5 中的计算过程一致. 如果在命令行输入

≫ [L, U] = lu(A)

则结果为

L =

0.0000	−0.5455	0.3200	1.0000
0.3333	−0.4545	1.0000	0.0000
0.6667	1.0000	0.0000	0.0000
1.0000	0.0000	0.0000	0.0000

U =

6.0000	1.0000	−6.0000	−5.0000
0.0000	−3.6667	4.0000	4.3333
0.0000	0.0000	6.8182	5.6364
0.0000	0.0000	0.0000	1.5600

此时 $\boldsymbol{L}$ 并非是单位下三角形矩阵, 这是由于按列选主元素三角分解后又乘 $\boldsymbol{P}^{-1}$ 的缘故.

(4) 库函数 chol() 实现对对称正定矩阵 Cholesky 分解, 即 $\boldsymbol{A}=\boldsymbol{R}^{\mathrm{T}}\boldsymbol{R}$, $\boldsymbol{R}$ 为上三角形矩阵. 命令行窗口输入一个对称正定矩阵 $\boldsymbol{A}$, 并对其做 Cholesky 分解

≫ A = [2, −1, 0; −1, 2, −1; 0, −1, 2]

≫ R = chol(A)

结果为

R =

扫描全能王 创建

1.4142　−0.7071　0.0000

0.0000　1.2247　−0.8165

0.0000　0.0000　1.1547

(5) 函数 norm() 实现向量和矩阵范数的计算, 对于向量 $\boldsymbol{v}$ 和矩阵 $\boldsymbol{A}$, 命令行窗口输入

≫ n = norm(v)

% 返回向量 $\boldsymbol{v}$ 的欧几里得范数 (2−范数)

≫ n = norm(v, p)

% 返回广义 $p-$范数 (p 可以是任意正实数或'inf', '-inf')

≫ n = norm(A)

% 返回矩阵 $\boldsymbol{A}$ 的 2−范数或最大奇异值, 该值近似于 max(svd(X)).

≫ n = norm(X, p)

% 返回矩阵 $\boldsymbol{A}$ 的 $p-$范数, 其中 p 为 1, 2, 'inf'

(6) 函数 cond() 实现矩阵条件数的计算, 命令行窗口输入

≫ C = cond(A)

% 返回 2−范数意义下条件数, 等于 $\boldsymbol{A}$ 的最大奇异值与最小奇异值之比

≫ C = cond(A, p)

% 返回 $p-$范数条件数, 其中 p 可以是 1, 2, 'inf' 或'fro'.

% 注意: 计算 2−范数条件数时, $\boldsymbol{A}$ 的行、列可以任意; 计算其他范数条件数时, $\boldsymbol{A}$ 只能是方阵

应用实例

1. 平面桁架结构系

平面桁架是由刚性元件通过结点互相联结而组成的力学结构, 它通常出现在桥梁结构和其他需要力学支撑的结构中. 图 2.1 是一个简单的静力桁架结构, 其中刚性元件 ($m = 5$) 通过结点 A, B, C, D 相连.

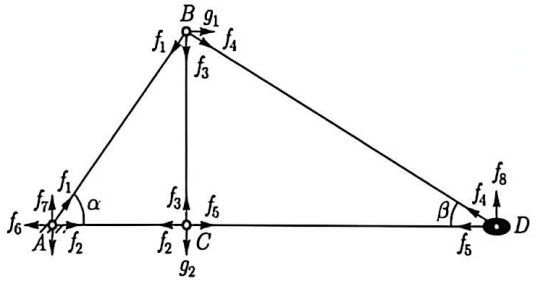


图 2.1　静力桁架结构

五个刚性元件的内力为 $\{f_1, f_2, \cdots, f_5\}$, 它们都处理为压力, 如果解是负的, 表示该力为张力. 桁架的左边由固定结点 A 支撑, 右边由滑轮 D 支撑, f_6, f_7, f_8 是外部支撑力, g_1, g_2 是外部负荷. 给定负荷时, 如何确定静力平衡时的五个内力及三个外部支撑力? 由于在静力平衡时, 每个结点处的水平方向合力与垂直方向的合力为零, 那么有

$$\text{结点 } A \begin{cases} f_1\cos\alpha + f_2 - f_6 = 0 \\ f_1\sin\alpha + f_7 = 0 \end{cases} \qquad \text{结点 } B \begin{cases} -f_1\cos\alpha + f_4\cos\beta + g_1 = 0 \\ -f_1\sin\alpha - f_3 - f_4\sin\beta = 0 \end{cases}$$

扫描全能王 创建

结点 C $\begin{cases} -f_2+f_5=0 \\ f_3-g_2=0 \end{cases}$ 结点 D $\begin{cases} -f_4\cos\beta-f_5=0 \\ f_4\sin\beta+f_8=0 \end{cases}$

设 $\boldsymbol{f}$ 表示未知力向量, 上述方程组可用矩阵表示为 $\boldsymbol{A}\boldsymbol{f}=\boldsymbol{b}$, 其中

$$\boldsymbol{A}=\begin{bmatrix} \cos\alpha & 1 & 0 & 0 & 0 & -1 & 0 & 0 \\ \sin\alpha & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ -\cos\alpha & 0 & 0 & \cos\beta & 0 & 0 & 0 & 0 \\ -\sin\alpha & 0 & -1 & -\sin\beta & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\cos\beta & -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \sin\beta & 0 & 0 & 0 & 1 \end{bmatrix}, \quad \boldsymbol{b}=\begin{bmatrix} 0 \\ 0 \\ -g_1 \\ 0 \\ 0 \\ g_2 \\ 0 \\ 0 \end{bmatrix}$$

若取 $\alpha=\dfrac{\pi}{3},\beta=\dfrac{\pi}{6}$, 外部负荷 $g_1=250\ \mathrm{N}, g_2=1\ 500\ \mathrm{N}$. 采用列主元素法或直接用矩阵左除运算 $\boldsymbol{A}\backslash\boldsymbol{b}$, 得各结点的内力如下:

$$\boldsymbol{f}=(-1\ 174,483.3,1\ 500,-966.5,483.3,103.8,1\ 017,483.3)^{\mathrm{T}}$$

2. 化学吸收过程

化学吸收过程一般是由一系列的一种物质渗入另一种相互接触的物质形成, 如蒸馏、浓缩和吸收过程. 液体的吸收过程可以用来从燃烧气体中除去二氧化硫 (SO_2). 图 2.2 显示了具有 $n=5$ 步的吸收过程. 液体从顶端以流速 L 注入, 气体由底部以流速 G 导入. 设 x_k 和 y_k 分别表示在第 k 步被吸收于液体和气体中的某成分的量, 且有

$$y_k=ax_k+b \quad (1\leqslant k\leqslant n)$$

设 x_f 表示液体中吸收的某成分的量, y_f 表示气体中吸收的某成分的量, H 表示液体的量. 经简化假设, 利用每一步中的分子平衡式, 得到吸收过程的下列方程组:

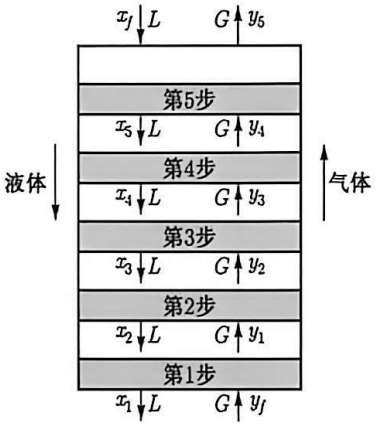


图 2.2 化学吸收过程图

$$\begin{cases} \tau\dfrac{\mathrm{d}x_1}{\mathrm{d}t}=K(y_f-b)-(1+\delta)x_1+x_2 \\ \tau\dfrac{\mathrm{d}x_2}{\mathrm{d}t}=\delta x_1-(1+\delta)x_2+x_3 \\ \tau\dfrac{\mathrm{d}x_3}{\mathrm{d}t}=\delta x_2-(1+\delta)x_3+x_4 \\ \tau\dfrac{\mathrm{d}x_4}{\mathrm{d}t}=\delta x_3-(1+\delta)x_4+x_5 \\ \tau\dfrac{\mathrm{d}x_5}{\mathrm{d}t}=\delta x_4-(1+\delta)x_5+x_f \end{cases}$$

其中 $\tau=H/L$ 是在每一步液体的滞留时间, $\delta=aG/L$ 是除去因子, $K=G/L$ 是气体与液

扫描全能王 创建

体之比. x 表示液体总量, 以滞留时间为单位时间, 则吸收过程可以表示为常微分方程组

$$\frac{\mathrm{d}x}{\mathrm{d}t} = \boldsymbol{A}\boldsymbol{x} - \boldsymbol{f}$$

其中

$$\boldsymbol{A} = \begin{bmatrix} -(1+\delta) & 1 & 0 & 0 & 0 \\ \delta & -(1+\delta) & 1 & 0 & 0 \\ 0 & \delta & -(1+\delta) & 1 & 0 \\ 0 & 0 & \delta & -(1+\delta) & 1 \\ 0 & 0 & 0 & \delta & -(1+\delta) \end{bmatrix}, \quad \boldsymbol{f} = \begin{bmatrix} K(b-y_f) \\ 0 \\ 0 \\ 0 \\ -x_f \end{bmatrix}$$

系数矩阵 $\boldsymbol{A}$ 是一个三对角矩阵. 为分析稳定态时的情况 (即常数解), 设 $\frac{\mathrm{d}x_k}{\mathrm{d}t} = 0$ $(1 \leqslant k \leqslant 5)$, 即需要求解线性方程组 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{f}$.

不失一般性, 设吸收过程的参数为 $a = 0.72, b = 0, K = 1.63, x_f = 0.01, y_f = 0.06$. 注意到系数矩阵中非零元素只有三个值, 用追赶法编写脚本程序 exp1.m 如下:

```
clear all;
n=5; a=0.72 ; b=0.0; K=1.63; xf=0.01;yf=0.06;
delta=a*K;
d=-(1+delta)*ones(n, 1);   % 对角线元素
u=ones(n, 1);   % 上面次对角线元素
w=delta*ones(n, 1);   % 下面次对角线元素
c=zeros(n, 1) ;   % 右端项, 第一和最后一个元素需要单独赋值
c(1)=K*(b-yf); c(n)=-xf;
for i=2:n   % 消元过程
w(i)=w(i)/d(i-1);   % 三角分解单位下三角形矩阵对角线下方元素
d(i)=d(i)-u(i-1)*w(i);   % 三角分解上三角形矩阵对角线元素
c(i)=c(i)-c(i-1)*w(i);   % 右端项消元结果仍然存储在向量 c 中
end
c(n)=c(n)/d(n);
for i=n-1:-1:1
c(i)=(c(i)-u(i)*c(i+1))/d(i);   % 方程组的解存储在向量 c 中
end
x=c'
y=a*x+b
```

运行结果为

```
x =
  0.075 4    0.066 2    0.055 3    0.042 5    0.027 6
y =
```


扫描全能王 创建

0.054 3　0.047 6　0.039 8　0.030 6　0.019 9

x, y 分别为各个阶段吸收于液体和气体中的某成分的量.

评注二

Gauss 消去法是解线性方程组直接法的基础. 若系数矩阵 $\boldsymbol{A}$ 非奇异, 当 $\left|a_{kk}^{(k)}\right|$ 很小或系数矩阵顺序主子式为零时, 采用列主元素法, 往往可得到较好的结果. 求解具有相同系数矩阵的一系列方程组时, 用直接三角分解法更为简便; 对于对称正定的系数矩阵, Cholesky 方法是有效的方法; 当系数矩阵是严格对角占优的三对角矩阵时, 追赶法是既快又数值稳定的常用方法, 若系数矩阵是分块三对角矩阵, 也有相应的追赶法.

直接法是目前求解中小规模线性方程组 ($n < 1\ 000$) 最常用的方法, 它的计算量为 $O(n^3)$. 直接法一般适用于系数矩阵稠密 (即矩阵的绝大多数元素都不为零) 而又不具有特殊结构的线性方程组. 如果系数矩阵具有某种特殊形式 (如矩阵中非零元素占很小百分比的稀疏矩阵、正定矩阵、对称矩阵和反称矩阵等), 无论在存储上还是在计算上都有专门的技术和方法, 详细可参阅 [16—18].

本章知识要点 直接Gauss消去法; 主元素法; 三角分解法; 追赶法; 向量和矩阵的范数; 条件数; 病态线性方程组.

算法历史及背后的数学家

习题二

1. 分别用 Gauss 消去法、列主元素法和全主元素法解下列方程组, 计算过程保留 3 位小数:

(1)
$$\begin{cases} 2x_1 + 6x_2 - 4x_3 = 4 \\ x_1 + 4x_2 - 5x_3 = 3 \\ 6x_1 - x_2 + 18x_3 = 2 \end{cases}$$

(2)
$$\begin{cases} 2x_1 + x_2 + 2x_3 = 6 \\ 4x_1 + 3x_2 + x_3 = 11 \\ 6x_1 + x_2 + 5x_3 = 13 \end{cases}$$

2. 用三角分解法求解题 1 中的方程组.

3. 用紧凑格式解下列方程组, 并写出 $\boldsymbol{L}, \boldsymbol{U}$ 矩阵:

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 1 & 4 & 9 & 16 \\ 1 & 8 & 27 & 64 \\ 1 & 16 & 81 & 256 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 2 \\ 10 \\ 44 \\ 190 \end{bmatrix}$$

扫描全能王 创建

4. 若

$$\boldsymbol{L}_k=\begin{bmatrix}1&&&&&\\&\ddots&&&&\\&&1&&&\\&&-l_{k+1,k}&1&&\\&&-l_{k+2,k}&\cdots&1&\\&&\vdots&&&\ddots\\&&-l_{n,k}&&\cdots&1\end{bmatrix}(k=1,2,\cdots,n-1)$$

求证: (1) $\boldsymbol{L}_k^{-1}=2\boldsymbol{I}-\boldsymbol{L}_k=\begin{bmatrix}1&&&&\\&\ddots&&&\\&&1&&\\&&l_{k+1,k}&1&\\&&l_{k+2,k}&&1\\&&\vdots&&&\ddots\\&&l_{n,k}&&&1\end{bmatrix}$

(2) $\boldsymbol{L}_1^{-1}\boldsymbol{L}_2^{-1}\cdots\boldsymbol{L}_{n-1}^{-1}=\begin{bmatrix}1&&&&\\l_{21}&1&&&\\l_{31}&l_{32}&1&&\\\vdots&\vdots&\vdots&\ddots&\\l_{n1}&l_{n2}&l_{n3}&\cdots&1\end{bmatrix}$

5. 用三角分解法求下列矩阵的逆矩阵:

(1) $\begin{bmatrix}1&1&-1\\2&1&0\\1&-1&0\end{bmatrix}$ (2) $\begin{bmatrix}2&2&3\\1&-1&0\\-1&2&1\end{bmatrix}$

6. 分别用平方根法和改进平方根法求解方程组

$$\begin{bmatrix}1&2&1&-3\\2&5&0&-5\\1&0&14&1\\-3&-5&1&15\end{bmatrix}\begin{bmatrix}x_1\\x_2\\x_3\\x_4\end{bmatrix}=\begin{bmatrix}1\\2\\16\\8\end{bmatrix}$$

7. 设有方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$, 其中

$$\boldsymbol{A}=\begin{bmatrix}2&1&0\\1&2&a\\0&a&2\end{bmatrix},\quad \boldsymbol{x}=(x_1,x_2,x_3)^{\mathrm{T}},\quad \boldsymbol{b}=(3,3,1)^{\mathrm{T}}$$

(1) 求出 $\boldsymbol{A}$ 能进行 Cholesky 分解, 即 $\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$ (其中 $\boldsymbol{L}$ 为下三角形矩阵) 的 a 的取值范围;

(2) 取 $a=1$, 对矩阵 $\boldsymbol{A}$ 进行 Cholesky 分解, 并用平方根法求解上述方程组, 计算过程保留 2 位小数.

扫描全能王 创建

8. 用追赶法解下列方程组:

(1)
$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 2.5 \end{bmatrix}$$

(2)
$$\begin{bmatrix} 136.01 & 90.860 & 0 & 0 \\ 90.860 & 98.810 & -67.590 & 0 \\ 0 & -67.590 & 132.01 & 46.260 \\ 0 & 0 & 46.260 & 177.17 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} -33.254 \\ 49.709 \\ 28.067 \\ -7.324\,4 \end{bmatrix}$$

9. 证明矩阵的 $F-$ 范数与向量的 $2-$ 范数相容.

10. 已知

$$\boldsymbol{A} = \begin{bmatrix} 6 & 13 & -17 \\ 13 & 29 & -38 \\ -17 & -38 & 50 \end{bmatrix}$$

求 $\mathrm{cond}(\boldsymbol{A})_1$ 及 $\mathrm{cond}(\boldsymbol{A})_\infty$, 并说明方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 是否病态.

11. 分别用 Gauss 消去法与列主元素法求解线性方程组

$$\begin{cases} 1.003x_1 + 58.09x_2 = 68.12 \\ 321.8x_1 + 5.550x_2 = 377.3 \end{cases}$$

计算过程取 4 位有效数字. 比较计算结果, 哪个算法的精度高?

12. 已知方程组

$$\begin{cases} x_1 + 0.99x_2 = 1 \\ 0.99x_1 + 0.98x_2 = 1 \end{cases}$$

的解为 $x_1 = 100, x_2 = -100$.

(1) 计算系数矩阵的条件数.

(2) 取 $\boldsymbol{x}_1^* = (1,0)^{\mathrm{T}}, \boldsymbol{x}_2^* = (100.5, -99.5)^{\mathrm{T}}$, 分别计算残量 $\boldsymbol{r}_i = \boldsymbol{b} - \boldsymbol{Ax}_i^*$ $(i = 1,2)$. 本题的计算结果说明了什么?

13. 求超定方程组

$$\begin{cases} 2x_1 + 4x_2 = 11 \\ 3x_1 - 5x_2 = 3 \\ x_1 + 2x_2 = 6 \\ 2x_1 + x_2 = 7 \end{cases}$$

的最小二乘解.

扫描全能王 创建

数值实验二

① 试用北太天元编程实现矩阵的列主元素三角分解, 并求 Pascal(帕斯卡) 矩阵

$$\boldsymbol{A}=\begin{bmatrix}1&1&1&1&1\\1&2&3&4&5\\1&3&6&10&15\\1&4&10&20&35\\1&5&15&35&70\end{bmatrix}$$

的逆矩阵 $\boldsymbol{A}^{-1}$, 用左除命令 $\boldsymbol{A}\backslash\boldsymbol{E}$ 检验你的结果.

2. 试用北太天元编程实现追赶法求解三对角方程组的算法, 并考虑图 2.3 中梯形电阻电路问题:

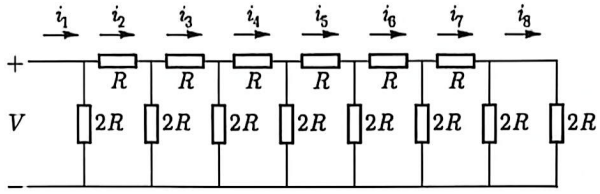


图 2.3

其中电路中的各个电流 $\{i_1,i_2,\cdots,i_8\}$ 满足下列线性方程组:

$$\begin{cases}2i_1-2i_2=V/R\\-2i_1+5i_2-2i_3=0\\-2i_2+5i_3-2i_4=0\\-2i_3+5i_4-2i_5=0\\-2i_4+5i_5-2i_6=0\\-2i_5+5i_6-2i_7=0\\-2i_6+5i_7-2i_8=0\\-2i_7+5i_8=0\end{cases}$$

设 $V=220\ \mathrm{V}, R=27\ \Omega$, 求各段电路的电流.

3. 方程组的性态和矩阵条件数的试验. 设有线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$, 其中系数矩阵 $\boldsymbol{A}=(a_{ij})_{n\times n}$ 的元素分别为 $a_{ij}=x_i^{j-1}\,(x_i=1+0.1i,i,j=1,2,\cdots,n)$ 或 $a_{ij}=\dfrac{1}{i+j-1}(i,j=1,2,\cdots,n)$, 右端向量 $\boldsymbol{b}=\left(\sum\limits_{j=1}^{n}a_{1j},\sum\limits_{j=1}^{n}a_{2j},\cdots,\sum\limits_{j=1}^{n}a_{nj}\right)^{\mathrm{T}}$. 利用北太天元中的库函数计算.

(1) 取 $n=5,10,20$, 分别求出系数矩阵的 2− 条件数, 判别它们是否是病态矩阵? 随着 n 的增大, 矩阵性态的变化如何?

(2) 分别取 $n=5,10,20$, 解上述两个线性方程组 $\boldsymbol{Ax}=\boldsymbol{b}$, 并将求得的解与精确解

扫描全能王 创建

$x=(1,1,\cdots,1)^{\mathrm{T}}$ 作比较, 说明了什么?

(3) 取 $n=10$, 对系数矩阵中的 a_{22} 和 a_{nn} 增加一个扰动 10^{-8}, 求解方程组 $(\boldsymbol{A}+\delta\boldsymbol{A})\boldsymbol{x}=\boldsymbol{b}$, 解的变化如何?

4. 已知 Wilson(威尔逊) 矩阵

$$\boldsymbol{A}=\begin{bmatrix}10&7&8&7\\7&5&6&5\\8&6&10&9\\7&5&9&10\end{bmatrix}$$

右端项 $\boldsymbol{b}=(32,23,33,31)^{\mathrm{T}}$, 则方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 有精确解 $\boldsymbol{x}=(1,1,\cdots,1)^{\mathrm{T}}$. 用北太天元中的库函数计算.

(1) 求 $\det(\boldsymbol{A})\,\mathrm{cond}(\boldsymbol{A})$, 及 $\boldsymbol{A}$ 的所有特征值.

(2) 令

$$\boldsymbol{A}+\delta\boldsymbol{A}=\begin{bmatrix}10&7.2&8.1&6.9\\7.08&5.07&6.02&5\\8.2&5.89&9.96&9.01\\6.98&5.04&8.97&9.98\end{bmatrix}$$

解扰动方程组 $(\boldsymbol{A}+\delta\boldsymbol{A})(\boldsymbol{x}+\delta\boldsymbol{x})=\boldsymbol{b}$, 并求出向量 $\delta\boldsymbol{x}$ 和 $\|\delta\boldsymbol{x}\|_2$, 从理论结果式 (2.22) 和实际计算结果两方面分析方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的解的相对误差 $\dfrac{\|\delta\boldsymbol{x}\|_2}{\|\boldsymbol{x}\|_2}$ 与 $\boldsymbol{A}$ 的相对误差 $\dfrac{\|\delta\boldsymbol{A}\|_2}{\|\boldsymbol{A}\|_2}$ 的关系.

(3) 利用函数 rand(n), 重新产生扰动矩阵 $\delta\boldsymbol{A}$, 使其各元素的绝对值不超过 $\dfrac{1}{2}\times10^{-4}$, 重复 (2).

扫描全能王 创建

/第三章/
解线性方程组的迭代法

随着计算机技术的发展、存储量的增大以及计算速度的提高, 直接法在计算机上可以求解的矩阵规模越来越大, 但直接法大多需要对系数矩阵进行分解, 运算中也很难保持矩阵的稀疏性. 而在实际应用中, 我们往往面对的是大型稀疏线性方程组的求解问题. 目前发展起来的求解稀疏线性方程组的方法主要有两大类: 一类是迭代法, 另一类是 (系数稀疏) 直接法. 后者是直接法和稀疏矩阵求解技巧的结合, 充分考虑零元素的分布特点, 采用灵活的主元选取策略, 使得分解完的矩阵因子尽可能保持原有的稀疏性, 并和问题本身密切相关. 我们这一章主要介绍迭代法.

迭代法是按照某种规则构造一个向量序列, 使其极限向量是线性方程组的解. 对于迭代法, 一般我们需要考虑

- 如何构造迭代序列?
- 构造的序列是否收敛, 在什么条件下收敛?
- 如果收敛, 收敛速度如何? 我们需要定量的刻画来比较不同方法的快慢.
- 解的误差估计和迭代过程中中断的处理.

一个方法是否有效要看得到具体某个精确度的近似解需要付出的代价. 通常以运算量和存储量的要求为标准. 在这个标准下, 直接法在很多情况下比迭代法好, 但是对于大型稀疏线性方程组来说, 迭代法更适用.

3.1 迭代法概述

迭代法的基本思想是构造一串收敛到解的序列, 即建立一种从已有近似解计算新的近似解的规则, 由不同的计算规则得到不同的迭代法. 本章着重介绍单步定常线性迭代法.

3.1.1 迭代法的一般形式

对线性方程组

$$\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b} \tag{3.1}$$

其中 $\boldsymbol{A}=(a_{ij})$ 为 $n\times n$ 非奇异矩阵, $\boldsymbol{b}=(b_1,b_2,\cdots,b_n)^{\mathrm{T}}$. 构造其形如

$$\boldsymbol{x}=\boldsymbol{M}\boldsymbol{x}+\boldsymbol{g} \tag{3.2}$$

的同解方程组, 其中 $\boldsymbol{M}$ 为 n 阶方阵, $\boldsymbol{g}\in\mathbf{R}^n$. 任取初始向量 $\boldsymbol{x}^{(0)}\in\mathbf{R}^n$, 代入迭代公式

扫描全能王 创建

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{M}\boldsymbol{x}^{(k)}+\boldsymbol{g}\quad(k=0,1,2,\cdots)\tag{3.3}$$

产生向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$, 当 k 充分大时, 以 $\boldsymbol{x}^{(k)}$ 作为方程组(3.1)的近似解, 这就是求解线性方程组的单步定常线性迭代法. 式 (3.3)中的 $\boldsymbol{M}$ 称为迭代矩阵.

为研究迭代法的收敛性, 必须先介绍向量序列与矩阵序列收敛的概念.

3.1.2 向量序列与矩阵序列的收敛性

由于 $\mathbf{R}^n$ 中的向量可与 $\mathbf{R}^n$ 中的点建立一一对应关系, 由点列的收敛概念及向量范数的等价性, 可得到向量序列的收敛概念.

定义 3.1 设 $\left\{\boldsymbol{x}^{(k)}\right\}$ 为 $\mathbf{R}^n$ 中的向量序列, $\boldsymbol{x}\in\mathbf{R}^n$, 如果

$$\lim_{k\to\infty}\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}\right\|=0\tag{3.4}$$

其中 $\|\cdot\|$ 为向量范数, 则称序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于 $\boldsymbol{x}$, 记为

$$\lim_{k\to\infty}\boldsymbol{x}^{(k)}=\boldsymbol{x}\tag{3.5}$$

n 维点列收敛的一种等价描述是其对应坐标序列均收敛, 向量序列也有类似的结果.

定理 3.1 $\mathbf{R}^n$ 中的向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于 $\mathbf{R}^n$ 中的向量 $\boldsymbol{x}$ 当且仅当

$$\lim_{k\to\infty}x_i^{(k)}=x_i\quad(i=1,2,\cdots,n)$$

其中 $\boldsymbol{x}^{(k)}=\left(x_1^{(k)},x_2^{(k)},\cdots,x_n^{(k)}\right)^{\mathrm{T}},\boldsymbol{x}=(x_1,x_2,\cdots,x_n)^{\mathrm{T}}$.

证明 由定义 3.1, $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于 $\boldsymbol{x}$, 即

$$\lim_{k\to\infty}\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}\right\|_\infty=0$$

而对任意 $1\leqslant i\leqslant n$, 有

$$0\leqslant\left|x_i^{(k)}-x_i\right|\leqslant\max_{1\leqslant j\leqslant n}\left|x_j^{(k)}-x_j\right|=\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}\right\|_\infty$$

由极限存在准则得

$$\lim_{k\to\infty}\left|x_i^{(k)}-x_i\right|=0\quad(i=1,2,\cdots,n)$$

即

$$\lim_{k\to\infty}x_i^{(k)}=x_i\quad(i=1,2,\cdots,n)$$

完全类似地, 可以定义矩阵序列的收敛.

定义 3.2 设 $\left\{\boldsymbol{A}^{(k)}\right\}$ 为 n 阶方阵序列, $\boldsymbol{A}$ 为 n 阶方阵. 如果

$$\lim_{k\to\infty}\left\|\boldsymbol{A}^{(k)}-\boldsymbol{A}\right\|=0\tag{3.6}$$

其中 $\|\cdot\|$ 为矩阵范数, 则称序列 $\left\{\boldsymbol{A}^{(k)}\right\}$ 收敛于矩阵 $\boldsymbol{A}$, 记为

$$\lim_{k\to\infty}\boldsymbol{A}^{(k)}=\boldsymbol{A}\tag{3.7}$$

扫描全能王 创建

定理 3.2　设 $\boldsymbol{A}^{(k)}=\left(a_{ij}^{(k)}\right)(k=1,2,\cdots),\boldsymbol{A}=(a_{ij})$ 均为 n 阶方阵，则矩阵序列 $\left\{\boldsymbol{A}^{(k)}\right\}$ 收敛于矩阵 $\boldsymbol{A}$ 的充要条件为

$$\lim_{k\to\infty}a_{ij}^{(k)}=a_{ij}\quad(i,j=1,2,\cdots,n)$$

证明留给读者.

定理 3.1和定理 3.2表明，向量序列和矩阵序列的收敛可以归结为对应分量或对应元素序列的收敛.

由以上讨论容易看出，若按式(3.3)产生的向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于向量 $\boldsymbol{x}$，则有

$$\boldsymbol{x}=\lim_{k\to\infty}\boldsymbol{x}^{(k+1)}=\lim_{k\to\infty}\left[\boldsymbol{M}\boldsymbol{x}^{(k)}+\boldsymbol{g}\right]=\boldsymbol{M}\boldsymbol{x}+\boldsymbol{g}$$

即 $\boldsymbol{x}$ 是方程组(3.1)的解.

3.2　几种基本的迭代法

3.2.1　Jacobi 迭代法

设方程组

$$\begin{cases}a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1\\a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2\\\cdots\cdots\cdots\cdots\\a_{n1}x_1+a_{n2}x_2+\cdots+a_{nn}x_n=b_n\end{cases}\tag{3.8}$$

的系数矩阵 $\boldsymbol{A}$ 非奇异，且 $a_{ii}\neq0(i=1,2,\cdots,n)$. 将方程组(3.8)变形为

$$\begin{cases}x_1=b_{12}x_2+b_{13}x_3+\cdots+b_{1n}x_n+g_1\\x_2=b_{21}x_1+b_{23}x_3+\cdots+b_{2n}x_n+g_2\\\cdots\cdots\cdots\cdots\\x_n=b_{n1}x_1+b_{n2}x_2+\cdots+b_{n\,n-1}x_{n-1}+g_n\end{cases}\tag{3.9}$$

其中 $b_{ij}=-\dfrac{a_{ij}}{a_{ii}}(i\neq j;i,j=1,2,\cdots,n),g_i=\dfrac{b_i}{a_{ii}}(i=1,2,\cdots,n)$. 记对角矩阵

$$\boldsymbol{D}=\operatorname{diag}(a_{11},a_{22},\cdots,a_{nn}),\quad\boldsymbol{D}^{-1}=\operatorname{diag}\left(\frac{1}{a_{11}},\frac{1}{a_{22}},\cdots,\frac{1}{a_{nn}}\right)$$

则

扫描全能王 创建

$$\boldsymbol{B}=\begin{bmatrix} 0 & b_{12} & \cdots & b_{1\,n-1} & b_{1n} \\ b_{21} & 0 & \cdots & b_{2\,n-1} & b_{2n} \\ \vdots & \vdots & & \vdots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{n\,n-1} & 0 \end{bmatrix}=\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{A},\quad \boldsymbol{g}=\begin{bmatrix} g_1 \\ g_2 \\ \vdots \\ g_n \end{bmatrix}=\boldsymbol{D}^{-1}\boldsymbol{b} \tag{3.10}$$

方程组(3.9)可简记为

$$\boldsymbol{x}=\boldsymbol{B}\boldsymbol{x}+\boldsymbol{g} \tag{3.11}$$

将初始向量 $\boldsymbol{x}^{(0)}$ 代入迭代公式

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{B}\boldsymbol{x}^{(k)}+\boldsymbol{g}\quad (k=0,1,\cdots,) \tag{3.12}$$

产生向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$, 公式(3.12)所给出的迭代法称为 Jacobi(雅可比) 迭代法, 又称简单迭代法. 称式(3.12)为 Jacobi 迭代法的迭代公式, $\boldsymbol{B}$ 为其迭代矩阵.

Jacobi 迭代法的计算过程如下:

算法 3.1 Jacobi 迭代法

---

1. 输入 $\boldsymbol{A}=(a_{ij}),\boldsymbol{b}=(b_1,b_2,\cdots,b_n)$, 维数 $n,\boldsymbol{x}^{(0)}=\left(x_1^{(0)},x_2^{(0)},\cdots,x_n^{(0)}\right)$, 误差限 ε, 最大容许迭代次数 N.
2. 置 $k=1$.
3. 对 $i=1,2,\cdots,n$,

$$x_i=\left(b_i-\sum_{\substack{j=1\\ j\neq i}}^{n}a_{ij}x_j^{(0)}\right)\Big/a_{ii}$$

4. 若 $\left\|\boldsymbol{x}-\boldsymbol{x}^{(0)}\right\|<\varepsilon$, 输出 $\boldsymbol{x}$, 停机; 否则转 5 .
5. 若 $k<N$, 置 $k+1\Rightarrow k,x_i\Rightarrow x_i^{(0)}(i=1,2,\cdots,n)$, 转 3 ; 否则, 输出失败信息, 停机.

例 3.1 用 Jacobi 迭代法求解线性方程组

$$\begin{cases} 10x_1- \quad x_2-2x_3=72 \\ -x_1+10x_2-2x_3=83 \\ -x_1- \quad x_2+5x_3=42 \end{cases}$$

解 由 Jacobi 迭代法的计算公式(3.12), 有

$$\begin{cases} x_1^{(k+1)}=0.1x_2^{(k)}+0.2x_3^{(k)}+7.2 \\ x_2^{(k+1)}=0.1x_1^{(k)}+0.2x_3^{(k)}+8.3 \\ x_3^{(k+1)}=0.2x_1^{(k)}+0.2x_2^{(k)}+8.4 \end{cases}$$

取 $\boldsymbol{x}^{(0)}=(0,0,0)^{\mathrm{T}}$, 代入上式得

扫描全能王 创建

$$x_1^{(1)} = 7.2, \quad x_2^{(1)} = 8.3, \quad x_3^{(1)} = 8.4$$

$$x_1^{(2)} = 0.1 \times 8.3 + 0.2 \times 8.4 + 7.2 = 9.71$$

$$x_2^{(2)} = 0.1 \times 7.2 + 0.2 \times 8.4 + 8.3 = 10.70$$

$$x_3^{(2)} = 0.2 \times 7.2 + 0.2 \times 8.3 + 8.4 = 11.50$$

如此继续下去, 迭代结果见表 3.1.

表 3.1　例 3.1Jacobi 迭代法数值结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.000 0	0.000 0	0.000 0
1	7.200 0	8.300 0	8.400 0
2	9.710 0	10.700 0	11.500 0
3	10.570 0	11.570 0	12.482 0
4	10.853 5	11.853 4	12.828 2
5	10.951 0	11.951 0	12.941 4
6	10.983 4	11.983 4	12.950 4
7	10.994 4	11.998 1	12.993 4
8	10.998 1	11.994 1	12.997 8
9	10.999 4	11.999 4	12.999 2

迭代 9 次, 得近似解 $\boldsymbol{x}^{(9)} = (10.999\,4, 11.999\,4, 12.999\,2)^{\mathrm{T}}$, 容易验证, 方程组的精确解为 $\boldsymbol{x} = (11, 12, 13)^{\mathrm{T}}$, 从表 3.1 可以看出, 当迭代次数增加时, 迭代结果越来越接近精确解.

3.2.2　Gauss–Seidel 迭代法

在 Jacobi 迭代法中, 计算 $x_i^{(k+1)}$ 时, 分量 $x_1^{(k+1)}, x_2^{(k+1)}, \cdots, x_{i-1}^{(k+1)}$ 已经算出, 可以考虑用新的分量 $x_l^{(k+1)}(l = 1, 2, \cdots, i-1)$ 代替旧的分量 $x_l^{(k)}(l = 1, 2, \cdots, i-1)$, 这就是 Gauss-Seidel(高斯–赛德尔) 迭代法. Gauss-Seidel 迭代法的计算公式为

$$\begin{cases} x_1^{(k+1)} = b_{12}x_2 + b_{13}x_3 + \cdots + b_{1n}x_n + g_1 \\ x_2^{(k+1)} = b_{21}x_1^{(k+1)} + b_{23}x_3 + \cdots + b_{2n}x_n + g_2 \\ \qquad\qquad\qquad\qquad \cdots\cdots\cdots\cdots \\ x_n^{(k+1)} = b_{n1}x_1^{(k+1)} + b_{n2}x_2^{(k+1)} + \cdots + b_{n\,n-1}x_{n-1}^{(k+1)} + g_n \end{cases} \tag{3.13}$$

而且近似向量 $\boldsymbol{x}^{(k)}$ 和 $\boldsymbol{x}^{(k+1)}$ 可以共用一组存储空间.

若将系数矩阵 $\boldsymbol{A}$ 表示为

$$\boldsymbol{A} = \boldsymbol{D} - \boldsymbol{L} - \boldsymbol{U}$$

其中

扫描全能王 创建

$$\boldsymbol{D}=\operatorname{diag}\left(a_{11},a_{22},\cdots,a_{nn}\right)$$

$$\boldsymbol{L}=\begin{bmatrix}0&&&&\\-a_{21}&0&&&\\-a_{31}&-a_{32}&0&&\\\vdots&\vdots&&\ddots&\\-a_{n1}&-a_{n2}&\cdots&-a_{n\,n-1}&0\end{bmatrix},\boldsymbol{U}=\begin{bmatrix}0&-a_{12}&-a_{13}&\cdots&-a_{1n}\\&0&-a_{23}&\cdots&-a_{2n}\\&&\ddots&&\vdots\\&&&0&-a_{n-1\,n}\\&&&&0\end{bmatrix}$$

并由式(3.10), 则迭代公式 (3.13)可以用矩阵表示为

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{D}^{-1}\boldsymbol{L}\boldsymbol{x}^{(k+1)}+\boldsymbol{D}^{-1}\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{D}^{-1}\boldsymbol{b}\tag{3.14}$$

移项得

$$\left(\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{L}\right)\boldsymbol{x}^{(k+1)}=\boldsymbol{D}^{-1}\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{D}^{-1}\boldsymbol{b}$$

因为 $\det(\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{L})=1$, 所以矩阵 $\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{L}$ 可逆, 且 $\left(\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{L}\right)^{-1}=\left[\boldsymbol{D}^{-1}(\boldsymbol{D}-\boldsymbol{L})\right]^{-1}=(\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{D}$, 故

$$\begin{aligned}\boldsymbol{x}^{(k+1)}&=\left(\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{L}\right)^{-1}\left[\boldsymbol{D}^{-1}\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{D}^{-1}\boldsymbol{b}\right]\\&=(\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{U}\boldsymbol{x}^{(k)}+(\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{b}\end{aligned}\tag{3.15}$$

式(3.15) 就是 Gauss-Seidel 迭代法的矩阵表示, 矩阵 $\boldsymbol{M}=(\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{U}$ 为 Gauss-Seidel 迭代法的迭代矩阵.

算法 3.2 Gauss-Seidel 迭代法

1. 输入 $\boldsymbol{A}=(a_{ij}),\boldsymbol{b}=(b_1,b_2,\cdots,b_n),\boldsymbol{x}^{(0)}=\left(x_1^{(0)},x_2^{(0)},\cdots,x_n^{(0)}\right)$, 维数 n, 误差限 ε, 最大容许迭代次数 N.
2. 置 $k=1$.
3. 计算

$$\begin{aligned}x_1&=\left(b_1-\sum_{j=2}^{n}a_{1j}x_j^{(0)}\right)\Big/a_{11}\\x_i&=\left(b_i-\sum_{j=1}^{i-1}a_{ij}x_j-\sum_{j=i+1}^{n}a_{ij}x_j^{(0)}\right)\Big/a_{ii}\quad(i=2,3,\cdots,n-1)\\x_n&=\left(b_n-\sum_{j=1}^{n-1}a_{nj}x_j\right)\Big/a_{nn}\end{aligned}$$

4. 若 $\left\|\boldsymbol{x}-\boldsymbol{x}^{(0)}\right\|<\varepsilon$, 则输出 $\boldsymbol{x}=(x_1,x_2,\cdots,x_n)$, 停机; 否则转 5.
5. 若 $k<N$, 置 $k+1\Rightarrow k,x_i\Rightarrow x_i^{(0)}(i=1,2,\cdots,n)$, 转 3 ; 否则输出失败信息, 停机.

扫描全能王 创建

用 Gauss-Seidel 迭代法解例 3.1. 仍取 $\boldsymbol{x}^{(0)}=(0,0,0)^{\mathrm{T}}$, 按式(3.14)计算得

$$
\begin{aligned}
x_1^{(1)} &= \frac{1}{10}\left(x_2^{(0)}+2x_3^{(0)}+b_1\right)=\frac{1}{10}\times 72=7.200\ 0\\
x_2^{(1)} &= \frac{1}{10}\left(x_1^{(1)}+2x_3^{(0)}+b_2\right)=\frac{1}{10}(7.200\ 0+83)=9.020\ 0\\
x_3^{(1)} &= \frac{1}{5}\left(x_1^{(1)}+x_2^{(1)}+b_3\right)=\frac{1}{5}(7.200\ 0+9.020\ 0+42)=11.644\ 0
\end{aligned}
$$

如此继续下去, 计算结果见表 3.2.

表 3.2　例 3.1Gauss-Seidel 迭代法数值结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0.000 0	0.000 0	0.000 0
1	7.200 0	9.020 0	11.644 0
2	10.430 8	11.671 9	12.820 5
3	10.931 3	11.957 2	12.977 8
4	10.991 3	11.994 7	12.997 2
5	10.998 9	11.999 3	12.999 6
6	10.999 9	11.999 9	13.000 0

计算结果表明, 用 Gauss-Seidel 迭代法求解例 3.1中的方程组比 Jacobi 迭代法效果好, 迭代 5 次所得到的结果与例 3.1中迭代 9 次所得的结果相仿. 事实上, 对有些问题 Gauss-Seidel 迭代法确实比 Jacobi 迭代法收敛得快, 但也有 Gauss-Seidel 迭代法比 Jacobi 迭代法收敛得慢, 甚至还有 Jacobi 迭代法收敛, Gauss-Seidel 迭代法发散的情形.

3.2.3　松弛 (Successive Over Relaxation) 迭代法

为了加速迭代向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 的收敛速度, 我们在 Gauss-Seidel 迭代法的基础上, 引入参数, 可以得到一种新的迭代法.

由式(3.14), Gauss-Seidel 迭代法可以改写为

$$
\begin{aligned}
\boldsymbol{x}^{(k+1)} &= \boldsymbol{D}^{-1}\left(\boldsymbol{L}\boldsymbol{x}^{(k+1)}+\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{b}\right)\\
&= \boldsymbol{x}^{(k)}+\boldsymbol{D}^{-1}\left(\boldsymbol{L}\boldsymbol{x}^{(k+1)}+\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{b}-\boldsymbol{D}\boldsymbol{x}^{(k)}\right)
\end{aligned}
$$

将上式等号右端的第二项看作修正项, 在其前面乘参数因子 ω, 即得到松弛迭代法 (简称松弛法, SOR 法)

$$
\begin{aligned}
\boldsymbol{x}^{(k+1)} &= \boldsymbol{x}^{(k)}+\omega\boldsymbol{D}^{-1}\left(\boldsymbol{L}\boldsymbol{x}^{(k+1)}+\boldsymbol{U}\boldsymbol{x}^{(k)}+\boldsymbol{b}-\boldsymbol{D}\boldsymbol{x}^{(k)}\right)\\
&= (1-\omega)\boldsymbol{x}^{(k)}+\omega\boldsymbol{D}^{-1}\boldsymbol{L}\boldsymbol{x}^{(k+1)}+\omega\boldsymbol{D}^{-1}\boldsymbol{U}\boldsymbol{x}^{(k)}+\omega\boldsymbol{D}^{-1}\boldsymbol{b}
\end{aligned}
\tag{3.16}
$$

因为 $\left|\boldsymbol{I}-\omega\boldsymbol{D}^{-1}\boldsymbol{L}\right|=1$, 所以矩阵 $\boldsymbol{I}-\omega\boldsymbol{D}^{-1}\boldsymbol{L}$ 可逆, 且

扫描全能王 创建

$$\left(I-\omega D^{-1}L\right)^{-1}=\left[D^{-1}(D-\omega L)\right]^{-1}=(D-\omega L)^{-1}D$$

代入式(3.16)可得

$$x^{(k+1)}=(D-\omega L)^{-1}[(1-\omega)D+\omega U]x^{(k)}+(D-\omega L)^{-1}\omega b \tag{3.17}$$

松弛法的迭代矩阵

$$M=(D-\omega L)^{-1}[(1-\omega)D+\omega U]$$

参数因子 ω 称为松弛因子. 当 $\omega<1$ 时, 方法称为低松弛法; 当 $\omega=1$ 时, 方法即为 Gauss-Seidel 迭代法; 当 $\omega>1$ 时, 方法称为超松弛法.

由式 (3.16) 知松弛法的计算公式为

$$x_i^{(k+1)}=(1-\omega)x_i^{(k)}+\frac{\omega}{a_{ii}}\left(b_i-\sum_{j=1}^{i-1}a_{ij}x_j^{(k+1)}-\sum_{j=i+1}^{n}a_{ij}x_j^{(k)}\right)\quad(i=1,2,\cdots,n) \tag{3.18}$$

具体算法如下:

算法 3.3 松弛迭代法

1. 输入 $A=(a_{ij}),b=(b_1,b_2,\cdots,b_n),x^{(0)}=\left(x_1^{(0)},x_2^{(0)},\cdots,x_n^{(0)}\right)$, 参数 ω, 误差限 ε, 最大容许迭代次数 N.
2. 置 $k=1$.
3. 计算:

$$x_1=(1-\omega)x_1^{(0)}+\omega\left(b_1-\sum_{j=2}^{n}a_{1j}x_j^{(0)}\right)\Big/a_{11}$$

$$x_i=(1-\omega)x_i^{(0)}+\omega\left(b_i-\sum_{j=1}^{i-1}a_{ij}x_j-\sum_{j=i+1}^{n}a_{ij}x_j^{(0)}\right)\Big/a_{ii}\quad(i=2,3,\cdots,n-1)$$

$$x_n=(1-\omega)x_n^{(0)}+\omega\left(b_n-\sum_{j=1}^{n-1}a_{nj}x_j\right)\Big/a_{nn}$$

4. 若 $\left\|x-x^{(0)}\right\|<\varepsilon$, 输出 $x=(x_1,x_2,\cdots,x_n)$, 停机, 否则转 5.
5. 若 $k<N$, 置 $k+1\Rightarrow k,x_i\Rightarrow x_i^{(0)}(i=1,2,\cdots,n)$, 转 3 ; 否则输出失败信息, 停机.

例 3.2 取 $x^{(0)}=(0,0,0)^{\mathrm{T}}$, 分别用 Gauss-Seidel 迭代法和松弛法 ($\omega=1.28$) 求解方程组

$$\begin{cases}2x_1-x_2=1\\-x_1+2x_2-x_3=0\\-x_2+2x_3=1.8\end{cases}$$

解 在北太天元中, 输入 A 和 b, 取 2 – 范数下的残差相对误差小于 10^{-4} 作为迭代停止标准. 令 $\omega=1$ 和 1.28, 调用相关函数 (参考 3.5节)

扫描全能王 创建

≫ sor(A, b, 1, 0.000 1);

≫ sor(A, b, 1.28, 0.000 1);

得计算结果如表 3.3:

表 3.3　例 3.2数值结果

k	Gauss-Seidel 迭代法 $x^{(k)}$			$\omega = 1.28$ 松弛法 $x^{(k)}$		
1	0.500 0	0.250 0	1.025 0	0.640 0	0.409 6	1.414 1
2	0.625 0	0.825 0	1.312 5	0.722 9	1.253 0	1.558 0
3	0.912 5	1.112 5	1.456 3	1.239 5	1.439 6	1.637 1
4	1.056 2	1.256 3	1.528 1	1.214 2	1.421 8	1.603 6
5	1.128 1	1.328 1	1.564 1	1.209 9	1.402 5	1.600 6
6	1.182 0	1.382 0	1.591 0	1.198 8	1.399 0	1.599 2
7	1.164 1	1.364 1	1.582 0	1.199 7	1.399 5	1.599 9
8	1.191 0	1.391 0	1.595 5	1.199 8	1.400 0	1.600 0
9	1.195 5	1.395 5	1.597 8			
10	1.197 8	1.397 8	1.598 9			
11	1.198 9	1.398 9	1.599 4			
12	1.199 4	1.399 4	1.599 7			
13	1.199 7	1.399 7	1.599 9			

由计算结果知松弛法迭代 8 次所得结果比 Gauss-Seidel 迭代法迭代 13 次的结果还要精确, 松弛因子 $\omega = 1.28$ 加速收敛效果明显.

松弛因子的选取对收敛速度影响极大, 但目前尚无可供实用的计算最佳松弛因子的方法. 实际计算时, 通常是根据系数矩阵的性质及实际计算经验, 通过试算来确定松弛因子的值. 特别当 $\boldsymbol{A}$ 为对称正定的三对角阵矩时, 有

$$\omega_{\text{opt}} = \frac{2}{1 + \sqrt{1 - \rho(B)^2}}$$

其中 $\rho(B)$ 为 Jacobi 迭代矩阵的谱半径.

为提高计算效率, 改善收敛性, 又产生了对称超松弛法, 简称 SSOR 法. 对称超松弛法实质上是将 $\boldsymbol{L}, \boldsymbol{U}$ 同等看待, 连续两次使用 SOR 迭代公式得到. 它的优点是对某些使用 SOR 法不收敛的问题, 可以构造出收敛的 SSOR 法; 松弛法对松弛因子十分敏感, SSOR 法则对松弛因子不敏感.

3.3　迭代法的收敛条件

3.3.1　矩阵的谱半径

迭代法的收敛性与迭代矩阵的特征值有关.

扫描全能王 创建

定义 3.3 设 $\boldsymbol{A}$ 为 n 阶方阵, $\lambda_i(i=1,2,\cdots,n)$ 为 $\boldsymbol{A}$ 的特征值, 称特征值绝对值的最大值为矩阵 $\boldsymbol{A}$ 的谱半径, 记为

$$\rho(\boldsymbol{A})=\max_{1\leqslant i\leqslant n}\{|\lambda_i|\} \tag{3.19}$$

$(\lambda_1,\lambda_2,\cdots,\lambda_n)$ 称为矩阵 $\boldsymbol{A}$ 的谱.

由特征值的定义容易得出, 矩阵

$$\boldsymbol{A}^k=\underbrace{\boldsymbol{A}\boldsymbol{A}\cdots\boldsymbol{A}}_{k\text{个}}$$

的谱是 $\left(\lambda_1^k,\lambda_2^k,\cdots,\lambda_n^k\right)(k=1,2,\cdots)$, 因而

$$\rho\left(\boldsymbol{A}^k\right)=[\rho(\boldsymbol{A})]^k \tag{3.20}$$

矩阵的谱半径与范数有以下关系.

定理 3.3 设 $\boldsymbol{A}$ 为任意 n 阶方阵, $\|\cdot\|$ 为任意由向量范数诱导出的矩阵范数, 则

$$\rho(\boldsymbol{A})\leqslant\|\boldsymbol{A}\|$$

证明 对 $\boldsymbol{A}$ 的任一特征值 λ_i 及相应的特征向量 $\boldsymbol{u}_i$, 都有

$$|\lambda_i|\,\|\boldsymbol{u}_i\|=\|\lambda_i\boldsymbol{u}_i\|=\|\boldsymbol{A}\boldsymbol{u}_i\|\leqslant\|\boldsymbol{A}\|\,\|\boldsymbol{u}_i\|$$

因为 $\boldsymbol{u}_i$ 为非零向量, 于是有

$$|\lambda_i|\leqslant\|\boldsymbol{A}\|$$

由 λ_i 的任意性即得

$$\rho(\boldsymbol{A})\leqslant\|\boldsymbol{A}\|$$

定理 3.4 设 $\boldsymbol{A}$ 为 n 阶方阵, 则对任意正数 ε, 存在一种矩阵范数 $\|\cdot\|$, 使得

$$\|\boldsymbol{A}\|\leqslant\rho(\boldsymbol{A})+\varepsilon$$

证明参看文献 [1].

对任意 n 阶方阵 $\boldsymbol{A}$, 一般不存在矩阵范数 $\|\cdot\|$, 使得 $\rho(\boldsymbol{A})=\|\boldsymbol{A}\|$. 但若 $\boldsymbol{A}$ 为对称矩阵, 则有 $\rho(\boldsymbol{A})=\|\boldsymbol{A}\|_2$.

下面的结论对建立迭代法的收敛条件十分重要.

定理 3.5 设 $\boldsymbol{A}$ 为 n 阶方阵, 则

$$\lim_{k\to\infty}\boldsymbol{A}^k=0$$

的充要条件为 $\rho(\boldsymbol{A})<1$.

证明 必要性. 若

$$\lim_{k\to\infty}\boldsymbol{A}^k=0$$

由定义 3.2, 有

$$\lim_{k\to\infty}\left\|\boldsymbol{A}^k\right\|=0$$

而

扫描全能王 创建

$$0 \leqslant \rho\left(\boldsymbol{A}^k\right) = [\rho(\boldsymbol{A})]^k \leqslant \left\|\boldsymbol{A}^k\right\|$$

于是由极限存在准则, 有

$$\lim_{k\to\infty} [\rho(\boldsymbol{A})]^k = 0$$

所以 $\rho(\boldsymbol{A}) < 1$.

充分性. 若 $\rho(\boldsymbol{A}) < 1$, 取 $\varepsilon = \dfrac{1-\rho(\boldsymbol{A})}{2} > 0$, 由定理 3.4, 存在一种矩阵范数 $\|\cdot\|$, 使得

$$\|\boldsymbol{A}\| \leqslant \rho(\boldsymbol{A}) + \varepsilon = \frac{1+\rho(\boldsymbol{A})}{2} < 1$$

而 $\left\|\boldsymbol{A}^k\right\| \leqslant \|\boldsymbol{A}\|^k$, 于是

$$\lim_{k\to\infty} \left\|\boldsymbol{A}^k\right\| = \lim_{k\to\infty} \|\boldsymbol{A}\|^k = 0$$

所以

$$\lim_{k\to\infty} \boldsymbol{A}^k = 0$$

3.3.2　迭代法的收敛条件

定理 3.6　*对任意初始向量 $\boldsymbol{x}^{(0)}$ 和右端项 $\boldsymbol{g}$, 由迭代格式*

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{M}\boldsymbol{x}^{(k)} + \boldsymbol{g} \quad (k = 0, 1, 2, \cdots) \tag{3.21}$$

产生的向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛的充要条件是

$$\rho(\boldsymbol{M}) < 1$$

证明　设存在 n 维向量 $\boldsymbol{x}^*$, 使得

$$\lim_{k\to\infty} \boldsymbol{x}^{(k)} = \boldsymbol{x}^*$$

则 $\boldsymbol{x}^*$ 满足

$$\boldsymbol{x}^* = \boldsymbol{M}\boldsymbol{x}^* + \boldsymbol{g}$$

由迭代公式 (3.21), 有

$$\begin{aligned}\boldsymbol{x}^{(k)} - \boldsymbol{x}^* &= \boldsymbol{M}\boldsymbol{x}^{(k-1)} + \boldsymbol{g} - \boldsymbol{M}\boldsymbol{x}^* - \boldsymbol{g} = \boldsymbol{M}\left(\boldsymbol{x}^{(k-1)} - \boldsymbol{x}^*\right) \\ &= \boldsymbol{M}^2\left(\boldsymbol{x}^{(k-2)} - \boldsymbol{x}^*\right) = \boldsymbol{M}^k\left(\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\right)\end{aligned}$$

于是有

$$\lim_{k\to\infty} \boldsymbol{M}^k\left(\boldsymbol{x}^{(0)} - \boldsymbol{x}^*\right) = \lim_{k\to\infty}\left(\boldsymbol{x}^{(k)} - \boldsymbol{x}^*\right) = 0$$

因为 $\boldsymbol{x}^{(0)}$ 为任意 n 维向量, 因此上式成立必须

$$\lim_{k\to\infty} \boldsymbol{M}^k = 0$$

由定理 3.5, 有

扫描全能王 创建

$$\rho(\boldsymbol{M})<1$$

反之, 若 $\rho(\boldsymbol{M})<1$, 则 $\lambda=1$ 不是 $\boldsymbol{M}$ 的特征值, 因而有 $|\boldsymbol{I}-\boldsymbol{M}|\neq 0$, 于是对任意 n 维向量 $\boldsymbol{g}$, 方程组 $(\boldsymbol{I}-\boldsymbol{M})\boldsymbol{x}=\boldsymbol{g}$ 有唯一解, 记为 $\boldsymbol{x}^*$, 即

$$\boldsymbol{x}^*=\boldsymbol{M}\boldsymbol{x}^*+\boldsymbol{g}$$

并且

$$\lim_{k\to\infty}\boldsymbol{M}^k=0$$

又因为

$$\boldsymbol{x}^{(k)}-\boldsymbol{x}^*=\boldsymbol{M}\left(\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^*\right)=\boldsymbol{M}^k\left(\boldsymbol{x}^{(0)}-\boldsymbol{x}^*\right)$$

所以, 对任意初始向量 $\boldsymbol{x}^{(0)}$, 都有

$$\lim_{k\to\infty}\left(\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\right)=\lim_{k\to\infty}\boldsymbol{M}^k\left(\boldsymbol{x}^{(0)}-\boldsymbol{x}^*\right)=0$$

即由迭代公式(3.21)产生的向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛.

定理 3.6 表明, 迭代法收敛与否只取决于迭代矩阵的谱半径, 与初始向量及方程组的右端项无关. 对同一方程组, 由于不同的迭代法迭代矩阵不同, 因此可能出现有的方法收敛, 有的方法发散的情形.

例 3.3 讨论用 Jacobi 迭代法与 Gauss-Seidel 迭代法求解方程组

$$\begin{cases} x_1+2x_2-2x_3=1 \\ x_1+x_2+x_3=2 \\ 2x_1+2x_2+x_3=3 \end{cases}$$

的收敛性.

解 由定理 3.6, 迭代法是否收敛等价于迭代矩阵的谱半径是否小于 1 , 故应先求迭代矩阵. 因为

$$\boldsymbol{A}=\begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix}\quad \boldsymbol{D}=\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\boldsymbol{L}=\begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & -2 & 0 \end{bmatrix}\quad \boldsymbol{U}=\begin{bmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix}$$

Jacobi 迭代法的迭代矩阵为

$$\boldsymbol{B}=\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{A}=\begin{bmatrix} 0 & -2 & 2 \\ -1 & 0 & -1 \\ -2 & -2 & 0 \end{bmatrix}$$

其特征方程为

扫描全能王 创建

$$|\lambda \boldsymbol{I}-\boldsymbol{B}|=\begin{vmatrix}\lambda & 2 & -2\\ 1 & \lambda & 1\\ 2 & 2 & \lambda\end{vmatrix}=\lambda^3=0$$

因此有 $\lambda_1=\lambda_2=\lambda_3=0$, 于是 $\rho(\boldsymbol{B})=0<1$, 所以 Jacobi 迭代法收敛.

如果用 Gauss-Seidel 迭代法, 由

$$\boldsymbol{D}-\boldsymbol{L}=\begin{bmatrix}1 & 0 & 0\\ 1 & 1 & 0\\ 2 & 2 & 1\end{bmatrix}$$

容易求出

$$(\boldsymbol{D}-\boldsymbol{L})^{-1}=\begin{bmatrix}1 & 0 & 0\\ -1 & 1 & 0\\ 0 & -2 & 1\end{bmatrix}$$

于是迭代矩阵为

$$\boldsymbol{M}=(\boldsymbol{D}-\boldsymbol{L})^{-1}\boldsymbol{U}=\begin{bmatrix}1 & 0 & 0\\ -1 & 1 & 0\\ 0 & -2 & 1\end{bmatrix}\begin{bmatrix}0 & -2 & 2\\ 0 & 0 & -1\\ 0 & 0 & 0\end{bmatrix}=\begin{bmatrix}0 & -2 & 2\\ 0 & 2 & -3\\ 0 & 0 & 2\end{bmatrix}$$

其特征方程为

$$\det(\lambda \boldsymbol{I}-\boldsymbol{M})=\begin{vmatrix}\lambda & 2 & -2\\ 0 & \lambda-2 & 3\\ 0 & 0 & \lambda-2\end{vmatrix}=\lambda(\lambda-2)^2=0$$

特征值为 $\lambda_1=0,\lambda_2=\lambda_3=2$, 故 $\rho(\boldsymbol{M})=2>1$, 所以, Gauss-Seidel 迭代法发散.

由定理 3.4及定理 3.6, 得

推论 3.1　*设迭代格式 $\boldsymbol{x}^{(k+1)}=\boldsymbol{M}\boldsymbol{x}^{(k)}+\boldsymbol{g}$, 若 $\|\boldsymbol{M}\|<1$, 则向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 收敛.*

推论 3.2　*松弛法收敛的必要条件是 $0<\omega<2$.*

证明　设松弛法的迭代矩阵 $\boldsymbol{M}$ 有特征值 $\lambda_1,\lambda_2,\cdots,\lambda_n$. 因为

$$|\det \boldsymbol{M}|=|\lambda_1\lambda_2\cdots\lambda_n|\leqslant[\rho(\boldsymbol{M})]^n$$

由定理 3.6, 松弛法收敛必有

$$|\det(\boldsymbol{M})|<1$$

又因为

$$|\det \boldsymbol{M}|=\left|(\boldsymbol{D}-\omega \boldsymbol{L})^{-1}\right||(1-\omega)\boldsymbol{D}+\omega \boldsymbol{U}|$$

$$\left|(\boldsymbol{D}-\omega \boldsymbol{L})^{-1}\right|=\frac{1}{a_{11}a_{22}\cdots a_{nn}}$$

$$|(1-\omega)\boldsymbol{D}+\omega \boldsymbol{U}|=(1-\omega)^n a_{11}a_{22}\cdots a_{nn}$$

于是有

扫描全能王 创建

$$|\det M| = |(1-\omega)^n| < 1$$

所以 $0 < \omega < 2$.

例 3.3 也说明了 $0 < \omega < 2$ 确实只是松弛法收敛的必要条件, 而非充分条件, 因为 Gauss-Seidel 迭代法即为 $\omega = 1$ 的情形.

定理 3.6虽然给出了判别迭代法收敛的充要条件, 但实际使用时很不方便. 因为求逆矩阵和特征值的难度并不亚于用直接方法求解线性方程组. 推论 3.1与推论 3.2使用起来方便得多, 但它们仅分别给出收敛的充分条件与必要条件, 许多情形下, 不能起作用. 如例 3.3中, 两个方法均有 $\|B\|_\infty > 1, \|M\|_\infty > 1$, 由推论 3.1无法判别收敛性. 下面对一些特殊的系数矩阵给出几个常用的判敛条件.

定义 3.4 *若 n 阶方阵 $\boldsymbol{A} = (a_{ij})$ 满足*

$$|a_{ii}| \geqslant \sum_{\substack{j=1 \\ j \neq i}}^{n} |a_{ij}| \quad (i = 1, 2, \cdots, n) \tag{3.22}$$

且至少有一个 i 值, 使上式中不等号严格成立, 则称 $\boldsymbol{A}$ 为弱对角占优矩阵. 若对所有 i, 式(3.22)中不等号均严格成立, 则称 $\boldsymbol{A}$ 为严格对角占优矩阵.

定义 3.5 *如果矩阵 $\boldsymbol{A}$ 不能通过行的互换和相应列的互换成为形式*

$$\boldsymbol{A} = \begin{bmatrix} \boldsymbol{A}_{11} & \boldsymbol{A}_{12} \\ \boldsymbol{O} & \boldsymbol{A}_{22} \end{bmatrix}$$

其中 $\boldsymbol{A}_{11}, \boldsymbol{A}_{22}$ 为方阵, 则称 $\boldsymbol{A}$ 是不可约的.

如例 3.2、例 3.3的系数矩阵是不可约的. 矩阵

$$\boldsymbol{A} = \begin{bmatrix} 2 & 1 & 0 \\ -1 & 3 & 0 \\ 4 & 1 & 2 \end{bmatrix}$$

是可约的. 一般地, 设 $\boldsymbol{A} = (a_{ij})$ 为 n 阶方阵 $(n \geqslant 2)$, 若存在非空集 $I \subset \{1, 2, \cdots, n\}$, 使得当 $i \in I$ 而 $j \notin I$ 时有 $a_{ij} = 0$, 则 $\boldsymbol{A}$ 是可约矩阵.

几个常用的充分收敛条件

设有线性方程组 $\boldsymbol{Ax} = \boldsymbol{b}$, 下列结论成立:

1. 若 $\boldsymbol{A}$ 为严格对角占优矩阵或不可约弱对角占优矩阵, 则 Jacobi 迭代法和 Gauss-Seidel 迭代法均收敛.

2. 若 $\boldsymbol{A}$ 为严格对角占优矩阵, $0 < \omega \leqslant 1$, 则松弛法收敛.

3. 若 $\boldsymbol{A}$ 为对称正定矩阵, $0 < \omega < 2$, 则松弛法收敛. 即若 $\boldsymbol{A}$ 是对称正定矩阵, 则松弛法收敛的充要条件为 $0 < \omega < 2$.

如例 3.1中, 系数矩阵

$$\boldsymbol{A} = \begin{bmatrix} 10 & -1 & -2 \\ -1 & 10 & -2 \\ -1 & -1 & 5 \end{bmatrix}$$

扫描全能王 创建

为严格对角占优矩阵, 故 Jacobi 迭代法与 Gauss-Seidel 迭代法均收敛. 例 3.2中, 系数矩阵

$$\boldsymbol{A}=\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

非严格对角占优, 但 $\boldsymbol{A}$ 为对称正定矩阵, $\omega=1.4$, 故松弛法收敛.

上述结论的证明可参看文献 [1],[3].

例 3.4　设有方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$, 其中

$$\boldsymbol{A}=\begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & 1 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & 1 \end{bmatrix}$$

讨论用三种迭代法求解的收敛性.

解　因为 $\boldsymbol{A}$ 为对称矩阵, 且其各阶主子式皆大于零, 故 $\boldsymbol{A}$ 为对称正定矩阵. 由判别条件 3, Gauss-Seidel 迭代法与松弛法 $(0<\omega<2)$ 均收敛. $\boldsymbol{A}$ 不是弱对角占优矩阵, 故不能用条件 1 判别 Jacobi 迭代法的收敛性. 容易算出, Jacobi 迭代法的迭代矩阵为

$$\boldsymbol{B}=\boldsymbol{I}-\boldsymbol{D}^{-1}\boldsymbol{A}=\begin{bmatrix} 0 & -\frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & 0 & -\frac{1}{2} \\ -\frac{1}{2} & -\frac{1}{2} & 0 \end{bmatrix}$$

其特征方程

$$\begin{aligned} |\lambda\boldsymbol{I}-\boldsymbol{B}| &= \begin{vmatrix} \lambda & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \lambda & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & \lambda \end{vmatrix} = \lambda^3+\frac{1}{4}-\frac{3}{4}\lambda \\ &= \left(\lambda-\frac{1}{2}\right)^2(\lambda+1)=0 \end{aligned}$$

有根 $\lambda_1=\lambda_2=\dfrac{1}{2}, \lambda_3=-1$, 因而 $\rho(\boldsymbol{B})=1$. 由定理 3.6知, Jacobi 迭代法不收敛.

值得指出的是, 改变方程组中方程的次序, 即将系数矩阵作行交换, 会改变迭代法的收敛性. 例如, 若方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的系数矩阵为

$$\boldsymbol{A}=\begin{bmatrix} 3 & -10 \\ 9 & -4 \end{bmatrix}$$

扫描全能王 创建

Jacobi 迭代法与 Gauss-Seidel 迭代法的迭代矩阵分别为

$$B=\begin{bmatrix} 0 & -\frac{10}{3} \\ -\frac{9}{4} & 0 \end{bmatrix} \quad M=\begin{bmatrix} 0 & \frac{10}{3} \\ 0 & \frac{15}{2} \end{bmatrix}$$

它们的谱半径为 $\rho(B)=\frac{\sqrt{30}}{2}, \rho(M)=\frac{15}{2}$. 由定理 3.6, 这两种迭代法均不收敛. 但若交换两个方程的次序, 得原方程组的同解方程组 $\boldsymbol{A}'\boldsymbol{x}=\boldsymbol{b}'$, 其中

$$\boldsymbol{A}'=\begin{bmatrix} 9 & -4 \\ 3 & -10 \end{bmatrix}$$

显然, $\boldsymbol{A}'$ 是严格对角占优矩阵, 因而对方程组 $\boldsymbol{A}'\boldsymbol{x}=\boldsymbol{b}'$ 用 Jacobi 迭代法与 Gauss-Seidel 迭代法求解均收敛.

3.3.3 误差估计

定理 3.7 设有迭代格式 $\boldsymbol{x}^{(k+1)}=\boldsymbol{M}\boldsymbol{x}^{(k)}+\boldsymbol{g}$, 若 $\|\boldsymbol{M}\|<1, \left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于 $\boldsymbol{x}^*$, 则有误差估计式

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\right\| \leqslant \frac{\|\boldsymbol{M}\|^k}{1-\|\boldsymbol{M}\|}\left\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}\right\| \tag{3.23}$$

证明 因为 $\rho(\boldsymbol{M}) \leqslant \|\boldsymbol{M}\|<1$, 故 $|\boldsymbol{I}-\boldsymbol{M}| \neq 0$, 于是 $(\boldsymbol{I}-\boldsymbol{M})^{-1}$ 存在, 方程组 $\boldsymbol{x}=\boldsymbol{M}\boldsymbol{x}+\boldsymbol{g}$ 有唯一解 $\boldsymbol{x}^*$, 且 $\boldsymbol{x}^*=(\boldsymbol{I}-\boldsymbol{M})^{-1}\boldsymbol{g}$. 从而有

$$\boldsymbol{x}^{(k)}-\boldsymbol{x}^*=\boldsymbol{M}\left(\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^*\right)=\boldsymbol{M}^k\left(\boldsymbol{x}^{(0)}-\boldsymbol{x}^*\right)$$

取范数得

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\right\| \leqslant \|\boldsymbol{M}\|^k\left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(*)}\right\| \tag{3.24}$$

因此,$\|\boldsymbol{M}\|<1, \left\{\boldsymbol{x}^{(k)}\right\}$ 收敛于 $\boldsymbol{x}^*$. 又因为

$$\begin{aligned}\left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(*)}\right\| &\leqslant \left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(1)}\right\|+\left\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(*)}\right\| \\ &\leqslant \left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(1)}\right\|+\|\boldsymbol{M}\|\left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(*)}\right\|\end{aligned}$$

移项得

$$\left\|\boldsymbol{x}^{(0)}-\boldsymbol{x}^{(*)}\right\| \leqslant \frac{1}{1-\|\boldsymbol{M}\|}\left\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}\right\| \tag{3.25}$$

将此式代入式 (3.24), 即得

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^*\right\| \leqslant \frac{\|\boldsymbol{M}\|^k}{1-\|\boldsymbol{M}\|}\left\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}\right\|$$

有了误差估计式(3.23), 根据事先给定的精度 ε, 可以估算出迭代的次数 k :

扫描全能王 创建

$$k \geqslant \frac{\ln \dfrac{\varepsilon(1-\|\boldsymbol{M}\|)}{\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}\|}}{\ln \|\boldsymbol{M}\|} \tag{3.26}$$

如例 3.1中, 若要求各分量的误差绝对值不超过 10^{-4}, 则由

$$\boldsymbol{M}=\begin{bmatrix} 0 & 0.1 & 0.2 \\ 0.1 & 0 & 0.2 \\ 0.2 & 0.2 & 0 \end{bmatrix}, \quad \boldsymbol{x}^{(0)}=\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad \boldsymbol{x}^{(1)}=\begin{bmatrix} 7.2 \\ 8.3 \\ 8.4 \end{bmatrix}$$

有 $\|\boldsymbol{M}\|_{\infty}=0.4, \left\|\boldsymbol{x}^{(1)}-\boldsymbol{x}^{(0)}\right\|_{\infty}=8.4$, 代入式 (3.26) 得

$$k \geqslant \frac{\ln \dfrac{10^{-4} \times (1-0.4)}{8.4}}{\ln 0.4}=12.932$$

所以需要迭代 13 次才能保证各分量误差绝对值不超过 10^{-4}.

实际计算时, 常常采取事后估计误差的方法, 即利用相邻两次迭代值之差是否达到精度作为停机准则. 因而下面的结论比定理 3.7 更实用.

定理 3.8　*在定理 3.7的条件下*, 有

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}\right\| \leqslant \frac{\|\boldsymbol{M}\|}{1-\|\boldsymbol{M}\|}\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{(k-1)}\right\| \tag{3.27}$$

证明　因为

$$\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}=\boldsymbol{M}\left(\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{*}\right)$$

取范数得

$$\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}\| \leqslant \|\boldsymbol{M}\|\|\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{*}\|$$

再由三角不等式及递推关系得

$$\begin{aligned} \|\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{*}\| &\leqslant \|\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{(k)}\|+\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}\| \\ &\leqslant \|\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{(k)}\|+\|\boldsymbol{M}\|\|\boldsymbol{x}^{(k-1)}-\boldsymbol{x}^{*}\| \end{aligned}$$

所以

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}\right\| \leqslant \frac{\|\boldsymbol{M}\|}{1-\|\boldsymbol{M}\|}\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{(k-1)}\right\|$$

由定理 3.8, 为使 $\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{*}\right\|<\varepsilon$, 只要

$$\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{(k-1)}\right\| \leqslant \frac{1-\|\boldsymbol{M}\|}{\|\boldsymbol{M}\|}\varepsilon$$

即可. 实际计算时, 当 $\|\boldsymbol{M}\|$ 不太接近 1 时, 可用 $\left\|\boldsymbol{x}^{(k)}-\boldsymbol{x}^{(k-1)}\right\|<\varepsilon$ 作为停机准则, $\boldsymbol{x}^{(k)}$ 即为满足精度之近似解.

扫描全能王 创建

3.4 最速下降法与共轭梯度法

在使用 SOR 迭代法求解线性方程组时, 需要确定松弛因子 ω, 而只有矩阵有较好性质时, 才有可能找到最佳松弛因子, 而计算最佳松弛因子时还需要 Jacobi 迭代矩阵的谱半径, 往往比较困难. 这一部分我们介绍一种不需要任何参数求解对称正定方程组的方法——共轭梯度法. 它是 20 世纪 50 年代初期由 Hestenes 和 Stiefel 首先提出的. 目前相关的方法和理论都非常成熟, 是求解大型稀疏线性方程组最受欢迎的方法之一. 我们先介绍和共轭梯度法密切相关的最速下降法.

3.4.1 最速下降法

当方程组(3.1)的系数矩阵 $\boldsymbol{A}$ 对称正定时, 任取 $\boldsymbol{x} \in \mathbf{R}^n$, 构造二次函数

$$\begin{aligned} \varphi(\boldsymbol{x}) &= \frac{1}{2}\langle \boldsymbol{A}\boldsymbol{x}, \boldsymbol{x}\rangle - \langle \boldsymbol{b}, \boldsymbol{x}\rangle \\ &= \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} a_{ij}x_i x_j - \sum_{j=1}^{n} b_j x_j \end{aligned} \tag{3.28}$$

利用 $\boldsymbol{A}$ 的对称性, 容易证明 φ 具有如下性质:

(1) φ 的梯度 $\nabla\varphi(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}$;

(2) 对于任意的 $\boldsymbol{x}, \boldsymbol{y} \in \mathbf{R}^n$, 实数 $\alpha \in \mathbf{R}$, 有

$$\begin{aligned} \varphi(\boldsymbol{x} + \alpha\boldsymbol{y}) &= \frac{1}{2}\langle \boldsymbol{A}(\boldsymbol{x} + \alpha\boldsymbol{y}), (\boldsymbol{x} + \alpha\boldsymbol{y})\rangle - \langle \boldsymbol{b}, (\boldsymbol{x} + \alpha\boldsymbol{y})\rangle \\ &= \varphi(\boldsymbol{x}) + \alpha\langle \boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}, \boldsymbol{y}\rangle + \frac{\alpha^2}{2}\langle \boldsymbol{A}\boldsymbol{y}, \boldsymbol{y}\rangle; \end{aligned}$$

(3) 若 $\boldsymbol{x}^* = \boldsymbol{A}^{-1}\boldsymbol{b}$ 是方程组(3.1)的解, 则

$$\varphi(\boldsymbol{x}^*) = -\frac{1}{2}\langle \boldsymbol{b}, \boldsymbol{x}^*\rangle = -\frac{1}{2}\langle \boldsymbol{A}\boldsymbol{x}^*, \boldsymbol{x}^*\rangle$$

且对于任意的 $\boldsymbol{x} \in \mathbf{R}^n$, 有

$$\begin{aligned} \varphi(\boldsymbol{x}) - \varphi(\boldsymbol{x}^*) &= \frac{1}{2}\langle \boldsymbol{A}\boldsymbol{x}, \boldsymbol{x}\rangle - \langle \boldsymbol{b}, \boldsymbol{x}\rangle + \frac{1}{2}\langle \boldsymbol{b}, \boldsymbol{x}^*\rangle \\ &= \frac{1}{2}\langle \boldsymbol{A}(\boldsymbol{x} - \boldsymbol{x}^*), (\boldsymbol{x} - \boldsymbol{x}^*)\rangle \end{aligned}$$

定理 3.9 设 $\boldsymbol{A}$ 对称正定, 则 $\boldsymbol{x}^*$ 是方程组 $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ 的解的充要条件是 $\boldsymbol{x}^*$ 满足

$$\varphi(\boldsymbol{x}^*) = \min_{\boldsymbol{x} \in \mathbf{R}^n} \varphi(\boldsymbol{x})$$

扫描全能王 创建

证明　必要性. 设 $\boldsymbol{A}\boldsymbol{x}^{*}=\boldsymbol{b}$, 则由性质 (3) 及 $\boldsymbol{A}$ 的对称正定性有

$$\varphi(\boldsymbol{x})-\varphi\left(\boldsymbol{x}^{*}\right)=\frac{1}{2}\left\langle A\left(\boldsymbol{x}-\boldsymbol{x}^{*}\right),\left(\boldsymbol{x}-\boldsymbol{x}^{*}\right)\right\rangle \geqslant 0$$

即 $\varphi\left(\boldsymbol{x}^{*}\right)=\min\limits_{\boldsymbol{x}\in\mathbf{R}^{n}}\varphi(\boldsymbol{x})$.

充分性. 设 $\varphi\left(\boldsymbol{x}^{*}\right)=\min\limits_{\boldsymbol{x}\in\mathbf{R}^{n}}\varphi(\boldsymbol{x})$, 则对于任意的 $\boldsymbol{y}\in\mathbf{R}^{n}$ 有

$$\left.\frac{\mathrm{d}\varphi\left(\boldsymbol{x}^{*}+\alpha\boldsymbol{y}\right)}{\mathrm{d}\alpha}\right|_{\alpha=0}=0$$

由性质 (2) 得

$$\left\langle\boldsymbol{A}\boldsymbol{x}^{*}-\boldsymbol{b},\boldsymbol{y}\right\rangle+\alpha\left\langle\boldsymbol{A}\boldsymbol{y},\boldsymbol{y}\right\rangle\big|_{\alpha=0}=0$$

即对于任意的 $\boldsymbol{y}\in\mathbf{R}^{n}$, 有 $\left\langle\boldsymbol{A}\boldsymbol{x}^{*}-\boldsymbol{b},\boldsymbol{y}\right\rangle=0$, 故必有 $\boldsymbol{A}\boldsymbol{x}^{*}=\boldsymbol{b}$.

定理 3.9将求解方程组(3.1)的问题转化为求函数 $\varphi(\boldsymbol{x})$ 的极小点问题, 即求向量 $\boldsymbol{x}^{*}\in\mathbf{R}^{n}$, 使得

$$\varphi\left(\boldsymbol{x}^{*}\right)=\min_{\boldsymbol{x}\in\mathbf{R}^{n}}\varphi(\boldsymbol{x}) \tag{3.29}$$

由微积分知识, 一个多元函数沿其负梯度方向函数值下降最快, 因此可以每次迭代时, 从点 $\boldsymbol{x}^{(k)}$ 出发, 沿着函数在该点处的负梯度方向求一个近似点 $\boldsymbol{x}^{(k+1)}$, 使得 $\varphi\left(\boldsymbol{x}^{(k+1)}\right)$ 在此方向达到极小值, 这就是最速下降法.

最速下降法具体计算过程如下: 设从 $\boldsymbol{x}^{(0)}$ 出发, 已迭代计算至 $\boldsymbol{x}^{(k)},\varphi(\boldsymbol{x})$ 在 $\boldsymbol{x}^{(k)}$ 处的负梯度方向 $\boldsymbol{r}^{(k)}=-\nabla\varphi\left(\boldsymbol{x}^{(k)}\right)=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(k)}$ (称为搜索方向), 沿此方向 φ 的函数值可表示为 $\varphi\left(\boldsymbol{x}^{(k)}+\alpha\boldsymbol{r}^{(k)}\right)$, 为求极小点令

$$\frac{\mathrm{d}\varphi\left(\boldsymbol{x}^{(k)}+\alpha\boldsymbol{r}^{(k)}\right)}{\mathrm{d}\alpha}=\left\langle\boldsymbol{A}\boldsymbol{x}^{(k)}-\boldsymbol{b},\boldsymbol{r}^{(k)}\right\rangle+\alpha\left\langle\boldsymbol{A}\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)}\right\rangle=0$$

得

$$\alpha_{k}=\frac{\left\langle\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)}\right\rangle}{\left\langle\boldsymbol{A}\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)}\right\rangle}$$

于是 $\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}+\alpha_{k}\boldsymbol{r}^{(k)}(k=1,2,\cdots)$, 且序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 满足 $\varphi\left(\boldsymbol{x}^{(k+1)}\right)<\varphi\left(\boldsymbol{x}^{(k)}\right)$ $(k=1,2,\cdots)$.

可以证明, 从任意初始点 $\boldsymbol{x}^{(0)}$ 出发, 最速下降法所得序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 均收敛于问题(3.29)的解, 即收敛于方程组(3.1)的解. 其收敛速度取决于比值 $\dfrac{\lambda_{1}-\lambda_{n}}{\lambda_{1}+\lambda_{n}}$, 其中 λ_{1},λ_{n} 分别是 $\boldsymbol{A}$ 的最大、最小特征值. 当 $\lambda_{1}\gg\lambda_{n}$ 时, 最速下降法收敛缓慢. 因此实际中很少使用. 然而它揭示了一种重要的思想, 开辟了一条全新的求解线性方程组的途径. 把上述方法稍加改进, 就可以得到著名的共轭梯度法.

扫描全能王 创建

3.4.2 共轭梯度法

在最速下降法中, 每一步迭代的搜索方向为负梯度方向 $\boldsymbol{r}^{(k)}$, 它只能保证 $\varphi(\boldsymbol{x})$ 局部下降最快, 从整体来看并非最优. 这就促使人们去寻找更好的下降方法. 当然, 我们希望每步确定新的下降方向所付出的代价不要太大, 共轭梯度法 (conjugate gradient method) 就是基于这一思想设计的. 共轭梯度法仍然是一维搜索法, 但不再沿负梯度方向 $\boldsymbol{r}^{(1)}, \boldsymbol{r}^{(2)}, \cdots, \boldsymbol{r}^{(k)}$ 搜索, 而是沿一组关于 $\boldsymbol{A}$ 共轭的方向 $\boldsymbol{d}^{(1)}, \boldsymbol{d}^{(2)}, \cdots, \boldsymbol{d}^{(k)}$ 搜索.

定义 3.6 设 $\boldsymbol{A}$ 对称正定, 若 $\mathbf{R}^n$ 中的向量组 $\left\{\boldsymbol{d}^{(0)}, \boldsymbol{d}^{(1)}, \cdots, \boldsymbol{d}^{(k)}\right\}$ 满足

$$\left\langle \boldsymbol{d}^{(i)}, \boldsymbol{A}\boldsymbol{d}^{(j)} \right\rangle = 0, \quad i \neq j$$

则称它是 $\mathbf{R}^n$ 中的一个 $\boldsymbol{A}-$ **共轭向量组**, 或称 $\boldsymbol{A}-$ **正交向量组**.

共轭梯度法算法如下: 设从 $\boldsymbol{x}^{(0)}$ 出发, 第一步仍选负梯度方向为下降方向. 假设已迭代计算至 $\boldsymbol{x}^{(k)}$, 下降方向不再取负梯度方向, 而是在负梯度方向和上一步下降方向 $\boldsymbol{d}^{(k-1)}$ 张成的二维平面上选取搜索方向

$$\boldsymbol{d}^{(k)} = -\nabla\varphi\left(\boldsymbol{x}^{(k)}\right) + \beta_{k-1}\boldsymbol{d}^{(k-1)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)} + \beta_{k-1}\boldsymbol{d}^{(k-1)}$$

我们要求 $\boldsymbol{d}^{(k)}$ 与 $\boldsymbol{d}^{(k-1)}$ 关于 $\boldsymbol{A}-$ 共轭, 即 $\left\langle \boldsymbol{d}^{(k)}, \boldsymbol{A}\boldsymbol{d}^{(k-1)} \right\rangle = 0$ 得

$$\beta_{k-1} = -\frac{\left\langle \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}, \boldsymbol{A}\boldsymbol{d}^{(k-1)} \right\rangle}{\left\langle \boldsymbol{d}^{(k-1)}, \boldsymbol{A}\boldsymbol{d}^{(k-1)} \right\rangle} \tag{3.30}$$

从而搜索方向

$$\boldsymbol{d}^{(k)} = \left(\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}\right) - \frac{\left\langle \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}, \boldsymbol{A}\boldsymbol{d}^{(k-1)} \right\rangle}{\left\langle \boldsymbol{d}^{(k-1)}, \boldsymbol{A}\boldsymbol{d}^{(k-1)} \right\rangle}\boldsymbol{d}^{(k-1)} \tag{3.31}$$

再沿 $\boldsymbol{d}^{(k)}$ 方向求 $\varphi(\boldsymbol{x})$ 的极小点 $\boldsymbol{x}^{(k+1)}$, 即

$$\varphi\left(\boldsymbol{x}^{(k+1)}\right) = \min_{\lambda>0}\varphi\left(\boldsymbol{x}^{(k)} + \lambda\boldsymbol{d}^{(k)}\right)$$

经计算可得

$$\lambda_k = \frac{\left\langle \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}, \boldsymbol{d}^{(k)} \right\rangle}{\left\langle \boldsymbol{d}^{(k)}, \boldsymbol{A}\boldsymbol{d}^{(k)} \right\rangle} \tag{3.32}$$

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \lambda_k\boldsymbol{d}^{(k)} \tag{3.33}$$

由此得方程组 (3.1) 的近似解序列 $\left\{\boldsymbol{x}^{(k)}\right\}$.

扫描全能王 创建

算法 3.4　共轭梯度法

1. 输入 $\boldsymbol{A}=(a_{ij}),\boldsymbol{b}=(b_1,b_2,\cdots,b_n)^{\mathrm{T}}$, 初始向量 $\boldsymbol{x}^{(0)}$, 误差限 ε.

2. 计算 $\boldsymbol{d}^{(0)}=\boldsymbol{r}^{(0)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(0)},\lambda_0=\left\langle \boldsymbol{r}^{(0)},\boldsymbol{d}^{(0)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(0)},\boldsymbol{A}\boldsymbol{d}^{(0)}\right\rangle,\boldsymbol{x}^{(1)}=\boldsymbol{x}^{(0)}+\lambda_0\boldsymbol{d}^{(0)}$.

3. 对于 $k=1,2,\cdots$, 计算

$$\boldsymbol{r}^{(k)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(k)}$$

$$\beta_{k-1}=-\left\langle \boldsymbol{r}^{(k)},\boldsymbol{A}\boldsymbol{d}^{(k-1)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(k-1)},\boldsymbol{A}\boldsymbol{d}^{(k-1)}\right\rangle,\quad \boldsymbol{d}^{(k)}=\boldsymbol{r}^{(k)}+\beta_{k-1}\boldsymbol{d}^{(k-1)}$$

$$\lambda_k=\left\langle \boldsymbol{r}^{(k)},\boldsymbol{d}^{(k)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(k)},\boldsymbol{A}\boldsymbol{d}^{(k)}\right\rangle,\quad \boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)}+\lambda_k\boldsymbol{d}^{(k)}$$

4. 若 $\boldsymbol{r}^{(k)}=0$ 或 $\left\|\boldsymbol{r}^{(k)}\right\|<\varepsilon$, 则终止计算, 输出 $\boldsymbol{x}^{*}=\boldsymbol{x}^{(k)}$. 否则转 3 .

例 3.5　用共轭梯度法求解例 3.2中的方程组.

解　例 3.2中方程组的系数矩阵和右端项为

$$\boldsymbol{A}=\begin{bmatrix}2&-1&0\\-1&2&-1\\0&-1&2\end{bmatrix},\quad \boldsymbol{b}=\begin{bmatrix}1\\0\\1.8\end{bmatrix}$$

精度要求 10^{-8}. 取初始向量 $\boldsymbol{x}^{(0)}=(0,0,0)^{\mathrm{T}}$, 利用北太天元计算得

$$\boldsymbol{d}^{(0)}=\boldsymbol{r}^{(0)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(0)}=(1.000\ 0,0,1.800\ 0)^{\mathrm{T}}$$

$$\lambda_0=\left\langle \boldsymbol{r}^{(0)},\boldsymbol{d}^{(0)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(0)},\boldsymbol{A}\boldsymbol{d}^{(0)}\right\rangle=0.500\ 0$$

$$\boldsymbol{x}^{(1)}=\boldsymbol{x}^{(0)}+\lambda_0\boldsymbol{d}^{(0)}=(0.500\ 0,0,0.900\ 0)^{\mathrm{T}}$$

$$\boldsymbol{r}^{(1)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(1)}=(0,1.400\ 0,0)^{\mathrm{T}}$$

$$\beta_0=-\left\langle \boldsymbol{r}^{(1)},\boldsymbol{A}\boldsymbol{d}^{(0)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(0)},\boldsymbol{A}\boldsymbol{d}^{(0)}\right\rangle=0.462\ 3$$

$$\boldsymbol{d}^{(1)}=\boldsymbol{r}^{(1)}+\beta_0\boldsymbol{d}^{(0)}=(0.462\ 3,1.400\ 0,0.832\ 1)^{\mathrm{T}}$$

$$\lambda_1=\left\langle \boldsymbol{r}^{(1)},\boldsymbol{d}^{(1)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(1)},\boldsymbol{A}\boldsymbol{d}^{(1)}\right\rangle=0.929\ 8$$

$$\boldsymbol{x}^{(2)}=\boldsymbol{x}^{(1)}+\lambda_1\boldsymbol{d}^{(1)}=(0.929\ 8,1.301\ 8,1.673\ 7)^{\mathrm{T}}$$

$$\boldsymbol{r}^{(2)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(2)}=(0.442\ 1,-0.000\ 0,0.245\ 6)^{\mathrm{T}}$$

$$\beta_1=-\left\langle \boldsymbol{r}^{(2)},\boldsymbol{A}\boldsymbol{d}^{(1)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(1)},\boldsymbol{A}\boldsymbol{d}^{(1)}\right\rangle=0.130\ 5$$

$$\boldsymbol{d}^{(2)}=\boldsymbol{r}^{(2)}+\beta_1\boldsymbol{d}^{(1)}=(0.502\ 4,0.182\ 7,-0.137\ 0)^{\mathrm{T}}$$

$$\lambda_2=\left\langle \boldsymbol{r}^{(2)},\boldsymbol{d}^{(2)}\right\rangle \Big/ \left\langle \boldsymbol{d}^{(2)},\boldsymbol{A}\boldsymbol{d}^{(2)}\right\rangle=0.537\ 7$$

扫描全能王 创建

$$\boldsymbol{x}^{(3)}=\boldsymbol{x}^{(2)}+\lambda_2\boldsymbol{d}^{(2)}=(1.200\ 0,1.400\ 0,1.600\ 0)^{\mathrm{T}}$$

$$\boldsymbol{r}^{(3)}=\boldsymbol{b}-\boldsymbol{A}\boldsymbol{x}^{(3)}=10^{-15}\times(0.000\ 0,-0.222\ 0,0.222\ 0)^{\mathrm{T}}$$

显然迭代到第 3 步即有 $\left\|\boldsymbol{r}^{(3)}\right\|_2<10^{-8}$, 满足迭代收敛条件, 终止迭代. 因此有

$$\boldsymbol{x}^*=\boldsymbol{x}^{(3)}=(1.200\ 0,1.400\ 0,1.600\ 0)^{\mathrm{T}}$$

如果利用 $\left\{\boldsymbol{d}^{(k)}\right\}$ 的 $\boldsymbol{A}-$ 共轭性, 式(3.30)、式(3.32) 可简化为

$$\beta_{k-1}=\frac{\left\langle\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)}\right\rangle}{\left\langle\boldsymbol{r}^{(k-1)},\boldsymbol{r}^{(k-1)}\right\rangle},\quad \lambda_k=\frac{\left\langle\boldsymbol{r}^{(k)},\boldsymbol{r}^{(k)}\right\rangle}{\left\langle\boldsymbol{d}^{(k)},\boldsymbol{A}\boldsymbol{d}^{(k)}\right\rangle} \tag{3.34}$$

例 3.5从 $\boldsymbol{x}^{(0)}$ 出发, 用共轭梯度法迭代了 3 步就得到了精确解. 理论上可以证明, 若计算过程是精确的, 则对于任何初值 $\boldsymbol{x}^{(0)}$, 用共轭梯度法求解问题(3.29)至多进行 n 步就能得到精确解. 因此共轭梯度法实质上是一种直接方法. 但在实际计算中, 由于舍入误差的出现, 破坏了它的有限终止性, 所以我们将它作为迭代法使用.

共轭梯度法的优点是存储量小, 计算简便. 它能充分利用矩阵 $\boldsymbol{A}$ 的稀疏性, 计算时只需存储 $\boldsymbol{A}$ 中的非零元素. 共轭梯度法具有超线性收敛性. 如果系数矩阵 $\boldsymbol{A}$ 的特征值比较均匀地分布在一个很长的区间内, 共轭梯度法可能收敛得比较缓慢.

对于非对称线性方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$, 可用共轭梯度法求解等价的对称正定方程组 $\boldsymbol{A}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{x}=\boldsymbol{A}^{\mathrm{T}}\boldsymbol{b}$, 收敛效果也是不错的.

如果方程组 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 的条件数比较大, 无论是 Jacobi 迭代法、松弛法, 还是共轭梯度法, 收敛都很缓慢, 为了改善收敛速度, 一般设法降低系数矩阵的条件数, 即采用预处理技术. 一种方法是在方程组两边同乘一个非奇异矩阵 $\boldsymbol{P}^{-1}$, 方程组等价地写为

$$\boldsymbol{P}^{-1}\boldsymbol{A}\boldsymbol{x}=\boldsymbol{P}^{-1}\boldsymbol{b}$$

使得条件数 $\operatorname{cond}\left(\boldsymbol{P}^{-1}\boldsymbol{A}\right)\ll\operatorname{cond}(\boldsymbol{A})$. 例如, 在共轭梯度法求解 $\boldsymbol{A}\boldsymbol{x}=\boldsymbol{b}$ 时, 若 $\boldsymbol{A}=\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}}$, 可将问题转化为求解同解方程组

$$\boldsymbol{L}^{-1}\boldsymbol{A}\boldsymbol{L}^{-\mathrm{T}}\boldsymbol{y}=\boldsymbol{L}^{-1}\boldsymbol{b},\quad \boldsymbol{x}=\boldsymbol{L}^{-\mathrm{T}}\boldsymbol{y}$$

用共轭梯度法求解方程组 $\boldsymbol{L}^{-1}\boldsymbol{A}\boldsymbol{L}^{-\mathrm{T}}\boldsymbol{y}=\boldsymbol{L}^{-1}\boldsymbol{b}$, 往往能改善收敛效果.

3.5 算法实现与应用实例

松弛法收敛性判别

利用北太天元中矩阵计算的方便性, 可编写如下脚本函数实现松弛迭代法收敛性的判别。

% 将函数存为 sorpb. m 文件

扫描全能王 创建

```
function [rho,k]=sorpb(A,b,omega)
    D= diag(diag(A));
% 取出 A 的对角元，构造对角矩阵 D
    L=D-tril(A);
%L 为 A 分裂 A=D-L-U 中的 L 矩阵
    U=D-triu(A);
%U 为 A 分裂 A=D-L-U 中的 U 矩阵
     M=(D-omega*L)/((1-omega)*D+omega*U);
% 迭代矩阵的计算
     rho=max(abs(eig(M)));
% 迭代矩阵谱半径的计算
     if(rho < 1) k=1
% k=1 为收敛.
     else k=0
% k=0 为发散.
     end
end
```

实际上，利用 Gauss-seidel 迭代法和 SOR 法计算时并不需要计算迭代矩阵，我们把 SOR 算法的脚本文件存在 sor.m 中

```
function y=sor(A,b,omega, tol)
n=length(b);
x=zeros(n,1);
x0=zeros(n,1);
err=10.0; step=1;
  while(err>tol & step<100)
    for i=1:n
    sum=0;
      for j=1:i-1
        sum=sum-A(i,j)*x(j);
      end    % end for j
      for j=i+1:n
        sum=sum-A(i,j)*x(j);
      end    % end for j
    sum=omega*(sum +b(i))/A(i,i);
    x(i)=(1-omega)*x(i)+sum;

    end   % end for i
    err=norm(x-x0);
    x0=x;
    step=step+1;
  end    % end for while 迭代步数不超过 100
  y=x;
end    % end for function
```


扫描全能王 创建

评注三

迭代法是求解大型线性方程组 (特别是系数矩阵为稀疏矩阵情形) 的有效方法, 具有存储空间小、程序简单等特点. Jacobi 法、Gauss-Seidel 法和 SOR 法是基本的迭代方法, 本章介绍的是最基本的内容. 迭代法的收敛性与系数矩阵的特性密切相关, 计算中应选用收敛速度快的迭代法. 当迭代法收敛速度慢时, 可通过整体校正、利用特征值外推和最小零偏差等方法加速收敛速度.

共轭梯度法最初是针对对称方程组提出的方法, 它具有超线性收敛性. 近年来相关的研究得到了很大的发展, 目前有关的理论和方法已经相当成熟. 大量的数值实验证实, 当系数矩阵 $\boldsymbol{A}$ 的条件数较小, 或 $\boldsymbol{A}$ 的特征值比较集中时, 用共轭梯度法迭代几步就能得到高精度的解.

共轭梯度法与预处理技术相结合, 往往会大大地改善收敛速度, 得到令人满意的结果. 对于非对称矩阵, 有计算效果不错的共轭梯度法方程残量法、共轭残量法、不完全分解预优共轭梯度法和分块 SSOR – CG 法等. 此外, 还有 Krylov 子空间上的共轭梯度型算法、基于 Arnoldi 迭代的广义最小残量法和利用 Lanczos 向量导出的双共轭梯度法等 [18]. 对于大型稀疏矩阵, 以 Galerkin 原理为基础建立的一类迭代法也是很有效的. 这些方法目前仍在发展中, 但大量的数值实验表明这些算法是很有效的. 上述方法并没有绝对的好坏之分, 方法的选择与具体问题有关. 对于偏微分方程离散后所得的大型代数方程组, 系数矩阵经常是稀疏的, 且很有规律, 常常可将系数矩阵分成几个数组存储或将系数矩阵分块, 将大型方程组降阶, 再用直接法或迭代法求解较低阶的方程组, 故上述各种方法都有相应特殊形式或分块形式.

本章知识要点 Jacobi迭代法;Gauss-Siedel迭代法;松弛迭代法(SOR法);迭代法收敛性的判别.

算法历史及背后的数学家

习题三

1. 分别用 Jacobi 迭代法和 Gauss-Seidel 迭代法求解下列方程组:

$$(1)\begin{cases} 10x_1-2x_2-x_3=3 \\ -2x_1+10x_2-x_3=15 \\ -x_1-2x_2+5x_3=10 \end{cases} \qquad (2)\begin{cases} 8x_1-3x_2+2x_3=20 \\ 4x_1+11x_2-x_3=33 \\ 2x_1+x_2+4x_3=12 \end{cases}$$

取初值 $\boldsymbol{x}^{(0)}=(0,0,0)^{\mathrm{T}}$, 计算结果精确到三位小数.

2. 分别用 Jacobi 迭代法、Gauss-Seidel 迭代法与 SOR 法 ($\omega=1.46$) 求解方程组

扫描全能王 创建

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

取初值 $x^{(0)} = (1,1,1,1)^{\mathrm{T}}$, 精确到 10^{-3}.

3. 用 Jacobi 迭代法和 Gauss-Seidel 迭代法求解方程组

$$\begin{cases} x_1+2x_2-2x_3=1 \\ x_1+x_2+x_3=1 \\ 2x_1+2x_2+x_3=1 \end{cases}$$

取初值 $x^{(0)} = (0,0,0)^{\mathrm{T}}$, 迭代 4 次, 并比较它们的计算结果.

4. 分别用 Gauss-Seidel 迭代法与 SOR 法 ($\omega = 1.25$) 求解方程组

$$\begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 24 \\ 30 \\ -24 \end{bmatrix}$$

取 $x^{(0)} = (1,1,1)^{\mathrm{T}}$, 迭代 7 次, 并比较它们的计算结果.

5. 若用 Jacobi 迭代法求解方程组

$$\begin{bmatrix} a & -1 & -3 \\ -1 & a & -2 \\ 3 & -2 & a \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

讨论实数 a 与收敛性的关系.

6. 求证用 Jacobi 迭代法求解方程组

$$\begin{cases} a_{11}x_1 + a_{12}x_2 = b_1 \\ a_{21}x_1 + a_{22}x_2 = b_2 \end{cases} \quad (a_{11}a_{22} \neq 0)$$

迭代收敛的充要条件是

$$\left| \frac{a_{12}a_{21}}{a_{11}a_{22}} \right| < 1$$

7. 设有方程组

$$\begin{cases} 10x_1 + 4x_2 + 4x_3 = 13 \\ 4x_1 + 10x_2 + 8x_3 = 11 \\ 4x_1 + 8x_2 + 10x_3 = 25 \end{cases}$$

扫描全能王 创建

（1）分别写出 Jacobi 迭代法、Gauss-Seidel 迭代法及 SOR 法 ($\omega = 1.35$) 的计算公式及迭代矩阵;

（2）对任意初值, 各迭代法是否收敛? 说明理由.

8. 设有方程组

$$\begin{bmatrix} 1.2 & -3.6 & -12 \\ -10 & 9 & 0.5 \\ 1 & -4 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

试写出收敛的迭代式, 并说明理由.

9. 设有方程组

$$\begin{bmatrix} 1 & a & a \\ 4a & 1 & 0 \\ a & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

（1）分别写出 Jacobi 迭代法和 Gauss-Seidel 迭代法的计算公式;

（2）用迭代收敛的充要条件给出使这两种迭代法都收敛的 a 的取值范围.

10. 分别用 Jacobi 迭代法和 Gauss-Seidel 迭代法求解方程组

$$\begin{cases} 20x_1 + 2x_2 + 3x_3 = 24 \\ x_1 + 8x_2 + x_3 = 12 \\ 2x_1 - 3x_2 + 15x_3 = 30 \end{cases}$$

取 $\boldsymbol{x}^{(0)} = (0,0,0)^{\mathrm{T}}$, 问迭代是否收敛? 若收敛, 需要迭代多少次, 才能保证各分量误差绝对值小于 10^{-6} ?

11. 证明严格对角占优矩阵是非奇异矩阵.

12. 证明对称矩阵

$$\boldsymbol{A} = \begin{bmatrix} 1 & a & a \\ a & 1 & a \\ a & a & 1 \end{bmatrix}$$

当 $-\dfrac{1}{2} < a < 1$ 时为正定矩阵, 且只有当 $-\dfrac{1}{2} < a < \dfrac{1}{2}$ 时, 用 Jacobi 迭代法求解方程组 $\boldsymbol{Ax} = \boldsymbol{b}$ 才收敛.

13. 分别用最速下降法和共轭梯度法求解方程组

$$\begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 24 \\ 30 \\ -24 \end{bmatrix}$$

扫描全能王 创建

数值实验三

1. 试分别用 Jacobi 迭代法、Gauss-Seidel 迭代法和共轭梯度法解线性方程组

$$\begin{bmatrix} 10 & 1 & 2 & 3 & 4 \\ 1 & 9 & -1 & 2 & -3 \\ 2 & -1 & 7 & 3 & -5 \\ 3 & 2 & 3 & 12 & -1 \\ 4 & -3 & -5 & -1 & 15 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} = \begin{bmatrix} 12 \\ -27 \\ 14 \\ -17 \\ 12 \end{bmatrix}$$

迭代初始向量取 $x^{(0)} = (0,0,0,0,0)^{\mathrm{T}}$.

2. 若仅保存系数矩阵 $A = (a_{ij})_{n\times n}$ 的非零元, 用共轭梯度法求解 $n = 10^5$ 阶的方程组 $Ax = b$, 其中

$$a_{ij} = \begin{cases} 3, & i = j \\ -1, & |i-j| = 1 \\ \dfrac{1}{2}, & i+j = n+1, i \neq \dfrac{n}{2}, \dfrac{n}{2}+1 \\ 0, & \text{其他} \end{cases} \quad (i,j = 1,2,\cdots,n),$$

$$b = (2.5, \underbrace{1.5,\cdots,1.5}_{(\frac{n}{2}-2)\text{个}}, 1.0, 1.0, \underbrace{1.5,\cdots,1.5}_{(\frac{n}{2}-2)\text{个}}, 2.5)^{\mathrm{T}}.$$

扫描全能王 创建

/第四章/

矩阵特征值与特征向量的计算

工程实践中有多种振动问题, 如桥梁或建筑物的振动、机械机件的振动、飞机机翼的振动等. 这些问题的求解常常归纳为求矩阵的特征值问题. 另外, 一些稳定性分析问题及相关分析问题也可以转化为求矩阵特征值与特征向量的问题.

本章仅对实矩阵进行讨论.

4.1 幂法和反幂法

4.1.1 幂法

幂法主要用于求矩阵的按模最大的特征值与相应的特征向量. 它是通过迭代产生向量序列, 由此计算特征值和特征向量.

设 $n \times n$ 实矩阵 $\boldsymbol{A}$ 的特征值 $\lambda_i\ (i=1,2,\cdots,n)$ 满足

$$|\lambda_1| > |\lambda_2| \geqslant \cdots \geqslant |\lambda_n|$$

且与 $\lambda_i (i=1,2,\cdots,n)$ 相应的特征向量 $\boldsymbol{u}_1, \boldsymbol{u}_2, \cdots, \boldsymbol{u}_n$ 线性无关.

给定初始向量 $\boldsymbol{x}^{(0)} \neq \boldsymbol{0}$, 由迭代公式

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{A}\boldsymbol{x}^{(k)} \quad (k=0,1,2,\cdots) \tag{4.1}$$

产生向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$, 可以证明, 当 k 充分大时, 有 $\lambda_1 \approx x_i^{(k+1)} / x_i^{(k)}$, 相应的特征向量为 $\boldsymbol{x}^{(k+1)}$.

为简便起见, 不妨设 $\|\boldsymbol{u}_i\| = 1 (i=1,2,\cdots,n)$. 因为 $\boldsymbol{u}_i$ $(i=1, 2, \cdots, n)$ 线性无关, 故必存在 n 个不全为零的数 $\alpha_i (i=1,2,\cdots,n)$, 使得 $\boldsymbol{x}^{(0)} = \sum\limits_{i=1}^{n} \alpha_i \boldsymbol{u}_i$. 由式(4.1) 得

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{A}\boldsymbol{x}^{(k)} = \boldsymbol{A}^{k+1}\boldsymbol{x}^{(0)} = \sum_{i=1}^{n} \boldsymbol{A}^{k+1}(\alpha_i \boldsymbol{u}_i) = \sum_{i=1}^{n} \alpha_i \lambda_i^{k+1} \boldsymbol{u}_i$$

即

$$\boldsymbol{x}^{(k+1)} = \lambda_1^{k+1}\left[\alpha_1 \boldsymbol{u}_1 + \left(\frac{\lambda_2}{\lambda_1}\right)^{k+1} \alpha_2 \boldsymbol{u}_2 + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k+1} \alpha_n \boldsymbol{u}_n\right] \tag{4.2}$$

设 $\alpha_1 \neq 0$, 由 $|\lambda_1| > |\lambda_i| (i=2,3,\cdots,n)$ 得

扫描全能王 创建

$$\lim_{k\to\infty}\left(\frac{\lambda_i}{\lambda_1}\right)^{k+1}\alpha_i\boldsymbol{u}_i=\boldsymbol{0}$$

于是

$$\lim_{k\to\infty}\sum_{i=2}^{n}\left(\frac{\lambda_i}{\lambda_1}\right)^{k+1}\alpha_i\boldsymbol{u}_i=\boldsymbol{0}$$

故只要 k 充分大, 就有

$$\boldsymbol{x}^{(k+1)}=\lambda_1^{k+1}\left[\alpha_1\boldsymbol{u}_1+\sum_{i=2}^{n}\left(\frac{\lambda_i}{\lambda_1}\right)^{k+1}\alpha_i\boldsymbol{u}_i\right]\approx\lambda_1^{k+1}\alpha_1\boldsymbol{u}_1$$

因此, 可把 $\boldsymbol{x}^{(k+1)}$ 作为与 λ_1 相应的特征向量的近似. 由

$$\boldsymbol{x}^{(k+1)}\approx\lambda_1^{k+1}\alpha_1\boldsymbol{u}_1,\quad \boldsymbol{x}^{(k)}\approx\lambda_1^{k}\alpha_1\boldsymbol{u}_1$$

又可得出

$$\lambda_1\approx\frac{x_i^{(k+1)}}{x_i^{(k)}}\quad(i=1,2,\cdots,n)\tag{4.3}$$

其中 $x_i^{(k)}$ 为 $\boldsymbol{x}^{(k)}$ 的第 i 个分量. 按式 (4.1)和式 (4.3) 计算矩阵 $\boldsymbol{A}$ 按模最大的特征值与相应的特征向量的方法称为幂法.

注 4.1　初始向量的选取影响幂法的计算结果, 容易验证如果初始向量刚好是某个非按模最大的特征值对应的特征向量, 则幂法得不到按模最大的特征值, 所以可以多选几个初始向量, 测试计算结果的可靠性.

注 4.2　如果 $\boldsymbol{x}^{(0)}$ 的选取恰恰使得 $\alpha_1=0$, 有时幂法计算仍能进行. 因为计算过程中舍入误差的影响, 迭代若干次后, 必然会产生一个向量 $\boldsymbol{x}^{(k)}$, 它在 $\boldsymbol{u}_1$ 方向上的分量不为零, 以后的计算就满足所设条件.

注4.3　因为 $\boldsymbol{x}^{(k)}\approx\lambda_1^k\alpha_1\boldsymbol{u}_1$, 计算过程中可能会出现溢出 ($|\lambda_1|>1$) 或成为 0 ($|\lambda_1|<1$) 的情形. 为避免这一情形的出现, 实际计算时每次迭代所求得的向量都要归一化.

因此, 幂法实际使用的计算公式是

$$\begin{cases}\boldsymbol{y}^{(k)}=\dfrac{\boldsymbol{x}^{(k)}}{x_r^{(k)}},\left|x_r^{(k)}\right|=\max\limits_{1\leqslant i\leqslant n}\left|x_i^{(k)}\right| & (k=0,1,2,\cdots)\\ \boldsymbol{x}^{(k+1)}=\boldsymbol{A}\boldsymbol{y}^{(k)}\\ \lambda_1\approx x_r^{(k+1)}\end{cases}\tag{4.4}$$

注 4.4　公式(4.4)中近似特征值的选取方法是直接取迭代后向量的第 r 个元素, 因为这个位置的元素迭代前被标准化为 1, 而其他位置需要计算比值.

实际上, 还可以采用以下计算公式, 设 $\boldsymbol{y}^0$ 是标准化后的初值向量,

$$\begin{cases}\boldsymbol{x}^{(k+1)}=\boldsymbol{A}\boldsymbol{y}^{(k)} & (k=0,1,2,\cdots)\\ \lambda_1\approx x_r^{(k+1)},\left|x_r^{(k+1)}\right|=\max\limits_{1\leqslant i\leqslant n}\left|x_i^{(k+1)}\right|\\ \boldsymbol{y}^{(k+1)}=\dfrac{\boldsymbol{x}^{(k+1)}}{x_r^{(k+1)}}\end{cases}\tag{4.5}$$

扫描全能王 创建

注 4.5 公式 (4.5)直接选取迭代后向量绝对值最大分量作为特征值近似值. 因为标准化的特征向量唯一, 所以迭代到一定步数以后, 绝对值最大的分量与标准化为 1 的分量位置重合. 以上两种取近似值的方法只影响前几步迭代结果, 最终收敛结果一致.

下面给出按照公式(4.4)幂法的计算流程, 读者可以自己编写按照公式(4.5)计算的流程.

算法 4.1 幂法

1. 输入 $\boldsymbol{A}=(a_{ij})$, 初始向量 $\boldsymbol{x}=(x_1,x_2,\cdots,x_n)$, 误差限 ε, 最大迭代次数 N.
2. 置 $k=1,\mu=0$.
3. 求整数 r, 使 $|x_r|=\max\limits_{1\leqslant i\leqslant n}|x_i|, x_r\Rightarrow\alpha$.
4. 计算

   $\boldsymbol{y}=\dfrac{\boldsymbol{x}}{\alpha}$

   $\boldsymbol{x}=\boldsymbol{A}\boldsymbol{y}$

   置 $x_r\Rightarrow\lambda$.
5. 若 $|\lambda-\mu|<\varepsilon$, 则输出 $\lambda,\boldsymbol{x}$, 停机; 否则, 转 6.
6. 若 $k<N$, 置 $k+1\Rightarrow k,\lambda\Rightarrow\mu$, 转 3 ; 否则, 输出失败信息, 停机.

例 4.1 用幂法求矩阵

$$\boldsymbol{A}=\begin{bmatrix}2&-1&0\\0&2&-1\\0&-1&2\end{bmatrix}$$

的按模最大的特征值和相应的特征向量. 取 $\boldsymbol{x}^{(0)}=(0,0,1)^{\mathrm{T}}$, 要求误差不超过 10^{-3}.

解 由算法 4.1, 得

$$\boldsymbol{y}^{(0)}=\boldsymbol{x}^{(0)}=(0,0,1)^{\mathrm{T}}$$

$$\boldsymbol{x}^{(1)}=\boldsymbol{A}\boldsymbol{y}^{(0)}=(0,-1,2)^{\mathrm{T}},\alpha=2,\lambda=2$$

$$\boldsymbol{y}^{(1)}=\frac{\boldsymbol{x}^{(1)}}{\alpha}=(0,-0.5,1)^{\mathrm{T}}$$

$$\boldsymbol{x}^{(2)}=\boldsymbol{A}\boldsymbol{y}^{(1)}=(0.5,-2,2.5)^{\mathrm{T}},\quad\alpha=2.5,\lambda=2.5$$

如此继续下去, 计算结果见表 4.1.

因为

$$2.999\ 697\ 3-2.999\ 092\ 4=0.000\ 604\ 9<10^{-3}$$

所以 $\lambda_1\approx 2.999\ 697\ 3$, 相应的特征向量为 $\boldsymbol{u}\approx(2.843\ 651\ 7,-2.999\ 394\ 6,2.\ 999\ 697\ 3)^{\mathrm{T}}$, 事实上, $\boldsymbol{A}$ 的特征值为 $\lambda_1=3,\lambda_2=2,\lambda_3=1$, 与 λ_1 相应的特征向量为 $(1,-1,1)^{\mathrm{T}}$.

幂法的收敛速度依赖于比值 $|\lambda_2/\lambda_1|$, 称其为收敛因子, 比值越小, 收敛越快. 例 4.1 中,

扫描全能王 创建

表 4.1　幂法迭代结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	λ	$y_1^{(k)}$	$y_2^{(k)}$	$y_3^{(k)}$
0	0	0	1	0	0	0	1
1	0	−1	2	2	0	−0.5	1
2	0.5	−2	2.5	2.5	0.2	−0.8	1
3	1.2	−2.6	2.8	2.8	0.428 571 4	−0.928 571 4	1
4	1.785 714 2	−2.857 142 8	2.928 571 4	2.928 571 4	0.609 756 0	−0.975 609 7	1
5	2.195 121 8	−2.951 394 0	2.975 609 7	2.975 609 7	0.737 704 8	−0.991 861 9	1
6	2.467 271 7	−2.983 723 8	2.991 861 9	2.991 861 9	0.824 660 9	−0.997 279 9	1
7	2.646 601 8	−2.994 559 8	2.997 279 9	2.997 279 9	0.883 001 2	−0.999 092 4	1
8	2.765 094 8	−2.998 184 8	2.999 092 4	2.999 092 4	0.921 977 2	−0.999 697 3	1
9	2.843 651 7	−2.999 394 6	2.999 697 3	2.999 697 3			

收敛因子 $|\lambda_2/\lambda_1| = 2/3$, 当迭代进行 9 次时, 所求特征值之误差不超过 10^{-3}. 读者可以用幂法自行计算矩阵

$$\boldsymbol{A} = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \end{bmatrix}$$

的按模最大的特征值, 取 $\boldsymbol{x}^{(0)} = (1,1,1)^{\mathrm{T}}$, 计算结果将表明, 只要迭代 5 次即可求得误差不超过 10^{-3} 的近似值. 这是因为该矩阵的收敛因子 $\left|\dfrac{\lambda_2}{\lambda_1}\right| \approx 0.065$, 因而收敛很快.

前面的讨论中, 我们假定 $|\lambda_1| > |\lambda_2|$. 如果按模最大的特征值有多个, 即矩阵 $\boldsymbol{A}$ 的特征值分布满足

$$|\lambda_1| = |\lambda_2| = \cdots = |\lambda_m| > |\lambda_{m+1}| \geqslant \cdots \geqslant |\lambda_n| \quad (m \geqslant 2)$$

幂法是否仍然有效? 下面就几种特殊情况进行讨论:

(1) 如果 λ_1 是 m 重根, 即 $\lambda_1 = \lambda_2 = \cdots = \lambda_m$, 矩阵 $\boldsymbol{A}$ 仍有 n 个线性无关的特征向量. 此时, 式 (4.2)变成

$$\boldsymbol{x}^{(k+1)} = \lambda_1^{k+1}\left[\alpha_1\boldsymbol{u}_1 + \alpha_2\boldsymbol{u}_2 + \cdots + \alpha_m\boldsymbol{u}_m + \left(\frac{\lambda_{m+1}}{\lambda_1}\right)^{k+1}\alpha_{m+1}\boldsymbol{u}_{m+1} + \cdots + \left(\frac{\lambda_n}{\lambda_1}\right)^{k+1}\alpha_n\boldsymbol{u}_n\right]$$

显然, 只要 $\alpha_1, \alpha_2, \cdots, a_m$ 不全为零, 当 k 充分大时, 就有

$$\boldsymbol{x}^{(k+1)} \approx \lambda_1^{(k+1)}\left(\alpha_1\boldsymbol{u}_1 + \alpha_2\boldsymbol{u}_2 + \cdots + \alpha_m\boldsymbol{u}_m\right)$$

因为 $\alpha_1\boldsymbol{u}_1 + \alpha_2\boldsymbol{u}_2 + \cdots + \alpha_m\boldsymbol{u}_m$ 也是矩阵 $\boldsymbol{A}$ 的相应于 λ_1 的特征向量, 因此仍有

$$\lambda_1 = \lambda_2 = \cdots = \lambda_m \approx \frac{x_i^{(k+1)}}{x_i^{(k)}} \quad (i = 1, 2, \cdots, n)$$

$\boldsymbol{x}^{(k+1)}$ 为相应的特征向量, 所以幂法对于按模最大的特征值是重根时仍然有效, 此时只能给出一个特征向量, 不同初始向量可能收敛到不同特征向量.

扫描全能王 创建

(2) 如果按模最大的特征值是一对相反数, 即 $\lambda_1=-\lambda_2,|\lambda_1|>|\lambda_3|$, 且矩阵 $\boldsymbol{A}$ 有 n 个线性无关的特征向量. 式(4.2) 变成

$$\boldsymbol{x}^{(k+1)}=\lambda_1^{k+1}\left[\alpha_1\boldsymbol{u}_1+(-1)^{k+1}\alpha_2\boldsymbol{u}_2+\left(\frac{\lambda_3}{\lambda_1}\right)^{k+1}\alpha_3\boldsymbol{u}_3+\cdots+\left(\frac{\lambda_n}{\lambda_1}\right)^{k+1}\alpha_n\boldsymbol{u}_n\right]$$

分析上式可知, $\left\{\boldsymbol{x}^{(k+1)}\right\}$ 是个摆动序列, 当 k 充分大时, 有

$$\boldsymbol{x}^{(2k-1)}\approx\lambda_1^{2k-1}\left(\alpha_1\boldsymbol{u}_1-\alpha_2\boldsymbol{u}_2\right)$$

$$\boldsymbol{x}^{(2k)}\approx\lambda_1^{2k}\left(\alpha_1\boldsymbol{u}_1+\alpha_2\boldsymbol{u}_2\right)$$

因此有

$$\lambda_1^2\approx\frac{x_i^{(k+2)}}{x_i^{(k)}}\quad(i=1,2,\cdots,n)$$

即

$$\lambda_{1,2}\approx\pm\sqrt{x_i^{(k+2)}/x_i^{(k)}}\quad(i=1,2,\cdots,n)\tag{4.6}$$

又由 $\boldsymbol{x}^{(k+1)}\approx\lambda_1^{k+1}\left[\alpha_1\boldsymbol{u}_1+(-1)^{k+1}\alpha_2\boldsymbol{u}_2\right],\boldsymbol{x}^{(k)}\approx\lambda_1^{k}\left[\alpha_1\boldsymbol{u}_1+(-1)^{k}\alpha_2\boldsymbol{u}_2\right]$ 可导出

$$\begin{cases}\boldsymbol{x}^{(k+1)}+\lambda_1\boldsymbol{x}^{(k)}\approx2\lambda_1^{k+1}\alpha_1\boldsymbol{u}_1\\ \boldsymbol{x}^{(k+1)}-\lambda_1\boldsymbol{x}^{(k)}\approx2\lambda_1^{k+1}(-1)^{k+1}\alpha_2\boldsymbol{u}_2\end{cases}\tag{4.7}$$

故在这种情况下, 仍可按幂法产生向量序列, 式 (4.6) (4.7) 为最大模特征值 λ_1,λ_2 与相应的特征向量的计算公式. 例如, 矩阵 $\boldsymbol{A}=\begin{bmatrix}4&-1&1\\16&-2&-2\\16&-3&-1\end{bmatrix}$, 若取 $\boldsymbol{x}^{(0)}=(1,0,0)^{\mathrm{T}}$, 用幂法计算得

$$\boldsymbol{x}^{(0)}=(1,0,0),\quad\boldsymbol{x}^{(1)}=(4,16,16)$$

$$\boldsymbol{x}^{(2)}=(16,0,0),\quad\boldsymbol{x}^{(3)}=(64,256,256)$$

$$\boldsymbol{x}^{(4)}=(256,0,0)\quad\boldsymbol{x}^{(5)}=(1\ 024,4\ 096,4\ 096)$$

$$\boldsymbol{x}^{(6)}=(4\ 096,0,0)\quad\boldsymbol{x}^{(7)}=(16\ 384,65\ 536,65\ 536)$$

显然有 $\left\{\boldsymbol{x}^{(k)}\right\}$ 是摆动的向量序列, $\left\{\boldsymbol{x}^{(2k)}\right\}$ 和 $\left\{\boldsymbol{x}^{(2k+1)}\right\}$ 是两个有公比的子列, 且 $\dfrac{x_i^{(k+2)}}{x_i^{(k)}}=16=\lambda_1^2$. 事实上 $\boldsymbol{A}$ 有特征值 $\lambda=4,-4$ 或 1 .

(3) 如果按模最大的特征值是一对共轭复数, $\lambda_1=\bar{\lambda}_2,\boldsymbol{u}_1=\bar{\boldsymbol{u}}_2$, 设 $\lambda_1=\lambda\mathrm{e}^{\mathrm{i}\theta}$, 且矩阵 $\boldsymbol{A}$ 有 n 个线性无关的特征向量. 式(4.2) 变成

$$\boldsymbol{x}^{(k+1)}=\lambda^{k+1}\left[\alpha_1\boldsymbol{u}_1\mathrm{e}^{\mathrm{i}(k+1)\theta}+\alpha_2\boldsymbol{u}_2\mathrm{e}^{-\mathrm{i}(k+1)\theta}+\left(\frac{\lambda_3}{\lambda}\right)^{k+1}\alpha_3\boldsymbol{u}_3+\cdots+\left(\frac{\lambda_n}{\lambda}\right)^{k+1}\alpha_n\boldsymbol{u}_n\right]$$

分析上式可知, $\left\{\boldsymbol{x}^{(k+1)}\right\}$ 是个振荡序列, 当 k 充分大时, 有

扫描全能王 创建

$$
\begin{aligned}
\boldsymbol{x}^{(k+1)} &\approx \lambda^{k+1}\left(\alpha_1\boldsymbol{u}_1\mathrm{e}^{\mathrm{i}(k+1)\theta}+\alpha_2\boldsymbol{u}_2\mathrm{e}^{-\mathrm{i}(k+1)\theta}\right)\\
\lambda_1\boldsymbol{x}^{(k)} &\approx \lambda^{k+1}\left(\alpha_1\boldsymbol{u}_1\mathrm{e}^{\mathrm{i}(k+1)\theta}+\alpha_2\boldsymbol{u}_2\mathrm{e}^{-\mathrm{i}(k-1)\theta}\right)\\
\lambda_2\boldsymbol{x}^{(k)} &\approx \lambda^{k+1}\left(\alpha_1\boldsymbol{u}_1\mathrm{e}^{\mathrm{i}(k-1)\theta}+\alpha_2\boldsymbol{u}_2\mathrm{e}^{-\mathrm{i}(k+1)\theta}\right)\\
\lambda_1\lambda_2\boldsymbol{x}^{(k-1)} &\approx \lambda^{k+1}\left(\alpha_1\boldsymbol{u}_1\mathrm{e}^{\mathrm{i}(k-1)\theta}+\alpha_2\boldsymbol{u}_2\mathrm{e}^{-\mathrm{i}(k-1)\theta}\right)
\end{aligned}
$$

因此有

$$
x_i^{(k+1)}-(\lambda_1+\lambda_2)x_i^{(k)}+\lambda_1\lambda_2x_i^{(k-1)}\approx 0\quad(i=1,2,\cdots,n) \tag{4.8}
$$

据式 (4.8), 至少选两组对应点, 或者采用最小二乘法计算 $\lambda_1+\lambda_2,\lambda_1\lambda_2$, 然后使用二次方程求根公式计算 λ_1,λ_2, 容易验证

$$
\begin{aligned}
\boldsymbol{x}^{(k+1)}-\lambda_1\boldsymbol{x}^{(k)} &\approx \lambda_2^k\left(\lambda_2-\lambda_1\right)\alpha_2\boldsymbol{u}_2\\
\boldsymbol{x}^{(k+1)}-\lambda_2\boldsymbol{x}^{(k)} &\approx \lambda_1^k\left(\lambda_1-\lambda_2\right)\alpha_1\boldsymbol{u}_1
\end{aligned} \tag{4.9}
$$

对于绝对值最大的特征值为一对共轭复数时, 仍可按幂法产生向量序列, 根据式 (4.8) (4.9) 计算绝对值最大的特征值 λ_1,λ_2 与相应的特征向量.

综上所述, 当 A 的特征值分布为 $|\lambda_1|>|\lambda_2|\geqslant\cdots\geqslant|\lambda_n|$ 或 $\lambda_1=\lambda_2=\cdots=\lambda_m,|\lambda_1|>|\lambda_{m+1}|\geqslant\cdots\geqslant|\lambda_n|$ 时, 用幂法可以计算出 λ_1 及相应的特征向量. 如果按式(4.1) 迭代所得向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 呈有规律的摆动, 则为 $\lambda_1=-\lambda_2$ 的情形. 如果按式(4.1) 迭代所得向量序列 $\left\{\boldsymbol{x}^{(k)}\right\}$ 呈无规律的振荡, 则为 $\lambda_1=\bar{\lambda}_2$ 的情形. 此外, 当矩阵 $\boldsymbol{A}$ 无 n 个线性无关的特征量时, 幂法收敛很慢, 亦应考虑改用其他方法.

幂法计算简便易行, 它是求大型稀疏矩阵按模最大特征值的常用方法.

4.1.2　幂法的加速

因为幂法的收敛速度是线性的, 而且依赖于收敛因子 $|\lambda_2/\lambda_1|$, 当收敛因子接近于 1 时, 幂法收敛很慢. 幂法的加速有许多方法, 下面介绍其中的两种.

(一) 原点移位法

容易看出, 矩阵 $\boldsymbol{A}$ 与 $\boldsymbol{A}-\lambda_0\boldsymbol{I}$ 的特征值有以下关系: 若 λ_i 是 $\boldsymbol{A}$ 的特征值, 则 $\lambda_i-\lambda_0$ 就是 $\boldsymbol{A}-\lambda_0\boldsymbol{I}$ 的特征值, 而且相应的特征向量不变. 如果对矩阵 $\boldsymbol{A}-\lambda_0\boldsymbol{I}$ 按式 (4.1)计算, 则有

$$
\begin{aligned}
\boldsymbol{x}^{(k+1)} &= \left(\boldsymbol{A}-\lambda_0\boldsymbol{I}\right)\boldsymbol{x}^{(k)}\\
&= \left(\lambda_1-\lambda_0\right)^{k+1}\left[\alpha_1\boldsymbol{u}_1+\left(\frac{\lambda_2-\lambda_0}{\lambda_1-\lambda_0}\right)^{k+1}\alpha_2\boldsymbol{u}_2+\cdots+\left(\frac{\lambda_n-\lambda_0}{\lambda_1-\lambda_0}\right)^{k+1}\alpha_n\boldsymbol{u}_n\right]
\end{aligned}
$$

适当地选取 λ_0, 使得 $|\lambda_1-\lambda_0|>|\lambda_i-\lambda_0|$, 且

$$
\left|\frac{\lambda_i-\lambda_0}{\lambda_1-\lambda_0}\right|<\left|\frac{\lambda_2}{\lambda_1}\right|\quad(i=2,3,\cdots,n)
$$

扫描全能王 创建

这样, 用幂法计算 $\boldsymbol{A}-\lambda_0\boldsymbol{I}$ 的最大模特征值 $\lambda_1-\lambda_0$ 及相应特征向量的收敛速度比对 $\boldsymbol{A}$ 用幂法计算要快. 这种加速收敛的方法称为原点移位法.

原点移位法使用简便, 不足之处在于 λ_0 的选取十分困难, 通常需要使用者对特征值的分布有一大概的了解, 才能粗略地估计 λ_0, 并通过计算不断进行修改. 但在一些简单情形, λ_0 是可以估计的. 如当矩阵 $\boldsymbol{A}$ 的特征值满足 $\lambda_1>\lambda_2\geqslant\lambda_3\geqslant\cdots\geqslant\lambda_n>0$(或 $\lambda_1<\lambda_2\leqslant\lambda_3\leqslant\cdots\leqslant\lambda_n<0$) 时, 取 $\lambda_0=\dfrac{1}{2}(\lambda_2+\lambda_n)$, 则有

$$|\lambda_i-\lambda_0|\leqslant|\lambda_2-\lambda_0|<|\lambda_1-\lambda_0|\quad(i=2,3,\cdots,n)$$

但

$$\frac{\lambda_2-\lambda_0}{\lambda_1-\lambda_0}=\frac{\lambda_2-\lambda_n}{2\lambda_1-\lambda_2-\lambda_n}=\frac{\lambda_2-\lambda_n}{\lambda_1-\lambda_n+\lambda_1-\lambda_2}<\frac{\lambda_2-\lambda_n}{\lambda_1-\lambda_n}<\frac{\lambda_2}{\lambda_1}$$

因此, 用原点移位法求 λ_1 可使收敛速度加快.

例 4.2 取 $\lambda_0=2.9$, 用原点移位法求矩阵

$$\boldsymbol{A}=\begin{bmatrix}-4 & 14 & 0\\ -5 & 13 & 0\\ -1 & 0 & 2.8\end{bmatrix}$$

的按模最大的特征值, 要求误差不超过 10^{-4}.

解

$$\boldsymbol{A}-\lambda_0\boldsymbol{I}=\begin{bmatrix}-6.9 & 14 & 0\\ -5 & 10.1 & 0\\ -1 & 0 & -0.1\end{bmatrix}$$

取 $\boldsymbol{x}^{(0)}=(1,1,1)^{\mathrm{T}}$, 按原点移位法进行计算, 结果见表 4.2.

表 4.2 原点移位法计算结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	λ	$y_1^{(k)}$	$y_2^{(k)}$	$y_3^{(k)}$
0	1	1	1	0	1	1	1
1	7.1	5.1	−1.1	7.1	1	0.718 309 8	−0.154 929 5
2	3.156 337 2	2.254 929 0	−0.984 507 1	3.156 337 2	1	0.714 413 2	−0.311 914 4
3	3.101 785 0	2.215 573 3	−0.968 808 6	3.101 785 0	1	0.714 289 7	−0.312 339 0
4	3.100 056 8	2.214 326	−0.968 766 1	3.100 056 8	1	0.714 285 6	−0.312 499 4
5	3.099 998 4	2.214 284 6	−0.968 750 1	3.099 998 4	1		

迭代 5 次后

$$|3.099\ 998\ 4-3.100\ 056\ 8|=0.000\ 058\ 4<10^{-4}$$

所以, 矩阵 $\boldsymbol{A}$ 的按模最大的特征值为

$$\lambda_1\approx3.099\ 998\ 4+2.9=5.999\ 998\ 4$$

相应的特征向量为 $\boldsymbol{x}\approx(3.099\ 998\ 4,2.214\ 284\ 6,-0.968\ 750\ 1)^{\mathrm{T}}$.

扫描全能王 创建

不难求出, $\boldsymbol{A}$ 的特征值为 $\lambda_1 = 6, \lambda_2 = 3, \lambda_3 = 2.8$, 若对 $\boldsymbol{A}$ 直接用幂法, 则收敛因子 $|\lambda_2/\lambda_1| = 1/2$, 而用原点移位法, 则有

$$\left|\frac{\lambda_2 - \lambda_0}{\lambda_1 - \lambda_0}\right| = \frac{0.1}{3.1} = \frac{1}{31}$$

因此收敛速度明显加快.

(二) 幂法的 Aitken (艾特肯) 加速

如果序列 $\{a_k\}$ 线性收敛到 a, 即

$$\lim_{k\to\infty} \frac{a_{k+1} - a}{a_k - a} = c \neq 0$$

则当 k 充分大时, 有

$$\frac{a_{k+2} - a}{a_{k+1} - a} \approx \frac{a_{k+1} - a}{a_k - a}$$

由此可解出

$$a \approx \frac{a_{k+2}a_k - a_{k+1}^2}{a_{k+2} - 2a_{k+1} + a_k} = a_k - \frac{(a_{k+1} - a_k)^2}{a_{k+2} - 2a_{k+1} + a_k} = \hat{a}_k$$

序列 $\{\hat{a}_k\}$ 比 $\{a_k\}$ 更快地收敛到 a, 这就是 Aitken 加速法. 将这一方法用于幂法所产生的序列 $\{a_k\}$, 可加快幂法的收敛速度.

算法 4.2　幂法 (Aitken 加速)

---

1. 输入 $\boldsymbol{A} = (a_{ij})$, 初始向量 $\boldsymbol{x} = (x_1, x_2, \cdots, x_n)$, 误差限 ε, 最大迭代次数 N.
2. 置 $k = 1, \alpha_0 = 0, \alpha_1 = 0, \lambda_0 = 1$.
3. 求整数 r, 使 $|x_r| = \max\limits_{1\leqslant i\leqslant n} |x_i|, x_r \Rightarrow \alpha$.
4. 计算

   $\boldsymbol{y} = \dfrac{\boldsymbol{x}}{\alpha}$

   $\boldsymbol{x} = \boldsymbol{A}\boldsymbol{y}$

   置 $x_r \Rightarrow \alpha_2$
5. 计算

$$\lambda = \alpha_0 - \frac{(\alpha_1 - \alpha_0)^2}{\alpha_2 - 2\alpha_1 + \alpha_0}$$

6. 若 $|\lambda - \lambda_0| < \varepsilon$, 输出 $\lambda, \boldsymbol{x}$, 停机; 否则转 7 .
7. 若 $k < N$, 置 $\alpha_1 \Rightarrow \alpha_0, \alpha_2 \Rightarrow \alpha_1, \lambda \Rightarrow \lambda_0, k+1 \Rightarrow k$, 转 3 ; 否则, 输出失败信息, 停机.

扫描全能王 创建

例 4.3 用 Aitken 加速法求矩阵

$$\boldsymbol{A}=\begin{bmatrix}2 & -1 & 0\\ 0 & 2 & -1\\ 0 & -1 & 2\end{bmatrix}$$

的按模最大的特征值与相应的特征向量. 取 $\boldsymbol{x}^{(0)}=(0,0,1)^{\mathrm{T}}$.

解 按算法 4.2 计算, 结果见表 4.3.

表 4.3 Aitken 加速法计算结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	α_k	$\lambda^{(k)}$
1	0	−1	2	2	
2	0.5	−2	2.5	2.5	
3	1.2	−2.6	2.8	2.8	3.25
4	1.785 714 2	−2.857 142 8	2.928 571 4	2.928 571 4	3.024 999 9
5	2.195 121 8	−2.951 394 0	2.975 609 7	2.975 609 7	3.002 747 2
6	2.467 271 7	−2.983 723 8	2.991 861 9	2.991 861 9	3.000 441 6

与例 4.1 的计算结果比较, 显然, Aitken 加速法的收敛速度比直接用幂法快.

算法 4.2 是用幂法迭代一步、加速一次, 通常也采用幂法迭代两步或三步、加速一次的方法.

4.1.3 反幂法

反幂法是计算矩阵按模最小的特征值及特征向量的方法, 也是修正特征值、求相应特征向量的最有效的方法.

设 $\boldsymbol{A}$ 为 $n\times n$ 非奇异矩阵, $\lambda,\boldsymbol{u}$ 为 $\boldsymbol{A}$ 的特征值与相应的特征向量, 则 $\boldsymbol{A}^{-1}$ 的特征值是 $\boldsymbol{A}$ 的特征值的倒数, 而相应的特征向量不变, 即

$$\boldsymbol{A}^{-1}\boldsymbol{u}=\frac{1}{\lambda}\boldsymbol{u}$$

因此, 若对矩阵 $\boldsymbol{A}^{-1}$ 用幂法, 即可计算出 $\boldsymbol{A}^{-1}$ 的按模最大的特征值, 其倒数恰为 $\boldsymbol{A}$ 的按模最小的特征值. 这就是反幂法的基本思想.

因为 $\boldsymbol{A}^{-1}$ 的计算比较麻烦, 而且往往不能保持矩阵 $\boldsymbol{A}$ 的一些好性质 (如稀疏性), 因此, 反幂法在实际运算时以求解方程组

$$\boldsymbol{A}\boldsymbol{x}^{(k+1)}=\boldsymbol{x}^{(k)} \tag{4.10}$$

代替幂法迭代

$$\boldsymbol{x}^{(k+1)}=\boldsymbol{A}^{-1}\boldsymbol{x}^{(k)}$$

求得 $\boldsymbol{x}^{(k+1)}$, 每迭代一次要解一个线性方程组 (4.10). 由于矩阵在迭代过程中不变, 故可对 $\boldsymbol{A}$ 先进行三角分解, 这样, 每次迭代只要解两个三角形方程组, 节省计算量.

扫描全能王 创建

反幂法计算的主要步骤如下：

1. 对 $\boldsymbol{A}$ 进行三角分解 $\boldsymbol{A}=\boldsymbol{L}\boldsymbol{U}$.

2. 求整数 r, 使得 $\left|x_r^{(k)}\right|=\max\limits_{1\leqslant i\leqslant n}\left|x_i^{(k)}\right|, \alpha=x_r^{(k)}$, 计算

$$\boldsymbol{y}^{(k)}=\frac{\boldsymbol{x}^{(k)}}{\alpha}$$

3. 解方程组

$$\boldsymbol{L}\boldsymbol{z}=\boldsymbol{y}^{(k)}$$

$$\boldsymbol{U}\boldsymbol{x}^{(k+1)}=\boldsymbol{z}$$

$$\lambda_n\approx\frac{1}{x_r^{(k+1)}}$$

反幂法的收敛因子为 $\left|\dfrac{\lambda_n}{\lambda_{n-1}}\right|$, 收敛因子越小, 收敛越快.

用带原点移位的反幂法来修正特征值, 并求相应的特征向量是非常有效的. 设已知 $\boldsymbol{A}$ 的一个特征值 λ 的近似值为 λ^*, 因为 λ^* 接近 λ, 一般应有

$$0<|\lambda-\lambda^*|\ll|\lambda_i-\lambda^*|\quad(\lambda_i\neq\lambda)$$

故 $\lambda-\lambda^*$ 是矩阵 $A-\lambda^*\boldsymbol{I}$ 的按模最小的特征值, 且由上式可知, 比值 $\dfrac{|\lambda-\lambda^*|}{|\lambda_i-\lambda^*|}(\lambda_i\neq\lambda)$ 较小. 因此, 对 $A-\lambda^*\boldsymbol{I}$ 用反幂法求 $\lambda-\lambda^*$ 一般收敛很快, 通常只要经过二三次迭代就能达到较高的精度.

算法 4.3　反幂法

1. 输入 $\boldsymbol{A}=(a_{ij})$, 近似值 λ^*, 初始向量 $\boldsymbol{x}=(x_1,x_2,\cdots,x_n)$, 误差限 ε, 最大容许迭代次数 N.

2. 置 $k=1,\mu=1$.

3. 作三角分解 $(\boldsymbol{A}-\lambda^*\boldsymbol{I})=\boldsymbol{L}\boldsymbol{U}$.

4. 求整数 r, 使得 $|x_r|=\max\limits_{1\leqslant i\leqslant n}|x_i|, x_r\Rightarrow\alpha$.

5. 计算

$$\boldsymbol{y}=\frac{\boldsymbol{x}}{\alpha}$$

$$\boldsymbol{L}\boldsymbol{z}=\boldsymbol{y}$$

$$\boldsymbol{U}\boldsymbol{x}=\boldsymbol{z}$$

置 $x_r\Rightarrow\beta$

6. 若 $\left|\dfrac{1}{\beta}-\dfrac{1}{\mu}\right|<\varepsilon$, 则置 $\lambda=\lambda^*+\dfrac{1}{\beta}$, 输出 $\lambda,\boldsymbol{x}$, 停机; 否则转 7.

7. 若 $k<N$, 则置 $k+1\Rightarrow k,\beta\Rightarrow\mu$, 转 4 ; 否则输出失败信息, 停机.

扫描全能王 创建

注 4.6 如果 $\lambda^* = 0$, 则算法 4.3 求出 $\boldsymbol{A}$ 的按模最小的特征值与特征向量.

例 4.4 用反幂法求矩阵

$$\boldsymbol{A} = \begin{bmatrix} 2 & -1 & 0 \\ 0 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

接近 2.93 的特征值, 并求相应的特征向量, 取 $\boldsymbol{x}^{(0)} = (0,0,1)^{\mathrm{T}}$.

解 对 $\boldsymbol{A} - 2.93\boldsymbol{I}$ 作三角分解得

$$\begin{aligned} \boldsymbol{A} - 2.93\boldsymbol{I} &= \begin{bmatrix} -0.93 & -1 & 0 \\ 0 & -0.93 & -1 \\ 0 & -1 & -0.93 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1/0.93 & 1 \end{bmatrix} \begin{bmatrix} -0.93 & -1 & 0 \\ 0 & -0.93 & -1 \\ 0 & 0 & -0.93 + 1/0.93 \end{bmatrix} \end{aligned}$$

按算法 4.3 计算的数值结果见表 4.4.

表 4.4 反幂法数值计算结果

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	α_k	λ
1	7.959 058 6	−7.401 924 5	6.883 789 8	7.959 059 5	3.075 268 8
2	12.692 313	−12.803 851	12.837 581	12.837 581	3.008 787 8
3	14.278 436	−14.267 629	14.266 268	14.278 436	3.000 095 4

迭代 3 次, 即得 $\lambda \approx 3.000\ 095\ 4$, 与准确值 $\lambda = 3$ 的误差小于 10^{-4}, 相应的特征向量为

$$\boldsymbol{u} \approx \frac{\boldsymbol{x}^{(3)}}{\alpha_3} = (1, -0.999\ 243\ 1, 0.999\ 147\ 8)^{\mathrm{T}}$$

与准确值 $\boldsymbol{u} = (1,-1,1)^{\mathrm{T}}$ 比较, 残量为 $\|\boldsymbol{r}\|_\infty < 0.001$.

4.2 Jacobi 方法

Jacobi 方法是求实对称矩阵的全部特征值及相应的特征向量的一种方法, 它基于以下两个结论:

1. 任意实对称矩阵 $\boldsymbol{A}$ 可以通过正交相似变换化成对角矩阵, 即存在正交矩阵 $\boldsymbol{Q}$, 使得

$$\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{Q} = \mathrm{diag}\,(\lambda_1, \lambda_2, \cdots, \lambda_n)$$

其中 $\lambda_i (i = 1,2,\cdots,n)$ 是 $\boldsymbol{A}$ 的特征值, $\boldsymbol{Q}$ 中各列即为相应的特征向量.

2. 在正交相似变换下, 矩阵元素的平方和不变. 设 $\boldsymbol{A} = (a_{ij})_{n\times n}$, $\boldsymbol{Q}$ 为正交矩阵, 记

扫描全能王 创建

$\boldsymbol{B}=\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{Q}=(b_{ij})_{n\times n}$, 则

$$\sum_{i,j=1}^{n}a_{ij}^{2}=\sum_{i,j=1}^{n}b_{ij}^{2} \tag{4.11}$$

Jacobi 方法的基本思想是通过一次正交变换, 将 $\boldsymbol{A}$ 中一对非零的非对角元素化成零, 并且使得非对角元素的平方和减小. 反复进行上述过程, 使变换后的矩阵的非对角元素的平方和趋于零, 从而使该矩阵近似为对角矩阵, 得到全部特征值和特征向量.

4.2.1　矩阵的旋转变换

设 $\boldsymbol{A}$ 为 n 阶实对称矩阵, 考虑矩阵

$$\boldsymbol{V}_{ij}(\varphi)=\begin{bmatrix}
1 & & & & & & & & \\
& \ddots & & & & & & & \\
& & 1 & & & & & & \\
& & & \cos\ \varphi & \cdots & \sin\ \varphi & & & i \\
& & & & 1 & & & & \\
& & & \vdots & \ddots & \vdots & & & \\
& & & & & 1 & & & \\
& & & -\sin\ \varphi & \cdots & \cos\ \varphi & & & j \\
& & & & & & 1 & & \\
& & & & & & & \ddots & \\
& & & & & & & & 1
\end{bmatrix} \tag{4.12}$$

它是将 n 阶单位矩阵中对角线上第 i 和第 j 个元换成 $\cos\ \varphi$, 非对角元 v_{ij} 和 v_{ji} 分别换或 $\sin\ \varphi$ 和 $-\sin\ \varphi$ 而得到的, 容易验证, $\boldsymbol{V}_{ij}(\varphi)$ 是正交矩阵, 若记

$$\boldsymbol{A}^{(1)}=\boldsymbol{V}_{ij}\boldsymbol{A}\boldsymbol{V}_{ij}^{\mathrm{T}}=\left(a_{ij}^{(1)}\right)$$

则有

$$\begin{cases}
a_{ii}^{(1)}=a_{ii}\cos^{2}\varphi+a_{jj}\sin^{2}\varphi+a_{ij}\sin 2\ \varphi \\
a_{ij}^{(1)}=a_{ii}\sin^{2}\varphi+a_{jj}\cos^{2}\varphi-a_{ij}\sin 2\ \varphi \\
a_{ik}^{(1)}=a_{ki}^{(1)}=a_{ik}\cos\ \varphi+a_{jk}\sin\ \varphi \quad (k\neq i,j) \\
a_{jk}^{(1)}=a_{kj}^{(1)}=-a_{ik}\sin\ \varphi+a_{jk}\cos\ \varphi \quad (k\neq i,j) \\
a_{kl}^{(1)}=a_{lk}^{(1)}=a_{kl} \quad (k,l\neq i,j) \\
a_{ij}^{(1)}=a_{ji}^{(1)}=\dfrac{1}{2}(a_{jj}-a_{ii})\sin 2\ \varphi+a_{ij}\cos 2\ \varphi
\end{cases} \tag{4.13}$$

如果 $a_{ij}\neq 0$, 取 φ 使得

$$\tan 2\ \varphi=\frac{2a_{ij}}{a_{ii}-a_{jj}} \quad \left(|\varphi|<\frac{\pi}{4}\right)$$

扫描全能王 创建

则有 $a_{ij}^{(1)}=a_{ji}^{(1)}=0$, 这样, 就得到一个使 $\boldsymbol{A}$ 中非零的非对角元素 $a_{ij}=a_{ji}$ 变成零的正交相似变换. 从几何上看, $\boldsymbol{V}_{ij}$ 相当于将 n 维空间中的 Ox_i 轴与 Ox_j 轴在 x_iOx_j 平面中作一旋转, 故 $\boldsymbol{V}_{ij}$ 称为旋转变换矩阵.

对 $\boldsymbol{A}^{(1)}$ 重复上述过程, 可得 $\boldsymbol{A}^{(2)}$, 如此继续下去, 得到一个矩阵序列 $\{\boldsymbol{A}^{(k)}\}$. 可以证明, 虽然这种变换不一定能使矩阵中非对角元中零元素的个数单调增加, 但可以保证非对角元的平方和递减, 我们以 $\boldsymbol{A}$ 与 $\boldsymbol{A}^{(1)}$ 为例进行讨论.

设 $\boldsymbol{E}(\boldsymbol{A})=\sum\limits_{k\neq l}a_{kl}^2, \boldsymbol{E}(\boldsymbol{A}^{(1)})=\sum\limits_{k\neq l}a_{kl}^{(1)^2}$, 由式(4.13)

$$
\begin{aligned}
&a_{kl}=a_{kl}^{(1)} \quad (k,l\neq i,j)\\
&a_{ki}^{(1)}=a_{ik}^{(1)}=a_{ik}\cos\varphi+a_{jk}\sin\varphi \quad (k\neq i,j)\\
&a_{jk}^{(1)}=a_{kj}^{(1)}=-a_{ik}\sin\varphi+a_{jk}\cos\varphi \quad (k\neq i,j)
\end{aligned}
$$

可得

$$
\begin{aligned}
\boldsymbol{E}\left(\boldsymbol{A}^{(1)}\right)&=\sum_{k\neq l,k,l\neq i,j}\left(a_{kl}^{(1)}\right)^2+2\sum_{k\neq i,j}\left[\left(a_{ik}^{(1)}\right)^2+\left(a_{jk}^{(1)}\right)^2\right]\\
&=\sum_{k\neq l,k,l\neq i,j}a_{kl}^2+2\sum_{k\neq i,j}\left(a_{ik}^2+a_{jk}^2\right) \qquad (4.14)\\
&=\boldsymbol{E}(\boldsymbol{A})-2a_{ij}^2<\boldsymbol{E}(\boldsymbol{A})
\end{aligned}
$$

式(4.14)表明, 在上述旋转变换下, 非对角元的平方和严格单调递减, 因而由式(4.11)即得, 对角元的平方和单调递增. 正是利用这一点, 导出了 Jacobi 方法.

4.2.2 Jacobi 方法

通过一系列旋转相似变换将 $\boldsymbol{A}$ 变成 $\boldsymbol{A}^{(k+1)}$, 求得 $\boldsymbol{A}$ 的全部特征值与特征向量的方法称为 Jacobi 方法. 如果在对 $\boldsymbol{A}$ 作相似变换的过程中, 每一步都选绝对值最大的非对角元素 $a_{ij}^{(k)}$, 以此确定旋转矩阵, 这种方法称为古典 Jacobi 方法. 其计算过程如下:

(1) 令 $k=0, \boldsymbol{A}^{(k)}=\boldsymbol{A}$.

(2) 求整数 i,j, 使得 $\left|a_{ij}^{(k)}\right|=\max\limits_{1\leqslant i,m\leqslant n;l\neq m}\left|a_{lm}^{(k)}\right|$.

(3) 计算旋转矩阵

$$
\begin{aligned}
&a=\cot 2\varphi=\frac{a_{ii}^{(k)}-a_{ij}^{(k)}}{2a_{ij}^{(k)}}\\
&b=\tan\varphi=\operatorname{sign}(a)\left(\sqrt{a^2+1}-|a|\right)\\
&c=\cos\varphi=\frac{1}{\sqrt{1+b^2}}, d=\sin\varphi=bc\\
&\boldsymbol{V}^{(k)}=\boldsymbol{V}_{ij}(\varphi)
\end{aligned}
$$

扫描全能王 创建

(4) 计算 $\boldsymbol{A}^{(k+1)}$

$$
\begin{aligned}
a_{ii}^{(k+1)} &= c^2 a_{ii}^{(k)} + d^2 a_{jj}^{(k)} + 2cda_{ij}^{(k)} \\
a_{jj}^{(k+1)} &= d^2 a_{ii}^{(k)} + c^2 a_{jj}^{(k)} - 2cda_{ij}^{(k)} \\
a_{il}^{(k+1)} &= a_{li}^{(k+1)} = ca_{il}^{(k)} + da_{jl}^{(k)} \quad (l = 1, 2, \cdots, n; l \neq i, j) \\
a_{jl}^{(k+1)} &= a_{lj}^{(k+1)} = -da_{il}^{(k)} + ca_{jl}^{(k)} \quad (l = 1, 2, \cdots, n; l \neq i, j) \\
a_{lm}^{(k+1)} &= a_{ml}^{(k+1)} = a_{lm}^{(k)} \quad (l, m = 1, 2, \cdots, n; l, m \neq i, j)
\end{aligned}
$$

置 $a_{ij}^{(k+1)} = a_{ji}^{(k+1)} = 0$

(5) 计算 $\boldsymbol{E}\left(\boldsymbol{A}^{(k+1)}\right) = \sum\limits_{l \neq m} \left(a_{lm}^{(k+1)}\right)^2$.

(6) 若 $\boldsymbol{E}\left(\boldsymbol{A}^{(k+1)}\right) < \varepsilon$, 则 $a_{11}^{(k+1)}, a_{22}^{(k+1)}, \cdots, a_{nn}^{(k+1)}$ 为特征值, $\boldsymbol{Q} = \boldsymbol{V}^{(0)}\boldsymbol{V}^{(1)}\cdots\boldsymbol{V}^{(k)}$ 的各列为相应的特征向量; 否则 $k+1 \Rightarrow k$, 返回 2 , 重复上述过程.

一般地, Jacobi 法不能在有限步内将 $\boldsymbol{A}$ 化成对角矩阵, 但有以下收敛性结果.

定理 4.1　设 $\boldsymbol{A}$ 为 n 阶实对称矩阵, 对 $\boldsymbol{A}$ 用古典 Jacobi 法得到序列 $\{\boldsymbol{A}^{(k)}\}$, 其中 $\boldsymbol{A}^{(0)} = \boldsymbol{A}$, 则

$$\lim_{k \to \infty} \boldsymbol{E}\left(\boldsymbol{A}^{(k)}\right) = 0$$

即古典 Jacobi 法收敛.

证明　由 Jacobi 法计算过程

$$\left|a_{ij}^{(k)}\right| = \max_{1 \leqslant l, m \leqslant n; l \neq m} \left|a_{lm}^{(k)}\right|$$

得

$$\left(a_{ij}^{(k)}\right)^2 \geqslant \frac{1}{n(n-1)} \boldsymbol{E}\left(\boldsymbol{A}^{(k)}\right) \tag{4.15}$$

另一方面, 由计算 $\boldsymbol{A}^{(k+1)}$ 的公式可以得出

$$
\begin{cases}
\left(a_{ii}^{(k+1)}\right)^2 + \left(a_{jj}^{(k+1)}\right)^2 = \left(a_{ii}^{(k)}\right)^2 + \left(a_{jj}^{(k)}\right)^2 + 2\left(a_{ij}^{(k)}\right)^2 \\
\left(a_{il}^{(k+1)}\right)^2 + \left(a_{jl}^{(k+1)}\right)^2 = \left(a_{il}^{(k)}\right)^2 + \left(a_{jl}^{(k)}\right)^2, \quad l \neq i, j
\end{cases}
$$

于是有 $\boldsymbol{E}\left(\boldsymbol{A}^{(k+1)}\right) = \boldsymbol{E}\left(\boldsymbol{A}^{(k)}\right) - 2\left(a_{ij}^{(k)}\right)^2$, 代入式(4.15) 得

$$\boldsymbol{E}\left(\boldsymbol{A}^{(k+1)}\right) \leqslant \left[1 - \frac{2}{n(n-1)}\right] \boldsymbol{E}(\boldsymbol{A}^{(k)}) \leqslant \left[1 - \frac{2}{n(n-1)}\right]^{k+1} \boldsymbol{E}(\boldsymbol{A})$$

因为 $1 - \dfrac{2}{n(n-1)} < 1$, 所以

$$\lim_{k \to \infty} \boldsymbol{E}\left(\boldsymbol{A}^{(k)}\right) = \lim_{k \to \infty} \left[1 - \frac{2}{n(n-1)}\right]^k \boldsymbol{E}(\boldsymbol{A}) = 0$$

扫描全能王 创建

定理 4.1表明, 古典 Jacobi 法是收敛的, 进一步分析还可以得出 Jacobi 法收敛较快. 另外, 这种方法对舍入误差有较强的稳定性, 因而解的精度高, 且所求得的特征向量正交性很好. 它的不足之处是运算量大, 且不能保持矩阵的特殊形状 (如稀疏性), 因此 Jacobi 法是求中小型稠密实对称矩阵的全部特征值与特征向量的较好方法.

例 4.5 用 Jacobi 方法求矩阵

$$\boldsymbol{A}=\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

的特征值.

解 首先取 $i=1,j=2$, 因为 $\cot 2\varphi=0$, 故有 $\varphi=\dfrac{\pi}{4}$, 于是 $\cos\varphi=\sin\varphi=\dfrac{1}{\sqrt{2}}$, 由式(4.12)得

$$\boldsymbol{V}^{(0)}=\boldsymbol{V}_{12}(\varphi)=\begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned}\boldsymbol{A}^{(1)}&=\boldsymbol{V}^{(0)}\boldsymbol{A}^{(0)}\boldsymbol{V}^{(0)\mathrm{T}}\\ &=\begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 2 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 2 \end{bmatrix}\begin{bmatrix} \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 \\ 0 & 0 & 1 \end{bmatrix}\\ &=\begin{bmatrix} 1 & 0 & -\frac{1}{\sqrt{2}} \\ 0 & 3 & -\frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 2 \end{bmatrix}\end{aligned}$$

再取 $i=1,j=3,\tan 2\varphi=\sqrt{2},\cos\varphi=0.888\ 07,\sin\varphi=0.459\ 70$, 则

$$\boldsymbol{V}^{(1)}=\begin{bmatrix} 0.888\ 07 & 0 & 0.459\ 70 \\ 0 & 1 & 0 \\ -0.459\ 70 & 0 & 0.888\ 07 \end{bmatrix}$$

$$\boldsymbol{A}^{(2)}=\boldsymbol{V}^{(1)}\boldsymbol{A}^{(1)}\boldsymbol{V}^{(1)T}=\begin{bmatrix} 0.633\ 97 & -0.325\ 06 & 0 \\ -0.325\ 06 & 3 & -0.627\ 96 \\ 0 & -0.627\ 96 & 2.366\ 03 \end{bmatrix}$$

下面应取 $i=2,j=3$, 重复上述过程. 如此继续下去, 可得

扫描全能王 创建

$$\boldsymbol{A}^{(5)}=\begin{bmatrix}0.585\ 79 & 0.002\ 038\ 3 & 0\\ 0.002\ 038\ 3 & 3.414\ 01 & 0.016\ 758\\ 0 & 0.016\ 758 & 2.000\ 20\end{bmatrix}$$

$$E\left(\boldsymbol{A}^{(5)}\right)=2\times\left[(0.002\ 038\ 3)^2+(0.016\ 758)^2\right]=5.699\ 7\times10^{-4}\approx0.000\ 569\ 97$$

所以 $\boldsymbol{A}$ 的特征值为

$$\lambda_1\approx3.414\ 01,\quad \lambda_2\approx2.000\ 20,\quad \lambda_3\approx0.585\ 79$$

与其准确值

$$\lambda_1=2+\sqrt{2}\approx3.414\ 213\ 6,\quad \lambda_2=2,\quad \lambda_3=2-\sqrt{2}\approx0.585\ 786\ 4$$

比较, 最大误差为 0.000 203 6.

古典 Jacobi 法在计算每个旋转矩阵 $\boldsymbol{V}^{(k)}$ 前, 都需要对 $\dfrac{1}{2}n(n-1)$ 个非对角元素进行比较, 从中找出绝对值最大的元素. 为减少运算量, 常用的一种改进方法是: 取定正序列 $\{\varepsilon_k\},\varepsilon_k\to0\ (k\to\infty)$, 以 ε_k 为限, 逐行检查非对角元, 若 $|a_{ij}|<\varepsilon_k$ 就跳过, 否则为 $\boldsymbol{V}_{ij}(\varphi)$ 消去元 a_{ij} 和 a_{ji}, 反复进行上述过程, 直到所有非对角元的绝对值均小于 ε_k, 再以 ε_{k+1} 为限, 进行第 $k+1$ 轮循环消元. 当 ε_k 充分小时, 所得到的矩阵的对角元即为 $\boldsymbol{A}$ 的全部特征值.

4.3　QR 方法

4.3.1　基本 QR 方法

20 世纪 60 年代出现的十大算法之一——QR 方法是目前计算一般中小型矩阵的全部特征值与特征向量的最有效的方法, 北太天元中的库函数 eig 就是采用这种方法. 这里仅讨论实矩阵, 并假定矩阵非奇异. 否则, 矩阵 $\boldsymbol{A}-\alpha\boldsymbol{I}(\alpha$ 不是 $\boldsymbol{A}$ 的特征值) 必定是非奇异的, 而由 $\boldsymbol{A}-\alpha\boldsymbol{I}$ 的特征值与特征向量容易得到 $\boldsymbol{A}$ 的特征值与特征向量.

因为任一非奇异实矩阵都可以分解成一个正交矩阵 $\boldsymbol{Q}$ 和一个上三角形矩阵 $\boldsymbol{R}$ 的乘积, 而且当 $\boldsymbol{R}$ 的对角元符号取定时, 分解是唯一的. 基本 QR 方法的基本思想是利用矩阵的 QR 分解, 通过迭代格式

$$\begin{cases}\boldsymbol{A}^{(k)}=\boldsymbol{Q}_k\boldsymbol{R}_k\\ \boldsymbol{A}^{(k+1)}=\boldsymbol{R}_k\boldsymbol{Q}_k\end{cases}\quad(k=1,2,\cdots)\tag{4.16}$$

将 $\boldsymbol{A}=\boldsymbol{A}^{(1)}$ 化成相似的上三角形矩阵（或分块上三角形矩阵）, 从而求出矩阵 $\boldsymbol{A}$ 的全部特征值与特征向量.

由式(4.16), $\boldsymbol{A}=\boldsymbol{A}^{(1)}=\boldsymbol{Q}_1\boldsymbol{R}_1$, 即 $\boldsymbol{Q}_1^{-1}\boldsymbol{A}=\boldsymbol{R}_1$, 于是

$$\boldsymbol{A}^{(2)}=\boldsymbol{R}_1\boldsymbol{Q}_1=\boldsymbol{Q}_1^{-1}\boldsymbol{A}\boldsymbol{Q}_1$$

即 $\boldsymbol{A}^{(2)}$ 与 $\boldsymbol{A}$ 相似. 同理可得, $\boldsymbol{A}^{(k)}\sim\boldsymbol{A}\ (k=2,3,\cdots)$. 故它们有相同的特征值.

扫描全能王 创建

可以证明, 在一定条件下, 基本 QR 方法产生的矩阵序列 $\{\boldsymbol{A}^{(k)}\}$ “基本” 收敛于一个上三角形矩阵 (或分块上三角形矩阵), 即主对角线 (或主对角线子块) 及其以下元素均收敛, 主对角线（或主对角线子块）以上元素可以不收敛. 特别地, 如果 $\boldsymbol{A}$ 是实对称矩阵, 则 $\{\boldsymbol{A}^{(k)}\}$ “基本” 收敛于对角矩阵.

因为上三角形矩阵的主对角元 (或分块上三角形矩阵中, 主对角子块的特征值) 即为该矩阵的特征值, 故当 k 充分大时, $\boldsymbol{A}^{(k)}$ 的主对角元 (或主对角子块的特征值) 就可以作为 $\boldsymbol{A}$ 的特征值的近似.

基本 QR 方法的主要运算是对矩阵作 QR 分解, 分解的方法有多种, 下面以 Schmidt 正交化方法为例说明.

设 $\boldsymbol{A}$ 为 n 阶非奇异实矩阵, 记为

$$\boldsymbol{A}=[\boldsymbol{a}_1\ \boldsymbol{a}_2\ \cdots\ \boldsymbol{a}_n]$$

其中 $\boldsymbol{a}_j=(a_{1j},a_{2j},\cdots,a_{nj})^{\mathrm{T}}\,(j=1,2,\cdots,n)$, 取

$$\boldsymbol{b}_1=\boldsymbol{a}_1/\|\boldsymbol{a}_1\|,\quad \boldsymbol{b}_2'=\boldsymbol{a}_2-\langle\boldsymbol{a}_2,\boldsymbol{b}_1\rangle\boldsymbol{b}_1$$

显然 $\boldsymbol{b}_1\perp\boldsymbol{b}_2'$, 再取 $\boldsymbol{b}_2=\boldsymbol{b}_2'/\|\boldsymbol{b}_2'\|$, 则

$$\|\boldsymbol{b}_1\|=\|\boldsymbol{b}_2\|=1,\quad \langle\boldsymbol{b}_1,\boldsymbol{b}_2\rangle=0$$

一般地, 取

$$\boldsymbol{b}_k'=\boldsymbol{a}_k-\sum_{i=1}^{k-1}\langle\boldsymbol{a}_k,\boldsymbol{b}_i\rangle\boldsymbol{b}_i \tag{4.17}$$

$$\boldsymbol{b}_k=\boldsymbol{b}_k'/\left\|\boldsymbol{b}_k'\right\|\quad(k=2,3,\cdots,n)$$

则向量组 $\boldsymbol{b}_1,\boldsymbol{b}_2,\cdots,\boldsymbol{b}_n$ 正交, 且 $\|\boldsymbol{b}_k\|=1\ (k=1,2,\cdots,n)$. 式(4.17) 可改写成

$$\boldsymbol{a}_k=\langle\boldsymbol{a}_k,\boldsymbol{b}_1\rangle\boldsymbol{b}_1+\langle\boldsymbol{a}_k,\boldsymbol{b}_2\rangle\boldsymbol{b}_2+\cdots+\langle\boldsymbol{a}_k,\boldsymbol{b}_{k-1}\rangle\boldsymbol{b}_{k-1}+\left\|\boldsymbol{b}_k'\right\|\boldsymbol{b}_k$$

于是

$$\begin{aligned}\boldsymbol{A}=[\boldsymbol{a}_1\ \boldsymbol{a}_2\ \cdots\ \boldsymbol{a}_n]&=[\boldsymbol{b}_1\ \boldsymbol{b}_2\ \cdots\ \boldsymbol{b}_n]\begin{bmatrix}\|\boldsymbol{a}_1\| & \langle\boldsymbol{a}_2,\boldsymbol{b}_1\rangle & \cdots & \langle\boldsymbol{a}_{n-1},\boldsymbol{b}_1\rangle & \langle\boldsymbol{a}_n,\boldsymbol{b}_1\rangle\\ & \left\|\boldsymbol{b}_2'\right\| & \cdots & \langle\boldsymbol{a}_{n-1},\boldsymbol{b}_2\rangle & \langle\boldsymbol{a}_n,\boldsymbol{b}_2\rangle\\ & & \ddots & & \vdots\\ & & & \left\|\boldsymbol{b}_{n-1}'\right\| & \langle\boldsymbol{a}_n,\boldsymbol{b}_{n-1}\rangle\\ & & & & \left\|\boldsymbol{b}_n'\right\|\end{bmatrix}\\&=\boldsymbol{Q}\boldsymbol{R}\end{aligned}$$

这就是用 Schmidt 正交化方法对矩阵进行 QR 分解的过程.

例 4.6　用 Schmidt 正交化方法将矩阵

$$\boldsymbol{A}=\begin{bmatrix}2 & -1 & 0\\ -1 & 2 & -1\\ 0 & -1 & 2\end{bmatrix}$$

扫描全能王 创建

进行 QR 分解.

解　$\boldsymbol{a}_1=(2,-1,0)^{\mathrm{T}},\boldsymbol{a}_2=(-1,2,-1)^{\mathrm{T}},\boldsymbol{a}_3=(0,-1,2)^{\mathrm{T}}$.

因而 $\boldsymbol{b}_1=\dfrac{\boldsymbol{a}_1}{\|\boldsymbol{a}_1\|}=\left(\dfrac{2}{\sqrt{5}},-\dfrac{1}{\sqrt{5}},0\right)^{\mathrm{T}}$, 按式(4.17) 计算

$$
\begin{aligned}
\boldsymbol{b}_2'&=\boldsymbol{a}_2-\langle\boldsymbol{a}_2,\boldsymbol{b}_1\rangle\boldsymbol{b}_1\\
&=(-1,2,-1)^{\mathrm{T}}+\frac{4}{\sqrt{5}}\left(\frac{2}{\sqrt{5}},-\frac{1}{\sqrt{5}},0\right)^{\mathrm{T}}=\left(\frac{3}{5},\frac{6}{5},-1\right)^{\mathrm{T}}\\
\boldsymbol{b}_2&=\frac{\boldsymbol{b}_2'}{\|\boldsymbol{b}_2'\|}=\left(\frac{3}{\sqrt{70}},\frac{6}{\sqrt{70}},-\frac{5}{\sqrt{70}}\right)^{\mathrm{T}}\\
\boldsymbol{b}_3'&=\boldsymbol{a}_3-\langle\boldsymbol{a}_3,\boldsymbol{b}_1\rangle\boldsymbol{b}_1-\langle\boldsymbol{a}_3,\boldsymbol{b}_2\rangle\boldsymbol{b}_2\\
&=(0,-1,2)^{\mathrm{T}}-\frac{1}{\sqrt{5}}\left(\frac{2}{\sqrt{5}},-\frac{1}{\sqrt{5}},0\right)^{\mathrm{T}}+\frac{16}{\sqrt{70}}\left(\frac{3}{\sqrt{70}},\frac{6}{\sqrt{70}},-\frac{5}{\sqrt{70}}\right)^{\mathrm{T}}\\
&=\left(\frac{2}{7},\frac{4}{7},\frac{6}{7}\right)^{\mathrm{T}}\\
\boldsymbol{b}_3&=\frac{\boldsymbol{b}_3'}{\|\boldsymbol{b}_3'\|}=\left(\frac{1}{\sqrt{14}},\frac{2}{\sqrt{14}},\frac{3}{\sqrt{14}}\right)^{\mathrm{T}}
\end{aligned}
$$

所以

$$
\boldsymbol{A}=\begin{bmatrix}2/\sqrt{5}&3/\sqrt{70}&1/\sqrt{14}\\-1/\sqrt{5}&6/\sqrt{70}&2/\sqrt{14}\\0&-5/\sqrt{70}&3/\sqrt{14}\end{bmatrix}\begin{bmatrix}\sqrt{5}&-4/\sqrt{5}&1/\sqrt{5}\\0&\sqrt{70}/5&-16/\sqrt{70}\\0&0&2\sqrt{40}/7\end{bmatrix}
$$

基本 QR 方法每次迭代都需作一次 QR 分解与矩阵乘法, 计算量大, 而且收敛速度慢. 因此实际使用的 QR 方法是先用一系列相似变换将 $\boldsymbol{A}$ 化成拟上三角形矩阵 (称为上 Hessenberg(海森伯格) 矩阵), 然后对此矩阵用基本 QR 方法. 因为拟上三角形矩阵中具有较多的零元素, 这样做可望减少运算量. 化 $\boldsymbol{A}$ 为相似的拟上三角形矩阵的方法有多种, 本节介绍其中的一种: Householder 变换.

4.3.2　Householder 变换

设向量 $\boldsymbol{w}=(w_1,w_2,\cdots,w_n)^{\mathrm{T}}$ 满足 $\|\boldsymbol{w}\|_2=\sqrt{w_1^2+w_2^2+\cdots+w_n^2}=1$, 则称

$$
\boldsymbol{H}=\boldsymbol{I}-2\boldsymbol{w}\boldsymbol{w}^{\mathrm{T}}=\begin{bmatrix}1-2w_1^2&-2w_1w_2&\cdots&-2w_1w_n\\-2w_2w_1&1-2w_2^2&\cdots&-2w_2w_n\\\vdots&\vdots&&\vdots\\-2w_nw_1&-2w_nw_2&\cdots&1-w_n^2\end{bmatrix}
$$

扫描全能王 创建

为 Householder(豪斯霍尔德) 矩阵或反射矩阵. 容易证明, Householder 矩阵具有以下性质:

1. $\boldsymbol{H}$ 是实对称的正交矩阵, 即 $\boldsymbol{H}^{-1}=\boldsymbol{H}^{\mathrm{T}}=\boldsymbol{H}$;

2. $\det \boldsymbol{H}=-1$;

3. $\boldsymbol{H}$ 仅有两个不等的特征值 ± 1, 其中 1 是 $n-1$ 重特征值, -1 是单重特征值, $\boldsymbol{w}$ 为其相应的特征向量;

4. 对任意的 $\boldsymbol{x} \in (\operatorname{span}\{\boldsymbol{w}\})^{\perp}, \alpha \in \mathbf{R}$, 有

$$\boldsymbol{H}(\boldsymbol{x}+\alpha \boldsymbol{w})=\boldsymbol{x}-\alpha \boldsymbol{w}$$

从图 4.1 可以看出, $\boldsymbol{x}+\alpha \boldsymbol{w}$ 与 $\boldsymbol{H}(\boldsymbol{x}+\alpha \boldsymbol{w})$ 关于超平面 $(\operatorname{span}\{\boldsymbol{w}\})^{\perp}$ 对称, 反射变换的名称由此而来.

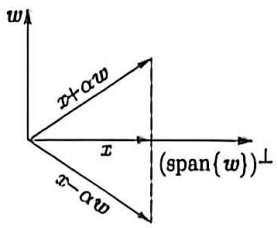


图 4.1

定理 4.2　设 $\boldsymbol{x}, \boldsymbol{y}$ 为 $\mathbf{R}^n$ 中任意非零向量, 且 $\|\boldsymbol{y}\|_2=1$, 则存在 Householder 矩阵 $\boldsymbol{H}$, 使得

$$\boldsymbol{H} \boldsymbol{x}=\pm\|\boldsymbol{x}\|_2 \boldsymbol{y} \tag{4.18}$$

证明　由性质 4 容易得出, 对任意 $\boldsymbol{x} \in \mathbf{R}^n, \boldsymbol{w}$ 与 $\boldsymbol{x}-\boldsymbol{H} \boldsymbol{x}$ 平行. 故要使式 (4.18)成立, 应取

$$\boldsymbol{w}=\frac{\boldsymbol{x}-(\pm\|\boldsymbol{x}\|_2 \boldsymbol{y})}{\left\|\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}\right\|_2}$$

令 $\boldsymbol{H}=\boldsymbol{I}-2 \boldsymbol{w} \boldsymbol{w}^{\mathrm{T}}$, 于是

$$\begin{aligned}\boldsymbol{H} \boldsymbol{x} &= \left(\boldsymbol{I}-2 \boldsymbol{w} \boldsymbol{w}^{\mathrm{T}}\right) \boldsymbol{x} \\ &= \boldsymbol{x}-2 \frac{\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}}{\left\|\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}\right\|_2^2}\left(\boldsymbol{x}^{\mathrm{T}} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}^{\mathrm{T}}\right) \boldsymbol{x}\end{aligned} \tag{4.19}$$

由 2– 范数的定义

$$\begin{aligned}\left\|\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}\right\|_2^2 &= \left(\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}\right)^{\mathrm{T}}\left(\boldsymbol{x} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}\right) \\ &= \boldsymbol{x}^{\mathrm{T}} \boldsymbol{x} \mp 2\|\boldsymbol{x}\|_2 \boldsymbol{y}^{\mathrm{T}} \boldsymbol{x}+\|\boldsymbol{x}\|_2^2 \\ &= 2\left(\boldsymbol{x}^{\mathrm{T}} \mp\|\boldsymbol{x}\|_2 \boldsymbol{y}^{\mathrm{T}}\right) \boldsymbol{x}\end{aligned}$$

将上式代入式(4.19) 即得

$$\boldsymbol{H} \boldsymbol{x}=\pm\|\boldsymbol{x}\|_2 \boldsymbol{y}$$

定理 4.2表明, 对任一非零向量 $\boldsymbol{x}$, 都可以构造一个 Householder 变换, 它将 $\boldsymbol{x}$ 变成事先给定的单位向量的倍数. 特别地, 若取 $\boldsymbol{y}=\boldsymbol{e}_i$, 则 $\boldsymbol{x}$ 经过 Householder 变换后可变成只有一个分量不为零.

实际计算时, 若 $\boldsymbol{x}=\left(x_1, x_2, \cdots, x_n\right)^{\mathrm{T}} \approx \boldsymbol{e}_i$, 则 $\boldsymbol{x}-\|\boldsymbol{x}\|_2 \boldsymbol{e}_i \operatorname{sign}\left(x_i\right) \approx \mathbf{0}$, 即

$$\left\|\boldsymbol{x}-\|\boldsymbol{x}\|_2 \boldsymbol{e}_i \operatorname{sign}\left(x_i\right)\right\|_2 \ll 1$$

从而在计算 $\boldsymbol{w}$ 时会产生较大的误差, 为此我们总取

$$\boldsymbol{w}=\frac{\boldsymbol{x}+\|\boldsymbol{x}\|_2 \boldsymbol{e}_i \operatorname{sign}\left(x_i\right)}{\left\|\boldsymbol{x}+\|\boldsymbol{x}\|_2 \boldsymbol{e}_i \operatorname{sign}\left(x_i\right)\right\|_2} \tag{4.20}$$

扫描全能王 创建

使得 Householder 矩阵 $\boldsymbol{H}=\boldsymbol{I}-2\boldsymbol{w}\boldsymbol{w}^{\mathrm{T}}$ 将 $\boldsymbol{x}$ 变成与 e_i 共线的向量, 即

$$\boldsymbol{H}\boldsymbol{x}=-\operatorname{sign}(x_i)\|\boldsymbol{x}\|_2 e_i$$

4.3.3　化一般矩阵为拟上三角形矩阵

称形如

$$\boldsymbol{H}=\begin{bmatrix} h_{11} & h_{12} & h_{13} & \cdots & h_{1,n-1} & h_{1n} \\ h_{21} & h_{22} & h_{23} & \cdots & h_{2,n-1} & h_{2n} \\ & h_{32} & h_{33} & \cdots & h_{3,n-1} & h_{3n} \\ & & & \ddots & \vdots & \vdots \\ & & & \ddots & h_{n-1,n-1} & h_{n-1,n} \\ & & & & h_{n,n-1} & h_{nn} \end{bmatrix} \tag{4.21}$$

的矩阵为拟上三角形矩阵, 也称为上 Hessenberg (海森堡) 矩阵. 如果次对角线元 $h_{i,i-1}(i=2,3,\cdots,n)$ 全不为零, 则称该矩阵为不可约的上 Hessenberg 矩阵. 下面讨论如何利用 Householder 变换将一般矩阵 $\boldsymbol{A}$ 相似变换成(4.21)的形式.

首先, 选取 Householder 矩阵 $\boldsymbol{H}_1$, 使得经 $\boldsymbol{H}_1$ 相似变换后的矩阵 $\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1$ 的第 1 列中有尽可能多的零元素. 为此, 应取 $\boldsymbol{H}_1$ 为如下形式

$$\boldsymbol{H}_1=\left[\begin{array}{c:ccc} 1 & 0 & \cdots & 0 \\ \hdashline 0 & & & \\ \vdots & & \tilde{\boldsymbol{H}}_1 & \\ 0 & & & \end{array}\right]$$

其中 $\tilde{\boldsymbol{H}}_1$ 为 $n-1$ 阶 Householder 矩阵. 于是有

$$\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1=\begin{bmatrix} a_{11} & \boldsymbol{a}_2^{\mathrm{T}}\tilde{\boldsymbol{H}}_1 \\ \tilde{\boldsymbol{H}}_1\boldsymbol{a}_1 & \tilde{\boldsymbol{H}}_1\boldsymbol{A}_{22}\tilde{\boldsymbol{H}}_1 \end{bmatrix}$$

其中

$$\boldsymbol{a}_1=(a_{21},a_{31},\cdots,a_{n1})^{\mathrm{T}},\boldsymbol{a}_2=(a_{12},a_{13},\cdots,a_{1n})^{\mathrm{T}}$$

$$\boldsymbol{A}_{22}=\begin{bmatrix} a_{22} & a_{23} & \cdots & a_{2n} \\ a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & & \vdots \\ a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}$$

由定理 4.2, 只要取 $\tilde{\boldsymbol{H}}_1$, 使得

$$\tilde{\boldsymbol{H}}_1\boldsymbol{a}_1=\alpha\underbrace{(1,0,\cdots,0)^{\mathrm{T}}}_{n-1\text{个元}} \tag{4.22}$$

就会使得变换后的矩阵 $\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1$ 的第 1 列出现 $n-2$ 个零元. 完全类似地, 可构造如下形

扫描全能王 创建

式的 Householder 矩阵

$$\boldsymbol{H}_2=\left[\begin{array}{cc:ccc}1&0&0&\cdots&0\\0&1&0&\cdots&0\\\hdashline0&0&&&\\\vdots&\vdots&&\tilde{\boldsymbol{H}}_2&\\0&0&&&\end{array}\right]$$

使得

$$\boldsymbol{H}_2\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1\boldsymbol{H}_2=\begin{bmatrix}*&*&*&\cdots&**&*&*&\cdots&*\\0&*&*&\cdots&*\\0&0&*&\cdots&*\\\vdots&\vdots&\vdots&&\vdots\\0&0&*&\cdots&*\end{bmatrix}$$

如此进行 $n-2$ 次, 可以构造 $n-2$ 个 Householder 矩阵 $\boldsymbol{H}_1,\boldsymbol{H}_2,\cdots,\boldsymbol{H}_{n-2}$ 使得

$$\boldsymbol{H}_{n-2}\cdots\boldsymbol{H}_2\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1\boldsymbol{H}_2\cdots\boldsymbol{H}_{n-2}=\boldsymbol{H}$$

其中 $\boldsymbol{H}$ 为形如式 (4.21) 的上 Hessenberg 矩阵. 特别地, 如果 $\boldsymbol{A}$ 为实对称矩阵, 则经过上述正交相似变换后所得到的矩阵 $\boldsymbol{H}$ 为三对角矩阵.

例 4.7　用 Householder 变换将矩阵

$$\boldsymbol{A}=\begin{bmatrix}5&-2&2\sqrt{2}&-3\sqrt{2}\\1&0&5\sqrt{2}/2&-\sqrt{2}/2\\0&-\sqrt{2}&1&0\\0&\sqrt{2}&-4&-1\end{bmatrix}$$

化成拟上三角形矩阵.

解　因为 $\boldsymbol{a}_1=(1,0,0)^{\mathrm{T}}$, 由式(4.22)可推得 $\boldsymbol{H}_1=\boldsymbol{I}$. 记

$$\boldsymbol{A}_{22}^{(1)}=\left[\begin{array}{c:cc}0&5\sqrt{2}/2&-\sqrt{2}/2\\\hdashline-\sqrt{2}&1&0\\\sqrt{2}&-4&-1\end{array}\right]$$

为使 Householder 矩阵 $\tilde{\boldsymbol{H}}_2$ 满足 $\tilde{\boldsymbol{H}}_2\begin{bmatrix}-\sqrt{2}\\\sqrt{2}\end{bmatrix}=\alpha\begin{bmatrix}1\\0\end{bmatrix}$, 由式 (4.22), 取

$$\boldsymbol{w}'=(-\sqrt{2},\sqrt{2})^{\mathrm{T}}-2(1,0)^{\mathrm{T}}=(-\sqrt{2}-2,\sqrt{2})^{\mathrm{T}}$$

$$\boldsymbol{w}=\frac{\boldsymbol{w}'}{\|\boldsymbol{w}'\|_2}=\left(-\frac{\sqrt{2+\sqrt{2}}}{2},\frac{\sqrt{2}}{2\sqrt{2+\sqrt{2}}}\right)^{\mathrm{T}}$$

扫描全能王 创建

$$\tilde{\boldsymbol{H}}_2 = \boldsymbol{I} - 2\boldsymbol{w}\boldsymbol{w}^{\mathrm{T}}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - 2 \begin{bmatrix} \frac{2+\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} \\ -\frac{\sqrt{2}}{4} & \frac{2-\sqrt{2}}{4} \end{bmatrix}$$

$$= \begin{bmatrix} -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$\boldsymbol{H}_2 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

于是有

$$\boldsymbol{H} = \boldsymbol{H}_2\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1\boldsymbol{H}_2$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix} \begin{bmatrix} 5 & -2 & 2\sqrt{2} & -3\sqrt{2} \\ 1 & 0 & \frac{5\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} \\ 0 & -\sqrt{2} & 1 & 0 \\ 0 & \sqrt{2} & -4 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ 0 & 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$= \begin{bmatrix} 5 & -2 & -5 & -1 \\ 1 & 0 & -3 & 2 \\ 0 & 2 & 2 & -3 \\ 0 & 0 & 1 & -2 \end{bmatrix}$$

4.3.4　拟上三角形矩阵的 QR 分解

因为拟上三角形矩阵 $\boldsymbol{H}$ 的特殊形状, 通常用 $n-1$ 个旋转变换 (又称 Givens(吉文斯) 变换) 可以将它化成上三角形矩阵, 从而得到 $\boldsymbol{H}$ 的 QR 分解式. 具体步骤为:

设 $h_{21} \neq 0$ (否则进行下一步), 取旋转矩阵

扫描全能王 创建

$$V_{21}=\begin{bmatrix}\cos\varphi_1 & \sin\varphi_1 & 0 & \cdots & 0\\ -\sin\varphi_1 & \cos\varphi_1 & 0 & \cdots & 0\\ & & 1 & \cdots & 0\\ & & & \ddots & \vdots\\ & & & & 1\end{bmatrix}$$

其中 $\cos\varphi_1=\dfrac{h_{11}}{r_1},\sin\varphi_1=\dfrac{h_{21}}{r_1},r_1=\sqrt{h_{11}^2+h_{21}^2}$, 则必有

$$V_{21}H=\begin{bmatrix}r_1 & h_{12}^{(2)} & h_{13}^{(2)} & \cdots & h_{1,n-1}^{(2)} & h_{1n}^{(2)}\\ 0 & h_{22}^{(2)} & h_{23}^{(2)} & \cdots & h_{2,n-1}^{(2)} & h_{2n}^{(2)}\\ 0 & h_{32}^{(2)} & h_{33}^{(2)} & \cdots & h_{3,n-1}^{(2)} & h_{3n}^{(2)}\\ & & \ddots & & \vdots & \vdots\\ & & & & h_{n,n-1}^{(2)} & h_{nn}^{(2)}\end{bmatrix}=H^{(2)}$$

设 $h_{32}^{(2)}\neq 0$ (否则进行下一步), 再取

$$V_{32}=\begin{bmatrix}1 & 0 & 0 & & & \\ 0 & \cos\varphi_2 & \sin\varphi_2 & & & \\ 0 & -\sin\varphi_2 & \cos\varphi_2 & & & \\ & & & 1 & & \\ & & & & \ddots & \\ & & & & & 1\end{bmatrix}$$

其中 $\cos\varphi_2=\dfrac{h_{22}^{(2)}}{r_2},\sin\varphi_2=\dfrac{h_{32}^{(2)}}{r_2},r_2=\sqrt{\left(h_{22}^{(2)}\right)^2+\left(h_{32}^{(2)}\right)^2}$, 则

$$V_{32}H^{(2)}=\begin{bmatrix}r_1 & h_{12}^{(3)} & h_{13}^{(3)} & \cdots & h_{1,n-1}^{(3)} & h_{1n}^{(3)}\\ & r_2 & h_{23}^{(3)} & \cdots & h_{2,n-1}^{(3)} & h_{2n}^{(3)}\\ & & h_{33}^{(3)} & \cdots & h_{3,n-1}^{(3)} & h_{3n}^{(3)}\\ & & h_{43}^{(3)} & \cdots & h_{4,n-1}^{(3)} & h_{4n}^{(3)}\\ & & & \ddots & \vdots & \vdots\\ & & & & h_{n,n-1}^{(3)} & h_{nn}^{(3)}\end{bmatrix}=H^{(3)}$$

假设上述过程已经进行了 $k-1$ 步, 有

$$H^{(k)}=V_{k,k-1}H^{(k-1)}=\left[\begin{array}{ccc:ccc}r_1 & \cdots & h_{1,k-1}^{(k)} & h_{1k}^{(k)} & \cdots & h_{1,n-1}^{(k)} & h_{1n}^{(k)}\\ & \ddots & & \vdots & & \vdots & \vdots\\ & & r_{k-1} & h_{k-1,k}^{(k)} & \cdots & h_{k-1,n-1}^{(k)} & h_{k-1,n}^{(k)}\\ \hdashline & & & h_{kk}^{(k)} & \cdots & h_{k,n-1}^{(k)} & h_{kn}^{(k)}\\ & & & h_{k+1,k}^{(k)} & \cdots & h_{k+1,n-1}^{(k)} & h_{k+1,n}^{(k)}\\ & & & & \ddots & \vdots & \vdots\\ & & & & & h_{n,n-1}^{(k)} & h_{nn}^{(k)}\end{array}\right]$$

扫描全能王 创建

设 $h_{k+1,k}^{(k)} \neq 0$, 取

$$\boldsymbol{V}_{k+1,k} = \begin{bmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ & & & \cos\varphi_k & \sin\varphi_k & & & \\ & & & -\sin\varphi_k & \cos\varphi_k & & & \\ & & & & & 1 & & \\ & & & & & & \ddots & \\ & & & & & & & 1 \end{bmatrix} \begin{matrix} \\ \\ \\ k \\ k+1 \\ \\ \\ \\ \end{matrix} \tag{4.23}$$

其中 $\cos\varphi_k = h_{kk}^{(k)}/r_k, \sin\varphi_k = h_{k+1,k}^{(k)}/r_k, r_k = \sqrt{\left(h_{kk}^{(k)}\right)^2 + \left(h_{k+1,k}^{(k)}\right)^2}$, 于是

$$\boldsymbol{V}_{k+1,k}\boldsymbol{H}^{(k)} = \left[\begin{array}{ccc:ccc} r_1 & \cdots & h_{1,k}^{(k+1)} & h_{1,k+1}^{(k+1)} & \cdots & h_{1,n}^{(k+1)} \\ & \ddots & & \vdots & & \vdots \\ & & r_k & h_{k,k+1}^{(k+1)} & \cdots & h_{k,n}^{(k+1)} \\ \hdashline & & & h_{k+1,k+1}^{(k+1)} & \cdots & h_{k+1,n}^{(k+1)} \\ & & & h_{k+2,k}^{(k+1)} & \cdots & h_{k+2,n}^{(k+1)} \\ & & & & \ddots & \vdots \\ & & & & & h_{n,n}^{(k+1)} \end{array}\right] = \boldsymbol{H}^{(k+1)}$$

因此, 最多作 $n-1$ 次旋转变换, 即得

$$\begin{aligned} \boldsymbol{H}^{(n)} &= \boldsymbol{V}_{n,n-1}\boldsymbol{V}_{n-1,n-2}\cdots\boldsymbol{V}_{21}\boldsymbol{H} \\ &= \begin{bmatrix} r_1 & h_{12}^{(n)} & h_{13}^{(n)} & \cdots & h_{1n}^{(n)} \\ & r_2 & h_{23}^{(n)} & \cdots & h_{2n}^{(n)} \\ & & \ddots & & \vdots \\ & & & & r_n \end{bmatrix} = \boldsymbol{R} \end{aligned}$$

因为 $\boldsymbol{V}_{i,i-1}(i=2,3,\cdots,n)$ 均为正交矩阵, 故

$$\boldsymbol{H} = \boldsymbol{V}_{21}^{\mathrm{T}}\boldsymbol{V}_{32}^{\mathrm{T}}\cdots\boldsymbol{V}_{n,n-1}^{\mathrm{T}}\boldsymbol{R} = \boldsymbol{Q}\boldsymbol{R}$$

其中 $\boldsymbol{Q} = \boldsymbol{V}_{21}^{\mathrm{T}}\boldsymbol{V}_{32}^{\mathrm{T}}\cdots\boldsymbol{V}_{n,n-1}^{\mathrm{T}}$ 仍为正交矩阵. 容易算出完成这一过程的运算量约为 $4n^2$, 比一般矩阵的 QR 分解的运算量 $O(n^3)$ 少了一个数量级.

不难证明, $\tilde{\boldsymbol{H}} = \boldsymbol{Q}\boldsymbol{R}$ 仍是拟上三角形矩阵, 于是可以按上述步骤一直迭代下去, 这样得到的 QR 方法的运算量比基本 QR 方法大为减少. 需要说明的是, 我们通常用 QR 方法计算特征值, 然后用反幂法求其相应的特征向量.

例 4.8　用 QR 方法求矩阵

扫描全能王 创建

$$\boldsymbol{A}=\begin{bmatrix}5 & -3 & 2\\ 6 & -4 & 4\\ 4 & -4 & 5\end{bmatrix}$$

的全部特征值.

解　首先将 $\boldsymbol{A}$ 化成拟上三角形矩阵. 取

$$\boldsymbol{w}'=(6,4)^{\mathrm{T}}+\sqrt{6^2+4^2}(1,0)^{\mathrm{T}}=(6+2\sqrt{13},4)^{\mathrm{T}}$$

$$\boldsymbol{w}=\boldsymbol{w}'/\|\boldsymbol{w}'\|_2=(0.957\,092,0.289\,784)^{\mathrm{T}}$$

$$\begin{aligned}\boldsymbol{I}-2\boldsymbol{w}\boldsymbol{w}^T&=\begin{bmatrix}1 & 0\\ 0 & 1\end{bmatrix}-2\begin{bmatrix}0.916\,025 & 0.277\,350\\ 0.277\,350 & 0.083\,974\,7\end{bmatrix}\\&=\begin{bmatrix}-0.832\,050 & -0.554\,700\\ -0.554\,700 & 0.832\,050\end{bmatrix}\end{aligned}$$

于是

$$\boldsymbol{H}_1=\begin{bmatrix}1 & 0 & 0\\ 0 & -0.832\,050 & -0.554\,700\\ 0 & -0.554\,700 & 0.832\,050\end{bmatrix}$$

$$\boldsymbol{H}=\boldsymbol{H}_1\boldsymbol{A}\boldsymbol{H}_1=\begin{bmatrix}5 & 1.386\,750 & 3.328\,200\\ -7.211\,102 & -1.230\,768 & -8.153\,840\\ 0 & -0.153\,846 & 2.230\,767\end{bmatrix}$$

$\boldsymbol{H}$ 即为与 $\boldsymbol{A}$ 相似的拟上三角形矩阵. 下面将 $\boldsymbol{H}$ 进行 QR 分解, 记 $\boldsymbol{H}^{(1)}=\boldsymbol{H}$, 按式 (4.23)

$$r_1=\sqrt{5^2+(-7.211\,102)^2}=8.774\,964$$

$$\cos\varphi_1=\frac{5}{r_1}=0.569\,803,\quad \sin\varphi_1=\frac{-7.211\,102}{r_1}=-0.821\,781$$

取

$$\boldsymbol{V}_{21}=\begin{bmatrix}0.569\,803 & -0.821\,781 & 0\\ 0.821\,781 & 0.569\,803\,0 & \\ 0 & 0 & 1\end{bmatrix}$$

于是

$$\boldsymbol{V}_{21}\boldsymbol{H}^{(1)}=\begin{bmatrix}8.774\,964 & 1.801\,596 & 8.597\,089\\ 0 & 0.438\,310 & -1.911\,030\\ 0 & -0.153\,846 & 2.230\,767\end{bmatrix}$$

扫描全能王 创建

再取

$$r_2=\sqrt{(0.438\ 310)^2+(-0.153\ 846)^2}=0.464\ 526$$

$$\cos\varphi_2=\frac{0.438\ 310}{r_2}=0.943\ 564,\quad \sin\varphi_2=\frac{-0.153\ 846}{r_2}=-0.331\ 189$$

$$\boldsymbol{V}_{32}=\begin{bmatrix}1 & 0 & 0\\ 0 & 0.943\ 564 & -0.331\ 189\\ 0 & 0.331\ 189 & 0.943\ 564\end{bmatrix}$$

于是

$$\boldsymbol{V}_{32}\boldsymbol{V}_{21}\boldsymbol{H}^{(1)}=\begin{bmatrix}8.774\ 964 & 1.801\ 596 & 8.597\ 089\\ 0 & 0.464\ 526 & -2.541\ 982\\ 0 & 0 & 1.471\ 953\end{bmatrix}=\boldsymbol{R}_1$$

$$\boldsymbol{Q}_1=\boldsymbol{V}_{21}^{\mathrm{T}}\boldsymbol{V}_{32}^{\mathrm{T}}=\begin{bmatrix}0.569\ 803 & 0.775\ 403 & 0.272\ 165\\ -0.821\ 781 & 0.537\ 643 & 0.188\ 712\\ 0 & -0.331\ 189 & 0.943\ 564\end{bmatrix}$$

第一次迭代得

$$\boldsymbol{H}^{(2)}=\boldsymbol{R}_1\boldsymbol{Q}_1\begin{bmatrix}3.519\ 482 & 4.925\ 491 & 10.840\ 117\\ -0.381\ 739 & 1.091\ 627 & -2.310\ 653\\ 0 & -0.487\ 495 & 1.388\ 883\end{bmatrix}$$

重复上述过程, 迭代 11 次得

$$\boldsymbol{H}^{(12)}=\begin{bmatrix}2.992\ 032 & -1.003\ 853 & 12.013\ 392\\ -0.007\ 496 & 2.004\ 695 & 1.941\ 971\\ 0 & -0.000\ 325 & 0.999\ 895\end{bmatrix}$$

所以 $\boldsymbol{A}$ 的特征值为

$$\lambda_1\approx 2.992\ 032,\quad \lambda_2\approx 2.004\ 695,\quad \lambda_3\approx 0.999\ 895$$

(准确值为 $\lambda_1=3,\lambda_2=2,\lambda_3=1$), 此时, $\boldsymbol{H}^{(12)}$ 的下三角非对角元的最大模为 0.007 496, 故 QR 方法“基本”收敛速度较慢.

4.3.5　带原点移位的 QR 方法

理论分析和实际计算均表明, QR 方法产生的矩阵序列 $\{\boldsymbol{H}^{(k)}\}$ 的右下角对角元 $h_{nn}^{(k)}$ 最先与 $\boldsymbol{A}$ 的特征值接近. 可以证明, 若矩阵 $\boldsymbol{A}$ 的特征值满足

$$|\lambda_1|\geqslant|\lambda_2|\geqslant\cdots\geqslant|\lambda_{n-1}|>|\lambda_n|$$

扫描全能王 创建

则 $\{\boldsymbol{H}^{(k)}\}$ 的右下角对角元 $h_{nn}^{(k)} \to \lambda_n (k \to \infty)$, 且收敛速度是线性的, 速率为 $\left|\dfrac{\lambda_n}{\lambda_{n-1}}\right|$. 于是考虑引入原点移位的技巧来加快收敛速度, 即取位移 μ, 使其满足

$$|\lambda_1 - \mu| \geqslant |\lambda_2 - \mu| \geqslant \cdots \geqslant |\lambda_{n-1} - \mu| > |\lambda_n - \mu|$$

且

$$\left|\frac{\lambda_n - \mu}{\lambda_{n-1} - \mu}\right| \ll 1$$

这样, 对 $\boldsymbol{H} - \mu \boldsymbol{I}$ 用 QR 方法就可以加快收敛速度, 这就是带原点移位的 QR 方法. 其具体步骤为:

1. 用 Householder 变换将矩阵 $\boldsymbol{A}$ 化成拟上三角形矩阵 $\boldsymbol{H}$;

2. 对 $k = 1, 2, \cdots$ 取位移 μ_k, 将 $\boldsymbol{H}^{(k)} - \mu_k \boldsymbol{I}$ 进行 QR 分解

$$\boldsymbol{H}^{(k)} - \mu_k \boldsymbol{I} = \boldsymbol{Q}_k \boldsymbol{R}_k$$

3. $\boldsymbol{H}^{(k+1)} = \boldsymbol{R}_k \boldsymbol{Q}_k + \mu_k \boldsymbol{I}$

关于每次迭代时位移 μ_k 的取法及 QR 方法的迭代准则, 可参看文献 [3].

至今为止, QR 方法仍是求解实矩阵特征值问题的最有效的数值方法, 它几乎可以适用于求所有矩阵的全部特征值和特征向量. 基本 QR 方法收敛较慢, 其收敛速度是线性的. 带原点移位的 QR 方法具有二次收敛性, 它主要用于特征值全为实数的矩阵特征值计算.

如果 $\boldsymbol{A}$ 为实对称矩阵, 在用 QR 方法求其特征值时, 充分利用对称性, 又引出了对称 QR 方法. 这是目前求解中小型稠密实对称矩阵的特征值与特征向量的最有效方法.

对步隐式 QR 方法的基本思想是: 先把原点移位量扩充到复数域, 每次迭代作两次原点移位的 QR 变换, 然后把它们结合在一起, 转化成实数运算. 此方法适用于求实矩阵的复数特征值, 计算效率很高, 在某些条件下可以达到三阶收敛性.

4.4 算法实现与应用实例

常用命令

北太天元软件中提供了有关特征值计算和矩阵分解相关的函数, 例如

≫ e = eig(A)

% 返回方阵 A 的特征值组成的向量

≫ [V, D] = eig(A)

% 返回以方阵 A 的特征值为对角线元素的对角矩阵 D, 右特征 (列) 向量组成的矩阵 V, 其中 A*V=V*D

≫ [V, D, W] = eig(A)

% 返回以方阵 A 的特征值为对角线元素的对角矩阵 D, 右特征 (列) 向量组成的矩阵 V 和左特征 (列) 向量组成的矩阵 W, 其中 W'*A=D*W'

≫ e = eig(A,B)

扫描全能王 创建

% 返回方阵 A 和 B 的广义特征值组成的对角矩阵

≫ [V, D] = eig(A,B)

% 返回方阵 A 和 B 的广义特征值组成的对角矩阵 D, 右特征 (列) 向量组成的矩阵 V, 其中 A*V=B*V*D

≫ [V, D, W] = eig(A,B)

% 返回方阵 A 和 B 的广义特征值组成的对角矩阵 D, 右特征 (列) 向量组成的矩阵 V 和左特征 (列) 向量组成的矩阵 W, 其中 W'*A=D*W'*B

≫ X = qr(A)

% 返回 QR 分解 A = Q*R 的上三角形矩阵 R 因子.

≫ [Q,R] = qr(A)

% 对 m×n 矩阵 A 执行 QR 分解, 满足 A = Q*R, 因子 R 是 m×n 上三角形矩阵, 因子 Q 是 m×m 正交矩阵

≫[Q,R,P] = qr(A)

% 返回一个置换矩阵 P, 满足 A*P = Q*R

应用实例

化学吸收过程

继续考虑第二章的化学吸收过程. 由于在每一步中, 液体吸收某成分的总量满足一阶常微分方程组, 该微分方程组可用矩阵形式表示为

$$\frac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t}=\boldsymbol{A}\boldsymbol{x}-\boldsymbol{f}$$

其中

$$\boldsymbol{A}=\begin{bmatrix}-(1+\delta) & 1 & 0 & 0 & 0\\ \delta & -(1+\delta) & 1 & 0 & 0\\ 0 & \delta & -(1+\delta) & 1 & 0\\ 0 & 0 & \delta & -(1+\delta) & 1\\ 0 & 0 & 0 & \delta & -(1+\delta)\end{bmatrix},\quad \boldsymbol{f}=\begin{bmatrix}K(b-y_f)\\ 0\\ 0\\ 0\\ -x_f\end{bmatrix}$$

在前面的讨论中, 我们设导数向量 $\dfrac{\mathrm{d}\boldsymbol{x}}{\mathrm{d}t}=\boldsymbol{0}$, 通过求解线性方程组, 获得稳定态时的吸收量. 但这一静态分析不能提供关于吸收过程的有价值的信息. 为对整个吸收过程有一个全面的了解, 需要求瞬态解, x_f, y_f 是常数时, 利用常数变异法可以得到瞬态解的形式为

$$\begin{aligned}\boldsymbol{x}(t)&=\exp(\boldsymbol{A}t)\boldsymbol{x}(0)+[\boldsymbol{I}-\exp(\boldsymbol{A}t)]\boldsymbol{A}^{-1}\boldsymbol{f}\\ &=\exp(\boldsymbol{A}t)(\boldsymbol{x}(0)-\boldsymbol{A}^{-1}\boldsymbol{f})+\boldsymbol{A}^{-1}\boldsymbol{f}\end{aligned}$$

为了分析瞬态解, 需求解系数矩阵 $\boldsymbol{A}$ 的特征值. 设 $\boldsymbol{D}=(\lambda_1,\lambda_2,\ \lambda_3,\lambda_4,\lambda_5)^{\mathrm{T}}$ 和 $\boldsymbol{U}=[\boldsymbol{u}_1,\boldsymbol{u}_2,\boldsymbol{u}_3,\boldsymbol{u}_4,\boldsymbol{u}_5]$ 分别由矩阵 $\boldsymbol{A}$ 的特征值和特征向量组成, 记 $\boldsymbol{x}_s=\boldsymbol{A}^{-1}\boldsymbol{f}$ 表示稳定解. 如果特征值已知, 则解可以表示为

扫描全能王 创建

$$x(t)=\sum_{k=1}^{5}c_k\exp(\lambda_k t)\,u_k+x_s$$

其中向量 $c=(c_1,c_2,c_3,c_4,c_5)^{\mathrm{T}}$ 依赖于初值条件 $x(0)$, 系数矩阵 A 和右端项 f. 上述表达式的前五项称为系统的自然模, 是解的瞬时部分, 主要决定解随时间的变化规律. 令 $t=0$, 可得向量 c 满足的线性方程组

$$Uc=x(0)-x_s$$

设第 k 个特征根被表示为

$$\lambda_k=a_k+\mathrm{i}b_k\quad(k=1,2,3,4,5)$$

a_k,b_k 分别是实部和虚部. 当时间增加时, 要使解的瞬时部分趋于零, 必须对于所有的 k 有 $a_k<0$, 即所有的特征值都必须位于复平面的左半平面内. 进一步地, 系统的解 $x(t)$ 趋于 x_s 的速度也由特征值确定. 若关心解的瞬时部分以怎样的速率衰变, 注意到第 k 个自然模以速率 $\exp(a_k)$ 趋于零, 假设衰变率为 $p/100$, 解得第 k 个自然模衰变时间

$$T_k=\frac{\ln(0.01p)}{a_k},\quad 1\leqslant k\leqslant 5$$

在整个吸收过程中, 解趋于它的稳定解的速度由衰变时间的最大值 $T_{\max}=\{T_k\}$ 决定.

所有参数选取同第二章, 并设瞬时衰变比例是 $p=1\%$, 编写求特征值、每个模的衰变时间及形式解中向量 c 的脚本 exp2.m 如下:

```
n=5; a=0.72 ; b=0.0; K=1.63; xf=0.01;yf=0.06; delta=a*K;    % 赋值
d=-(1+delta)*ones(n,1)    % 对角线元素
A=diag(d)+diag(ones(n-1,1),1)+diag(delta*ones(n-1,1), -1);    % 系数矩阵是三对角矩阵
f=zeros(n,1) ;    % 右端项, 第一和最后一个元素需要单独赋值
f(1)=K*(b-yf); f(n)=-xf;
x=A\f;    % 稳态解
x0=[0.0573 0.0662 0.0495 0.0425 0.0257 ]';    % x(t) 的初始值
e=eig(A)'    % 求特征值
[U D ] =eig(A);    % 求特征值和特征向量
T=log(0.01)./e    % 计算每个自然模的衰变时间
c=(U\(x0-x))'    % 计算瞬时衰变项系数
```

取初始值 $x(0)=$ [0.0573 0.0662 0.0495 0.0425 0.0257], 运行结果为

e = −4.0500 −0.2972 −1.0903 −2.1736 −3.2569

T = 1.1371 15.4941 4.2239 2.1187 1.4140

c = 0.010023681324195 0.010041927409977 0.009969653868603 0.010029567003990 0.010038892844931

可以得到衰变时间 $T_{\max}=15.494\,1$.

扫描全能王 创建

评注四

幂法是求矩阵最大模的特征值和相应特征向量的有效而又简单的方法, 特别适用于大型矩阵或稀疏矩阵, 也是计算矩阵谱半径的有效方法. 但它的收敛速度是线性的, 令人不太满意, 一般用原点移位法或 Aitken 外推加速技术加速收敛. 反幂法是计算矩阵最小模的特征值及其特征向量的有效方法, 它的计算过程需要求解线性方程组, 一般采用直接三角分解法. 结合原点移位法, 反幂法可以修正某一指定的特征值及特征向量. 若知道矩阵特征值的大致分布情况, 带移位的反幂法可以求出每一个特征值及其特征向量.

Jacobi 方法是一个经典方法, 大多数情况下它与对称矩阵的 QR 方法相比, 没有多大的优势, 但当矩阵几乎接近对角形时, Jacobi 方法仍是很有效的方法, 而且它便于并行计算, 近来又引起人们的关注.

QR 方法是至今最有效的求矩阵特征值的方法, 它可以求出全部特征值与特征向量. 当矩阵 $\boldsymbol{A}$ 是对称矩阵时, 可以找到正交矩阵 $\boldsymbol{Q}$ 使得 $\boldsymbol{Q}^{\mathrm{T}}\boldsymbol{A}\boldsymbol{Q}=\boldsymbol{T}$ 为三对角矩阵, 使得存储空间和计算量都大为降低, 从而 QR 方法的计算公式可以简单得多 [18]. 一般情况下, QR 方法的收敛速度依赖于特征值的比, 且是一阶收敛的. 带原点位移的 QR 方法具有二阶精度. 对于对称矩阵, 适当选择位移量, 可以达到三阶收敛. 此外还有利用实 Schur(舒尔) 分解所获得的不变子空间, 求矩阵特征值和特征向量的方法 [19].

本章知识要点 幂法; 反幂法;Jacobi 方法.

算法历史及背后的数学家

习题四

1. 用幂法计算下列矩阵的按模最大的特征值及相应的特征向量, 取 $\boldsymbol{x}^{(0)}=(1,0,0)^{\mathrm{T}}$:

$$(1)\begin{bmatrix}-4 & 14 & 0\\ -5 & 13 & 0\\ -1 & 0 & 2\end{bmatrix};\quad (2)\begin{bmatrix}4 & -1 & 1\\ -1 & 3 & -2\\ 1 & -2 & 3\end{bmatrix}.$$

2. 用带 Aitken 加速的幂法计算第 1 题中矩阵的最大模特征值及相应的特征向量, 要求结果有 4 位有效数字.

3. 求矩阵

$$\boldsymbol{A}=\begin{bmatrix}1 & 2 & 3\\ 2 & 3 & 4\\ 3 & 4 & 5\end{bmatrix}$$

的接近 9.6 的特征值及相应的特征向量.

4. 用反幂法计算矩阵

扫描全能王 创建

$$\boldsymbol{A}=\begin{bmatrix}2&1&0\\1&3&1\\0&1&4\end{bmatrix}$$

相应于特征值 1.267 9 的特征向量.

5. 已知矩阵

$$\boldsymbol{A}=\begin{bmatrix}6&-12&6\\-21&-3&24\\-12&-12&51\end{bmatrix}$$

(1) 用反幂法求 $\boldsymbol{A}$ 的按模最小的特征值, 迭代 2 次, 计算中取 4 位有效数字.

(2) 以 (1) 的计算结果为近似值, 再用反幂法改进这个特征值, 并求相应的特征向量.

6. 设矩阵 $\boldsymbol{A}$ 的特征值满足

$$\lambda_1>\lambda_2\geqslant\lambda_3\geqslant\cdots\geqslant\lambda_n$$

求证当取 $\lambda_0=\dfrac{\lambda_2+\lambda_n}{2}$ 时, 用原点移位法求 λ_1 的收敛速度最快.

7. 设 $\boldsymbol{A}$ 为 n 阶实对称矩阵. 对任意非零向量 $\boldsymbol{x}\in\mathbf{R}^n$, 称 $\dfrac{\langle\boldsymbol{Ax},\boldsymbol{x}\rangle}{\langle\boldsymbol{x},\boldsymbol{x}\rangle}=\dfrac{\boldsymbol{x}^{\mathrm{T}}\boldsymbol{Ax}}{\boldsymbol{x}^{\mathrm{T}}\boldsymbol{x}}$ 为 $\boldsymbol{A}$ 的 Rayleigh (瑞利) 商, 求证:

$$\min_{\boldsymbol{x}\neq\boldsymbol{0}}\frac{\langle\boldsymbol{Ax},\boldsymbol{x}\rangle}{\langle\boldsymbol{x},\boldsymbol{x}\rangle}=\min\lambda(\boldsymbol{A}),\quad\max_{\boldsymbol{x}\neq\boldsymbol{0}}\frac{\langle\boldsymbol{Ax},\boldsymbol{x}\rangle}{\langle\boldsymbol{x},\boldsymbol{x}\rangle}=\max\lambda(\boldsymbol{A})$$

8. 用 Jacobi 方法求下列矩阵的全部特征值:

(1) $\begin{bmatrix}4&2&2\\2&5&1\\2&1&6\end{bmatrix}$; (2) $\begin{bmatrix}2&-1&0\\-1&-2&-1\\0&-1&2\end{bmatrix}$.

9. 用 Householder 变换将下列矩阵化成相似的拟上三角形矩阵:

(1) $\begin{bmatrix}-4&-3&-7\\2&3&2\\4&2&7\end{bmatrix}$; (2) $\begin{bmatrix}4&3&0&4\\3&5&-1&2\\0&-1&2&1\\4&2&1&-1\end{bmatrix}$.

10. 设 λ 是实对称矩阵 $\boldsymbol{A}$ 的特征值, 相应的特征向量为 $\boldsymbol{u}$, 且 $\|\boldsymbol{u}\|_2=1$, 若存在正交矩阵 $\boldsymbol{P}$, 使得 $\boldsymbol{Pu}=\boldsymbol{e}_1=(1,0,\cdots,0)^{\mathrm{T}}$, 求证矩阵 $\boldsymbol{B}=\boldsymbol{PAP}^{\mathrm{T}}$ 的第 1 行和第 1 列除 λ 外, 其余元素均为零.

11. 用 QR 方法求下列矩阵的全部特征值:

(1) $\begin{bmatrix}2&1&0\\1&3&1\\0&1&4\end{bmatrix}$; (2) $\begin{bmatrix}3&-1&0\\-1&2&-1\\0&-1&1\end{bmatrix}$.

扫描全能王 创建

数值实验四

1. 设 $\boldsymbol{A}=\begin{bmatrix} 6 & -4 & 18 \\ 20 & -6 & -6 \\ 22 & -22 & 11 \end{bmatrix}$, 取 $\boldsymbol{x}^{(0)}=(1,1,1)^{\mathrm{T}}$, 求 $\boldsymbol{A}$ 的按模最大的特征值和特征向量, 要求误差小于 10^{-6}.

(1) 直接用幂法计算, 出现什么现象, 说明原因;

(2) 用归一化的幂法计算.

2. 设 $\boldsymbol{A}=\begin{bmatrix} 12 & 6 & -6 \\ 6 & 16 & 2 \\ -6 & 2 & 16 \end{bmatrix}$, 取 $\boldsymbol{x}^{(0)}=(1,1,1)^{\mathrm{T}}$, 先用幂法迭代 3 次, 得到 $\boldsymbol{A}$ 的按模最大特征值的近似值, 取 λ^* 为其整数部分, 再用反幂法计算 $\boldsymbol{A}$ 的按模最大特征值的更精确的近似值, 要求误差小于 10^{-10}.

3. 根据习题 7 介绍的 Rayleigh 商方法, 对于数值实验题 2 中的矩阵 $\boldsymbol{A}$, 用归一化的幂法所得向量 $\{\boldsymbol{x}^{(k)}\}$ 的 Rayleigh 商作为 $\boldsymbol{A}$ 的按模最大特征值的近似值, 将计算结果与归一化的幂法得到的结果进行比较, 并说明原因.

4. 已知矩阵

$$\begin{bmatrix} 190 & 66 & -84 & 30 \\ 66 & 303 & 42 & -36 \\ 336 & -168 & 147 & -112 \\ 30 & -36 & 28 & 291 \end{bmatrix}$$

(1) 用幂法求矩阵的按模最大的特征值与特征向量;

(2) 用反幂法求矩阵的按模最小的特征值与特征向量;

(3) 用 QR 方法求矩阵的所有特征值与特征向量, 并用函数 [V,D]= eig(A) 检验你的结果.

扫描全能王 创建

/第五章/

插值法

生产实践中常常出现这样的问题: 给出一批离散样点, 要求作出一条通过这些点的光滑曲线, 以便满足设计要求或进行加工. 反映在数学上, 即已知函数在一些点上的值, 寻求它的分析表达式. 因为由函数的表格形式不能直接得出表中未列点处的函数值, 也不便于研究函数的性质. 此外, 有些函数虽有表达式, 但因式子复杂, 不易计算其值和进行理论分析, 也需要构造一个简单函数来近似它.

解决这种问题的方法有两类: 一类是给出函数 $f(x)$ 的一些样点值, 选定一个便于计算的函数形式, 如多项式、分式线性函数及三角多项式等, 要求它通过已知样点, 由此确定函数 $\varphi(x)$ 作为 $f(x)$ 的近似. 这就是插值法. 另一类方法在选定近似函数的形式后, 不要求近似函数过已知样点, 只要求在某种意义下它在这些点上的总偏差最小. 这类方法称为曲线(数据) 拟合法.

本章主要讨论构造插值多项式的几种常用方法及其误差.

5.1 Lagrange 插值

5.1.1 多项式插值

用插值法求函数的近似表达式时, 首先要选定近似函数的形式. 可供选择的函数很多, 常用的是多项式函数. 因为多项式函数计算简便, 只需用加、减、乘等运算, 而且其导数与积分仍为多项式. 用多项式作为研究插值的工具, 称为代数插值.

定义 5.1 已知函数 $f(x)$ 在区间 $[a,b]$ 上 $n+1$ 个不同点 $x_0,x_1,\cdots,x_n$ 处的函数值 $y_i=f(x_i)\,(i=0,1,\cdots,n)$, 求一个至多 n 次的多项式

$$\varphi_n(x)=a_0+a_1x+\cdots+a_nx^n \tag{5.1}$$

使其在给定点处与 $f(x)$ 同值, 即满足插值条件

$$\varphi_n(x_i)=f(x_i)=y_i \quad (i=0,1,\cdots,n) \tag{5.2}$$

$\varphi_n(x)$ 称为插值多项式,$x_i(i=0,1,\cdots,n)$ 称为插值节点, 简称节点,$[a,b]$ 称为插值区间.

从几何上看 (如图 5.1 所示),n 次多项式插值就是过 $n+1$ 个点 $(x_i,f(x_i))\,(i=0,1,\cdots,n)$, 作一条多项式曲线 $y=\varphi_n(x)$ 近似曲线 $y=f(x)$.n 次多项式 (5.1)有 $n+1$ 个待定系数, 由插值条件式 (5.2) 恰好给出 $n+1$ 个方程

扫描全能王 创建

$$\begin{cases} a_0 + a_1 x_0 + a_2 x_0^2 + \cdots + a_n x_0^n = y_0 \\ a_0 + a_1 x_1 + a_2 x_1^2 + \cdots + a_n x_1^n = y_1 \\ \cdots\cdots\cdots\cdots \\ a_0 + a_1 x_n + a_2 x_n^n + \cdots + a_n x_n^n = y_n \end{cases} \tag{5.3}$$

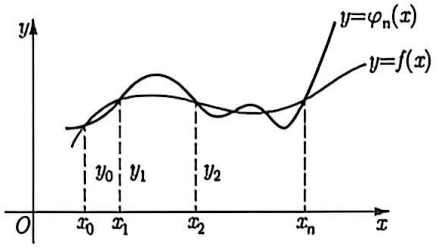


图 5.1　插值函数示意图

记此方程组的系数矩阵为 $\boldsymbol{A}$, 则

$$\det(\boldsymbol{A}) = \begin{vmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^n \\ 1 & x_1 & x_1^2 & \cdots & x_1^n \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^n \\ 1 & x_n & x_n^2 & \cdots & x_n^n \end{vmatrix}$$

是 Vandermonde(范德蒙德) 行列式. 当 $x_0, x_1, \cdots, x_n$ 互不相同时, 此行列式值不为零. 对于满足插值条件的多项式, 我们关心三个问题:

1: 插值多项式是否存在唯一?

2: 插值多项式对原函数的近似效果如何? 即误差估计.

3: 如何给出插值多项式?

实际上, 可以从两个角度证明满足插值要求(5.2) 式的插值多项式(5.1)的存在唯一性.

注 5.1　(1) Vandermonde 行列式非零, 方程组(5.3) 有唯一解. 这表明, 只要 $n+1$ 个节点互不相同, 满足插值要求式 (5.2) 的插值多项式 (5.1) 存在唯一.

(2) 假设方程组 (5.3) 对应的齐次方程组有非零解, 则说明 $n+1$ 个点 $x_i\,(i = 0, 1, \cdots, n)$ 是 n 阶多项式方程 $\varphi_n(x) = 0$ 的根, 根据代数学基本定理, 这是不可能的, 所以齐次方程只有零解, 方程(5.3) 解存在唯一.

5.1.2　插值多项式的误差估计

定义 5.2　插值多项式与被插函数之间的差

$$R_n(x) = f(x) - \varphi_n(x)$$

称为截断误差, 又称为插值余项.

定理 5.1　设 $x_0, x_1, \cdots, x_n$ 是区间 $[a, b]$ 上的互异节点,$\varphi_n(x)$ 是过这组节点的 n 次插值多项式. 如果 $f(x)$ 在 $[a, b]$ 上 $n+1$ 次连续可导, 则对 $[a, b]$ 内任意点 x, 插值余项为

扫描全能王 创建

$$R_n(x)=\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x),\quad \xi\in(a,b) \tag{5.4}$$

证明 设

$$R_n(x)=f(x)-\varphi_n(x)=K(x)\omega_{n+1}(x)$$

其中 $\omega_{n+1}(x)=\prod_{j=0}^{n}(x-x_j)$, 由插值条件式(5.2),$R_n(x_i)=0(i=0,1,\cdots,n)$. 因为 $f(x)$ 在区间 $[a,b]$ 上 $n+1$ 次连续可导, 对 $[a,b]$ 上任意点 $x\neq x_i(i=0,1,\cdots,n)$, 构造辅助函数

$$\psi(t)=f(t)-\varphi_n(t)-K(x)\omega_{n+1}(t)$$

显然

$$\psi(x)=f(x)-\varphi_n(x)-K(x)\omega_{n+1}(x)=0$$

故函数 $\psi(t)$ 在区间 $[a,b]$ 内至少有 $n+2$ 个零点 $(x,x_0,x_1,\cdots,x_n)$. 由 Rolle (罗尔) 定理, 函数

$$\psi'(t)=f'(t)-\varphi_n'(t)-K(x)\omega_{n+1}'(t)$$

在 (a,b) 内至少有 $n+1$ 个零点. 反复使用 Rolle 定理, 可以得出

$$\psi^{(n+1)}(t)=f^{(n+1)}(t)-K(x)\omega_{n+1}^{(n+1)}(t)$$

在 (a,b) 内至少有一个零点, 设为 ξ, 即

$$\psi^{(n+1)}(\xi)=0$$

因为 $\varphi_n(t)$ 为至多 n 次的多项式,$\omega_{n+1}(t)$ 为最高次项系数为 1 的 $n+1$ 次多项式, 因而

$$\varphi_n^{(n+1)}(t)\equiv 0,\quad \omega_{n+1}^{(n+1)}(t)=(n+1)!$$

于是有

$$\psi^{(n+1)}(\xi)=f^{(n+1)}(\xi)-K(x)(n+1)!=0$$

所以

$$R_n(x)=\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x)=\frac{f^{(n+1)}(\xi)}{(n+1)!}(x-x_0)(x-x_1)\cdots(x-x_n).$$

通过证明方程组(5.3)解的存在唯一性, 已经给出了一种确定插值多项式 $\varphi_n(x)$ 的方法, 即通过求解线性方程组(5.3). 这种方法虽然思想简单, 但是计算量较大, 当系数矩阵病态时, 误差很大, 没有实用性. 下面介绍几种常用的确定插值多项式的方法.

5.1.3 Lagrange 插值多项式

先讨论只有两个节点 $x_0,x_1(n=1)$ 的插值多项式. 由前所述, 插值多项式应设为 $\varphi_1(x)=a_0+a_1x$, 且满足插值条件

$$\varphi_1(x_0)=a_0+a_1x_0=y_0=f(x_0)$$

$$\varphi_1(x_1)=a_0+a_1x_1=y_1=f(x_1)$$

解此方程组得

扫描全能王 创建

$$a_0=\frac{y_1x_0-y_0x_1}{x_0-x_1},\quad a_1=\frac{y_0-y_1}{x_0-x_1}$$

所以, 两个节点的一次插值多项式为

$$\varphi_1(x)=\frac{y_1x_0-y_0x_1}{x_0-x_1}+\frac{y_0-y_1}{x_0-x_1}x \tag{5.5}$$

如图 5.2 所示, 这是用过两点 $(x_0,y_0),(x_1,y_1)$ 的直线 $y=\varphi_1(x)$ 近似曲线 $y=f(x)$, 故这种插值又称为线性插值.

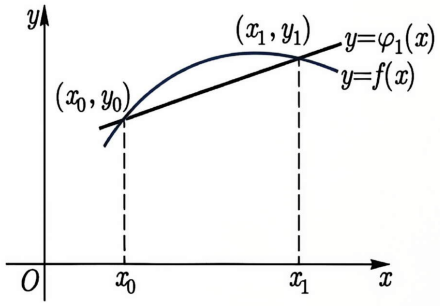


图 5.2　线性插值函数

如果将式 (5.5) 改写成以下形式

$$\varphi_1(x)=y_0\frac{x-x_1}{x_0-x_1}+y_1\frac{x-x_0}{x_1-x_0} \tag{5.6}$$

式(5.6)中,$\varphi_1(x)$ 被表成两个线性函数的线性组合. 记

$$l_0(x)=\frac{x-x_1}{x_0-x_1},\quad l_1(x)=\frac{x-x_0}{x_1-x_0}$$

显然, 它们满足

$$\begin{aligned}&l_0(x_0)=1,\quad l_0(x_1)=0\\&l_1(x_0)=0,\quad l_1(x_1)=1\end{aligned}$$

即 $l_i(x)$ $(i=0,1)$ 在对应的插值点 x_i 处取值为 1, 在其他点处取值为 0. 不难想象, 以对应点处的函数值为系数对它们作线性组合所得的函数, 不仅仍是线性的, 且必定满足插值条件. 由此得到启发, 当节点增多到 $n+1$ 个时, 可以先构造 n 次多项式 $l_i(x)(i=0,1,\cdots,n)$, 它们满足

$$l_i(x_j)=\begin{cases}0, & j\neq i\\ 1, & j=i\end{cases} \tag{5.7}$$

然后以对应点处的函数值为系数作线性组合, 即得所要求的插值多项式. 下面推导 $l_i(x)(i=0,1,\cdots,n)$ 的表达式. 由式 (5.7), 多项式 $l_i(x)$ 有 n 个根 $x_j(j=0,1,\cdots,n;j\neq i)$, 且 $l_i(x_i)=1$, 故它必定是以下形式

$$\begin{aligned}l_i(x)&=\frac{(x-x_0)\cdots(x-x_{i-1})(x-x_{i+1})\cdots(x-x_n)}{(x_i-x_0)\cdots(x_i-x_{i-1})(x_i-x_{i+1})\cdots(x_i-x_n)}\\&=\prod_{\substack{j=0\\j\neq i}}^{n}\frac{x-x_j}{x_i-x_j}\quad(i=0,1,\cdots,n)\end{aligned} \tag{5.8}$$

扫描全能王 创建

这些函数称为 Lagrange (拉格朗日) 插值基函数. 利用它们立即得出插值问题的解

$$\varphi_n(x)=\sum_{i=0}^{n}y_il_i(x)=\sum_{i=0}^{n}y_i\left(\prod_{\substack{j=0\\j\neq i}}^{n}\frac{x-x_j}{x_i-x_j}\right) \tag{5.9}$$

事实上, 因为每个插值基函数 $l_i(x)(i=0,1,\cdots,n)$ 都是 n 次多项式, 故 $\varphi_n(x)$ 是至多 n 次多项式. 由式 (5.7) 又得

$$\varphi_n\left(x_k\right)=\sum_{i=0}^{n}y_il_i\left(x_k\right)=y_k\quad(k=0,1,\cdots,n)$$

即 $\varphi_n(x)$ 满足插值条件式(5.2). 式 (5.9) 称为 n 次 Lagrange 插值多项式. 为了以后便于区别其他类型插值多项式, 常用 $L_n(x)$ 代替 $\varphi_n(x)$.

定义 5.3 由 Lagrange 插值基函数得到的插值多项式, 即

$$L_n(x)=\sum_{i=0}^{n}y_il_i(x) \tag{5.10}$$

称为 Lagrange 插值多项式.

由前面讨论的结果,$n+1$ 个节点的 n 次 Lagrange 插值多项式存在唯一, 式(5.4)为插值余项. 式(5.10)形式规律性强, 容易编制程序. 算法 5.1 用于计算非节点处的函数近似值.

算法 5.1 Lagrange 插值

1. 输入 $x,x_i,y_i(i=0,1,\cdots,n)$.
2. 对 $i=0,1,\cdots,n,l_i=\prod\limits_{\substack{j=0\\j\neq i}}^{n}\dfrac{x-x_j}{x_i-x_j}$.
3. $L=\sum\limits_{i=0}^{n}y_il_i$.
4. 输出 $L\approx f(x)$, 停机.

因为 $\omega_{n+1}(x)=\prod\limits_{j=0}^{n}(x-x_j)$, 求导可得 $\omega_{n+1}'\left(x_i\right)=\prod\limits_{\substack{j=0\\j\neq i}}^{n}\left(x_i-x_j\right)$, 于是式 (5.10) 还可以写成

$$L_n(x)=\sum_{i=0}^{n}\frac{\omega_{n+1}(x)}{(x-x_i)\,\omega_{n+1}'\left(x_i\right)}y_i$$

特别地, 当 $n=1$ 时, 两点一次 Lagrange 插值多项式为

$$L_1(x)=\frac{x-x_1}{x_0-x_1}y_0+\frac{x-x_0}{x_1-x_0}y_1$$

当 $n=2$ 时, 即给定三个节点 x_0,x_1,x_2 处的函数值 y_0,y_1,y_2, 三个插值基函数为

扫描全能王 创建

$$l_0(x)=\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}$$

$$l_1(x)=\frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}$$

$$l_2(x)=\frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}$$

二次 Lagrange 插值多项式为

$$L_2(x)=y_0l_0(x)+y_1l_1(x)+y_2l_2(x)$$

如图 5.3 所示, 这是一条过点 $(x_i,y_i)(i=0,1,2)$ 的抛物线. 这说明用二次插值多项式近似函数, 其几何意义是用过曲线上三点的抛物线近似曲线, 故三点二次插值又称为抛物线插值.

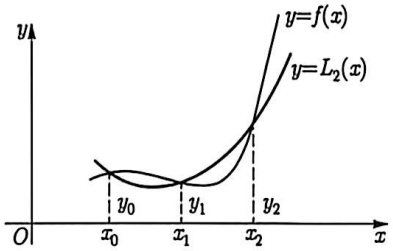


图 5.3　二次插值函数

例 5.1　已知函数 $y=\ln x$ 的函数表如下:

x	10	11	12	13	14
$y=\ln x$	2.302 6	2.397 9	2.484 9	2.564 9	2.639 1

分别用 Lagrange 线性插值和抛物线插值求 $\ln 11.5$ 的近似值, 并估计截断误差.

解　线性插值. 取两个节点 $x_0=11,x_1=12$, 插值基函数为

$$l_0(x)=\frac{x-x_1}{x_0-x_1}=-(x-12),\quad l_1(x)=\frac{x-x_0}{x_1-x_0}=x-11$$

由式 (5.6)

$$L_1(x)=-2.397\ 9(x-12)+2.484\ 9(x-11)$$

将 $x=11.5$ 代入, 即得

$$\ln 11.5\approx L_1(11.5)=2.397\ 9\times 0.5+2.484\ 9\times 0.5=2.441\ 4$$

按误差估计式 (5.4)

$$R_1(x)=\frac{(\ln x)''_\xi}{2!}(x-11)(x-12)$$

因为 $(\ln x)''=-\dfrac{1}{x^2},\xi$ 在 11 与 12 之间, 故

$$\left|(\ln x)''_\xi\right|=\frac{1}{\xi^2}\leqslant\frac{1}{11^2}=0.008\ 264\ 5$$

扫描全能王 创建

于是

$$|R_1(11.5)| \leqslant \frac{1}{2} \times 0.008\ 264\ 5 \times 0.5 \times 0.5 = 1.033\ 06 \times 10^{-3}$$

抛物线插值. 取 $x_0 = 11, x_1 = 12, x_2 = 13$, 插值多项式为

$$\begin{aligned} L_2(x) &= 2.397\ 9\frac{(x-12)(x-13)}{(11-12)(11-13)} + 2.484\ 9\frac{(x-11)(x-13)}{(12-11)(12-13)} + 2.564\ 9\frac{(x-11)(x-12)}{(13-11)(13-12)} \\ &= 1.198\ 95(x-12)(x-13) - 2.484\ 9(x-11)(x-13) + 1.282\ 45(x-11)(x-12) \end{aligned}$$

所以

$$\begin{aligned} \ln 11.5 &\approx L_2(11.5) \\ &= 1.198\ 95 \times (-0.5) \times (-1.5) - 2.484\ 9 \times 0.5 \times (-1.5) + 1.282\ 45 \times 0.5 \times (-0.5) \\ &= 2.442\ 275 \end{aligned}$$

因为 $(\ln x)''' = \dfrac{2}{x^3}$, 于是

$$\max_{11 \leqslant x \leqslant 13} |(\ln x)'''| \leqslant \frac{2}{11^3} = 0.150\ 3 \times 10^{-2}$$

因此用抛物线插值计算的误差为

$$\begin{aligned} |R_2(11.5)| &= \frac{|(\ln x)'''_\xi|}{3!} |(11.5-11)(11.5-12)(11.5-13)| \\ &\leqslant \frac{1}{6} \times 0.150\ 3 \times 10^{-2} \times 0.5 \times 0.5 \times 1.5 \\ &= 9.393\ 8 \times 10^{-5} \end{aligned}$$

计算可得 $\ln 11.5 = 2.442\ 347$.

例 5.2 设 $x_0, x_1, \cdots, x_n$ 为 $n+1$ 个互异节点,$l_i(x)$ $(i = 0, 1, \cdots, n)$ 为这组节点上的 Lagrange 插值基函数, 试证:$\sum\limits_{i=0}^{n} x_i^k l_i(x) \equiv x^k$ $(k = 0, 1, \cdots, n)$.

证明 由上式左端的表达式, 设 $f_k(x) = x^k$ $(k = 0, 1, \cdots, n)$, 则 $f_k(x)$ 对于这 $n+1$ 个节点的 Lagrange 插值函数为

$$L_{k,n}(x) = \sum_{i=0}^{n} x_i^k l_i(x) \quad (k = 0, 1, \cdots, n)$$

对于任意 x, 由于 $f_k^{(m)}(x) \equiv 0$ $(m > k)$, 则插值函数 $L_{k,n}(x)$ 的余项为

$$R_{k,n}(x) = f_k(x) - L_{k,n}(x) = \frac{f_k^{(n+1)}(\xi)}{(n+1)!} \omega_{n+1}(x) \equiv 0 \quad (k = 0, 1, \cdots, n)$$

所以

$$L_{k,n}(x) = \sum_{i=0}^{n} x_i^k l_i(x) \equiv f_k(x) = x^k \quad (k = 0, 1, \cdots, n)$$

扫描全能王 创建

5.2　Newton 插值

如果将图 5.2中直线用点斜式方程表示, 即把线性插值公式改写成以下形式

$$\varphi_1(x) = y_0 + \frac{y_1 - y_0}{x_1 - x_0}(x - x_0) \tag{5.11}$$

由此导出插值多项式的又一种表示形式——Newton 插值公式. 在导出 Newton 公式前, 先介绍公式表示中所需要用到的差商、差分的概念及性质.

5.2.1　差商

定义 5.4　设有函数 $f(x), x_0, x_1, x_2, \cdots$ 为一系列互不相等的点, 称 $\dfrac{f(x_i) - f(x_j)}{x_i - x_j}\ (i \neq j)$ 为 $f(x)$ 关于点 x_i, x_j 的一阶差商 (也称均差), 记为 $f[x_i, x_j]$, 即

$$f[x_i, x_j] = \frac{f(x_i) - f(x_j)}{x_i - x_j}$$

类似于高阶导数的定义, 称一阶差商的差商

$$\frac{f[x_i, x_j] - f[x_j, x_k]}{x_i - x_k}$$

为 $f(x)$ 关于点 x_i, x_j, x_k 的二阶差商, 记为 $f[x_i, x_j, x_k]$. 一般地, 称

$$\frac{f[x_0, x_1, \cdots, x_{k-1}] - f[x_1, x_2, \cdots, x_k]}{x_0 - x_k}$$

为 $f(x)$ 关于点 $x_0, x_1, \cdots, x_k$ 的 k 阶差商, 记为

$$f[x_0, x_1, \cdots, x_k] = \frac{f[x_0, x_1, \cdots, x_{k-1}] - f[x_1, x_2, \cdots, x_k]}{x_0 - x_k}$$

注 5.2　可以用差商符号表示一个点上的函数值, 即 $f[x_0, x_0] = f(x_0)$, 可以看作函数 $f(x)$ 在点 x_0 的零阶差商.

容易证明, 差商具有下述性质:

(1) 各阶差商均具有线性性, 即若 $f(x) = a\varphi(x) + b\psi(x)$, 则对任意正整数 k, 都有

$$f[x_0, x_1, \cdots, x_k] = a\varphi[x_0, x_1, \cdots, x_k] + b\psi[x_0, x_1, \cdots, x_k]$$

(2) k 阶差商 $f[x_0, x_1, \cdots, x_k]$ 可表示成 $f(x_0), f(x_1), \cdots, f(x_k)$ 的线性组合

$$f[x_0, x_1, \cdots, x_k] = \sum_{i=0}^{k} \frac{f(x_i)}{\omega'_{k+1}(x_i)}$$

其中 $\omega'_{k+1}(x_i) = \prod\limits_{\substack{j=0 \\ j \neq i}}^{n} (x_i - x_j)$.

扫描全能王 创建

$k=1$ 是显然的, 当 $k=2$ 时, 容易验证

$$\begin{aligned} f[x_0,x_1,x_2] &= \frac{f[x_0,x_1]-f[x_1,x_2]}{x_0-x_2} \\ &= \frac{1}{x_0-x_2}\left[\frac{f(x_0)-f(x_1)}{x_0-x_1}-\frac{f(x_1)-f(x_2)}{x_1-x_2}\right] \\ &= \frac{f(x_0)}{(x_0-x_1)(x_0-x_2)}+\frac{f(x_1)}{(x_1-x_0)(x_1-x_2)}+\frac{f(x_2)}{(x_2-x_0)(x_2-x_1)} \end{aligned}$$

一般情形可用数学归纳法证明. 实际上, 通过比较 Newton 插值多项式和 Lagrange 插值多项式可以很容易证明这个性质.

(3) 各阶差商均具有对称性, 即改变节点的位置, 差商值不变. 如

$$f[x_i,x_j]=f[x_j,x_i]$$

$$f[x_i,x_j,x_k]=f[x_i,x_k,x_j]=f[x_j,x_i,x_k]$$

(4) 若 $f(x)$ 是 n 次多项式, 则一阶差商 $f[x,x_i]$ 是 $n-1$ 次多项式.

事实上, 如果 $f(x)$ 是 n 次多项式, 则 $P(x)=f(x)-f(x_i)$ 也是 n 次多项式, 且 $P(x_i)=0$. 于是 $P(x)$ 可分解为

$$P(x)=(x-x_i)P_{n-1}(x)$$

其中 $P_{n-1}(x)$ 为 $n-1$ 次多项式. 所以

$$f[x,x_i]=\frac{f(x)-f(x_i)}{x-x_i}=\frac{(x-x_i)P_{n-1}(x)}{x-x_i}=P_{n-1}(x)$$

为 $n-1$ 次多项式.

(5) 差商与导数的关系

$$f[x_0,x_1,\cdots,x_n]=\frac{f^{(n)}(\xi)}{n!} \tag{5.12}$$

其中 $\xi\in(\alpha,\beta),\alpha=\min\limits_{0\leqslant i\leqslant n}\{x_i\},\beta=\max\limits_{0\leqslant i\leqslant n}\{x_i\}$.

按照定义计算各阶差商, 可按表 5.1 进行.

表 5.1 差 商 表

x_i	$f(x_i)$	一阶差商	二阶差商	三阶差商	四阶差商	$\cdots$
x_0	$f(x_0)$					
x_1	$f(x_1)$	$f[x_0,x_1]$				
x_2	$f(x_2)$	$f[x_1,x_2]$	$f[x_0,x_1,x_2]$			
x_3	$f(x_3)$	$f[x_2,x_3]$	$f[x_1,x_2,x_3]$	$f[x_0,x_1,x_2,x_3]$		
x_4	$f(x_4)$	$f[x_3,x_4]$	$f[x_2,x_3,x_4]$	$f[x_1,x_2,x_3,x_4]$	$f[x_0,x_1,x_2,x_3,x_4]$	
x_5	$f(x_5)$	$f[x_4,x_5]$	$f[x_3,x_4,x_5]$	$f[x_2,x_3,x_4,x_5]$	$f[x_1,x_2,x_3,x_4,x_5]$	

扫描全能王 创建

5.2.2　Newton 插值公式

按定义 5.4, 线性插值公式 (5.11) 可表示成

$$\varphi_1(x) = f(x_0) + (x - x_0) f[x_0, x_1] \tag{5.13}$$

式 (5.13) 称为一次 Newton 插值多项式. 一般地, 由各阶差商的定义, 依次可得

$$f(x) = f(x_0) + (x - x_0) f[x, x_0]$$

$$f[x, x_0] = f[x_0, x_1] + (x - x_1) f[x, x_0, x_1]$$

$$f[x, x_0, x_1] = f[x_0, x_1, x_2] + (x - x_2) f[x, x_0, x_1, x_2]$$

$$\cdots$$

$$f[x, x_0, \cdots, x_{n-1}] = f[x_0, x_1, \cdots, x_n] + (x - x_n) f[x, x_0, \cdots, x_n]$$

把以上表达式从下到上依次回代, 即得

$$\begin{aligned} f(x) =& f(x_0) + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2] + \cdots + \\ & (x - x_0)(x - x_1) \cdots (x - x_{n-1}) f[x_0, x_1, \cdots, x_n] + \\ & (x - x_0)(x - x_1) \cdots (x - x_n) f[x, x_0, x_1, \cdots, x_n] \end{aligned} \tag{5.14}$$

记

$$\begin{aligned} N_n(x) =& f(x_0) + (x - x_0) f[x_0, x_1] + (x - x_0)(x - x_1) f[x_0, x_1, x_2] + \cdots + \\ & (x - x_0)(x - x_1) \cdots (x - x_{n-1}) f[x_0, x_1, \cdots, x_n] \end{aligned} \tag{5.15}$$

于是有截断误差

$$\begin{aligned} R_n(x) &= (x - x_0)(x - x_1) \cdots (x - x_n) f[x, x_0, x_1, \cdots, x_n] \\ &= \omega_{n+1}(x) f[x, x_0, x_1, \cdots, x_n] \end{aligned} \tag{5.16}$$

则

$$f(x) = N_n(x) + R_n(x)$$

显然,$N_n(x)$ 是至多 n 次的多项式. 而由

$$R_n(x_i) = \omega_{n+1}(x_i) f[x_i, x_0, x_1, \cdots, x_n] = 0 \quad (i = 0, 1, \cdots, n)$$

得 $f(x_i) = N_n(x_i)$ $(i = 0, 1, \cdots, n)$. 这表明 $N_n(x)$ 满足插值条件式 (5.2), 因而它是 $f(x)$ 的 n 次插值多项式. 这种形式的插值多项式称为 Newton 插值多项式.

定义 5.5　*已知函数 $f(x)$ 在 $n+1$ 个不同节点 $x_0, x_1, \cdots, x_n$ 处的函数值 $y_0, y_1, \cdots, y_n$,* 记

$$\psi_0(x) = 1, \psi_i(x) = \psi_{i-1}(x)(x - x_{i-1}), \quad i = 1, 2, \cdots, n$$

称

扫描全能王 创建

$$N_n(x)=\sum_{i=0}^{n} f[x_0,x_1,\cdots,x_i]\psi_i(x)$$

为 Newton 插值多项式.

注 5.3　可见, 不同类型的 n 次插值多项式表现为不同基函数的线性组合:

一般插值多项式基函数: $\psi_i(x)=x^i, i=0,1,\cdots,n.$

Lagrange 插值多项式基函数: $\psi_i(x)=l_i(x), i=0,1,\cdots,n.$

Newton 插值多项式基函数: $\psi_0(x)=1, \psi_i(x)=\psi_i(x)(x-x_{i-1}), i=1,2,\cdots,n.$

x_0 点展开的 Taylor 多项式基函数: $\psi_i(x)=(x-x_0)^i, i=0,1,\cdots,n.$

注 5.4　Newton 插值多项式的优点是: 每增加一个节点, 插值多项式只增加一项, 即

$$N_{n+1}(x)=N_n(x)+(x-x_0)(x-x_1)\cdots(x-x_n)f[x_0,x_1,\cdots,x_{n+1}]$$

因此便于递推运算. 而且 Newton 插值的计算量小于 Lagrange 插值.

注 5.5　由插值多项式的唯一性可知,n 次 Newton 插值多项式与 n 次 Lagrange 插值多项式是相等的, 即 $N_n(x)=L_n(x)$, 它们只是表示形式不同. 因此 Newton 余项与 Lagrange 余项也是相等的, 即

$$R_n(x)=\omega_{n+1}(x)f[x,x_0,x_1,\cdots,x_n]=\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x),\quad \xi\in(a,b)$$

由此可得差商与导数的关系

$$f[x_0,x_1,\cdots,x_n]=\frac{f^{(n)}(\xi)}{n!}$$

注 5.6　比较 n 次 Lagrange 插值多项式和 Newton 插值多项式 n 阶项的系数, 可得

$$f[x_0,x_1,\cdots,x_k]=\sum_{i=0}^{k}\frac{f(x_i)}{\omega'_{k+1}(x_i)}$$

Newton 插值多项式可按表 5.2 计算.

表 5.2　利用差商表确定 Newton 插值多项式

x_i	$y_i=f(x_i)$	一阶差商	二阶差商	$\cdots$	n 阶差商	$\psi_i(x)$
x_0	$\underline{y_0}$					1
x_1	y_1	$\underline{f[x_0,x_1]}$				$x-x_0$
x_2	y_2	$f[x_1,x_2]$	$\underline{f[x_0,x_1,x_2]}$			$\prod_{j=0}^{1}(x-x_j)$
x_3	y_3	$f[x_2,x_3]$	$f[x_1,x_2,x_3]$			$\prod_{j=0}^{2}(x-x_j)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$		$\vdots$
x_n	y_n	$f[x_{n-1},x_n]$	$f[x_{n-2},x_{n-1},x_n]$		$\underline{f[x_0,x_1,\cdots,x_n]}$	$\prod_{j=0}^{n-1}(x-x_j)$

n 次 Newton 插值多项式 $N_n(x)$ 为表 5.2 中对角线上的差商值与右端基函数乘积的和.

扫描全能王 创建

算法 5.2 给出了用 Newton 插值计算非节点处函数近似值的过程.

算法 5.2　Newton 插值方法

1. 输入数据 $x, x_i, y_i (i=0,1,\cdots,n)$.
2. 置 $f_k = y_k (k=0,1,\cdots,n)$.
3. 计算各阶差商. 对 $i=1,2,\cdots,n$

$$\frac{f_{k-1}-f_k}{x_{k-i}-x_k} \Rightarrow f_k \quad (k=n,n-1,\cdots,i)$$

4. 采用秦九韶算法的思想简化计算 Newton 插值多项式

$$\begin{aligned} p =& f_0 + \sum_{k=0}^{n} f_k \left[\prod_{j=0}^{k-1} (x-x_j) \right] \\ =& \{ [(f_n (x-x_{n-1}) + f_{n-1})(x-x_{n-2}) + f_{n-2}](x-x_{n-3}) + \cdots \} (x-x_0) + f_0 \end{aligned}$$

5. 输出 $p \approx f(x)$, 停机.

例 5.3　用 Newton 插值公式计算例 5.1 中的 $\ln 11.5$.

解　如果仍取点 $x_0=11, x_1=12, x_2=13$ 作抛物线插值, 按表 5.2 计算, 结果如下:

x_i	$y_i=\ln x_i$	一阶差商	二阶差商	$\psi_i(x)$
11	2.397 9			1
12	2.484 9	0.087 0		$x-11$
13	2.564 9	0.080 0	−0.003 5	$(x-11)(x-12)$

$$N_2(x) = 2.397\ 9 + 0.087\ 0(x-11) - 0.003\ 5(x-11)(x-12)$$

所以

$$\begin{aligned} \ln 11.5 &\approx N_2(11.5) \\ &= 2.397\ 9 + 0.087\ 0 \times 0.5 + 0.003\ 5 \times 0.5 \times 0.5 \\ &= 2.442\ 275 \end{aligned}$$

若加节点 $x=10,14, \ln 10 = 2.302\ 6, \ln 14 = 2.639\ 1$, 用 $\ln x$ 的四次 Newton 插值多项式近似, 则按表 5.2 计算结果如下:

x_i	$y_i=\ln x_i$	一阶差商	二阶差商	三阶差商	四阶差商	$\psi_i(x)$
10	2.302 6					1
11	2.397 9	0.095 3				$x-10$
12	2.484 9	0.087 0	−0.004 15			$(x-10)(x-11)$
13	2.564 9	0.080 0	−0.003 50	0.000 22		$\prod_{k=10}^{12}(x-k)$
14	2.639 1	0.074 2	−0.002 90	0.000 20	−0.000 005	$\prod_{k=10}^{13}(x-k)$

扫描全能王 创建

所以

$$\begin{aligned}\ln 11.5 &\approx N_4(11.5)\\ &= 2.302\,6+0.095\,3\times 1.5-0.004\,15\times 1.5\times 0.5+0.000\,22\times 1.5\times\\ &\quad 0.5\times(-0.5)-0.000\,005\times 1.5\times 0.5\times(-0.5)\times(-1.5)\\ &= 2.442\,352\,2\end{aligned}$$

5.2.3 差分

当节点等距时, 即相邻两个节点之差 (称为步长) 为常数, Newton 插值公式的形式会更简单. 此时关于节点间函数的平均变化率 (差商) 可用函数值之差 (差分) 来表示.

定义 5.6 设有等距节点 $x_k=x_0+kh(k=0,1,\cdots,n)$, 步长 h 为常数,$f_k=f(x_k)$. 称相邻两个节点 x_k,x_{k+1} 处的函数值的增量 $f_{k+1}-f_k(k=0,1,\cdots,n-1)$ 为函数 $f(x)$ 在点 x_k 处以 h 为步长的一阶向前差分, 记为 Δf_k, 即

$$\Delta f_k=f_{k+1}-f_k\quad(k=0,1,\cdots,n-1)$$

类似地, 定义差分的差分为高阶差分. 如二阶向前差分为

$$\Delta^2 f_k=\Delta f_{k+1}-\Delta f_k\quad(k=0,1,\cdots,n-2)$$

一般地,m 阶向前差分为

$$\Delta^m f_k=\Delta^{m-1}f_{k+1}-\Delta^{m-1}f_k\quad(m=2,3,\cdots)$$

计算各阶向前差分可按表 5.3 进行.

表 5.3 向前差分表

x_k	$f_k=f(x_k)$	Δf_k	$\Delta^2 f_k$	$\Delta^3 f_k$	$\Delta^4 f_k$	$\cdots$
x_0	f_0					
x_1	f_1	Δf_0				
x_2	f_2	Δf_1	$\Delta^2 f_0$			
x_3	f_3	Δf_2	$\Delta^2 f_1$	$\Delta^3 f_0$		
x_4	f_4	Δf_3	$\Delta^2 f_2$	$\Delta^3 f_1$	$\Delta^4 f_0$	
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	

根据点的相对位置, 也可以定义向后差分和中心差分:

定义 5.7 称 $f(x_k)$ 与其前面点 $f(x_{k-1})$ 的差为函数 $f(x)$ 在点 x_k 处以 h 为步长的一阶向后差分, 记为 ∇f_k, 即

$$\nabla f_k=f_k-f_{k-1}$$

称 $f(x_k)$ 前后对称中点的差为函数 $f(x)$ 在 x_k 处以 h 为步长的中心差分, 记为 δf_k, 即

$$\delta f_k=f\left(x_k+\frac{h}{2}\right)-f\left(x_k-\frac{h}{2}\right)$$

扫描全能王 创建

同样可以根据递推定义给出 m 阶向后差分与 m 阶中心差分公式

$$\nabla^m f_k = \nabla^{m-1} f_k - \nabla^{m-1} f_{k-1}$$

$$\delta^m f_k = \delta^{m-1} f_{k+\frac{1}{2}} - \delta^{m-1} f_{k-\frac{1}{2}}$$

计算时可分别按表 5.4 和表 5.5 排列.

表 5.4　向后差商表

x_k	$f_k = f(x_k)$	∇f_k	$\nabla^2 f_k$	$\nabla^3 f_k$	$\nabla^4 f_k$	$\cdots$
x_0	f_0					
x_1	f_1	∇f_1				
x_2	f_2	∇f_2	$\nabla^2 f_2$			
x_3	f_3	∇f_3	$\nabla^2 f_3$	$\nabla^3 f_3$		
x_4	f_4	∇f_4	$\nabla^2 f_4$	$\nabla^3 f_4$	$\nabla^4 f_4$	

表 5.5　中心差商表

x_k	$f_k = f(x_i)$	δ	δ^2	δ^3	δ^4	$\cdots$
x_0	f_0					
x_1	f_1	$\delta f_{\frac{1}{2}}$				
x_2	f_2	$\delta f_{1\frac{1}{2}}$	$\delta^2 f_1$			
x_3	f_3	$\delta f_{2\frac{1}{2}}$	$\delta^2 f_2$	$\delta^3 f_{1\frac{1}{2}}$		
x_4	f_4	$\delta f_{3\frac{1}{2}}$	$\delta^2 f_3$	$\delta^3 f_{2\frac{1}{2}}$	$\delta^4 f_2$	

由差分表 5.3—5.5 可以看出, 差分值所在的列数为差分的阶数. 要确定某个差分值是哪个点的差分, 对向前差分则为左上斜线上函数值对应的自变量值; 向后差分则为差分值所在行的自变量值; 对中心差分, 只要将左上斜线上函数值所对应的自变量值加上 $mh/2$ 即得, 其中 m 为差分的阶数,h 为步长.

例如, 按例 5.1 中所给出的 $y = \ln x$ 的函数表, 可列出 $h = 1$ 的差分表如下:

表 5.6　$y = \ln x$ 的差分表

x_i	$f_i = \ln x_i$	一阶差分	二阶差分	三阶差分	四阶差分
10	2.302 6				
11	2.397 9	0.095 3			
12	2.484 9	0.087 0	−0.008 3		
13	2.564 9	0.080 0	−0.007 0	0.001 3	
14	2.639 1	0.074 2	−0.005 8	0.001 2	−0.000 1

扫描全能王 创建

表中值 $-0.007\ 0$ 左上斜线函数值对应的自变量值为 11, 所在行的自变量值为 13, $m=2, h=1$, 故

$$-0.007\ 0=\Delta^2 \ln 11=\nabla^2 \ln 13=\delta^2 \ln 12$$

同理可得

$$0.001\ 3=\Delta^3 \ln 10=\nabla^3 \ln 13=\delta^3 \ln 11.5$$

差分具有以下性质:

(1) 各阶差分均具有线性性, 即若 $f(x)=a\varphi(x)+b\psi(x)$, 则对任意正整数 m, 都有

$$\Delta^m f_k=a\Delta^m \varphi_k+b\Delta^m \psi_k$$

(2) 各阶差分均可表成函数值的线性组合, 例如

$$\Delta^2 f_k=\Delta f_{k+1}-\Delta f_k=f_{k+2}-2f_{k+1}+f_k$$

$$\Delta^3 f_k=\Delta^2 f_{k+1}-\Delta^2 f_k=f_{k+3}-3f_{k+2}+3f_{k+1}-f_k$$

$$\nabla^2 f_k=\nabla f_k-\nabla f_{k-1}=f_k-2f_{k-1}+f_{k-2}$$

一般地

$$\Delta^m f_k=\sum_{j=0}^{m}(-1)^j\begin{pmatrix} m \\ j \end{pmatrix} f_{k+m-j}$$

$$\nabla^m f_k=\sum_{j=0}^{m}(-1)^j\begin{pmatrix} m \\ j \end{pmatrix} f_{k-j}$$

其中 $\begin{pmatrix} m \\ j \end{pmatrix}=\mathrm{C}_m^j=\dfrac{m!}{j!(m-j)!}$ 为二项展开式的系数.

(3) 各种差分之间可以互化, 这由差分表即可看出. 向后差分与中心差分化成向前差分的公式如下:

$$\nabla^m f_k=\Delta^m f_{k-m}$$

$$\delta^m f_k=\Delta^m f_{k-\frac{m}{2}}$$

例如,$\nabla^2 f_4=\nabla f_4-\nabla f_3=f_4-2f_3+f_2=\Delta^2 f_2=\Delta^2 f_{4-2}$.

(4) 可用差分表示差商. 例如

$$f[x_k, x_{k+1}]=\frac{f(x_k)-f(x_{k+1})}{x_k-x_{k+1}}=\frac{\Delta f_k}{h}=\frac{\nabla f_{k+1}}{h}$$

$$f[x_k, x_{k+1}, x_{k+2}]=\frac{f[x_k, x_{k+1}]-f[x_{k+1}, x_{k+2}]}{x_k-x_{k+2}}=\frac{\Delta^2 f_k}{2h^2}=\frac{\nabla^2 f_{k+2}}{2h^2}$$

一般地, 有

扫描全能王 创建

$$f[x_k, x_{k+1}, \cdots, x_{k+m}] = \frac{1}{m!h^m}\Delta^m f_k = \frac{1}{m!h^m}\nabla^m f_{k+m} \quad (m = 1, 2, \cdots) \tag{5.17}$$

(5) 结合式 (5.12), 可得差分与导数的关系

$$\Delta^m f_k = m!h^m f[x_k, x_{k+1}, \cdots, x_{k+m}] = h^m f^{(m)}(\xi), \quad x_k < \xi < x_{k+m}$$

5.2.4　等距节点插值公式

如果插值节点是等距的, 则 Newton 插值公式可用差分表示. 设已知节点 $x_k = x_0 + kh(k = 0, 1, 2, \cdots, n)$, 利用差分和差商之间的关系式 (5.17) 把 Newton 插值公式 (5.15)中的差商用差分替换, 得

$$\begin{aligned} N_n(x) =& f(x_0) + f[x_0, x_1](x - x_0) + \cdots + \\ & f[x_0, x_1, \cdots, x_n](x - x_0)(x - x_1)\cdots(x - x_{n-1}) \\ =& f_0 + \frac{1}{h}\Delta f_0(x - x_0) + \frac{\Delta^2 f_0}{2!h^2}(x - x_0)(x - x_1) + \cdots + \\ & \frac{\Delta^n f_0}{n!h^n}(x - x_0)(x - x_1) + \cdots + (x - x_{n-1}) \end{aligned}$$

若令 $x = x_0 + th$, 则上式又可变形为

$$\begin{aligned} N_n(x_0 + th) =& f_0 + t\Delta f_0 + \frac{t(t-1)}{2!}\Delta^2 f_0 + \cdots + \\ & \frac{t(t-1)\cdots(t-n+1)}{n!}\Delta^n f_0 \end{aligned} \tag{5.18}$$

式 (5.18) 称为 Newton 向前插值公式, 其余项为

$$\begin{aligned} R_n(x) &= \frac{f^{n+1}(\xi)}{(n+1)!}(x - x_0)(x - x_1)\cdots(x - x_n) \\ &= \frac{t(t-1)\cdots(t-n)}{(n+1)!}h^{n+1}f^{n+1}(\xi), \quad \xi \in (x_0, x_n) \end{aligned} \tag{5.19}$$

完全类似地, 也可以用向后差分表示 Newton 插值公式. 令 $x = x_n + th, x \in [x_0, x_n]$, 则有

$$N_n(x_n + th) = f_n + t\nabla f_n + \frac{t(t+1)}{2!}\nabla^2 f_n + \cdots + \frac{t(t+1)\cdots(t+n-1)}{n!}\nabla^n f_n \tag{5.20}$$

式 (5.20) 称为 Newton 向后插值公式, 其余项为

$$R_n(x) = \frac{t(t+1)\cdots(t+n)}{(n+1)!}h^{n+1}f^{(n+1)}(\xi), \quad \xi \in (x_0, x_n)$$

Newton 向前、向后插值公式均是 Newton 插值公式在等距节点时的变形. 实际计算时, 也可列表进行. 将表 5.7 中对角线上的差分值与对应行右端因子乘积求和即得 Newton 向前插值公式, 而 Newton 向后插值公式则为最后的节点所在行的各阶差分值与对应列下端因子乘积之和.

扫描全能王 创建

表 5.7 差分型 Newton 插值

x_i	$f_i=f(x_i)$	一阶差分	二阶差分	三阶差分	$\cdots$	n 阶差分	
x_0	f_0						1
x_1	f_1	$\Delta f_0\,(\nabla f_1)$					t
x_2	f_2	$\Delta f_1\,(\nabla f_2)$	$\Delta^2 f_0\,(\nabla^2 f_2)$				$\frac{t(t-1)}{2}$
x_3	f_3	$\Delta f_2\,(\nabla f_3)$	$\Delta^2 f_1\,(\nabla^2 f_3)$	$\Delta^3 f_0\,(\nabla^3 f_3)$			$\frac{1}{3!}\prod_{j=0}^{2}(t-j)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$			$\vdots$
x_n	f_n	$\Delta f_{n-1}\,(\nabla f_n)$	$\Delta^2 f_{n-2}\,(\nabla^2 f_n)$	$\Delta^3 f_{n-3}\,(\nabla^3 f_n)$		$\Delta^n f_0\,(\nabla^n f_n)$	$\frac{1}{n!}\prod_{j=0}^{n-1}(t-j)$
	1	t	$\frac{t(t+1)}{2}$	$\frac{1}{3!}\prod_{j=0}^{2}(t+j)$	$\cdots$	$\frac{1}{n!}\prod_{j=0}^{n-1}(t+j)$	

例 5.4 已知 $f(x)=\sin x$ 的函数表如下

x	0.4	0.5	0.6	0.7
$\sin x$	0.389 42	0.479 43	0.564 64	0.644 22

分别用 Newton 向前、向后插值公式求 $\sin 0.578\ 91$ 的近似值.

解 取 $x_0=0.4, x_1=0.5, x_2=0.6, x_3=0.7$, 按表 5.7 计算得

x_i	$f_i=\sin x_i$	一阶差分	二阶差分	三阶差分	
0.4	0.389 42				1
0.5	0.479 43	0.090 01			t
0.6	0.564 64	0.085 21	−0.004 80		$\frac{1}{2}t(t-1)$
0.7	0.644 22	0.079 58	−0.005 63	−0.000 83	$\frac{t}{3!}(t-1)(t-2)$
	1	t	$\frac{t}{2}(t+1)$	$\frac{t}{3!}(t+1)(t+2)$	

Newton 向前插值公式为

$$N_3(x_0+th)=0.389\ 42+0.090\ 01t-\frac{1}{2}\times 0.004\ 80t(t-1)-\frac{1}{6}\times 0.000\ 83t(t-1)(t-2)$$

将 $t=\dfrac{x-x_0}{h}=\dfrac{0.578\ 91-0.4}{0.1}=1.789\ 1$ 代入上式得

$$\begin{aligned}\sin 0.578\ 91 \approx & N_3(0.578\ 91)\\ =&0.389\ 42+0.090\ 01\times 1.789\ 1-\frac{1}{2}\times 0.004\ 80\times 1.789\ 1\times 0.789\ 1+\\ &\frac{1}{6}\times 0.000\ 83\times 1.789\ 1\times 0.789\ 1\times 0.210\ 9\\ =&0.547\ 11\end{aligned}$$

扫描全能王 创建

由式(5.19), 误差为

$$|R_3(0.578\ 91)| = \left|\frac{0.1^4}{4!} \times 1.789\ 1 \times 0.789\ 1 \times (-0.210\ 9) \times (-1.210\ 9)\sin\xi\right| < 2 \times 10^{-6}$$

Newton 向后插值公式为

$$N_3(x_4 + th) = 0.644\ 22 + 0.079\ 58t - \frac{0.005\ 63}{2}t(t+1) - \frac{0.000\ 83}{3!}t(t+1)(t+2)$$

将 $t = \dfrac{x - x_4}{h} = -1.210\ 9$ 代入上式得

$$\begin{aligned}\sin 0.578\ 91 \approx & 0.644\ 22 - 0.079\ 58 \times 1.210\ 9 - \\ & \frac{1}{2} \times 0.005\ 63 \times (-1.210\ 9) \times (-0.210\ 9) - \\ & \frac{1}{3!} \times 0.000\ 83 \times (-1.210\ 9) \times (-0.210\ 9) \times 0.789\ 1 \\ = & 0.547\ 11\end{aligned}$$

查表可得 $\sin 0.578\ 91 = 0.547\ 111\ 8$.

如果取 $x_0 = 0.4, x_1 = 0.5, x_2 = 0.6$, 用二阶 Newton 向后插值公式, 则得

$$N_2(x) = 0.564\ 64 + 0.085\ 21t - \frac{1}{2} \times 0.004\ 80t(t+1)$$

将 $t = \dfrac{x - x_2}{h} = -0.210\ 9$ 代入上式得

$$\sin 0.578\ 91 \approx 0.564\ 64 + 0.085\ 21 \times (-0.210\ 9) - 0.002\ 40 \times (-0.210\ 9) \times 0.789\ 1 = 0.547\ 07$$

其误差为

$$|R_2(5.789\ 1)| = \frac{0.1^3}{3!}|(-0.210\ 9) \times 0.789\ 1 \times 1.789\ 1\cos\xi| \leqslant 0.5 \times 10^{-4}$$

为便于递推计算, 当插值点 x 接近数据表头时, 一般用向前插值公式, 而当插值点 x 接近数据表尾时, 则采用向后插值公式.

5.3　分段线性插值

在代数插值中, 为了提高插值多项式对函数的逼近程度, 常常增加节点个数, 即提高多项式的次数, 但这样做往往不能达到预想的结果. 例如, 函数 $f(x) = \dfrac{1}{1+x^2}$, 如果在区间 $[-5, 5]$ 上取 11 个等距节点 $x_k = -5 + k(k = 0, 1, \cdots, 10)$, 由 Lagrange 插值公式可得到 $f(x)$ 的 10 次插值多项式 $L_{10}(x)$. 如图 5.4 所示,$L_{10}(x)$ 仅在区间中部能较好地逼近函数 $f(x)$, 在其他部位差异较大, 而且越接近端点, 逼近效果越差. 可以证明, 当节点无限加密时,$L_n(x)$ 也只能在很小范围内收敛. 这一现象称为 Runge (龙格) 现象, 它表明通过增加节点来提高逼近程度是不宜的. 因而一般不采用高次多项式插值.

扫描全能王 创建

直观上容易想象, 如果不用多项式曲线, 而是将曲线 $y=f(x)$ 的两个相邻的点用线段连接 (见图 5.5), 这样得到的折线必定能较好地近似曲线. 而且只要 $f(x)$ 连续, 节点越密, 近似程度越好. 由此得到启发, 为提高精度, 在加密节点时, 可以把节点分成若干段, 分段用低次多项式近似函数, 这就是分段插值的思想. 用折线近似曲线, 相当于分段用线性插值, 称为分段线性插值.

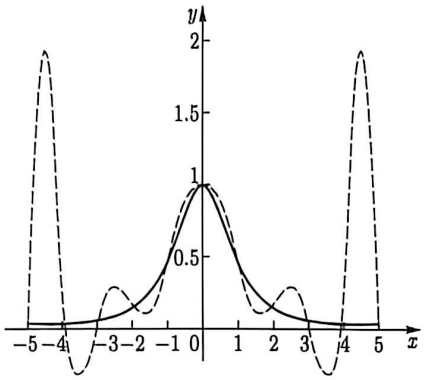


图 5.4 Runge 现象

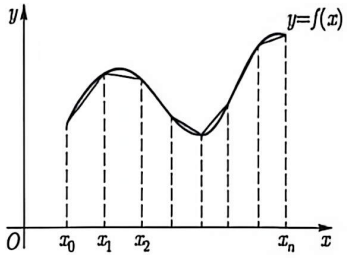


图 5.5 分段线性插值

定义 5.8 设在区间 $[a,b]$ 上给定 $n+1$ 个节点 $a=x_0<x_1<\cdots<x_{n-1}<x_n=b$ 及节点上的函数值 $y_i=f(x_i)\,(i=0,1,\cdots,n)$, 作一个插值函数 $\varphi(x)$, 使其满足

(1) $\varphi(x_i)=y_i\ (i=0,1,\cdots,n)$;

(2) 在每个小区间 $[x_i,x_{i+1}]\,(i=0,1,\cdots,n-1)$ 上,$\varphi(x)$ 是线性函数,

则称函数 $\varphi(x)$ 为 $[a,b]$ 上关于数据 $(x_i,y_i)\,(i=0,1,\cdots,n)$ 的分段线性插值函数.

由 Lagrange 线性插值公式容易写出 $\varphi(x)$ 的分段表达式

$$\varphi(x)=\frac{x-x_{i+1}}{x_i-x_{i+1}}y_i+\frac{x-x_i}{x_{i+1}-x_i}y_{i+1},\quad x_i\leqslant x\leqslant x_{i+1}\ (i=0,1,\cdots,n-1)\tag{5.21}$$

也可以通过构造基函数的方法来求 $\varphi(x)$.

首先构造一组基函数 $l_i(x)(i=0,1,\cdots,n)$, 每个 $l_i(x)$ 满足:

(1) $l_i(x_j)=\begin{cases}0, & j\neq i\\ 1, & j=i\end{cases}\quad(i,j=0,1,\cdots,n)$;

(2) $l_i(x)$ 在每个小区间 $[x_j,x_{j+1}]\,(j=0,1,\cdots n-1)$ 上是线性函数.

这组函数称为分段线性插值基函数, 其图形如图 5.6 所示.

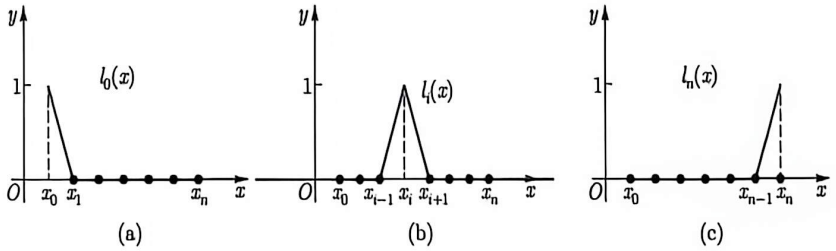


图 5.6 分段线性插值基函数

扫描全能王 创建

由图 5.6可直接写出 $l_i(x)$ 的表达式如下：

$$l_0(x)=\begin{cases}\dfrac{x-x_1}{x_0-x_1}, & x\in[x_0,x_1]\\ 0, & x\in(x_1,x_n]\end{cases}$$

$$l_i(x)=\begin{cases}\dfrac{x-x_{i-1}}{x_i-x_{i-1}}, & x\in[x_{i-1},x_i]\\ \dfrac{x-x_{i+1}}{x_i-x_{i+1}}, & x\in(x_i,x_{i+1}]\\ 0, & x\in[x_0,x_{i-1})\cup(x_{i+1},x_n]\end{cases}\qquad(i=1,2,\cdots,n-1)\tag{5.22}$$

$$l_n(x)=\begin{cases}\dfrac{x-x_{n-1}}{x_n-x_{n-1}}, & x\in[x_{n-1},x_n]\\ 0, & x\in[x_0,x_{n-1})\end{cases}$$

类似于 Lagrange 插值多项式的构造, 函数

$$\varphi(x)=\sum_{i=0}^{n}y_il_i(x)$$

就是所求的分段线性插值函数. 它与式 (5.21) 表示同一个函数.

例5.5　已知函数 $y=f(x)=\dfrac{1}{1+x^2}$, 在 $[0,5]$ 上取等距节点 $x_i=0+i\,(i=0,1,\cdots,5)$. 求分段线性插值函数, 并由此计算 $f(4.5)$ 的近似值.

解　节点处函数值如下

x_i	0	1	2	3	4	5
$y_i=\dfrac{1}{1+x_i^2}$	1.000 00	0.500 00	0.200 00	0.100 00	0.058 82	0.038 46

由式 (5.21), 在区间 $[4,5]$ 上分段线性插值函数为

$$\begin{aligned}\varphi(x)&=\frac{x-5}{4-5}\times0.058\ 82+\frac{x-4}{5-4}\times0.038\ 46\\&=0.038\ 46(x-4)-0.058\ 82(x-5)\end{aligned}$$

将 $x=4.5$ 代入, 得

$$f(4.5)\approx\varphi(4.5)=0.038\ 46\times0.5+0.058\ 82\times0.5=0.048\ 64$$

与精确值 $f(4.5)=0.047\ 06$ 比较, 结果是比较令人满意的.

分段线性插值的误差估计如下.

定理 5.2　如果 $f(x)$ 在 $[a,b]$ 上二阶连续可微, 则分段线性插值函数 $\varphi(x)$ 的余项有以下估计

$$|R(x)|=|f(x)-\varphi(x)|\leqslant\frac{h^2}{8}M$$

其中 $h=\max\limits_{0\leqslant i\leqslant n-1}(x_{i+1}-x_i)$, $M=\max\limits_{a\leqslant x\leqslant b}|f''(x)|$.

扫描全能王 创建

证明 因为在每个小区间 $[x_i, x_{i+1}]\,(i=0,1,\cdots,n-1)$ 上,$\varphi(x)$ 是 $f(x)$ 的线性插值,故由式(5.4), 对任意 $x \in [x_i, x_{i+1}]$ 有

$$R(x) = f(x) - \varphi(x) = \frac{f''(\xi)}{2}(x - x_i)(x - x_{i+1}), \quad \xi \in (x_i, x_{i+1})$$

又因为

$$(x - x_i)(x - x_{i+1}) = \left(x - \frac{x_i + x_{i+1}}{2}\right)^2 - \frac{(x_i - x_{i+1})^2}{4} \leqslant 0$$

因而

$$\max_{x_i \leqslant x \leqslant x_{i+1}} |(x - x_i)(x - x_{i+1})| = \frac{(x_i - x_{i+1})^2}{4}$$

于是

$$|R(x)| \leqslant \frac{|f''(\xi)|}{2} \frac{(x_i - x_{i+1})^2}{4} \leqslant \frac{(x_i - x_{i+1})^2}{8} \max_{x_i \leqslant x \leqslant x_{i+1}} |f''(x)|$$

所以, 对任意 $x \in [a, b]$, 都有

$$|R(x)| \leqslant \max_{0 \leqslant i \leqslant n-1} \left\{ \frac{(x_i - x_{i+1})^2}{8} \max_{x_i \leqslant x \leqslant x_{i+1}} |f''(x)| \right\} \leqslant \frac{h^2}{8} M$$

分段线插值简便易行, 定理 5.2 还表明, 当节点加密时, 分段线性插值的误差变小, 收敛性有保证. 另一方面, 在分段线性插值中, 每个小区间上的插值函数只依赖于本段的节点值, 因而每个节点只影响到节点邻近的一两个小区间, 计算过程中数据误差基本上不扩大, 从而保证了节点数增加时插值过程的稳定性. 但分段线性插值函数仅在 $[a, b]$ 上连续, 一般地, 在节点处插值函数不可微, 这就不能满足有些工程技术问题的光滑度要求.

5.4 Hermite 插值

如果对插值函数, 不仅要求它在节点处与函数同值, 而且要求它与函数有相同的一阶、二阶甚至更高阶的导数值, 这就是 Hermite (埃尔米特) 插值问题. 本节主要讨论在节点处插值函数与函数的值及一阶导数值均相等的 Hermite 插值.

5:4.1 Hermite 插值

定义 5.9 设已知函数 $y = f(x)$ 在 $n+1$ 个互异节点 $x_0, x_1, \cdots, x_n$ 上的函数值 $y_i = f(x_i)(i=0,1,\cdots,n)$ 和导数值 $y_i' = f'(x_i)\,(i=0,1,\cdots,n)$, 要求一个至多 $2n+1$ 次的多项式 $H(x)$, 使得

$$H(x_i) = y_i, \quad H'(x_i) = y_i' \quad (i=0,1,\cdots,n) \tag{5.23}$$

满足条件(5.23) 的多项式 $H(x)$ 称为 Hermite 插值多项式.

扫描全能王 创建

我们仍采用构造插值基函数的方法来求 Hermite 插值多项式. 可以设想, 如果有两组函数 $h_i(x), H_i(x)(i=0,1,\cdots,n)$, 它们满足:

(1) $h_i(x), H_i(x)(i=0,1,\cdots,n)$ 都是至多 $2n+1$ 次多项式;

(2)

$$\begin{aligned} &h_i(x_j)=\begin{cases}0, & j\neq i,\\ 1, & j=i,\end{cases} \quad h_i'(x_j)=0 \quad (j=0,1,\cdots,n)\\ &H_i(x_j)=0, \quad H_i'(x_j)=\begin{cases}0, & j\neq i,\\ 1, & j=i\end{cases} \quad (j=0,1,\cdots,n)\end{aligned} \tag{5.24}$$

则多项式

$$H(x)=\sum_{i=0}^{n}\left[y_i h_i(x)+y_i' H_i(x)\right]$$

必定满足插值条件式 (5.23), 且次数不超过 $2n+1$. 按条件式 (5.24) ,$h_i(x)$ 在 x_j $(j\neq i)$ 处函数值与导数值均为 0 , 故它们应含因子 $(x-x_j)^2(j\neq i)$, 因此可以设

$$h_i(x)=\left[a+b(x-x_i)\right]l_i^2(x) \tag{5.25}$$

其中 $l_i(x)=\dfrac{\omega_{n+1}(x)}{(x-x_i)\omega_{n+1}'(x_i)}$ 为 Lagrange 插值基函数. 由条件式 (5.24),$h_i(x)$ 还应满足

$$h_i(x_i)=1, \quad h_i'(x_i)=0$$

代入式 (5.25), 得

$$\begin{aligned} h_i(x_i)&=a l_i^2(x_i)=a=1\\ h_i'(x_i)&=b l_i^2(x_i)+2\left[a+b(x_i-x_i)\right]l_i(x_i)l_i'(x_i)\\ &=b+2a l_i'(x_i)=0\end{aligned}$$

其解为 $b=-2l_i'(x_i)$. 所以

$$h_i(x)=\left[1-2(x-x_i)l_i'(x_i)\right]l_i^2(x) \quad (i=0,1,\cdots,n) \tag{5.26}$$

同理, 由于 $H_i(x)$ 在 x_j $(j\neq i)$ 处的函数值与导数值均为 0 , 而 $H_i(x_i)=0$, 故可设

$$H_i(x)=c(x-x_i)l_i^2(x)$$

代入条件式 (5.24) 得

$$H_i'(x_i)=c l_i^2(x_i)=1$$

于是 $c=1$, 因此

$$H_i(x)=(x-x_i)l_i^2(x) \quad (i=0,1,\cdots,n) \tag{5.27}$$

所以 Hermite 插值多项式为

扫描全能王 创建

$$
\begin{aligned}
H(x) &= \sum_{i=0}^{n} \left[y_i h_i(x) + y_i' H_i(x) \right] \\
&= \sum_{i=0}^{n} \left\{ \left[1 - 2(x - x_i) l_i'(x_i) \right] l_i^2(x) y_i + (x - x_i) l_i^2(x) y_i' \right\}
\end{aligned}
$$

特别地, 当 $n = 1$ 时, 有

$$
\begin{aligned}
h_0(x) &= \left(1 + 2\frac{x - x_0}{x_1 - x_0} \right) \left(\frac{x - x_1}{x_0 - x_1} \right)^2 \\
h_1(x) &= \left(1 + 2\frac{x - x_1}{x_0 - x_1} \right) \left(\frac{x - x_0}{x_1 - x_0} \right)^2 \\
H_0(x) &= (x - x_0) \left(\frac{x - x_1}{x_0 - x_1} \right)^2 \\
H_1(x) &= (x - x_1) \left(\frac{x - x_0}{x_1 - x_0} \right)^2
\end{aligned}
$$

所以, 两个节点的三次 Hermite 插值多项式为

$$
\begin{aligned}
H(x) = &\left(1 + 2\frac{x - x_0}{x_1 - x_0} \right) \left(\frac{x - x_1}{x_0 - x_1} \right)^2 y_0 + \left(1 + 2\frac{x - x_1}{x_0 - x_1} \right) \left(\frac{x - x_0}{x_1 - x_0} \right)^2 y_1 + \\
&(x - x_0) \left(\frac{x - x_1}{x_0 - x_1} \right)^2 y_0' + (x - x_1) \left(\frac{x - x_0}{x_1 - x_0} \right)^2 y_1'
\end{aligned} \tag{5.28}
$$

5.4.2 误差估计

与 Lagrange 多项式插值的误差估计类似, 可导出 Hermite 插值的误差估计.

定理 5.3 设 $x_0, x_1, \cdots, x_n$ 为区间 $[a, b]$ 上的互异节点,$H(x)$ 为 $f(x)$ 的过这组节点的 $2n + 1$ 次 Hermite 插值多项式. 如果 $f(x)$ 在 $[a, b]$ 上 $2n + 2$ 次连续可导, 则对任意 $x \in [a, b]$, 插值余项为

$$
R(x) = f(x) - H(x) = \frac{f^{(2n+2)}(\xi)}{(2n + 2)!} \omega_{n+1}^2(x), \quad \xi \in (a, b) \tag{5.29}
$$

请读者仿定理,5.1 的证明思路自行证明此定理. 提示：设 $R(x) = K(x)\omega_{n+1}^2(x)$, 做辅助函数为

$$
\psi(t) = f(t) - H(t) - K(x)\omega_{n+1}^2(t)
$$

特别地, 三次 Hermite 插值余项为

$$
R(x) = \frac{f^{(4)}(\xi)}{4!} (x - x_0)^2 (x - x_1)^2 \tag{5.30}
$$

Hermit 插值多项式的唯一性证明除了可以用代数学基本定理反证, 也可以利用误差余项估计式.

扫描全能王 创建

定理 5.4　设 $x_0, x_1, \cdots, x_n$ 为区间 $[a,b]$ 上的互异节点，$f(x) \in C^{(1)}[a,b]$，则满足插值条件式 (5.23) 的 $2n+1$ 次 Hermite 多项式是唯一的.

证明　假设 $H(x)$ 与 $\bar{H}(x)$ 是满足相同插值要求的 $2n+1$ 次 Hermite 多项式，容易看出，$\bar{H}(x)$ 也是 $H(x)$ 的 $2n+1$ 次 Hermite 插值多项式. 由余项公式 (5.29)

$$H(x) - \bar{H}(x) = \frac{H^{(2n+2)}(\xi)}{(2n+2)!}\omega_{n+1}^2(x)$$

因为 $H(x)$ 为至多 $2n+1$ 次多项式，故 $H^{(2n+2)}(x) \equiv 0$. 于是 $H(x) - \bar{H}(x) \equiv 0$，所以 Hermite 插值多项式是唯一的.

由此导出以下结论：

(1) 至多 $2n+1$ 次的多项式在任意 $n+1$ 个互异节点上的 Hermite 多项式就是其自身.

(2) $\sum_{i=0}^{n} h_i(x) = 1$.

事实上，令 $f(x) = 1$，则有 $f(x_i) = 1, f'(x_i) = 0$ $(i = 0,1,\cdots,n)$. 显然 $H(x) = \sum_{i=0}^{n} h_i(x)$ 满足这组插值条件，由定理 5.4即得结论.

例 5.6　按下列数据求 Hermite 插值多项式

x_i	0	1
y_i	0	1
y_i'	3	9

解　将表中数据代入式 (5.28)，即得 Hermite 插值多项式

$$\begin{aligned} H(x) = &\left(1+2\frac{x-0}{1-0}\right)\left(\frac{x-1}{0-1}\right)^2 \times 0 + \left(1+2\frac{x-1}{0-1}\right)\left(\frac{x-0}{1-0}\right)^2 \times 1 + \\ &(x-0)\left(\frac{x-1}{0-1}\right)^2 \times 3 + (x-1)\left(\frac{x-0}{1-0}\right)^2 \times 9 \\ =&(3-2x)x^2 + 3x(x-1)^2 + 9x^2(x-1) \\ =&10x^3 - 12x^2 + 3x \end{aligned}$$

5.4.3　分段三次 Hermite 插值

当节点较多时，为避免多项式次数过高而引起非节点处的偏离过大，仍采用分段插值的方法. 给定下面函数表

x_i	x_0	$<$	x_1	$<$	x_2	$<$	$\cdots$	$<$	x_n
y_i	y_0		y_1		y_2		$\cdots$		y_n
y_i'	y_0'		y_1'		y_2'		$\cdots$		y_n'

扫描全能王 创建

若把节点两两分段, 在每一小段上作三次 Hermite 插值, 就得到一个分段三次 Hermite 插值函数 $H(x)$, 它满足:

(1) $H(x_i)=y_i, H'(x_i)=y_i' \quad (i=0,1,\cdots,n)$;

(2) 在每个小区间 $[x_i,x_{i+1}]\,(i=0,1,\cdots,n-1)$ 上,$H(x)$ 是三次多项式.

由式 (5.28) 可直接写出分段三次 Hermite 插值函数的分段表达式

$$\begin{aligned} H(x)=&\left(1+2\frac{x-x_i}{x_{i+1}-x_i}\right)\left(\frac{x-x_{i+1}}{x_i-x_{i+1}}\right)^2 y_i+\left(1+2\frac{x-x_{i+1}}{x_i-x_{i+1}}\right)\left(\frac{x-x_i}{x_{i+1}-x_i}\right)^2 y_{i+1}+\\ &(x-x_i)\left(\frac{x-x_{i+1}}{x_i-x_{i+1}}\right)^2 y_i'+(x-x_{i+1})\left(\frac{x-x_i}{x_{i+1}-x_i}\right)^2 y_{i+1}'\\ &x\in[x_i,x_{i+1}]\ \ (i=0,1,\cdots,n-1) \end{aligned} \tag{5.31}$$

也可以通过构造基函数给出分段三次 Hermite 插值函数的表达式. 参照分段线性插值与 Hermite 插值基函数公式 (5.26) 和式 (5.27), 可得出分段三次 Hermite 插值的基函数为

$$h_0(x)=\begin{cases}\left(1+2\dfrac{x-x_0}{x_1-x_0}\right)\left(\dfrac{x-x_1}{x_0-x_1}\right)^2, & x\in[x_0,x_1]\\ 0, & x\in[x_1,x_n]\end{cases}$$

$$h_i(x)=\begin{cases}\left(1+2\dfrac{x-x_i}{x_{i-1}-x_i}\right)\left(\dfrac{x-x_{i-1}}{x_i-x_{i-1}}\right)^2, & x\in[x_{i-1},x_i]\\ \left(1+2\dfrac{x-x_i}{x_{i+1}-x_i}\right)\left(\dfrac{x-x_{i+1}}{x_i-x_{i+1}}\right)^2, & x\in(x_i,x_{i+1}]\quad (i=1,\cdots,n-1)\\ 0, & x\in[x_0,x_{i-1})\cup(x_{i+1},x_n]\end{cases} \tag{5.32}$$

$$h_n(x)=\begin{cases}\left(1+2\dfrac{x-x_n}{x_{n-1}-x_n}\right)\left(\dfrac{x-x_{n-1}}{x_n-x_{n-1}}\right)^2, & x\in[x_{n-1},x_n]\\ 0, & x\in[x_0,x_{n-1})\end{cases}$$

$$H_0(x)=\begin{cases}(x-x_0)\left(\dfrac{x-x_1}{x_0-x_1}\right)^2, & x\in[x_0,x_1]\\ 0, & x\in(x_1,x_n]\end{cases}$$

$$H_i(x)=\begin{cases}(x-x_i)\left(\dfrac{x-x_{i-1}}{x_i-x_{i-1}}\right)^2, & x\in[x_{i-1},x_i]\\ (x-x_i)\left(\dfrac{x-x_{i+1}}{x_i-x_{i+1}}\right)^2, & x\in(x_i,x_{i+1}]\quad (i=1,\cdots,n-1)\\ 0, & x\in[x_0,x_{i-1})\cup(x_{i+1},x_n]\end{cases} \tag{5.33}$$

$$H_n(x)=\begin{cases}(x-x_n)\left(\dfrac{x-x_{n-1}}{x_n-x_{n-1}}\right)^2, & x\in[x_{n-1},x_n]\\ 0, & x\in[x_0,x_{n-1})\end{cases}$$

分段三次 Hermite 插值函数为

扫描全能王 创建

$$H(x)=\sum_{i=0}^{n}\left[y_i h_i(x)+y_i' H_i(x)\right]$$

由余项公式 (5.30)可以导出, 分段三次 Hermite 插值的误差有如下估计

$$|R(x)|=|f(x)-H(x)|\leqslant\frac{h^4}{384}\max_{a\leqslant x\leqslant b}\left|f^{(4)}(x)\right| \tag{5.34}$$

其中 $h=\max\limits_{0\leqslant i\leqslant n-1}\left(x_{i+1}-x_i\right)$.

分段三次 Hermite 插值函数是插值区间上的光滑函数, 它与函数 $f(x)$ 在节点处密合程度较好.

5.4.4　Hermite 插值的一般形式

Hermite 插值问题的一般形式是: 已知函数 $f(x)$ 在 $n+1$ 个互异节点 $x_0,x_1,\cdots,x_n$ 处的函数值 $y_i=f\left(x_i\right)(i=0,1,\cdots,n)$ 及某些节点上的导数值 $y_{i_k}'=f\left(x_{i_k}\right)(k=0,1,\cdots,m;m\leqslant n;0\leqslant i_k\leqslant n)$ 求一个至多 $n+m+1$ 次的多项式 $H(x)$, 使得

$$\begin{aligned}&H\left(x_i\right)=y_i\quad(i=0,1,\cdots,n)\\&H'\left(x_{i_k}\right)=y_{i_k}'\quad(k=0,1,\cdots,m)\end{aligned} \tag{5.35}$$

与前面的讨论类似, 可以证明满足条件 (5.35) 的 Hermite 插值多项式是存在唯一的, 且有

定理 5.5　设 $x_0,x_1,\cdots,x_n$ 为区间 $[a,b]$ 上的互异节点,$H(x)$ 为 $f(x)$ 的满足条件式 (5.35) 的 Hermite 插值多项式. 若 $f(x)\in C^{(n+m+2)}[a,b]$, 则对 $\forall x\in[a,b]$, 有

$$R(x)=f(x)-H(x)=\frac{f^{(m+n+2)}(\xi)}{(m+n+2)!}\omega_{n+1}(x)\prod_{k=0}^{m}\left(x-x_{i_k}\right) \tag{5.36}$$

证明　对 $\forall x\in[a,b],x\neq x_i\ (i=0,1,\cdots,n)$, 令 $R(x)=K(x)\omega_{n+1}(x)\prod\limits_{k=0}^{m}\left(x-x_{i_k}\right)$, 构造辅助函数

$$\psi(t)=f(t)-H(t)-K(x)\omega_{n+1}(t)\prod_{k=0}^{m}\left(t-x_{i_k}\right)$$

显然,$\psi(t)\in C^{(n+m+2)}[a,b]$ 且 $\psi(x)=0,\psi\left(x_i\right)=0\ (i=0,1,\cdots,n)$. 由 Rolle 定理,$\psi'(t)$ 在 $[a,b]$ 上至少有 $n+1$ 个异于节点 $x_i\ (i=0,1,\cdots,n)$ 与 x 的零点. 由插值条件 $f'\left(x_{i_k}\right)=H'\left(x_{i_k}\right)$ 可得

$$\begin{aligned}\psi'\left(x_{i_l}\right)=&f'\left(x_{i_l}\right)-H'\left(x_{i_l}\right)-\\&K(x)\left[\omega_{n+1}'\left(x_{i_l}\right)\prod_{k=0}^{m}\left(x_{i_l}-x_{i_k}\right)+\omega_{n+1}\left(x_{i_l}\right)\prod_{\substack{k=0\\k\neq l}}^{m}\left(x_{i_l}-x_{i_k}\right)\right]\\=&0\quad(l=0,1,\cdots,m)\end{aligned}$$

扫描全能王 创建

因此 $\psi'(t)$ 在 $[a,b]$ 上至少有 $n+m+2$ 个零点. 反复用 Rolle 定理可得出,$\psi^{(n+m+2)}(t)$ 在 (a,b) 上至少有一个零点, 即存在 $\xi\in(a,b)$, 使得

$$\psi^{(n+m+2)}(\xi)=f^{(n+m+2)}(\xi)-K(x)(n+m+2)!=0$$

于是

$$R(x)=\frac{f^{(n+m+2)}(\xi)}{(n+m+2)!}\omega_{n+1}(x)\prod_{k=0}^{m}(x-x_{i_k}),\quad \xi\in(a,b)$$

当 $x=x_i$ $(i=0,1,\cdots,n)$ 时, 上式自然成立. 所以, 对任意 $x\in[a,b]$, 式(5.36)都成立.

一般形式的 Hermite 插值多项式的求法仍可采用构造基函数的方法, 但用以下方法计算往往更简便. 设

$$H(x)=\varphi_n(x)+P_m(x)\omega_{n+1}(x)$$

其中 $\varphi_n(x)$ 为 $f(x)$ 的满足条件 $\varphi_n(x_i)=y_i$ $(i=0,1,\cdots,n)$ 的 n 次插值多项式,$\omega_{n+1}(x)=\prod_{j=0}^{n}(x-x_j)$, $P_m(x)$ 为待定的 m 次多项式, 显然

$$H(x_i)=\varphi_n(x_i)+P_m(x_i)\omega_{n+1}(x_i)=y_i$$

于是由导数值插值条件, 有

$$H'(x_{i_k})=\varphi_n'(x_{i_k})+P_m(x_{i_k})\omega_{n+1}'(x_{i_k})=y_{i_k}'$$

由此确定 $P_m(x)$, 即可求得 Hermite 插值多项式, 这种方法称为降阶法.

例 5.7 按下列数据求 Hermite 插值多项式

x_i	0	1	2
y_i	0	1	1
y_i'	0	1	

解 采用降阶法, 利用 Lagrange 插值或 Newton 插值容易求出满足函数值插值条件的插值多项式为

$$\varphi_2(x)=x-\frac{1}{2}x(x-1)=\frac{1}{2}x(3-x)$$

设

$$H(x)=\varphi_2(x)+(a+bx)x(x-1)(x-2)$$

求导得

$$\begin{aligned}H'(x)=&\frac{3}{2}-x+bx(x-1)(x-2)+\\&(a+bx)[(x-1)(x-2)+x(x-2)+x(x-1)]\end{aligned}$$

代入导数值插值条件得

$$H'(0)=\frac{3}{2}+2a=0,\quad H'(1)=\frac{1}{2}+\left(b-\frac{3}{4}\right)(-1)=1$$

扫描全能王 创建

求解得 $a=-\frac{3}{4}, b=\frac{1}{4}$, 故所求 Hermite 插值多项式为

$$H(x)=\frac{1}{2}x(3-x)+\left(\frac{1}{4}x-\frac{3}{4}\right)x(x-1)(x-2)=\frac{x^2}{4}(x-3)^2$$

5.5 样 条 插 值

许多工程技术中提出的计算问题对插值函数的光滑性有较高要求, 如飞机的机翼外形, 内燃机的进、排气门的凸轮曲线, 都要求曲线具有较高的光滑程度, 不仅要连续, 而且要有连续的曲率, 即二阶导数连续. 这就导致了样条插值的产生.

5.5.1 样条函数的概念

所谓样条 (spline) 本来是工程设计中使用的一种绘图工具, 它是富有弹性的细木条或细金属条. 绘图员利用它把一些已知点连接成一条光滑曲线（称为样条曲线), 并使连接点处有连续的曲率. 三次样条插值就是由此抽象出来的.

数学上将具有一定光滑性的分段多项式称为样条函数. 具体地说, 给定区间 $[a,b]$ 的一个分划

$$\Delta: a=x_0<x_1<\cdots<x_{n-1}<x_n=b$$

定义 5.10　*如果函数 $S(x)$ 满足:*

(1) *在每个小区间 $[x_i,x_{i+1}]\,(i=0,1,\cdots,n-1)$ 上 $S(x)$ 是 m 次多项式;*

(2) *$S(x)$ 在 $[a,b]$ 上具有 $m-1$ 阶连续导数,*

则称 $S(x)$ 为关于分划 Δ 的 m 次样条函数, 其图形为 m 次样条曲线.

显然, 折线是一次样条曲线. 绘图员用“样条”绘出的图形为三次样条曲线, 这也是最常用的样条函数.

5.5.2 三次样条插值

利用样条函数进行插值, 即取插值函数为样条函数, 称为样条插值. 例如, 分段线性插值是一次样条插值. 本节只介绍三次样条插值.

定义 5.11　*已知函数 $y=f(x)$ 在区间 $[a,b]$ 上的 $n+1$ 个节点*

$$a=x_0<x_1<\cdots<x_n=b$$

上的值 $y_i=f(x_i)\,(i=0,1,\cdots,n)$, 插值函数 $S(x)$ 满足

(1) $S(x_i)=y_i\quad(i=0,1,\cdots,n)$;

(2) *在每个小区间 $[x_j,x_{j+1}]\,(j=0,1,\cdots,n-1)$ 上 $S(x)$ 是三次多项式, 记为 $S_j(x)$;*

(3) *$S(x)$ 在 $[a,b]$ 上二阶连续可微,*

则函数 $S(x)$ 称为 $f(x)$ 的三次样条插值函数.

扫描全能王 创建

可以证明, 在一定的边界条件下, 三次样条插值问题的解是存在唯一的. 下面讨论三次样条插值函数的求法.

由于 $S(x)$ 在每个小区间上都是三次多项式, 故共有 $4n$ 个参数. 为简化计算, 取节点上的导数值或二阶导数值为参数, 来导出三次样条插值函数的表达式.

1. 以节点处的二阶导数值为参数的三次样条插值函数

设 $S''(x_i)=M_i\ (i=0,1,\cdots,n)$. 因为在小区间 $[x_j,x_{j+1}]$ 上 $S(x)=S_j(x)$ 是三次多项式, 故 $S_j''(x)$ 为线性函数, 由 Lagrange 插值公式得

$$S_j''(x)=M_{j+1}\frac{x-x_j}{h_j}-M_j\frac{x-x_{j+1}}{h_j}$$

其中 $h_j=x_{j+1}-x_j$. 将上式积分两次, 并代入插值条件 $S_j(x_j)=y_j, S_j(x_{j+1})=y_{j+1}$, 得

$$\begin{aligned}S_j(x)=&M_{j+1}\frac{(x-x_j)^3}{6h_j}-M_j\frac{(x-x_{j+1})^3}{6h_j}+\left(y_{j+1}-\frac{M_{j+1}h_j^2}{6}\right)\frac{x-x_j}{h_j}-\\&\left(y_j-\frac{M_jh_j^2}{6}\right)\frac{x-x_{j+1}}{h_j}\quad(j=0,1,\cdots,n-1)\end{aligned}\tag{5.37}$$

求导得

$$\begin{aligned}S_j'(x)=&\frac{M_{j+1}}{2h_j}(x-x_j)^2-\frac{M_j}{2h_j}(x-x_{j+1})^2+\frac{1}{h_j}\left(y_{j+1}-\frac{M_{j+1}h_j^2}{6}\right)-\frac{1}{h_j}\left(y_j-\frac{M_jh_j^2}{6}\right)\\=&\frac{M_{j+1}}{2h_j}(x-x_j)^2-\frac{M_j}{2h_j}(x-x_{j+1})^2+\frac{y_{j+1}-y_j}{h_j}-\frac{h_j}{6}(M_{j+1}-M_j)\\&(j=0,1,\cdots,n-1)\end{aligned}\tag{5.38}$$

因为 $S(x)$ 在节点处一阶导数连续, 故应有

$$S'(x_j+0)=S_j'(x_j+0)=S'(x_j-0)=S_{j-1}'(x_j-0)\quad(j=1,2,\cdots,n-1)$$

即

$$-\frac{h_j}{2}M_j+\frac{y_{j+1}-y_j}{h_j}-\frac{h_j}{6}(M_{j+1}-M_j)=\frac{h_{j-1}}{2}M_j+\frac{y_j-y_{j-1}}{h_{j-1}}-\frac{h_{j-1}}{6}(M_j-M_{j-1})$$

化简得

$$\frac{h_{j-1}}{6}M_{j-1}+\frac{h_{j-1}+h_j}{3}M_j+\frac{h_j}{6}M_{j+1}=\frac{y_{j+1}-y_j}{h_j}-\frac{y_j-y_{j-1}}{h_{j-1}}\quad(j=1,2,\cdots,n-1)\tag{5.39}$$

令

$$\begin{aligned}\alpha_j&=\frac{h_{j-1}}{h_{j-1}+h_j}\\\beta_j&=1-\alpha_j=\frac{h_j}{h_{j-1}+h_j}\\c_j&=\frac{6}{h_{j-1}+h_j}\left(\frac{y_{j+1}-y_j}{h_j}-\frac{y_j-y_{j-1}}{h_{j-1}}\right)\end{aligned}\tag{5.40}$$

扫描全能王 创建

则式(5.39) 改写成

$$\alpha_j M_{j-1} + 2M_j + \beta_j M_{j+1} = c_j \quad (j = 1, 2, \cdots, n-1) \tag{5.41}$$

式 (5.41) 给出了含 $n+1$ 个参数 $M_0, M_1, \cdots, M_n$ 的 $n-1$ 个方程, 称为三次样条的 M 关系式, 或按其力学意义, 称为**三弯矩方程**.

为完全确定这些参数, 需要根据问题的具体情况, 在区间的两个端点处给出条件, 称为边界条件. 常用的边界条件有以下三种:

(1) 给定两端点处的导数值 $S'(a) = y'_0, S'(b) = y'_n$. 特别地, 当 $y'_0 = y'_n = 0$ 时, 样条曲线在端点处呈水平状态.

(2) 给定两端点处的二阶导数值 $S''(a) = y''_0, S''(b) = y''_n$. 特别地, 当 $y''_0 = y''_n = 0$ 时, 称为自然边界条件.

(3) 如果 $f(x)$ 是以 $b-a$ 为周期的周期函数, 则 $S(x)$ 也应是具有同样周期的周期函数, 在端点处需满足

$$S'(a+0) = S'(b-0), \quad S''(a+0) = S''(b-0)$$

将这三种边界条件中的任一种与三弯矩方程 (5.41) 联立, 即可求出参数 $M_i\ (i = 0, 1, \cdots, n)$.

如果问题要求 $S(x)$ 满足边界条件 (1), 由式 (5.38)得

$$S'(a) = y'_0 = -\frac{h_0}{2} M_0 + \frac{y_1 - y_0}{h_0} - \frac{h_0}{6}(M_1 - M_0)$$

$$S'(b) = y'_n = \frac{h_{n-1}}{2} M_n + \frac{y_n - y_{n-1}}{h_{n-1}} - \frac{h_{n-1}}{6}(M_n - M_{n-1})$$

化简得

$$\begin{aligned} 2M_0 + M_1 &= \frac{6}{h_0}\left(\frac{y_1 - y_0}{h_0} - y'_0\right) \\ M_{n-1} + 2M_n &= \frac{6}{h_{n-1}}\left(y'_n - \frac{y_n - y_{n-1}}{h_{n-1}}\right) \end{aligned} \tag{5.42}$$

式 (5.42) 和式 (5.41) 联立, 即得关于参数 $M_i\ (i = 0, 1, \cdots, n)$ 的 $n+1$ 阶线性方程组, 其矩阵形式为

$$\begin{bmatrix} 2 & \beta_0 & & & & \\ \alpha_1 & 2 & \beta_1 & & & \\ & \alpha_2 & 2 & \beta_2 & & \\ & & \ddots & \ddots & \ddots & \\ & & & \alpha_{n-1} & 2 & \beta_{n-1} \\ & & & & \alpha_n & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix}$$

其中

$$\beta_0 = 1, \alpha_n = 1, c_0 = \frac{6}{h_0}\left(\frac{y_1 - y_0}{h_0} - y'_0\right), c_n = \frac{6}{h_{n-1}}\left(y'_n - \frac{y_n - y_{n-1}}{h_{n-1}}\right).$$

这是三对角方程组, 可用追赶法求解.

扫描全能王 创建

如果问题要求 $S(x)$ 满足边界条件 (2), 即

$$M_0 = y_0'', \quad M_n = y_n''$$

此时方程组 (5.41) 中实际只有 $n-1$ 个未知数, 其矩阵形式的

$$\begin{bmatrix} 2 & \beta_1 & & & & \\ \alpha_2 & 2 & \beta_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & \alpha_{n-2} & 2 & \beta_{n-2} & \\ & & & \alpha_{n-1} & 2 & \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} c_1 - \alpha_1 y_0'' \\ c_2 \\ \vdots \\ c_{n-2} \\ c_{n-1} - \beta_{n-1} y_n'' \end{bmatrix} \tag{5.43}$$

这仍是三对角方程组.

如果问题要求 $S(x)$ 满足周期边界条件 (3), 将边界条件 $S'(x_0+0) = S'(x_n-0), M_0 = M_n$ 与方程组 (5.41) 联立, 得到 n 阶方程组

$$\begin{bmatrix} 2 & \beta_1 & & & \alpha_1 \\ \alpha_2 & 2 & \beta_2 & & \\ & \ddots & \ddots & \ddots & \\ & & \alpha_{n-1} & 2 & \beta_{n-1} \\ \beta_n & & & \alpha_n & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-1} \\ M_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_{n-1} \\ c_n \end{bmatrix}$$

其中 $\alpha_n = \dfrac{h_{n-1}}{h_0+h_{n-1}}, \beta_n = \dfrac{h_0}{h_0+h_{n-1}}, c_n = \dfrac{6}{h_0+h_{n-1}}\left(\dfrac{y_1-y_0}{h_0} - \dfrac{y_n-y_{n-1}}{h_{n-1}}\right)$. 其求解过程类似于求解三对角方程组的追赶法.

容易验证, 这三种情形所得到的方程组的系数矩阵均为严格对角占优矩阵, 因此方程组有唯一解. 将解代入式 (5.37), 即得三次样条插值函数.

2. 以节点处的导数值为参数的三次样条插值函数

类似地, 也可以节点处的导数值为参数来导出三次样条插值函数的表达式.

设 $S'(x_j) = m_j$ $(j = 0, 1, \cdots, n)$. 按分段三次 Hermite 插值公式(5.31),$S(x)$ 可表示成

$$\begin{aligned} S_j(x) = &\left(1 + 2\frac{x-x_j}{h_j}\right)\left(\frac{x-x_{j+1}}{h_j}\right)^2 y_j + \left(1 - 2\frac{x-x_{j+1}}{h_j}\right)\left(\frac{x-x_j}{h_j}\right)^2 y_{j+1} + \\ &(x-x_j)\left(\frac{x-x_{j+1}}{h_j}\right)^2 m_j + (x-x_{j+1})\left(\frac{x-x_j}{h_j}\right)^2 m_{j+1}, \quad x \in [x_j, x_{j+1}] \end{aligned} \tag{5.44}$$

由二阶导数在节点处连续

$$S_j''(x_j+0) = S_{j-1}''(x_j-0) \quad (j = 1, 2, \cdots, n-1)$$

可导出关于参数 m_j 的方程组

$$\beta_j m_{j-1} + 2m_j + \alpha_j m_{j+1} = d_j \quad (j = 1, 2, \cdots, n-1) \tag{5.45}$$

其中

扫描全能王 创建

$$\alpha_j = \frac{h_{j-1}}{h_{j-1}+h_j}$$

$$\beta_j = 1-\alpha_j = \frac{h_j}{h_{j-1}+h_j} \tag{5.46}$$

$$d_j = 3\left[\frac{\beta_j}{h_{j-1}}(y_j - y_{j-1}) + \frac{\alpha_j}{h_j}(y_{j+1}-y_j)\right]$$

式 (5.45) 称为三次样条的 m 关系式, 或按其力学意义称为**三转角方程**.

方程组 (5.45) 含有 $n+1$ 个未知数,$n-1$ 个方程, 与对 M 关系式的讨论类似, 增加边界条件 (1), (2) 后, 可得关于参数 m_j 的三对角方程组; 增加周期边界条件 (3) 后, 得 n 阶方程组. 这些方程组的系数矩阵均非奇异, 故方程组有唯一解. 解出参数 m_j $(j=0,1,\cdots,n)$ 后代入式 (5.44), 即得三次样条插值函数.

例 5.8　已知函数表

j	0	1	2	3
x_j	0	1	2	3
y_j	0	2	3	16

求满足边界条件 $y'(0)=1, y'(3)=0$ 的三次样条插值函数.

解　如果用三弯矩方程求解, 由已知 $h_j=1$ $(j=0,1,2)$, 按式 (5.40) 和式(5.42) 计算方程组的系数及右端项, 结果如下:

j	α_j	β_j	c_j
0		1	6
1	1/2	1/2	−3
2	1/2	1/2	36
3	1		−78

将上述数据代入式 (5.41)和式 (5.42) 得方程组

$$\begin{bmatrix} 2 & 1 & 0 & 0 \\ 1/2 & 2 & 1/2 & 0 \\ 0 & 1/2 & 2 & 1/2 \\ 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} M_0 \\ M_1 \\ M_2 \\ M_3 \end{bmatrix} = \begin{bmatrix} 6 \\ -3 \\ 36 \\ -78 \end{bmatrix}$$

其解为 $M_0=\frac{28}{3}, M_1=-\frac{38}{3}, M_2=\frac{106}{3}, M_4=\frac{170}{3}$. 将此解代入式 (5.37), 得

$$\begin{aligned} S_0(x) &= -\frac{38}{3}\times\frac{(x-0)^3}{6} - \frac{28}{3}\times\frac{(x-1)^3}{6} + \left(2+\frac{38}{3\times 6}\right)(x-0) - \left(0-\frac{28}{3\times 6}\right)(x-1) \\ &= -\frac{19}{9}x^3 - \frac{14}{9}(x-1)^3 + \frac{37}{9}x + \frac{14}{9}(x-1) \\ &= \frac{x}{3}\left(-11x^2+14x+3\right), \qquad x\in[0,1] \end{aligned}$$

扫描全能王 创建

类似有

$$S_1(x)=\frac{1}{3}\left(24x^3-91x^2+108x-35\right),\qquad x\in[1,2]$$

$$S_2(x)=\frac{1}{3}\left(-46x^3+329x^2-732x+525\right),\quad x\in[2,3]$$

如果用三转角方程求解, 由式 (5.46)

$$\alpha_1=\alpha_2=\beta_1=\beta_2=\frac{1}{2}$$

$$d_1=3\left[\frac{1}{2}(y_1-y_0)+\frac{1}{2}(y_2-y_1)\right]=\frac{9}{2}$$

$$d_2=3\left[\frac{1}{2}(y_2-y_1)+\frac{1}{2}(y_3-y_2)\right]=21$$

将上述数据及 $m_0=1,m_3=0$ 代入式 (5.45), 得三转角方程

$$\frac{1}{2}+2m_1+\frac{1}{2}m_2=\frac{9}{2},\quad \frac{1}{2}m_1+2m_2=21$$

其解为 $m_1=-\dfrac{2}{3},m_2=\dfrac{32}{3}$. 将此解代入式 (5.44), 即得三次样条插值函数的分段表示式

$$S(x)=\begin{cases}\dfrac{x}{3}\left(-11x^2+14x+3\right), & 0\leqslant x\leqslant 1\\ \dfrac{1}{3}\left(24x^3-91x^2+108x-35\right), & 1\leqslant x\leqslant 2\\ \dfrac{1}{3}\left(-46x^3+329x^2-732x+525\right), & 2\leqslant x\leqslant 3\end{cases}$$

比较上面两种解法, 对第一种边界条件, 用三转角方程计算较简便.

小结计算三次样条插值函数的步骤如下:

(1) 根据给定的点 (x_j,y_j) 及相应的边界条件计算 α_j,β_j,c_j 或 d_j. 一般地, 对第一种边界条件用三转角方程, 对第二种边界条件用三弯矩方程较为简便;

(2) 解方程组 (5.41) 或 (5.45), 求出参数 $\{M_j\}$ 或 $\{m_j\}$;

(3) 将求得的参数代入式 (5.37) 或式 (5.44), 即得三次样条插值函数 $S(x)$ 的分段表示式.

三次样条插值函数有较好的收敛性.

定理 5.6 设 $f(x)$ 在 $[a,b]$ 上四次连续可微,$S(x)$ 为 $f(x)$ 在 $[a,b]$ 上的三次样条插值函数, 则

$$\left|f^{(l)}(x)-S^{(l)}(x)\right|\leqslant c_l h^{4-l}\max_{a\leqslant x\leqslant b}\left|f^{(4)}(x)\right|,\quad x\in[a,b]\quad (l=0,1,2,3)$$

其中 $h=\max\limits_{0\leqslant j\leqslant n-1}h_j, c_0=\dfrac{5}{384}, c_1=\dfrac{1}{24}, c_2=\dfrac{3}{8}, c_3=\dfrac{\beta+\beta^{-1}}{2}, \beta=\dfrac{\max\limits_{0\leqslant j\leqslant n-1}h_j}{\min\limits_{0\leqslant j\leqslant n-1}h_j}$ 称为分划比.

证明略.

定理 5.6 表明, 当分划小区间的长度趋于零时, $S(x)$ 及其一至二阶导数分别一致收敛

扫描全能王 创建

到 $f(x)$ 及其一至二阶导数, 因而, 为提高精度只需加密分划节点, 不需要提高样条函数的次数. 由于样条插值有这样好的性质, 因此应用十分广泛. 在外形设计及计算机辅助设计的许多领域中, 样条插值都是十分有效的数学工具.

5.6 二 维 插 值

在科学研究和生产实际问题中, 经常遇到二维或更高维的插值问题, 如数字地形的模拟, 飞机、汽车外形的设计等. 本节将模仿一维插值函数方法, 讨论矩形区域上的简单插值.

二维插值的一般提法是: 给定了二元函数 $u(x,y)$ 的一组离散值

$$u_{ij}=u\left(x_i,y_j\right)\quad(i=0,1,\cdots,n;j=0,1,\cdots,m)$$

要求构造关于 x 为 n 次、y 为 m 次的二元多项式

$$P_{n,m}(x,y)=\sum_{i=0}^{n}\sum_{j=0}^{m}a_{ij}x^iy^j$$

满足插值条件

$$P_{n,m}\left(x_i,y_j\right)=u_{ij}=u\left(x_i,y_j\right)\quad(i=0,1,\cdots,n;j=0,1,\cdots,m)\tag{5.47}$$

我们称二元多项式 $P_{n,m}(x,y)$ 为插值多项式,(x_i,y_j) $(i=0,1,\cdots,n;j=0,1,\cdots,m)$ 为插值节点.

5.6.1 Lagrange 插值

设平面矩形区域 $D=\{(x,y)\mid a\leqslant x\leqslant b,c\leqslant y\leqslant d\}$, 相应 x,y 的剖分为

$$\Delta_x:a=x_0<x_1<\cdots<x_n=b$$

$$\Delta_y:c=y_0<y_1<\cdots<y_m=d$$

则

$$\Delta=\Delta_x\times\Delta_y=\{(x_i,y_j):0\leqslant i\leqslant n,0\leqslant j\leqslant m\}\tag{5.48}$$

构成 D 的一个剖分节点.

根据一维 Lagrange 插值多项式的方法, 若关于 x 为 n 次的多项式 $l_i(x)$ 和关于 y 为 m 次的多项式 $\tilde{l}_j(y)$ 分别是关于节点 Δ_x 和 Δ_y 的 Lagrange 插值基函数, 那么关于 x 为 n 次、y 为 m 次的二元多项式 $l_{ij}(x,y)=l_i(x)\tilde{l}_j(y)$ 满足

$$l_{ij}\left(x_s,y_t\right)=l_i\left(x_s\right)\tilde{l}_j\left(y_t\right)=\delta_{is}\delta_{jt}=\begin{cases}1, & s=i,t=j\\0, & s\neq i\text{ 或 }t\neq j\end{cases}$$

$$(i,s=0,1,\cdots,n;j,t=0,1,\cdots,m)$$

且

$$P_{n,m}(x,y)=\sum_{i=0}^{n}\sum_{j=0}^{m}u\left(x_i,y_j\right)l_{ij}(x,y)\tag{5.49}$$

扫描全能王 创建

就是满足插值条件 (5.47) 的关于 x 为 n 次、y 为 m 次的二元插值多项式.

可以证明: 满足插值条件 (5.47) 的形如式 (5.49) 的插值多项式是唯一的, 而且当 $u(x,y)$ 关于 x 有 $n+1$ 阶连续导数, 关于 y 有 $m+1$ 阶连续导数时, 有

$$\max_{\substack{a\leqslant x\leqslant b\\ c\leqslant y\leqslant d}}|u(x,y)-P_{n,m}(x,y)|=O\{\max_{\substack{1\leqslant i\leqslant n\\ 1\leqslant j\leqslant m}}\left(h_i^{n+1},h_j^{m+1}\right)\}.$$

若给定的节点很多, 直接求一个二元插值多项式, 必然使得多项式次数很高, 而高次多项式容易产生 Runge 现象, 近似效果并不一定好. 因此经常采用分片低次二元多项式插值, 即将区域分成若干个小矩形或小三角形区域, 然后在每一小区域上求一个次数较低的二元插值多项式, 再将它们在小区域的边界上连接起来.

5.6.2 分片双线性插值

分片双线性插值是一维分段线性插值的直接推广. 在矩形区域 D 上

$$P_{1,1}(x,y)=\sum_{i=0}^{n}\sum_{j=0}^{m}u\left(x_i,y_j\right)\varphi_{ij}(x,y)$$

其中 $\varphi_{ij}(x,y)=l_i(x)\tilde{l}_j(y),l_i(x),\tilde{l}_j(y)$ $(0\leqslant i\leqslant n,0\leqslant j\leqslant m)$ 由式 (5.22) 给出.

若 D 上典型小区域 A_{ij} 的顶点为 $\{(x_i,y_j),(x_{i+1},y_j),(x_i,y_{j+1}),(x_{i+1},y_{j+1})\}$ $(0\leqslant i\leqslant n-1,0\leqslant j\leqslant m-1)$, 类似于式 (5.21), 分片插值多项式 $P_{1,1}(x,y)$ 在小区域 A_{ij} 上的具体形式为

$$\begin{aligned}P_{1,1}(x,y)=&u_{ij}\frac{(x-x_{i+1})(y-y_{j+1})}{(x_i-x_{i+1})(y_j-y_{j+1})}+u_{i+1,j}\frac{(x-x_i)(y-y_{j+1})}{(x_{i+1}-x_i)(y_j-y_{j+1})}+\\&u_{i,j+1}\frac{(x-x_{i+1})(y-y_j)}{(x_i-x_{i+1})(y_{j+1}-y_j)}+u_{i+1,j+1}\frac{(x-x_i)(y-y_j)}{(x_{i+1}-x_i)(y_{j+1}-y_j)},\\&(x,y)\in A_{ij},\quad 0\leqslant i\leqslant n-1,\quad 0\leqslant j\leqslant m-1\end{aligned}\tag{5.50}$$

由于在各个小矩形区域的边界上分别有 x 固定或 y 固定, 分片双线性插值是一条直线, 相邻小矩形区域上的分片插值在共有的边界上有两个公共的端点, 从而是两条重合的直线. 因此分片双线性插值在连接处是连续的, 但不能保证光滑性.

5.6.3 分片双三次 Hermite 插值

如果希望分片插值多项式在连接处有连续的偏导数, 我们考虑带有导数插值条件的分片双三次 Hermite 插值.

设平面矩形区域 $D=\{(x,y)\mid a\leqslant x\leqslant b,c\leqslant y\leqslant d\}$ 的剖分如 (5.48), 在节点 (x_i,y_j) 处给出数据

$$u_{ij}=u\left(x_i,y_j\right),\quad u'_{x,ij}=\frac{\partial u\left(x_i,y_j\right)}{\partial x},\quad u'_{y,ij}=\frac{\partial u\left(x_i,y_j\right)}{\partial y}$$

$$u''_{xy,ij}=\frac{\partial^2 u\left(x_i,y_j\right)}{\partial x\partial y}\quad(0\leqslant i\leqslant n,0\leqslant j\leqslant m)$$

扫描全能王 创建

求矩形区域 D 上的分片双三次多项式 $H_{3,3}(x,y)$ 满足

$$\begin{cases} H_{3,3}(x_i,y_j)=u_{ij} \\ \dfrac{\partial H_{3,3}(x_i,y_j)}{\partial x}=u'_{x,ij} \\ \dfrac{\partial H_{3,3}(x_i,y_j)}{\partial y}=u'_{y,ij} \\ \dfrac{\partial^2 H_{3,3}(x_i,y_j)}{\partial x\partial y}=u''_{xy,ij} \end{cases} \quad (0\leqslant i\leqslant n, 0\leqslant j\leqslant m) \tag{5.51}$$

满足条件 (5.51) 的多项式称为分片双三次 Hermite 插值.

利用式 (5.32) 和 (5.33) 给出的一维分段三次 Hermite 插值基函数 $\{h_i(x),H_i(x)\}$, 可得分片双三次 Hermite 插值

$$H_{3,3}(x,y)=\sum_{i=0}^{n}\sum_{j=0}^{m}u_{ij}h_i(x)\tilde{h}_j(y)+\sum_{i=0}^{n}\sum_{j=0}^{m}u'_{x,ij}H_i(x)\tilde{h}_j(y)+$$

$$\sum_{i=0}^{n}\sum_{j=0}^{m}u'_{y,ij}h_i(x)\tilde{H}_j(y)+\sum_{i=0}^{n}\sum_{j=0}^{m}u''_{xy,ij}H_i(x)\tilde{H}_j(y)$$

另外, 常用的分片二维样条插值也有很好的光滑性, 此处不再赘述.

5.7　快速 Fourier 变换

工程技术中常把复杂的波形分解成谐波的叠加, 数学上则表示为用三角函数的叠加来近似周期函数, 即三角函数插值. 快速 Fourier (傅里叶) 变换 (FFT) 是充分利用三角函数的周期性, 去掉重复运算, 以达到减少运算量的一种常用方法.

5.7.1　三角函数插值

三角函数插值的基本问题: 设已知 $f(x)$ 在区间 $[0,2\pi]$ 上 $N+1$ 个等距节点 $x_l=\dfrac{2\pi}{N}l$ $(l=0,1,\cdots,N)$ 处的值为 $f\left(\dfrac{2\pi}{N}l\right)=f_l$, 且 $f_0=f_N$, 求函数

$$P(x)=\sum_{k=0}^{N-1}c_k\mathrm{e}^{\mathrm{i}kx} \tag{5.52}$$

使其在节点处与 $f(x)$ 同值, 即满足插值条件

$$P(x_l)=f(x_l)=f_l \quad (l=0,1,\cdots,N) \tag{5.53}$$

显然, 函数 $P(x)$ 自然满足 $P(x_0)=P(x_N)$, 这是因为 $\mathrm{e}^{\mathrm{i}kx}$ $(k=0,1,\cdots,N-1)$ 都以 2π 为周期. 因此, 由插值条件式 (5.53) 可得 N 阶线性方程组

扫描全能王 创建

$$\boldsymbol{A}\boldsymbol{c}=\boldsymbol{f} \tag{5.54}$$

其中

$$\boldsymbol{A}=\begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \mathrm{e}^{\mathrm{i}\frac{2\pi}{N}} & \mathrm{e}^{\mathrm{i}2\frac{2\pi}{N}} & \cdots & \mathrm{e}^{\mathrm{i}(N-1)\frac{2\pi}{N}} \\ 1 & \mathrm{e}^{\mathrm{i}\frac{4\pi}{N}} & \mathrm{e}^{\mathrm{i}2\frac{4\pi}{N}} & \cdots & \mathrm{e}^{\mathrm{i}(N-1)\frac{4\pi}{N}} \\ \vdots & \vdots & \vdots & & \vdots \\ 1 & \mathrm{e}^{\mathrm{i}\frac{(N-1)2\pi}{N}} & \mathrm{e}^{\mathrm{i}2\frac{(N-1)2\pi}{N}} & \cdots & \mathrm{e}^{\mathrm{i}(N-1)\frac{(N-1)2\pi}{N}} \end{bmatrix}$$

$$\boldsymbol{c}=(c_0,c_1,\cdots,c_{N-1})^{\mathrm{T}},\quad \boldsymbol{f}=(f_0,f_1,\cdots,f_{N-1})^{\mathrm{T}}$$

记矩阵 $\boldsymbol{A}$ 的第 j 列向量为

$$\boldsymbol{a}_j=\left(1,\mathrm{e}^{\mathrm{i}j\frac{2\pi}{N}},\mathrm{e}^{\mathrm{i}j\frac{4\pi}{N}},\cdots,\mathrm{e}^{\mathrm{i}j\frac{(N-1)2\pi}{N}}\right)^{\mathrm{T}}\quad (j=0,1,\cdots,N-1)$$

由复向量的内积运算, 有

$$\begin{aligned}(\boldsymbol{a}_j,\boldsymbol{a}_k)&=\sum_{l=0}^{N-1}\mathrm{e}^{\mathrm{i}j\frac{2l\pi}{N}}\overline{\mathrm{e}^{\mathrm{i}k\frac{2l\pi}{N}}}\\&=\sum_{i=0}^{N-1}\mathrm{e}^{\mathrm{i}(j-k)\frac{2l\pi}{N}}=\begin{cases}N, & j=k\\0, & j\neq k\end{cases}\end{aligned}$$

因此, 矩阵 $\boldsymbol{A}$ 的列向量为正交向量组, 故方程组 (5.54) 有解. 以向量 $\overline{\boldsymbol{a}}_k^{\mathrm{T}}$ 左乘方程组 (5.54), 有

$$\overline{\boldsymbol{a}}_k^{\mathrm{T}}\boldsymbol{A}\boldsymbol{c}=Nc_k=\overline{\boldsymbol{a}}_k^{\mathrm{T}}\boldsymbol{f}=\sum_{l=0}^{N-1}f_l\mathrm{e}^{-\mathrm{i}k\frac{2l\pi}{N}}$$

由此可得方程组 (5.54) 的解

$$c_k=\frac{1}{N}\sum_{l=0}^{N-1}f_l\mathrm{e}^{-\mathrm{i}\frac{2l\pi}{N}}\quad (k=0,1,\cdots,N-1) \tag{5.55}$$

将式 (5.55) 代入式 (5.52), 即得函数 $f(x)$ 的三角插值函数. 通过式 (5.55) 由数据 $\{f_l\}_{l=0}^{N-1}$ 求系数 $\{c_k\}_{k=0}^{N-1}$ 的过程称为 $f(x)$ 的有限离散 Fourier 变换, 简称 DFT (discrete Fourier transform).

反之, 由式 (5.54) 容易得出

$$f_l=\sum_{k=0}^{N-1}c_k\mathrm{e}^{\mathrm{i}k\frac{2l\pi}{N}}\quad (l=0,1,\cdots,N-1) \tag{5.56}$$

按式 (5.56) 由数据 $\{c_k\}_{k=0}^{N-1}$ 求出 $\{f_l\}_{l=0}^{N-1}$ 的过程称为有限离散 Fourier 反变换.

无论是有限离散 Fourier 变换还是其反变换, 都只是作复数加法与乘法运算, 计算是简便易行的. 但当 N 较大时, 由于每求一个 c_k 就要作 N 次复数乘法与 $N-1$ 次加法运算, 整个过程需作 N^2 次乘法、$N(N-1)$ 次加法与 N 次除法, 故计算量十分可观. 事实上, 由

扫描全能王 创建

三角函数的周期性、式 (5.55) 与式 (5.56) 的计算中有许多是相同的, 如果加以合并, 就可以减少运算次数, 特别是乘法运算次数. 基于这一想法, 提出了快速 Fourier 变换算法.

5.7.2　快速 Fourier 变换

DFT 运算很重要, 但在很长一段时间里, 由于 DFT 运算复杂, 并没有得到真正的运用, 而频谱分析仍大多采用模拟信号滤波的方法解决. 直到 1965 年, 当 Cooley 和 Tukey 首次提出 DFT 运算的一种快速算法以后, 情况才发生了根本变化. 人们开始认识到 DFT 运算的一些内在规律, 从而很快地发展和完善了一套高速有效的运算方法, 被称为快速 Fourier 变换算法. FFT 的出现, 使 DFT 的运算大大简化.

取 $N=2^m, m$ 为正整数, 令 $w_N=\mathrm{e}^{-\mathrm{i}\frac{2\pi}{N}}, a_l=\dfrac{f_l}{N}, w_N$ 通常被称为指数因子或旋转因子, 则式 (5.55) 可改写成

$$c_k=\frac{1}{N}\sum_{l=0}^{N-1}f_l\mathrm{e}^{-\mathrm{i}k\frac{2l\pi}{N}}=\sum_{l=0}^{N-1}a_l w_N^{kl}\quad(k=0,1,\cdots,N-1)\tag{5.57}$$

如果直接利用式 (5.57) 计算 $\{c_k\}$ 需要 N^2 次乘法运算. 快速 Fourier 变换的基本思想是: 充分利用指数因子的性质, 减少重复计算.

下面我们列出在简化重复计算过程中用到的指数因子的性质:

(1) 指数因子的特殊值

$w_N^0=w_N^N=w_N^{kN}=1,\quad w_N^{\frac{N}{2}}=-1,\quad w_N^{\frac{N}{4}}=-\mathrm{i},\quad w_N^{\frac{3N}{4}}=\mathrm{i}$

(2) 指数因子的周期性

指数因子 $w_N=e^{-\mathrm{i}\frac{2\pi}{N}}$ 以 N 为周期, 即

$$w_N^{kn}=w_N^{k(n+N)}=w_N^{(k+N)n}=w_N{}^{(k+IN)(n+mN)}$$

(3) 指数因子的可约性

$$w_{2N}^k=w_N^{\frac{k}{2}},\quad w_{mN}^{mk}=w_N^k$$

$$w_N^{kn}=w_{mN}^{mkn}=w_{\frac{N}{m}}^{\frac{kn}{m}}$$

(4) 指数因子的半对称性

指数因子 $w_N=\mathrm{e}^{-\mathrm{i}\frac{2\pi}{N}}$ 以半个周期 $N/2$ 为反对称, 即

$$w_N^{k+\frac{N}{2}}=-w_N^k$$

(5) 指数因子的共轭对称性

$$\left(w_N^{kn}\right)^*=w_N^{-kn}=w_N^{(N-k)n}=w_N^{k(N-n)}$$

将 $x(-n)=x^*(n)$ 称为共轭偶对称, $x(-n)=-x^*(n)$ 称为共轭奇对称.

(6) 指数因子的正交性

$$\sum_{k=0}^{N-1}w_N^{kn}=\begin{cases}N, & n=rN, r\text{ 为整数}\\ 0, & \text{其他 } n\end{cases}$$

扫描全能王 创建

利用上面性质可以将 N 个点 DFT 化为 2 个 $N/2$ 个点的 DFT, 具体为

$$\begin{aligned} c_k &= \sum_{l=0}^{N-1} a_l w_N^{kl} \quad (k=0,1,\cdots,N-1) \\ &= \sum_{r=0}^{N/2-1} a_{2r} w_N^{k2r} + \sum_{r=0}^{N/2-1} a_{2r+1} w_N^{k(2r+1)} \\ &= \sum_{r=0}^{N/2-1} a_{2r} w_{N/2}^{kr} + w_N^k \sum_{r=0}^{N/2-1} a_{2r+1} w_{N/2}^{kr} \end{aligned} \tag{5.58}$$

令

$$X_k^1 = \sum_{r=0}^{N/2-1} a_{2r} w_{N/2}^{kr}, \quad X_k^2 = \sum_{r=0}^{N/2-1} a_{2r+1} w_{N/2}^{kr}$$

注 5.7 $\{X_k^1\}$ 为 $N/2$ 个偶数位置数 $(a_0, a_2, \cdots, a_{N-2})$ 的 DFT, 而 $\{X_k^2\}$ 为 $N/2$ 个奇数位置数 $(a_1, a_3, \cdots, a_{N-1})$ 的 DFT.

容易验证

$$X_{N/2+k}^1 = X_k^1, \quad X_{N/2+k}^2 = X_k^2 \quad (k=0,1,\cdots,N/2-1)$$

如果 $\{X_k^1\}, \{X_k^2\}$ 已求出, 则有

$$\begin{aligned} c_k &= X_k^1 + w_N^k X_k^2 \qquad (k=0,1,\cdots,N/2-1) \\ c_{N/2+k} &= X_k^1 - w_N^k X_k^2 \qquad (k=0,1,\cdots,N/2-1) \end{aligned} \tag{5.59}$$

利用式 (5.59) 计算 $\{c_k\}$ 只需要 $N/2$ 次乘法运算.

继续利用这种化简的思想计算 $\{X_k^1\}, \{X_k^2\}$, 每次化简都把 DFT 化为 2 个半数的 DFT, 直到计算 $N/2$ 组 2 个数的 DFT. 基于这种思想的化简就是快速 Fourier 变换 (FFT).

递归在计算机运算中效率不够高, 实际进行 FFT 时, 我们通常用分割、组装两个步骤进行.

下面我们以 $N=8$ 为例进行说明 [6].

设 $\boldsymbol{a} = (a_0, a_1, \cdots, a_7)^{\mathrm{T}}$, 则

(1) 分割 (重排序)

(1.1) 第一步分割:

$$\boldsymbol{a}_e = (a_0, a_2, a_4, a_6)^{\mathrm{T}}, \quad \boldsymbol{a}_o = (a_1, a_3, a_5, a_7)^{\mathrm{T}}$$

(1.2) 第二步分割:

$$\begin{aligned} \boldsymbol{a}_{ee} &= (a_0, a_4)^{\mathrm{T}}, \quad \boldsymbol{a}_{eo} = (a_2, a_6)^{\mathrm{T}} \\ \boldsymbol{a}_{oe} &= (a_1, a_5)^{\mathrm{T}}, \quad \boldsymbol{a}_{oo} = (a_3, a_7)^{\mathrm{T}} \end{aligned}$$

(1.3) 第三步分割:

$$\begin{aligned} a_{eee} &= a_0, \quad a_{eeo} = a_4, \quad a_{eoe} = a_2, \quad a_{eoo} = a_6 \\ a_{oee} &= a_1, \quad a_{oeo} = a_5, \quad a_{ooe} = a_3, \quad a_{ooo} = a_7 \end{aligned}$$

(2) 组装

(2.1) 第一步组装:

扫描全能王 创建

$$
\begin{aligned}
\boldsymbol{c}_{ee} &= \left(a_0 + \omega_2^0 a_4, a_0 - \omega_2^0 a_4\right)^{\mathrm{T}} \\
\boldsymbol{c}_{eo} &= \left(a_2 + \omega_2^0 a_6, a_2 - \omega_2^0 a_6\right)^{\mathrm{T}} \\
\boldsymbol{c}_{oe} &= \left(a_1 + \omega_2^0 a_5, a_1 - \omega_2^0 a_5\right)^{\mathrm{T}} \\
\boldsymbol{c}_{oo} &= \left(a_3 + \omega_2^0 a_7, a_3 - \omega_2^0 a_7\right)^{\mathrm{T}}
\end{aligned}
$$

(2.2) 第二步组装:

$$
\boldsymbol{c}_e = \begin{pmatrix} \boldsymbol{c}_{ee} + \boldsymbol{w}_4 \circ \boldsymbol{c}_{eo} \\ \boldsymbol{c}_{ee} - \boldsymbol{w}_4 \circ \boldsymbol{c}_{eo} \end{pmatrix}, \quad \boldsymbol{c}_o = \begin{pmatrix} \boldsymbol{c}_{oe} + \boldsymbol{w}_4 \circ \boldsymbol{c}_{oo} \\ \boldsymbol{c}_{oe} - \boldsymbol{w}_4 \circ \boldsymbol{c}_{oo} \end{pmatrix}
$$

(2.3) 第三步组装:

$$
\boldsymbol{c} = \begin{pmatrix} \boldsymbol{c}_e + \boldsymbol{w}_8 \circ \boldsymbol{c}_o \\ \boldsymbol{c}_e - \boldsymbol{w}_8 \circ \boldsymbol{c}_o \end{pmatrix}
$$

这里 $\boldsymbol{w}_4 \overset{\text{def}}{=} \left(w_4^0, w_4^1\right)^{\mathrm{T}}, \boldsymbol{w}_8 \overset{\text{def}}{=} \left(w_8^0, w_8^1, w_8^2, w_8^3\right)^{\mathrm{T}}, \boldsymbol{X} \circ \boldsymbol{Y} \overset{\text{def}}{=} \left(x_j y_j\right)_j$ 为各分量分别相乘得到的向量. 整个过程可简单地图示如下:

$$
\boldsymbol{a} \xrightarrow{\text{重排序}} \begin{pmatrix} a_0 \\ a_4 \\ a_2 \\ a_6 \\ a_1 \\ a_5 \\ a_3 \\ a_7 \end{pmatrix} \xrightarrow{\text{第一步组装}} \begin{pmatrix} \boldsymbol{c}_{ee} \\ \boldsymbol{c}_{eo} \\ \boldsymbol{c}_{oe} \\ \boldsymbol{c}_{oo} \end{pmatrix} \xrightarrow{\text{第二步组装}} \begin{pmatrix} \boldsymbol{c}_e \\ \boldsymbol{c}_o \end{pmatrix} \xrightarrow{\text{第三步组装}} \boldsymbol{c}
$$

如果令 e 对应 $0, o$ 对应 1, 比较重排序之后的指标, 我们可以发现前后指标对应的正好是二进制位的逆序关系, 即

$$
\begin{array}{ll}
0 = 000_2; & 000_2 = 0; \\
1 = 001_2; & 100_2 = 4; \\
2 = 010_2; & 010_2 = 2; \\
3 = 011_2; \xrightarrow{\text{逆序}} & 110_2 = 6; \\
4 = 100_2; & 001_2 = 1; \\
5 = 101_2; & 101_2 = 5; \\
6 = 110_2; & 011_2 = 3; \\
7 = 111_2; & 111_2 = 7.
\end{array} \tag{5.60}
$$

显然, 如果 $N = 2^m$, 一共需要分组 m 次, 组装 m 次, 每次的乘法运算量为 $N/2$, 所以 FFT 总的计算量为 $\dfrac{N}{2} \log_2 N$.

可以比较一下 FFT 与 DFT 的计算工作量, 取 $N = 1\ 024 \approx 2^{10} \approx 1\ 000$, 直接计算 DFT 的复数乘法次数为

扫描全能王 创建

$$N^2 = 1\ 024^2 = 1\ 048\ 576 \text{ (约一百万)}$$

FFT 计算 DFT 的复数乘法次数为

$$\frac{N}{2}\log_2 N = 512 \times 10 = 5\ 120 \text{ (约五千)}$$

二者复数乘法次数的比值为

$$\frac{N^2}{\frac{N}{2}\log_2 N} = 204.8 \text{ (约两百)}$$

采用 FFT 计算 DFT, 比直接计算 DFT 的效率高得多.

5.8 算法实现与应用实例

北太天元在插件 spline 中提供了计算样条插值的若干个函数. 我们可以采用北太天元实现 Lagrange 插值运算, 编写脚本文件 lagrange.m 如下：

```
function y=lagrange( xp, yp, x )
n=length(xp);    % 数据 xp 的长度
m=length(x);    % 数据 x 的长度
y=zeros(1,m);    % y 和 x 长度一样, 且初值为 0
for i=1:n
  t=ones(1,m);    % 存储 lagrange 基函数
  for j=1:i-1    % 开始计算第 i 个基函数
    t=t.*(x-xp(j))/(xp(i)-xp(j));
  end
  for j=i+1:n
    t=t.*(x-xp(j))/(xp(i)-xp(j));
  end
  y=y+yp(i)*t;    % 组合第 i 个基函数
end
  % plot(xp, yp,'*',x, y) 可以实现插值结果可视化
```

其中 xp, yp 是插值数据; x 为待计算函数值的点组成的数组. 例如, 在命令行窗口中输入

≫xp=[11　12　13];

≫yp=[2.3979　2.4849　2.5649]

≫x=[11: 0.1: 13]

输入命令：

≫y= lagrange (xp, yp, x)

即可得到函数在相应点处的一组近似值 y.

也可以采用 Newton 插值方法, 具体通过 Newton.m 脚本文件实现

扫描全能王 创建

```
function y=Newton( xp, yp, x)
n=length(xp) ;   % 数据 xp 的长度
fp=yp;   % 后面用 fp 存储各阶差商
for i=2:n   % 此处开始计算各阶差商
  for k=n:-1:i   % 注意这里一定是倒序, 否则原来的点就被覆盖了
    fp(k)=(fp(k)-fp(k-1))/(xp(k)-xp(k-i+1));   % i 从 2 开始的
  end
end   % 注意向量序数从 1 开始,fp(k) 对应的是 k-1 阶差商
y=zeros(size(x));  % 利用秦九韶算法计算 Newton 插值
for k=n:-1:2
  y=(y+fp(k)).*(x-xp(k-1));
end
y=y+fp(1);
```

通过 linearinterp.m 脚本文件, 可以实现分段线性插值：

```
function y=linearinterp( xp, yp, x )
n=length(xp);   % 数据 xp 的长度
m=length(x);   % 数据 x 的长度
y=zeros(1,m);   % y 和 x 长度一样, 且初值为 0
for j=1:m
  for i=1:n-1
    if (x(j)>xp(i)&&x(j)<=xp(i+1))
      y(j) = ((x(j)-xp(i+1))/(xp(i)-xp(i+1)))*yp(i)+(((x(j)-xp(i))/(xp(i+1)-xp(i)))*yp
      (i+1));
    end
  end
end
```

例 5.9　利用 Lagrange 插值和分段线性插值近似来自极坐标曲线 $\rho=5\sin\frac{\theta}{3}$ 的数据, 并作出平滑的曲线.

解　方程为极坐标方程, 由于曲线有自交点, 且 x 与 y 的关系非一一对应的, 我们选 θ 为自变量, 分别对参数方程 $x(\theta)$ 和 $y(\theta)$ 进行插值, 然后在直角坐标 xOy 平面上作图.

首先根据函数可直接生成插值数据, 为使曲线较光滑, 再生成更细密的待计算的插值点, 可画出原曲线图.

```
≫thetap =0 : 0.5 : 3 * pi;   % 取以 0.5 为间隔的 θ 的等分节点
≫r=5 *sin ( thetap/3) ;   % 计算等分点上的极径
≫xp=r . * cos (thetap ) ;   % 计算直角坐标系下的 x 坐标
≫yp=r . * sin(thetap);   % 计算直角坐标系下的 y 坐标
≫theta =0: 0.05: 3 * pi; r0=5 * sin ( theta/ 3);   % 取更密的等分点
≫x0=r0.* cos ( theta) ; y0=r0. * sin ( theta );   % 计算直角坐标系下坐标
≫plot (xp ,yp ,'.', x0, y0,'-');   % 画插值节点及原函数曲线
```


扫描全能王 创建

我们可以利用函数 lagrange(), linearinterp() 分别实现 Lagrange 插值、分段线性插值的计算. 在直角坐标系下插值结果计算结果如图 5.7、图 5.8 所示.

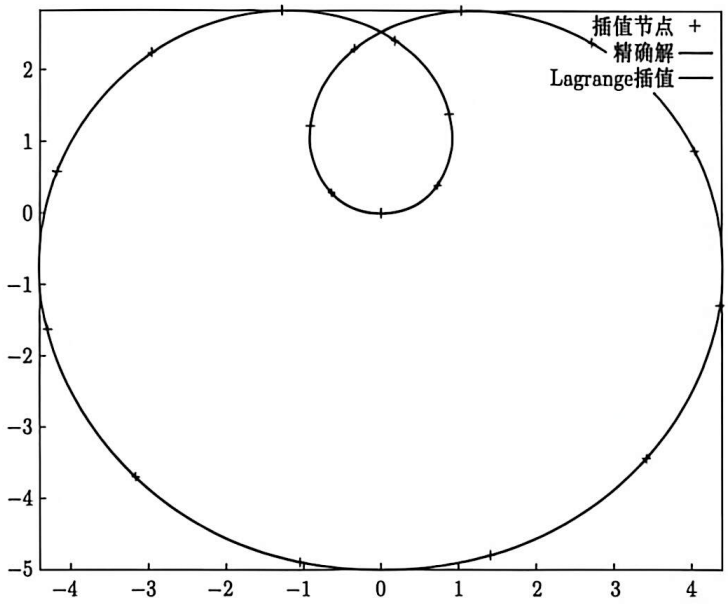


图 5.7 Lagrange 插值结果

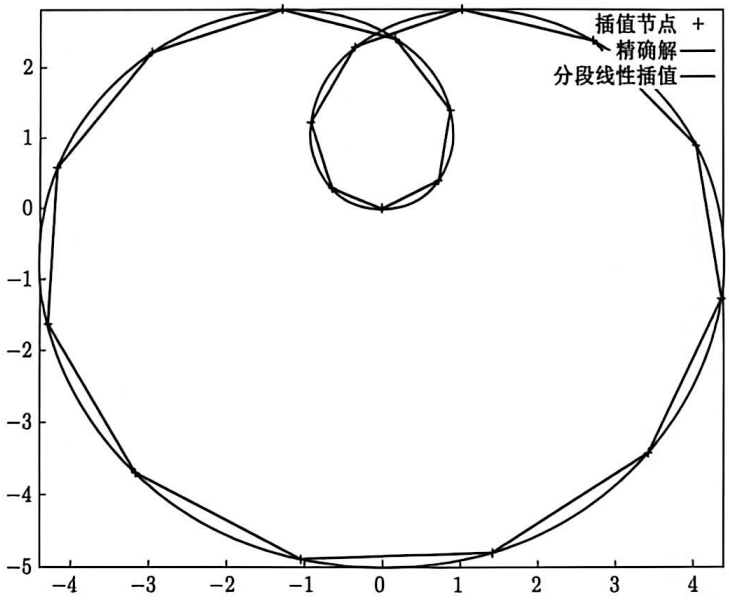


图 5.8 分段线性插值结果

≫x1= lagrange (thetap , xp, theta) ; y1= lagrange (thetap, yp, theta);

≫x2= linearinterp (thetap, xp, theta); y2=linearinterp (thetap, yp, theta);

≫plot (xp, yp ,′.′, x1, y1 ,′−′)

≫plot(xp, yp,′.′, x2, y2,′−′)

从图形可见 Lagrange 插值效果较理想, 分段线性插值需要更多的数据才能有较好的近

扫描全能王 创建

似效果.

此外，北太天元中的函数 c=fft(f) 可以实现快速傅里叶变换, f=ifft(c) 可进行快速傅里叶反变换.

评注五

函数插值是数值积分和数值微分的基础, 也是微分方程数值解法的常用工具. 本章讨论了常用的 Lagrange 插值公式、Newton 插值公式和 Hermite 插值公式, 给出了各插值公式的误差表达式. 若被插值函数充分光滑, 由插值误差表达式可以给出插值近似值的误差限. 由于高次插值的 Runge 现象, 实际计算中很少采用高次插值, 而分段低次插值却受到重视, 如分段线性插值、分段三次 Hermite 插值和三次样条插值. 只要插值节点足够稠密, 分段低次插值是能够达到总体精度要求的.

样条插值的理论和方法自 20 世纪 60 年代以来有了飞速的发展. 本章仅介绍了三弯矩算法和三转角算法. 分段三次样条插值不仅是光滑的, 而且函数及其一、二阶导数均具有一致收敛性. 至于更便于计算的 B– 样条函数, 读者可参阅文献 [22].

二维及多元函数的插值有着广泛的应用领域. 利用构造基函数的方法很容易将一维插值方法推广到多维的情形. 常用的方法是分片双一次插值、分片双三次 Hermite 插值和双三次样条插值. 在飞机、轮船等的外形设计中常用的是 Coons (昆氏) 曲面插值. 详细可参阅文献 [23].

本章知识要点　Lagrange插值法；Newton插值法；Hermit插值；代数精确度；截断误差；分段线性插值.

算法历史及背后的数学家

习题五

1. 已知函数表如下:

x_i	-1	0	1
$y_i = 2^{x_i}$	0.5	1	2

(1) 利用线性插值计算 $2^{0.3}$ 的近似值并估计误差;

(2) 利用二次插值计算 $2^{0.3}$ 的近似值并估计误差.

2. 已知函数表如下:

x_i	0.527	0.727	0.807	0.927
y_i	0.010 75	0.012 19	0.011 88	0.014 26

扫描全能王 创建

用二次插值求 $y(0.7)$ 的近似值.

3. 已知函数表如下:

x_i	1	3	4	6
$f(x_i)$	-7	5	8	14

试求其三阶 Lagrange 插值多项式, 并以此计算 $f(2)$ 的近似值.

4. 已知函数表如下:

x_i	0.40	0.55	0.65	0.80	0.90
$y_i = \operatorname{sh} x_i$	0.410 75	0.578 15	0.696 75	0.888 11	1.026 52

利用四次插值计算 $\operatorname{sh} 0.569$ 的近似值, 并估计误差.

5. 已知函数表如下:

x_i	2.0	2.1	2.2	2.3	2.4
$y_i = \sqrt{x_i}$	1.414 214	1.449 138	1.483 240	1.516 575	1.549 193

试用多项式插值计算 $\sqrt{2.15}$ 的近似值, 要求误差不超过 $\dfrac{1}{2} \times 10^{-5}$.

6. 已知 $f(x) = \mathrm{e}^{x^2-1}$ 的函数表如下:

x_i	1	1.1	1.2	1.3	1.4
$f(x_i)$	1.000 00	1.233 68	1.552 71	1.993 72	2.611 70

(1) 利用四次 Lagrange 插值计算 $f(1.25)$ 的近似值, 并估计误差;

(2) 求 $f(x)$ 的四次 Newton 插值多项式, 并计算 $f(1.25)$ 的近似值;

(3) 增加节点 $x_5 = 1.5, f(x_5) = 3.490\ 34$, 求 $f(x)$ 的五次 Newton 插值多项式, 并计算 $f(1.25)$ 的近似值.

7. 设 $x_0, x_1, \cdots, x_n$ 为 $n+1$ 个互异节点, $l_j(x)$ $(j = 0, 1, \cdots, n)$ 为这组节点上的 n 次 Lagrange 插值基函数, 试证:

(1) $\displaystyle\sum_{j=0}^{n} (x_j - x)^k l_j(x) \equiv 0 \quad (k = 1, 2, \cdots, n)$;

(2) 设 $P(x)$ 是任意一个最高次项系数为 1 的 $n+1$ 次多项式, 则

$$P(x) - \sum_{j=0}^{n} P(x_j)\, l_j(x) = \omega_{n+1}(x) = \prod_{j=0}^{n} (x - x_j)$$

8. 已知函数表如下:

x_i	1.615	1.634	1.702	1.828
$f(x_i)$	2.414 50	2.462 59	2.652 71	3.030 35

扫描全能王 创建

试求其三阶 Newton 插值多项式, 并以此计算 $f(1.682)$ 的近似值.

9. 设有函数 $\varphi(x)=\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}\mathrm{e}^{-\frac{1}{2}t^2}\,\mathrm{d}t$ 的函数表如下:

x	0.10	0.11	0.12	0.13
$\varphi(x)$	0.539 828	0.543 795	0.547 758	0.551 717

试给出 $\varphi(0.123\ 6)$ 的值, 要求截断误差不超过 $\frac{1}{2}\times10^{-5}$. 如果要求结果有 6 位有效数字, $\varphi(0.123\ 6)$ 为多少?

10. 已知函数表如下:

x_i	1	2	3	4
$f(x_i)$	3	5	9	15

分别用 Newton 向前、向后插值公式计算 $f(1.5), f(3.7)$ 的近似值.

11. 已知 $y=\cos x$ 的函数表如下:

x_i	0.0	0.1	0.2	0.3	0.4	0.5	0.6
$y_i=\cos x_i$	1.000 00	0.995 00	0.980 07	0.955 34	0.921 06	0.877 58	0.825 34

分别用四阶 Newton 向前、向后插值公式计算 $\cos 0.048$ 和 $\cos 0.575$ 的近似值.

12. 在 $[0,\pi/2]$ 上给出 $f(x)=\sin x$ 的等距节点函数表, 为保证用分段线性插值求非节点处的正弦值时, 其截断误差不超过 10^{-6}, 函数表的步长至少应取多大? 若改用分段二次插值求非节点处的正弦值, 步长应取多大?

13. 设 $f(x)=x^7+5x^5+1$, 求差商:

(1) $f[2^0,2^1,\cdots,2^7]$;

(2) $f\left[2^0,2^1,\cdots,2^k\right]\ (k\geqslant 8)$.

14. 设 $f(x)$ 是一个 n 次多项式, $f(x)=\sum_{k=0}^{n}a_kx^k\ (a_n\neq0)$, 试证:

$$f[x_0,x_1,\cdots,x_m,x]=\begin{cases}n-m-1\text{ 次多项式}, & m\leqslant n-1\\ a_n, & m=n-1\\ 0, & m>n-1\end{cases}$$

15. 设 $f(x)=3x\mathrm{e}^x-\mathrm{e}^{2x}$, 节点 $x_0=1,x_1=1.05$, 求 $f(x)$ 的三次 Hermite 插值多项式及 $f(1.03)$ 的近似值, 并估计误差.

16. 给定函数表

扫描全能王 创建

x_i	0	1	2
$f(x_i)$	1	2	1
$f'(x_i)$		0	-1

且已知 $f(x)$ 在 $[0,2]$ 上五阶连续可导, 求 $f(x)$ 的四次 Hermite 插值多项式 $H_4(x)$, 并写出余项 $f(x)-H_4(x)$ 的表示式.

17. 已知函数表如下:

x_i	0	1	2	3
y_i	0	0	0	0

边界条件为

(1) $m_0=1, m_3=0$;

(2) $M_0=1, M_3=0$,

分别求三次样条插值函数.

18. 已知函数表如下：

x_i	0.25	0.30	0.39	0.45	0.53
y_i	0.500 0	0.547 7	0.624 5	0.672 8	0.278 0

求自然边界条件下的三次样条插值函数.

19. 已知函数表如下:

x_i	1	2	3
$f(x_i)$	2	4	8

求满足边界条件 $f'(1)=1.386\ 3, f'(3)=5.545\ 2$ 的三次样条插值函数.

20. 已知 $f(x)=\mathrm{e}^{-x}$, 取节点 $x_0=0, x_1=0.2, x_2=0.6, x_3=1.0$.

(1) 求 $f(x)$ 在自然边界条件下的三次样条插值函数;

(2) 求 $f(x)$ 满足边界条件 $f'(0)=-1, f'(1)=-0.367\ 9$ 的三次样条插值函数;

(3) 利用样条插值函数计算 $\int_0^1 f(x)\mathrm{d}x$ 的近似值.

21. 设 $f_l=(l-2)^2$ $(l=0,1,2,3)$, 分别用 DFT 法和 FFT 法求 $\{c_k\}_{k=0}^3$, 并比较它们的计算量.

22. 试对 $N=2^4=16$ 给出 FFT 的计算公式及流程图.

数值实验五

1. 试应用北太天元软件编写脚本函数实现 Newton 插值, 要求能输出插值多项式. 对函数 $f(x)=\dfrac{1}{1+4x^2}$ 在区间 $[-5,5]$ 上实现 10 次多项式插值.

扫描全能王 创建

(1) 输出插值多项式;

(2) 在区间 $[-5,5]$ 内均匀插入 99 个节点, 计算这些节点上函数 $f(x)$ 的近似值, 并在同一张图中画出原函数和插值多项式的图形;

(3) 观察 Runge 现象, 计算插值函数在各节点上的误差, 并画出误差图.

2. 试应用北太天元软件编写函数分别实现分段线性插值、分段三次 Hermite 插值和三次样条插值. 将区间 $[-5,5]$ 分成 10 等份, 对上述函数 $f(x)=\dfrac{1}{1+4x^2}$ 调用编写的程序, 画出相应的插值函数图形, 计算误差比较各种插值的精确度.

3. 已知直升机旋转机翼外形轮廓线上某些点的数据:

k	x_k	y_k	k	x_k	y_k
1	520	0	8	3.1	9.4
2	280	−30	9	39.62	28.44
3	156.6	−36	10	78	35
4	78	−35	11	156.6	36
5	39.62	−28.44	12	280	30
6	3.1	−9.4	13	520	0
7	0	0			

以 k 为自变量, 用表中 13 个节点上的数据, 分别作出 $x(k),y(k)$ 的样条插值. 平均增加节点至 121 个, 在 xOy 平面上绘制出较光滑的机翼外形曲线图.

4. 试分别编写矩形网格上的双线性插值和三次样条插值, 对平面区域 $D=[-3,3]\times[-2,2]$ 上函数 $f(x,y)=(x^2-2x)\,\mathrm{e}^{-x^2-y^2-xy}$, 先以 0.5 为步长产生较粗糙的网格, 生成插值数据, 再以 0.2 为步长产生细密的网格. 调用编写的函数作双线性插值和三次样条插值, 计算函数的近似值, 画出近似曲面图形. 进一步将近似值与精确值比较, 求出各网格点上的绝对误差, 画出误差图形.

扫描全能王 创建

/第六章/

函数逼近

曲线拟合又称函数逼近, 是求近似函数的又一类数值方法. 它不要求近似函数在节点处与函数同值, 即不要求近似曲线过已知点, 只要求它尽可能反映给定数据点的基本趋势, 在某种意义下与函数最“逼近”. 下面我们先举例说明.

例 6.1 给定一组实验数据如下:

i	1	2	3	4
x_i	2	4	6	8
y_i	1.1	2.8	4.9	7.2

求 x, y 的函数关系.

解 先作草图. 如图 6.1所示, 这些点的分布接近一条直线, 因此可设想, y 为 x 的一次函数. 设

$$y = a_1 x + a_0 \tag{6.1}$$

从图 6.1不难看出, 无论 a_0, a_1 取何值, 直线都不可能同时过全部数据点. 怎样选取 a_0, a_1, 才能使直线 (6.1)“最好”地反映数据点的基本趋势? 首先要建立好坏的标准.

假设 a_0, a_1 已经确定, $y_i^* = a_1 x_i + a_0$ $(i = 1, 2, 3, 4)$ 为由近似函数求得的近似值, 它与观测值 y_i 之差

$$\delta_i = y_i - y_i^* = y_i - a_1 x_i - a_0 \quad (i = 1, 2, 3, 4) \tag{6.2}$$

称为残差. 显然, 残差的大小可作为衡量近似函数好坏的标准. 常用的准则有以下三种:

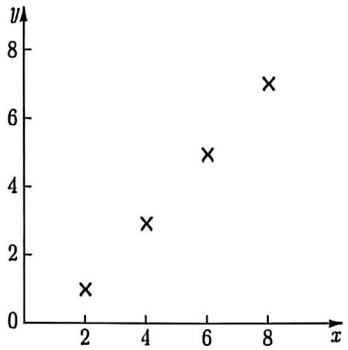


图 6.1 例 6.1 中数据点的位置

扫描全能王 创建

(1) 使残差的绝对值之和最小, 即

$$\min \sum_i |\delta_i|$$

(2) 使残差的最大绝对值最小, 即

$$\min \max_i |\delta_i|$$

(3) 使残差的平方和最小, 即

$$\min \sum_i \delta_i^2$$

准则 (1) 的提出很自然, 也合理, 但实际使用不便. 按准则 (2) 求近似函数的方法称为函数的最佳一致逼近. 按准则 (3) 确定参数, 求得近似函数的方法称为最佳平方逼近, 也称曲线拟合 (或数据拟合) 的最小二乘法. 它的计算比较简便, 是实践中常用的一种函数逼近方法.

6.1　数据拟合的最小二乘法

数据拟合的最小二乘问题是: 根据给定的数据组 $(x_i, y_i)\,(i = 1, 2, \cdots, n)$, 选取近似函数形式, 即给定函数类 H, 求函数 $\varphi(x) \in H$, 使得

$$\sum_{i=1}^{n} \delta_i^2 = \sum_{i=1}^{n} [y_i - \varphi(x_i)]^2$$

为最小, 即

$$\sum_{i=1}^{n} [y_i - \varphi(x_i)]^2 = \min_{\psi \in H} \sum_{i=1}^{n} [y_i - \psi(x_i)]^2$$

这种求近似函数的方法称为数据拟合的最小二乘法, 函数 $\varphi(x)$ 称为这组数据的最小二乘函数. 通常取 H 为一些比较简单函数的集合, 如低次多项式、指数函数等. 例 6.1中取 H 为一次多项式集合.

6.1.1　多项式拟合

对给定的数据组 $(x_i, y_i)\,(i = 1, 2, \cdots, n)$, 求一个 m 次多项式 $(m < n)$

$$P_m(x) = a_0 + a_1 x + \cdots + a_m x^m \tag{6.3}$$

使得

扫描全能王 创建

$$\sum_{i=1}^{n}\delta_i^2=\sum_{i=1}^{n}\left[y_i-P_m\left(x_i\right)\right]^2=F\left(a_0,a_1,\cdots,a_m\right) \tag{6.4}$$

为最小, 即选取参数 $a_i\ (i=0,1,\cdots,m)$, 使得

$$F\left(a_0,a_1,\cdots,a_m\right)=\sum_{i=1}^{n}\left[y_i-P_m\left(x_i\right)\right]^2=\min_{\psi\in H}\sum_{i=1}^{n}\left[y_i-\psi\left(x_i\right)\right]^2$$

其中 H 为至多 m 次多项式集合. 这就是数据的多项式拟合, $P_m(x)$ 称为这组数据的最小二乘 m 次拟合多项式.

与 2.5 节求超定线性方程组的最小二乘解的方法相同, 由多元函数取极值的必要条件, 得方程组

$$\frac{\partial F}{\partial a_j}=-2\sum_{i=1}^{n}\left[y_i-\sum_{k=0}^{m}a_kx_i^k\right]x_i^j=0\quad(j=0,1,\cdots,m)$$

移项得

$$\sum_{k=0}^{m}a_k\left(\sum_{i=1}^{n}x_i^{k+j}\right)=\sum_{i=1}^{n}y_ix_i^j\quad(j=0,1,\cdots,m)$$

即

$$\begin{cases}na_0+a_1\sum\limits_{i=1}^{n}x_i+a_2\sum\limits_{i=1}^{n}x_i^2+\cdots+a_m\sum\limits_{i=1}^{n}x_i^m=\sum\limits_{i=1}^{n}y_i\\ a_0\sum\limits_{i=1}^{n}x_i+a_1\sum\limits_{i=1}^{n}x_i^2+a_2\sum\limits_{i=1}^{n}x_i^3+\cdots+a_m\sum\limits_{i=1}^{n}x_i^{m+1}=\sum\limits_{i=1}^{n}y_ix_i\\ \cdots\cdots\cdots\cdots\\ a_0\sum\limits_{i=1}^{n}x_i^m+a_1\sum\limits_{i=1}^{n}x_i^{m+1}+a_2\sum\limits_{i=1}^{n}x_i^{m+2}+\cdots+a_m\sum\limits_{i=1}^{n}x_i^{2m}=\sum\limits_{i=1}^{n}y_ix_i^m\end{cases} \tag{6.5}$$

这是最小二乘拟合多项式的系数 $a_k\ (k=0,1,\cdots,m)$ 应满足的方程组, 称为正则方程组或法方程组. 由函数组 $\{1,x,x^2,\cdots,x^m\}$ 的线性无关性可以证明, 方程组(6.5)存在唯一解, 且解所对应的多项式(6.3)必定是已给数据组 $(x_i,y_i)\ (i=1,2,\cdots,n)$ 的最小二乘 m 次拟合多项式.

如图 6.1表明, 可用一次多项式(6.1)拟合例 6.1中数据组所给出的函数关系. 将数据代入正则方程组 (6.5)得

$$\begin{cases}4a_0+20a_1=16\\ 20a_0+120a_1=100.4\end{cases}$$

其解为 $a_0=-1.1,a_1=1.02$, 所以

$$y=1.02x-1.1$$

就是所给数据组的最小二乘拟合直线.

扫描全能王 创建

例 6.2　求数据表

i	1	2	3	4	5	6	7	8	9
x_i	-1	-0.75	-0.5	-0.25	0	0.25	0.5	0.75	1
y_i	$-0.220\ 9$	$0.329\ 5$	$0.882\ 6$	$1.439\ 2$	$2.000\ 3$	$2.564\ 5$	$3.133\ 4$	$3.760\ 1$	$4.283\ 6$

的最小二乘二次拟合多项式.

解　设二次拟合多项式为 $P_2(x)=a_0+a_1x+a_2x^2$, 将数据表代入式(6.5), 得此问题的正则方程组

$$\begin{cases} 9a_0+0+3.75a_2=18.172\ 3 \\ 0+3.75a_1+0=8.484\ 2 \\ 3.75a_0+0+2.765\ 6a_2=7.617\ 3 \end{cases}$$

其解为 $a_0=2.003\ 4, a_1=2.262\ 5, a_2=0.037\ 8$. 所以此数据组的最小二乘二次拟合多项式为

$$P_2(x)=2.003\ 4+2.262\ 5x+0.037\ 8x^2$$

6.1.2　指数拟合

如果数据点 $(x_i,y_i)\,(i=1,2,\cdots,n)$ 的分布近似指数曲线, 则可考虑用指数函数

$$y=b\mathrm{e}^{ax} \tag{6.6}$$

去拟合数据, 按最小二乘原理, a,b 的选取应使得

$$F(a,b)=\sum_{i=1}^{n}(y_i-b\mathrm{e}^{ax_i})^2$$

为最小. 由此导出的正则方程组是关于参数 a,b 的非线性方程组, 称其为非线性最小二乘问题, 而多项式拟合中正则方程组为线性方程组, 称为线性最小二乘问题. 非线性最小二乘问题的求解比较复杂. 但若对式(6.6)两端取对数, 相当于作变换 $\tilde{y}=\ln y$, 则有

$$\ln y=ax+\ln b$$

上式右端是线性函数. 因此, 如果数据点 $(x_i,y_i)\,(i=1,2,\cdots,n)$ 的分布近似于指数曲线, 则数据组 $(x_i,\ln y_i)\,(i=1,2,\cdots,n)$ 的分布就近似于直线. 故可以先求出数据组 $(x_i,\ln y_i)$ 的最小二乘拟合直线 $\tilde{y}=a_0+a_1x$, 取指数即得数据组 $(x_i,y_i)\ (i=1,2,\cdots,n)$ 的最小二乘拟合指数函数 $y=\mathrm{e}^{a_0+a_1x}=\mathrm{e}^{a_0}\mathrm{e}^{a_1x}$.

例 6.3　设一发射源的发射强度公式形为

$$I=I_0\mathrm{e}^{-\alpha t}$$

现测得 I 与 t 的数据如下表:

t_i	0.2	0.3	0.4	0.5	0.6	0.7	0.8
I_i	3.16	2.38	1.75	1.34	1.00	0.74	0.56

扫描全能王 创建

试用最小二乘法确定 I_0 与 α.

解 先求数据表

t_i	0.2	0.3	0.4	0.5	0.6	0.7	0.8
$\ln I_i$	1.150 6	0.867 1	0.559 6	0.292 7	0.000 0	−0.301 1	−0.579 8

的最小二乘拟合直线. 将此表数据代入式(6.5), 得正则方程组

$$\begin{cases} 7a_0 + 3.5a_1 = 1.989\ 1 \\ 3.5a_0 + 2.03a_1 = 0.185\ 8 \end{cases}$$

其解为 $a_0 = 1.73, a_1 = -2.89$. 所以

$$I_0 = \mathrm{e}^{a_0} = 5.64, \quad \alpha = -a_1 = 2.89$$

发射强度公式近似为

$$I = 5.64\mathrm{e}^{-2.89t}$$

6.1.3 分式线性拟合

如果数据点 $(x_i, y_i)\,(i = 1, 2, \cdots, n)$ 的分布近似于分式线性函数 $y = \dfrac{1}{ax + b}$ 的图像, 怎样求这种形式的最小二乘拟合函数? 与指数拟合类似, 可先作变换

$$\tilde{y} = \frac{1}{y} = ax + b$$

将问题线性化, 按数据组 $\left(x_i, \dfrac{1}{y_i}\right)(i = 1, 2, \cdots, n)$ 求其最小二乘一次拟合多项式 $\tilde{y} = ax + b$, 然后取倒数即得数据组 $(x_i, y_i)\,(i = 1, 2, \cdots, n)$ 的最小二乘拟合函数.

若取拟合函数的形式为 $y = \dfrac{x}{ax + b}$, 则应先作倒数变换

$$\tilde{y} = \frac{1}{y} = \frac{ax + b}{x} = a + \frac{b}{x}$$

同时令 $t = \dfrac{1}{x}$, 于是 $\tilde{y} = a + bt$ 为线性拟合问题, 然后按数据组

$$(t_i, \tilde{y}_i) = \left(\frac{1}{x_i}, \frac{1}{y_i}\right) \quad (i = 1, 2, \cdots, n)$$

求其最小二乘拟合直线 $\tilde{y} = a + bt = a + \dfrac{b}{x}$, 再取倒数得所求拟合函数

$$y = \frac{1}{\tilde{y}} = \frac{1}{a + \dfrac{b}{x}} = \frac{x}{ax + b}$$

扫描全能王 创建

综上所述, 求数据组的最小二乘拟合函数的步骤为:

(1) 由给定数据确定近似函数的表达形式, 一般可通过描点观察或经验估计得到;

(2) 按最小二乘原则确定表达式中的参数, 即由残差平方和最小导出正则方程组, 求解得参数. 值得提出的是: 一些简单的非线性最小二乘问题通常需先作变换将问题化为线性最小二乘问题再求解.

注 6.1　(1) 在指数拟合中, 先作变量代换对新变量求最小二乘拟合函数, 然后再还原所得的近似函数与直接对原变量按最小二乘原则求得的拟合函数是不同的. 但由于实际计算时, 人们关心的是问题的简化, 就把这两者的区别忽略了.

(2) 实际问题中, 由于各点的观测数据精度不同, 常常引入加权方差, 即确定参数的准则为使得

$$\sum_{i=1}^{n}\omega_i\delta_i^2$$

最小, 其中 $\omega_i(i=1,2,\cdots,n)$ 为加权系数.

6.1.4　线性最小二乘法的一般形式

一般地, 设给定数据组 $(x_i,y_i)\,(i=1,2,\cdots,n),\varphi_0(x),\varphi_1(x),\cdots,\varphi_m(x)$ 为已知的一组 $[a,b]$ 上线性无关的函数, 选取近似函数为

$$\varphi(x)=a_0\varphi_0(x)+a_1\varphi_1(x)+\cdots+a_m\varphi_m(x) \tag{6.7}$$

若把数据直接代入, 实际上就是超定线性方程组求解问题. 要求残差向量在 2-范数意义最小，即使得

$$\begin{aligned}\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]^2&=\sum_{i=1}^{n}\omega_i\left[y_i-\sum_{k=0}^{m}a_k\varphi_k\left(x_i\right)\right]^2\\&=\min_{\psi\in H}\sum_{i=1}^{n}\omega_i\left[y_i-\psi\left(x_i\right)\right]^2\end{aligned} \tag{6.8}$$

其中 $\omega_i>0\ (i=1,2,\cdots,n)$ 为权系数, H 为 $\varphi_0(x),\varphi_1(x),\cdots,\varphi_m(x)$ 的线性组合的全体, 这就是线性最小二乘法的一般形式. 特别地, 取 $\varphi_k(x)=x^k\ (k=0,1,\cdots,m\ )$, 就是多项式拟合.

与多项式拟合的讨论类似, 上述问题的正则方程组为

$$\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]\varphi_j\left(x_i\right)=0\quad(j=0,1,\cdots,m)$$

即

$$\sum_{k=0}^{m}a_k\left[\sum_{i=1}^{n}\omega_i\varphi_k\left(x_i\right)\varphi_j\left(x_i\right)\right]=\sum_{i=1}^{n}\omega_iy_i\varphi_j\left(x_i\right)\quad(j=0,1,\cdots,m) \tag{6.9}$$

扫描全能王 创建

如果记 $(\varphi_k,\varphi_j)=\sum\limits_{i=1}^{n}\omega_i\varphi_k(x_i)\varphi_j(x_i),(y,\varphi_j)=\sum\limits_{i=1}^{n}\omega_i y_i\varphi_j(x_i)$, 方程组(6.9)可表示成矩阵形式

$$\begin{bmatrix}(\varphi_0,\varphi_0) & (\varphi_0,\varphi_1) & \cdots & (\varphi_0,\varphi_m)\\ (\varphi_1,\varphi_0) & (\varphi_1,\varphi_1) & \cdots & (\varphi_1,\varphi_m)\\ \vdots & \vdots & & \vdots\\ (\varphi_m,\varphi_0) & (\varphi_m,\varphi_1) & \cdots & (\varphi_m,\varphi_m)\end{bmatrix}\begin{bmatrix}a_0\\ a_1\\ \vdots\\ a_m\end{bmatrix}=\begin{bmatrix}(y,\varphi_0)\\ (y,\varphi_1)\\ \vdots\\ (y,\varphi_m)\end{bmatrix}\tag{6.10}$$

由 $\varphi_k(x)\ (k=0,1,\cdots,m)$ 线性无关可导出式(6.10)的系数矩阵非奇异, 从而保证了方程组(6.9) 的解存在、唯一.

定理 6.1 设 $a_0,a_1,\cdots,a_m$ 为方程组(6.9)的解, 则函数 $\varphi(x)=\sum\limits_{k=0}^{m}a_k\varphi_k(x)$ 满足关系式(6.8), 即它是数据组 $(x_i,y_i)\ (i=1,2,\cdots,n)$ 的最小二乘函数.

证明 对任意 $\psi(x)=\sum\limits_{k=0}^{m}c_k\varphi_k(x)\in H$, 有

$$\begin{aligned}\sum_{i=1}^{n}\omega_i\left[y_i-\psi\left(x_i\right)\right]^2&=\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)+\varphi\left(x_i\right)-\psi\left(x_i\right)\right]^2\\&=\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]^2+2\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]\left[\varphi\left(x_i\right)-\right.\\&\quad\left.\psi\left(x_i\right)\right]+\sum_{i=1}^{n}\omega_i\left[\varphi\left(x_i\right)-\psi\left(x_i\right)\right]^2\end{aligned}$$

由式(6.9)可得

$$\begin{aligned}&\sum_{i=1}^{n}\omega_i\left[y-\varphi\left(x_i\right)\right]\left[\varphi\left(x_i\right)-\psi\left(x_i\right)\right]\\&=\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]\left[\sum_{k=0}^{m}\left(a_k-c_k\right)\varphi_k\left(x_i\right)\right]\\&=\sum_{k=0}^{m}\left(a_k-c_k\right)\left[\sum_{i=1}^{n}\omega_i\left(y_i-\varphi\left(x_i\right)\right)\varphi_k\left(x_i\right)\right]=0\end{aligned}$$

从而有

$$\begin{aligned}\sum_{i=1}^{n}\omega_i\left[y_i-\psi\left(x_i\right)\right]^2&=\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]^2+\sum_{i=1}^{n}\omega_i\left[\varphi\left(x_i\right)-\psi\left(x_i\right)\right]^2\\&\geqslant\sum_{i=1}^{n}\omega_l\left[y_i-\varphi\left(x_i\right)\right]^2\end{aligned}$$

即

$$\sum_{i=1}^{n}\omega_i\left[y_i-\varphi\left(x_i\right)\right]^2=\min_{\psi\in H}\sum_{i=1}^{n}\omega_i\left[y_i-\psi\left(x_i\right)\right]^2$$

扫描全能王 创建

所以 $\varphi(x)$ 是数据组 $(x_i, y_i)(i=1,2,\cdots,n)$ 的最小二乘函数.

例 6.4　已知数据表

x	−0.980 0	−0.730 0	−0.480 0	−0.230 0	0.020 0	0.270 0	0.520 0	0.770 0
y	5.238 9	3.256 4	1.542 7	0.966 2	1.629 8	2.405 7	2.400 4	2.279 8

由函数 $\varphi_0(x)=\mathrm{e}^{x^2}, \varphi_1(x)=\sin 5x, \varphi_2(x)=\ln(x+1.5)$ 线性组合而成, 求最小二乘拟合函数 $\varphi(x)=a\mathrm{e}^{x^2}+b\sin 5x+c\ln(x+1.5)$.

解　显然函数组 $\varphi_0(x), \varphi_1(x), \varphi_2(x)$ 线性无关, 计算它们在各节点处的值

$$\varphi_0(x_j)=\left[\begin{array}{cccccccc} 2.612\,7 & 1.703\,9 & 1.259\,1 & 1.054\,3 & 1.000\,4 & 1.075\,6 & 1.310\,5 & 1.809\,2 \end{array}\right],$$

$$\varphi_1(x_j)=\left[\begin{array}{cccccccc} 0.982\,5 & 0.486\,8 & -0.675\,5 & -0.912\,8 & 0.099\,8 & 0.975\,7 & 0.515\,5 & -0.650\,6 \end{array}\right],$$

$$\varphi_2(x_j)=\left[\begin{array}{cccccccc} -0.653\,9 & -0.261\,4 & 0.019\,8 & 0.239\,0 & 0.418\,7 & 0.571\,0 & 0.703\,1 & 0.819\,8 \end{array}\right],$$

按式(6.10)计算得正则方程组

$$\left[\begin{array}{ccc} 19.575\,0 & 2.231\,3 & 1.560\,7 \\ 2.231\,3 & 4.142\,6 & -0.573\,2 \\ 1.560\,7 & -0.573\,2 & 2.221\,2 \end{array}\right]\left[\begin{array}{c} a \\ b \\ c \end{array}\right]=\left[\begin{array}{c} 33.685\,9 \\ 7.072\,2 \\ 1.597\,2 \end{array}\right]$$

解正则方程组得 $a=0.650\,3, b=0.785\,4, c=-0.237\,8$, 从而最小二乘拟合函数为

$$\varphi(x)=0.650\,3\mathrm{e}^{x^2}+0.785\,4\sin 5x-0.237\,8\ln(x+1.5)$$

最小二乘法的正则方程组一般是病态的, 当 m 较大时更是如此, 这使得按上述过程求解时误差较大. 如果适当地选取 $\varphi_k(x)$ $(k=0,1,\cdots,m)$, 使得

$$\begin{aligned} &(\varphi_k,\varphi_k)=\sum_{i=1}^{n}\omega_i\varphi_k^2(x_i)>0 \quad (k=0,1,\cdots,m) \\ &(\varphi_k,\varphi_j)=0 \qquad\qquad\qquad\quad (k\neq j) \end{aligned} \tag{6.11}$$

则方程组(6.10)的系数矩阵成对角形, 正则方程组(6.9)极易解出, 其解为

$$a_k=\frac{(y,\varphi_k)}{(\varphi_k,\varphi_k)}=\frac{\displaystyle\sum_{i=1}^{n}\omega_i y_i\varphi_k(x_i)}{\displaystyle\sum_{i=1}^{n}\omega_i\varphi_k^2(x_i)} \quad (k=0,1,\cdots,m) \tag{6.12}$$

最小二乘函数为

$$\varphi(x)=\sum_{k=0}^{m}a_k\varphi_k(x)=\sum_{k=0}^{m}\frac{(y,\varphi_k)}{(\varphi_k,\varphi_k)}\varphi_k(x) \tag{6.13}$$

定义 6.1　称满足条件式(6.11)的函数族 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_m(x)\}$ 为以 $\{\omega_i\}(i=1,2,\cdots,n)$ 为权关于点集 $\{x_1,x_2,\cdots,x_n\}$ 的正交函数族.

扫描全能王 创建

可以验证下列多项式系

$$\begin{cases} \varphi_0(x) = 1 \\ \varphi_1(x) = x - \alpha_1 \\ \varphi_k(x) = (x - \alpha_k)\varphi_{k-1}(x) - \beta_k \varphi_{k-2}(x) \quad (k = 2, 3, \cdots, m) \end{cases} \tag{6.14}$$

是以 ω_i $(i = 1, 2, \cdots, n)$ 为权关于点集 $\{x_1, x_2, \cdots, x_n\}$ 的正交函数族, 其中

$$\begin{aligned} \alpha_k &= \frac{(x\varphi_{k-1}, \varphi_{k-1})}{(\varphi_{k-1}, \varphi_{k-1})} = \frac{\sum_{i=1}^{n} \omega_i x_i \varphi_{k-1}^2(x_i)}{\sum_{i=1}^{n} \omega_i \varphi_{k-1}^2(x_i)} \quad (k = 1, 2, \cdots, m) \\ \beta_k &= \frac{(\varphi_{k-1}, \varphi_{k-1})}{(\varphi_{k-2}, \varphi_{k-2})} = \frac{\sum_{i=1}^{n} \omega_i \varphi_{k-1}^2(x_i)}{\sum_{i=1}^{n} \omega_i \varphi_{k-2}^2(x_i)} \quad (k = 2, 3, \cdots, m) \end{aligned} \tag{6.15}$$

于是, 求数据组 (x_i, y_i) $(i = 1, 2, \cdots, n)$ 带权 ω_i $(i = 1, 2, \cdots, n)$ 的最小二乘拟合多项式按以下过程进行:

(1) 按式 (6.14)和式(6.15)构造正交函数族 $\{\varphi_0(x), \varphi_1(x), \cdots, \varphi_m(x)\}$;

(2) 由式 (6.12) 计算出正则方程组 (6.9) 的解;

(3) 按式 (6.13)写出最小二乘 m 次拟合多项式.

例 6.5　利用正交函数族求例 6.2所给数据表的最小二乘二次拟合多项式.

解　按式(6.14)、式(6.15) 计算, 得

$$\varphi_0(x) = 1$$

$$\alpha_1 = \frac{(x\varphi_0, \varphi_0)}{(\varphi_0, \varphi_0)} = \frac{\sum_{i=1}^{9} x_i}{\sum_{i=1}^{9} 1} = \frac{0}{9} = 0$$

$$\varphi_1(x) = x$$

$$\alpha_2 = \frac{(x\varphi_1, \varphi_1)}{(\varphi_1, \varphi_1)} = \frac{\sum_{i=1}^{9} x_i^3}{\sum_{i=1}^{9} x_i^2} = 0$$

扫描全能王 创建

$$\beta_2=\frac{(\varphi_1,\varphi_2)}{(\varphi_0,\varphi_0)}=\frac{\sum_{i=1}^{9}x_i^2}{\sum_{i=1}^{9}1}=\frac{3.75}{9}=0.416\ 67$$

$$\varphi_2(x)=x^2-0.416\ 67$$

由式(6.12)

$$a_0=\frac{(y,\varphi_0)}{(\varphi_0,\varphi_0)}=\frac{\sum_{i=1}^{9}y_i}{\sum_{i=1}^{9}1}=\frac{18.172\ 3}{9}=2.019\ 14$$

$$a_1=\frac{(y,\varphi_1)}{(\varphi_1,\varphi_1)}=\frac{\sum_{i=1}^{9}y_ix_i}{\sum_{i=1}^{9}x_i^2}=\frac{8.484\ 2}{3.75}=2.262\ 5$$

$$a_2=\frac{(y,\varphi_2)}{(\varphi_2,\varphi_2)}=\frac{\sum_{i=1}^{9}y_i\left(x_i^2-0.416\ 67\right)}{\sum_{i=1}^{9}\left(x_i^2-0.416\ 67\right)^2}=\frac{0.045\ 45}{1.203\ 1}=0.037\ 8$$

将上述结果代入式(6.13), 得最小二乘二次拟合多项式为

$$\begin{aligned}\varphi(x)&=a_0\varphi_0(x)+a_1\varphi_1(x)+a_2\varphi_2(x)\\&=2.019\ 14+2.262\ 5x+0.037\ 8\left(x^2-0.416\ 67\right)\\&=2.003\ 4+2.262\ 5x+0.037\ 8x^2\end{aligned}$$

这与例 6.2的计算结果相同.

6.2　正交多项式

如前所述, 利用正交函数系可以简化最小二乘法的求解, 并提高解的精度. 正交多项式系, 由于其计算简便, 是函数逼近的重要工具.

6.2.1　基本概念

定义 6.2　若函数系 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_n(x),\cdots\}$ 满足

扫描全能王 创建

$$(\varphi_j,\varphi_k)=\int_a^b\omega(x)\varphi_j(x)\varphi_k(x)\mathrm{d}x=\begin{cases}0, & j\neq k\\ \alpha_k>0, & j=k\end{cases}\quad(j,k=0,1,2,\cdots)$$

则称此函数系为区间 $[a,b]$ 上关于权函数 $\omega(x)$ 的正交函数系. 特别地, 若 $\alpha_k=1$ $(k=0,1,2,\cdots)$, 则称其为标准正交函数系.

例如, 三角函数系 $\{1,\cos x,\sin x,\cos 2x,\sin 2x,\cdots,\cos nx,\sin nx,\cdots\}$ 就是区间 $[-\pi,\pi]$ 上关于权函数 $\omega(x)\equiv 1$ 的正交函数系.

正交函数系具有以下性质.

定理 6.2 区间 $[a,b]$ 上关于权函数 $\omega(x)$ 的正交函数系 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_n(x)\}$ 必定是线性无关的.

证明 用反证法. 假设 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_n(x)\}$ 线性相关, 即存在不全为零的实数 $c_0,c_1,\cdots,c_n$, 使得

$$c_0\varphi_0(x)+c_1\varphi_1(x)+\cdots+c_n\varphi_n(x)\equiv 0,\quad x\in[a,b]$$

不妨设 $c_i\neq 0$, 以 $\omega(x)\varphi_i(x)$ 乘上式后, 在区间 $[a,b]$ 上积分得

$$c_0(\varphi_0,\varphi_i)+c_1(\varphi_1,\varphi_i)+\cdots+c_n(\varphi_n,\varphi_i)=c_i(\varphi_i,\varphi_i)=0$$

因为 $(\varphi_i,\varphi_i)>0$, 故 $c_i=0$, 导出矛盾. 所以 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_n(x)\}$ 在 $[a,b]$ 上线性无关.

如果正交函数系 $\{\varphi_i(x)\}$ 中的函数均为代数多项式, 则称其为正交多项式系. 由定理 6.2, 正交多项式系是线性无关的.

定理 6.3 设 $\varphi_k(x)$ $(k=0,1,2,\cdots)$ 是最高次项系数不为零的 k 次多项式, 则 $\{\varphi_k(x)\}$ 是 $[a,b]$ 上关于权函数 $\omega(x)$ 的正交多项式系的充要条件是对任意至多 $k-1$ 次的多项式 $Q_{k-1}(x)$, 均有

$$(\varphi_k,Q_{k-1})=\int_a^b\omega(x)\varphi_k(x)Q_{k-1}(x)\mathrm{d}x=0\quad(k=1,2,\cdots)$$

证明 充分性. 若对任意至多 $k-1$ 次多项式 $Q_{k-1}(x)$, 都有

$$\int_a^b\omega(x)\varphi_k(x)Q_{k-1}(x)\mathrm{d}x=0\quad(k=1,2,\cdots)$$

特别地, 对多项式 $\varphi_j(x)(j=0,1,\cdots,k-1)$, 有

$$\int_a^b\omega(x)\varphi_k(x)\varphi_j(x)\mathrm{d}x=0$$

即对任意 $k\neq j$, 都有 $(\varphi_k,\varphi_j)=0$. 另一方面, 由 $\varphi_k(x)$ 的最高次项系数不为零可得 $\varphi_k^2(x)\geqslant 0$, 且 $\varphi_k^2(x)\neq 0$ $(k=0,1,\cdots)$, 根据定积分性质有

$$(\varphi_k,\varphi_k)=\int_a^b\omega(x)\varphi_k^2(x)\mathrm{d}x>0$$

所以, $\{\varphi_k(x)\}$ 为正交多项式系.

必要性. 设 $\{\varphi_k(x)\}$ 是 $[a,b]$ 上关于权函数 $\omega(x)$ 的正交多项式系, 由定理 6.2, $\varphi_0(x)$,

扫描全能王 创建

$\varphi_1(x),\cdots,\varphi_{k-1}(x)$ 在 $[a,b]$ 上线性无关, 因而任一至多 $k-1$ 次多项式 $Q_{k-1}(x)$ 均能表示成它们的线性组合. 设

$$Q_{k-1}(x)=\sum_{j=0}^{k-1}b_j\varphi_j(x)$$

于是

$$\begin{aligned}\int_a^b\omega(x)\varphi_k(x)Q_{k-1}(x)\mathrm{d}x&=\int_a^b\omega(x)\varphi_k(x)\left[\sum_{j=0}^{k-1}b_j\varphi_j(x)\right]\mathrm{d}x\\&=\sum_{j=0}^{k-1}b_j\int_a^b\omega(x)\varphi_k(x)\varphi_j(x)\mathrm{d}x=0\end{aligned}$$

6.2.2　Gram–Schmidt 方法

构造正交多项式的一般方法由以下定理给出.

定理 6.4　按以下方式定义的多项式集合 $\{\varphi_0(x),\varphi_1(x),\cdots,\varphi_n(x)\}$ 是区间 $[a,b]$ 上关于权函数 $\omega(x)\geqslant 0(\omega(x)\neq 0)$ 的正交函数族.

$$\begin{aligned}&\varphi_0(x)=1\\&\varphi_1(x)=x-\alpha_1\\&\varphi_k(x)=(x-\alpha_k)\varphi_{k-1}(x)-\beta_k\varphi_{k-2}(x)\quad(k=1,2,\cdots,n-1)\end{aligned}\tag{6.16}$$

其中

$$\alpha_k=\frac{(x\varphi_{k-1},\varphi_{k-1})}{(\varphi_{k-1},\varphi_{k-1})}=\frac{\int_a^b\omega(x)x\varphi_{k-1}^2(x)\mathrm{d}x}{\int_a^b\omega(x)\varphi_{k-1}^2(x)\mathrm{d}x}\quad(k=1,2,\cdots,n-1)$$

$$\beta_k=\frac{(\varphi_{k-1},\varphi_{k-1})}{(\varphi_{k-2},\varphi_{k-2})}=\frac{\int_a^b\omega(x)\varphi_{k-1}^2(x)\mathrm{d}x}{\int_a^b\omega(x)\varphi_{k-2}^2(x)\mathrm{d}x}\quad(k=1,2,\cdots,n-1)$$

证明　显然, $\varphi_k(x)$ $(k=0,1,\cdots,n)$ 为 k 次多项式, 且 k 次项系数为 1 .

因为 $\varphi_k(x)\neq 0$ $(k=0,1,\cdots,n),\omega(x)\geqslant 0,\omega(x)\neq 0$, 由定积分性质

$$(\varphi_k,\varphi_k)=\int_a^b\omega(x)\varphi_k^2(x)\mathrm{d}x>0$$

实际上，式 (6.16)正是 Gram-Schmidt (格拉姆–施密特) 正交化的结果, 容易验证

$$(\varphi_0,\varphi_1)=0\quad(x\varphi_i,\varphi_j)=(\varphi_i,x\varphi_j)$$

设

$$\varphi_{k+1}(x)=x\varphi_k(x)-\sum_{j=0}^{k}a_j^{(k)}\varphi_j(x)$$

根据 Schmidt 正交化过程，需要

$$(\varphi_{k+1},\varphi_j)=0,\quad j=0,1,2,\cdots,k,$$

扫描全能王 创建

可得

$$a_j^{(k)} = \frac{(x\varphi_k, \varphi_j)}{(\varphi_j, \varphi_j)} = \frac{(\varphi_k, x\varphi_j)}{(\varphi_j, \varphi_j)}$$

当 $j = k$ 时

$$a_k^{(k)} = \frac{(x\varphi_k, \varphi_k)}{(\varphi_k, \varphi_k)} \overset{\text{def}}{=} \alpha_k$$

当 $j = k-1$ 时

$$a_{k-1}^{(k)} = \frac{(\varphi_k, x\varphi_{k-1})}{(\varphi_{k-1}, \varphi_{k-1})} = \frac{(\varphi_k, \varphi_k)}{(\varphi_{k-1}, \varphi_{k-1})} \overset{\text{def}}{=} \beta_k$$

当 $j < k-1$ 时

$$a_j^{(k)} = 0$$

得证.

下面介绍几种常用的正交多项式.

6.2.3 常用的正交多项式

(一) Legendre(勒让德) 多项式

Legendre 多项式的一般表示式为

$$P_n(x) = \frac{1}{2^n n!} \frac{\mathrm{d}^n}{\mathrm{d}x^n} \left[\left(x^2 - 1\right)^n \right] \quad (n = 0, 1, 2 \cdots) \tag{6.17}$$

具体表达为

$$\begin{aligned} P_0(x) &= 1 \\ P_1(x) &= x \\ P_2(x) &= \frac{1}{2}\left(3x^2 - 1\right) \\ P_3(x) &= \frac{1}{2}\left(5x^3 - 3x\right) \\ P_4(x) &= \frac{1}{8}\left(35x^4 - 30x^2 + 3\right) \\ &\cdots\cdots\cdots \end{aligned} \tag{6.18}$$

$$P_n(x) = \sum_{k=0}^{\left[\frac{n}{2}\right]} \frac{(-1)^k (2n-2k)!}{2^n k!(n-k)!(n-2k)!} x^{n-2k} \quad (n = 0, 1, 2, \cdots)$$

Legendre 多项式有以下性质:

(1) $\{P_n(x)\}$ 是区间 $[-1, 1]$ 上关于权函数 $\omega(x) \equiv 1$ 的正交函数系, 且

$$(P_n, P_m) = \int_{-1}^{1} P_n(x) P_m(x) \mathrm{d}x = \begin{cases} 0, & n \neq m \\ \dfrac{2}{2n+1}, & n = m \end{cases} \tag{6.19}$$

扫描全能王 创建

(2) Legendre 多项式满足递推公式

$$(n+1)P_{n+1}(x)=(2n+1)xP_n(x)-nP_{n-1}(x)\quad(n=1,2,\cdots)\tag{6.20}$$

通过变量替换由 Legendre 多项式(6.18)可以得到在任意区间 $[a,b]$ 上关于权函数 $\omega(x)\equiv 1$ 的正交多项式系.

令 $x=\dfrac{b+a}{2}+\dfrac{b-a}{2}t$, 当 x 在区间 $[a,b]$ 上变化时, 对应的 t 在 $[-1,1]$ 上变化, 故

$$\tilde{P}_n(x)=P_n(t)=P_n\left[\frac{2x-(b+a)}{b-a}\right]\quad(n=0,1,2,\cdots)\tag{6.21}$$

为 $[a,b]$ 上的正交多项式. 例如, 区间 $[0,1]$ 上的正交多项式为

$$\tilde{P}_0(x)=P_0(2x-1)=1$$

$$\tilde{P}_1(x)=P_1(2x-1)=2x-1$$

$$\tilde{P}_2(x)=\frac{1}{2}\left[3(2x-1)^2-1\right]=6x^2-6x+1$$

$$\tilde{P}_3(x)=\frac{1}{2}\left[5(2x-1)^3-3(2x-1)\right]=20x^3-30x^2+12x-1$$

$$\cdots\cdots\cdots\cdots$$

(二) 第一类 Chebyshev (切比雪夫) 多项式

第一类 Chebyshev 多项式的一般表示式为

$$T_n(x)=\cos(n\arccos x)\quad(-1\leqslant x\leqslant 1,n=0,1,2\cdots)\tag{6.22}$$

令 $x=\cos\theta$, 当 x 在 $[-1,1]$ 上变化时, 对应的 θ 在 $[0,\pi]$ 上变化, 式(6.22) 可改写成

$$T_n(x)=\cos n\theta$$

具体表达为

$$\begin{aligned}
&T_0(x)=\cos 0=1\\
&T_1(x)=\cos\theta=x\\
&T_2(x)=\cos 2\theta=2\cos^2\theta-1=2x^2-1\\
&T_3(x)=\cos 3\theta=4x^3-3x\\
&T_4(x)=\cos 4\theta=8x^4-8x^2+1\\
&\cdots\\
&T_n(x)=\frac{n}{2}\sum_{k=0}^{\left[\frac{n}{2}\right]}(-1)^k\frac{(n-k-1)!}{k!(n-2k)!}(2x)^{n-2k}
\end{aligned}\tag{6.23}$$

由式(6.23)容易看出, $T_n(x)$ 是首项系数为 2^{n-1} 的 n 次多项式.

第一类 Chebyshev 多项式有以下性质:

扫描全能王 创建

(1) $\{T_n(x)\}$ 是区间 $[-1,1]$ 上关于权函数 $\omega(x)=(1-x^2)^{-\frac{1}{2}}$ 的正交多项式系, 且

$$(T_n,T_m)=\int_{-1}^{1}\frac{T_n(x)T_m(x)}{\sqrt{1-x^2}}\,\mathrm{d}x=\begin{cases}0, & n\neq m\\ \dfrac{\pi}{2}, & n=m\neq 0\\ \pi, & n=m=0\end{cases}\tag{6.24}$$

(2) 第一类 Chebyshev 多项式满足递推公式

$$T_{n+1}(x)=2xT_n(x)-T_{n-1}(x)\quad(n=1,2,\cdots)\tag{6.25}$$

(3) $T_n(x)$ 在 $[-1,1]$ 上有 n 个互异零点

$$x_i=\cos\frac{2i-1}{2n}\pi\quad(i=1,2,\cdots,n)$$

(4) $T_n(x)$ 在 $[-1,1]$ 上有 $n+1$ 个极值点

$$x_k=\cos\frac{k\pi}{n}\quad(k=0,1,\cdots,n).$$

且 $|T_n(x_k)|=1\quad(k=0,1,\cdots,n).$

性质 (3) 和性质 (4) 由余弦函数性质即得.

(三) Laguerre (拉盖尔) 多项式

Laguerre 多项式定义为

$$L_n(x)=\mathrm{e}^{x}\frac{\mathrm{d}^n}{\mathrm{d}x^n}\left(x^n\mathrm{e}^{-x}\right)\quad(n=0,1,2\cdots)\tag{6.26}$$

其具体表达式为

$$L_0(x)=1$$

$$L_1(x)=-x+1$$

$$L_2(x)=x^2-4x+2$$

$$L_3(x)=-x^3+9x^2-18x+6$$

$$\cdots$$

$$L_n(x)=\sum_{k=0}^{n}(-1)^k\begin{pmatrix}n\\k\end{pmatrix}\frac{n!}{k!}x^k\quad(n=1,2,\cdots)$$

Laguerre 多项式有以下性质:

(1) $\{L_n(x)\}$ 是区间 $[0,+\infty)$ 上关于权函数 $\omega(x)=\mathrm{e}^{-x}$ 的正交多项式系, 且

$$(L_n,L_m)=\int_0^{+\infty}\mathrm{e}^{-x}L_n(x)L_m(x)\mathrm{d}x=\begin{cases}0, & n\neq m\\ (n!)^2, & n=m\end{cases}\tag{6.27}$$

(2) Laguerre 多项式满足递推公式

$$L_{n+1}(x)=(2n+1-x)L_n(x)-x^2L_{n-1}(x)\quad(n=1,2,\cdots)\tag{6.28}$$

扫描全能王 创建

6.3　函数的最佳平方逼近

有些函数由于表达式比较复杂而不易计算和研究, 也需要用简单函数去近似它, 这是函数逼近所研究的问题. 本节讨论如何用最小二乘法求连续函数的近似函数.

定义 6.3　设函数 $f(x)$ 在区间 $[a,b]$ 上连续, $\varphi_i(x)$ $(i=0,1,\cdots,m)$ 为定义在 $[a,b]$ 上的一组线性无关的连续函数. $H=\operatorname{Span}\{\varphi_0,\varphi_1,\cdots,\varphi_m\}$. 若函数

$$\varphi(x)=a_0\varphi_0(x)+a_1\varphi_1(x)+\cdots+a_m\varphi_m(x) \tag{6.29}$$

使得

$$\begin{aligned} F(a_0,a_1,\cdots,a_n)&=\int_a^b\omega(x)\left[f(x)-\sum_{k=0}^m a_k\varphi_k(x)\right]^2\mathrm{d}x\\ &=\min_{\psi\in H}\int_a^b\omega(x)[f(x)-\psi(x)]^2\,\mathrm{d}x \end{aligned} \tag{6.30}$$

其中 $\omega(x)$ 为权函数, 则称 $\varphi(x)$ 为函数 $f(x)$ 在 H 中关于权函数 $\omega(x)$ 的最佳平方逼近函数.

特别地, 如果 $\varphi_k(x)$ 为 k 次多项式 $(k=0,1,\cdots,m)$, 则 $\varphi(x)$ 称为 $f(x)$ 在 $[a,b]$ 上关于权函数 $\omega(x)$ 的 m 次最佳平方逼近多项式或最小二乘逼近多项式.

在具体问题中, 权函数 $\omega(x)$ 是给定的, 如果没有特别指明, 就表示 $\omega(x)\equiv 1$.

与离散的情形类似, 由多元函数取极值的必要条件

$$\frac{\partial F}{\partial a_j}=-2\int_a^b\omega(x)\left[f(x)-\sum_{k=0}^m a_k\varphi_k(x)\right]\varphi_j(x)\mathrm{d}x=0\quad(j=0,1,\cdots,m) \tag{6.31}$$

得正则方程组

$$\sum_{k=0}^m a_k\int_a^b\omega(x)\varphi_k(x)\varphi_j(x)\mathrm{d}x=\int_a^b\omega(x)f(x)\varphi_j(x)\mathrm{d}x\quad(j=0,1,\cdots,m)$$

改写成矩阵形式为

$$\begin{bmatrix} (\varphi_0,\varphi_0) & (\varphi_0,\varphi_1) & \cdots & (\varphi_0,\varphi_m)\\ (\varphi_1,\varphi_0) & (\varphi_1,\varphi_1) & \cdots & (\varphi_1,\varphi_m)\\ \vdots & \vdots & & \vdots\\ (\varphi_m,\varphi_0) & (\varphi_m,\varphi_1) & \cdots & (\varphi_m,\varphi_m) \end{bmatrix}\begin{bmatrix} a_0\\ a_1\\ \vdots\\ a_m \end{bmatrix}=\begin{bmatrix} (f,\varphi_0)\\ (f,\varphi_1)\\ \vdots\\ (f,\varphi_m) \end{bmatrix} \tag{6.32}$$

其中 $(\varphi,\psi)=\int_a^b\omega(x)\varphi(x)\psi(x)\mathrm{d}x$. 由 $\varphi_0,\varphi_1,\cdots,\varphi_m$ 线性无关, 可导出方程组(6.32)的系数矩阵非奇异, 故解存在、唯一.

定理 6.5　设 $f(x)$ 在 $[a,b]$ 上连续, $\varphi(x)=\sum_{k=0}^m a_k\varphi_k(x)$ 是 $f(x)$ 在 H 中关于权函数 $\omega(x)$ 的最佳平方逼近函数的充要条件是

扫描全能王 创建

$$(f-\varphi,\varphi_j)=0 \quad (j=0,1,\cdots,m) \tag{6.33}$$

证明 必要性. 设 $\varphi(x)$ 是 $f(x)$ 在 H 中关于权函数 $\omega(x)$ 的最佳平方逼近函数, 且存在 $i(0\leqslant i\leqslant m)$, 使得

$$(f-\varphi,\varphi_i)=r\neq 0$$

令

$$\psi(x)=\varphi(x)+\frac{r}{(\varphi_i,\varphi_i)}\varphi_i=\sum_{\substack{k=0\\k\neq i}}^{m}a_k\varphi_k(x)+\left[a_i+\frac{r}{(\varphi_i,\varphi_i)}\right]\varphi_i(x)$$

显然, $\psi(x)\in H$, 且

$$\begin{aligned}&\int_a^b\omega(x)[f(x)-\psi(x)]^2\,\mathrm{d}x\\ =&(f-\psi,f-\psi)\\ =&(f-\varphi,f-\varphi)-2\left(f-\varphi,\frac{r}{(\varphi_i,\varphi_i)}\varphi_i\right)+\frac{r^2}{(\varphi_i,\varphi_i)^2}(\varphi_i,\varphi_i)\\ =&(f-\varphi,f-\varphi)-\frac{r^2}{(\varphi_i,\varphi_i)}<(f-\varphi,f-\varphi)\end{aligned}$$

这与 $\varphi(x)$ 是最佳平方逼近函数矛盾, 故必要性成立.

充分性. 设式(6.33)成立, 则对任意 $\psi(x)\in H$, 记 $\psi=\sum\limits_{k=0}^{m}b_k\varphi_k$, 都有

$$\begin{aligned}(f-\psi,f-\psi)&=(f-\varphi+\varphi-\psi,f-\varphi+\varphi-\psi)\\ &=(f-\varphi,f-\varphi)+2(f-\varphi,\varphi-\psi)+(\varphi-\psi,\varphi-\psi)\end{aligned}$$

因为

$$(f-\varphi,\varphi-\psi)=\left(f-\varphi,\sum_{k=0}^{m}(a_k-b_k)\varphi_k\right)=\sum_{k=0}^{m}(a_k-b_k)(f-\varphi,\varphi_k)=0$$

$$(\varphi-\psi,\varphi-\psi)=\int_a^b\omega(x)[\varphi(x)-\psi(x)]^2\,\mathrm{d}x\geqslant 0$$

所以

$$(f-\psi,f-\psi)\geqslant(f-\varphi,f-\varphi)$$

故 $\varphi(x)$ 是 $f(x)$ 在 H 中关于权函数 $\omega(x)$ 的最佳平方逼近函数.

注 6.2 定理 6.5实际上是投影定理, 对数据拟合的最小二乘法本质上也是投影定理, 将方程组(6.32)的解代入式(6.29) 所得的函数 $\varphi(x)$ 必定是 $f(x)$ 在 H 中关于权函数 $\omega(x)$ 的最佳平方逼近函数, 且是唯一的.

如果取 $\varphi_k(x)$ 为 k 次多项式, 且 $\varphi_0(x),\varphi_1(x),\cdots,\varphi_m(x)$ 为 $[a,b]$ 上关于权函数 $\omega(x)$ 的正交多项式系, 则求函数 $f(x)$ 的 m 次最佳平方逼近多项式就归结为求解正则方程组

扫描全能王 创建

$$\begin{bmatrix} (\varphi_0,\varphi_0) & & & \\ & (\varphi_1,\varphi_1) & & \\ & & \ddots & \\ & & & (\varphi_m,\varphi_m) \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_m \end{bmatrix} = \begin{bmatrix} (f,\varphi_0) \\ (f,\varphi_1) \\ \vdots \\ (f,\varphi_m) \end{bmatrix} \tag{6.34}$$

容易得出, 方程组(6.34)的解为

$$a_k = \frac{(f,\varphi_k)}{(\varphi_k,\varphi_k)} \quad (k=0,1,\cdots,m) \tag{6.35}$$

所以, $f(x)$ 在 $[a,b]$ 上的 m 次最佳平方逼近多项式为

$$\varphi(x) = \sum_{k=0}^{m} \frac{(f,\varphi_k)}{(\varphi_k,\varphi_k)} \varphi_k(x) \tag{6.36}$$

例 6.6 求 $f(x)=\sin \pi x$ 在 $[0,1]$ 上的二次最佳平方逼近多项式.

解 若取 $\varphi_0(x)=1, \varphi_1(x)=x, \varphi_2(x)=x^2$, 容易求出此问题的正则方程组为三元 Hilbert 方程组, 故先按式(6.16) 构造正交多项式.

$$\varphi_0(x)=1$$

$$\alpha_1 = \frac{(x\varphi_0,\varphi_0)}{(\varphi_0,\varphi_0)} = \frac{\int_0^1 x \,\mathrm{d}x}{\int_0^1 1 \,\mathrm{d}x} = \frac{1}{2}$$

$$\varphi_1(x) = x - \frac{1}{2}$$

$$\alpha_2 = \frac{(x\varphi_1,\varphi_1)}{(\varphi_1,\varphi_1)} = \frac{\int_0^1 x\left(x-\frac{1}{2}\right)^2 \mathrm{d}x}{\int_0^1 \left(x-\frac{1}{2}\right)^2 \mathrm{d}x} = \frac{1}{2}$$

$$\beta_2 = \frac{(\varphi_1,\varphi_1)}{(\varphi_0,\varphi_0)} = \frac{\int_0^1 \left(x-\frac{1}{2}\right)^2 \mathrm{d}x}{\int_0^1 1 \,\mathrm{d}x} = \frac{1}{12}$$

$$\varphi_2(x) = \left(x-\frac{1}{2}\right)^2 - \frac{1}{12} = x^2 - x + \frac{1}{6}$$

于是

$$(\varphi_0,\varphi_0)=1$$

$$(\varphi_1,\varphi_1) = \int_0^1 \left(x-\frac{1}{2}\right)^2 \mathrm{d}x = \frac{1}{12}$$

$$(\varphi_2,\varphi_2) = \int_0^1 \left(x^2-x+\frac{1}{6}\right)^2 \mathrm{d}x = \frac{1}{180}$$

$$(f,\varphi_0) = \int_0^1 \sin \pi x \,\mathrm{d}x = \frac{2}{\pi}$$

扫描全能王 创建

$$(f,\varphi_1)=\int_0^1\left(x-\frac{1}{2}\right)\sin\pi x\,\mathrm{d}x=0$$

$$(f,\varphi_2)=\int_0^1\left(x^2-x+\frac{1}{6}\right)\sin\pi x\,\mathrm{d}x=\frac{\pi^2-12}{3\pi^3}$$

所以, $f(x)=\sin\pi x$ 在 $[0,1]$ 上的二次最佳平方逼近多项式为

$$\begin{aligned}\varphi(x)&=\frac{(f,\varphi_0)}{(\varphi_0,\varphi_0)}\varphi_0(x)+\frac{(f,\varphi_1)}{(\varphi_1,\varphi_1)}\varphi_1(x)+\frac{(f,\varphi_2)}{(\varphi_2,\varphi_2)}\varphi_2(x)\\&=\frac{2}{\pi}+\frac{\dfrac{\pi^2-12}{3\pi^3}}{\dfrac{1}{180}}\left(x^2-x+\frac{1}{6}\right)\\&=\frac{60\pi^2-720}{\pi^3}\left(x^2-x+\frac{1}{6}\right)+\frac{2}{\pi}\\&\approx-4.122\,5x^2+4.122\,5x-0.050\,47\end{aligned}$$

例 6.7　求 $f(x)=\mathrm{e}^x$ 在 $[-1,1]$ 上的三次最佳平方逼近多项式.

解　Legendre 多项式是 $[-1,1]$ 上的正交多项式系, 故可取

$$\varphi_0(x)=P_0(x)=1$$

$$\varphi_1(x)=P_1(x)=x$$

$$\varphi_2(x)=P_2(x)=\frac{1}{2}\left(3x^2-1\right)$$

$$\varphi_3(x)=P_3(x)=\frac{1}{2}\left(5x^3-3x\right)$$

由式(6.34)

$$(\varphi_k,\varphi_k)=\frac{2}{2k+1}\quad(k=0,1,2,3)$$

$$(f,\varphi_0)=\int_{-1}^1\mathrm{e}^x\,\mathrm{d}x=\mathrm{e}-\mathrm{e}^{-1}\approx2.350\,4$$

$$(f,\varphi_1)=\int_{-1}^1x\mathrm{e}^x\,\mathrm{d}x=2\mathrm{e}^{-1}\approx0.735\,8$$

$$(f,\varphi_2)=\int_{-1}^1\frac{1}{2}\left(3x^2-1\right)\mathrm{e}^x\,\mathrm{d}x=\mathrm{e}-7\mathrm{e}^{-1}\approx0.143\,1$$

$$(f,\varphi_3)=\int_{-1}^1\frac{1}{2}\left(5x^3-3x\right)\mathrm{e}^x\,\mathrm{d}x=-5\mathrm{e}+37\mathrm{e}^{-1}\approx0.020\,13$$

于是正则方程组之解为

$$a_0=\frac{1}{2}(f,\varphi_0)\approx1.175\,2,\quad a_1=\frac{3}{2}(f,\varphi_1)\approx1.103\,7$$

扫描全能王 创建

$$a_2=\frac{5}{2}(f,\varphi_2)\approx 0.357\ 8,\quad a_3=\frac{7}{2}(f,\varphi_3)\approx 0.070\ 64$$

所以函数 $f(x)=\mathrm{e}^x$ 在 $[-1,1]$ 上的三次最佳平方逼近多项式为

$$\begin{aligned}\varphi(x)&=1.175\ 2\varphi_0(x)+1.103\ 7\varphi_1(x)+0.357\ 8\varphi_2(x)+0.070\ 46\varphi_3(x)\\&=0.176\ 2x^3+0.536\ 7x^2+0.998\ 0x+0.996\ 3\end{aligned}$$

如果求 $f(x)=\mathrm{e}^x$ 在 $[0,1]$ 上的二次最佳平方逼近多项式, 则可用变量替换, 从 Legendre 多项式得出 $[0,1]$ 上的正交多项式 $\tilde{P}_n(x)=P_n(2x-1)$. 取

$$\varphi_0(x)=\tilde{P}_0(x)=1$$

$$\varphi_1(x)=\tilde{P}_1(x)=2x-1$$

$$\varphi_2(x)=\tilde{P}_2(x)=6x^2-6x+1$$

于是

$$(\varphi_0,\varphi_0)=\int_0^1 \mathrm{d}x=1$$

$$(\varphi_1,\varphi_1)=\int_0^1(2x-1)^2\ \mathrm{d}x=\frac{1}{3}$$

$$(\varphi_2,\varphi_2)=\int_0^1\left(6x^2-6x+1\right)^2\ \mathrm{d}x=\frac{1}{5}$$

$$(f,\varphi_0)=\int_0^1 \mathrm{e}^x\ \mathrm{d}x=\mathrm{e}-1\approx 1.718\ 3$$

$$(f,\varphi_1)=\int_0^1 \mathrm{e}^x(2x-1)\mathrm{d}x=3-\mathrm{e}\approx 0.281\ 7$$

$$(f,\varphi_2)=\int_0^1 \mathrm{e}^x\left(6x^2-6x+1\right)\mathrm{d}x=7\mathrm{e}-19\approx 0.027\ 97$$

由式(6.36)得 $f(x)=\mathrm{e}^x$ 在 $[0,1]$ 上的二次最佳平方逼近多项式为

$$\begin{aligned}\varphi(x)&=1.718\ 3\varphi_0(x)+0.845\ 1\varphi_1(x)+0.139\ 9\varphi_2(x)\\&=0.839\ 4x^2+0.850\ 8x+1.013\ 1\end{aligned}$$

6.4　算法实现与应用实例

6.4.1　线性最小二乘拟合问题

由于噪声干扰, 函数值的获取有一定的误差. 针对下面函数

$$f(x)=0.123\ 5\mathrm{e}^{-0.213\ 8x}+0.546\ 7\mathrm{e}^{-0.178\ 5x}\cdot\sin(1.257\ 4x),\quad x\in(0,10)$$

扫描全能王 创建

构造带有一定的误差的函数值, 试用多项式拟合这组数据.

北太天元软件中的函数 polyfit( ) 可直接实现最小二乘多项式拟合, 其调用格式为

≫ p = polyfit(x, y, n)

其中 x, y 为拟合数据, n 为拟和多项式的最高次数, p 是一个向量, 各分量表示拟合多项式按幂次由高次到低次的系数.

我们采用北太天元软件解决该问题. 首先利用 rand() 函数, 产生带有一定误差的拟合数据:

≫ x = 0 : 0.2 : 10;

≫ y = 0.1235 ∗ exp(−0.2138 ∗ x) + 0.5467 ∗ exp(−0.1785 ∗ x). ∗ sin (1.2574 ∗ x) + 0.1 ∗ (rand(1, 51) − 0.5);

再计算准确值, 并将拟合数据与函数显示于图 6.2 中.

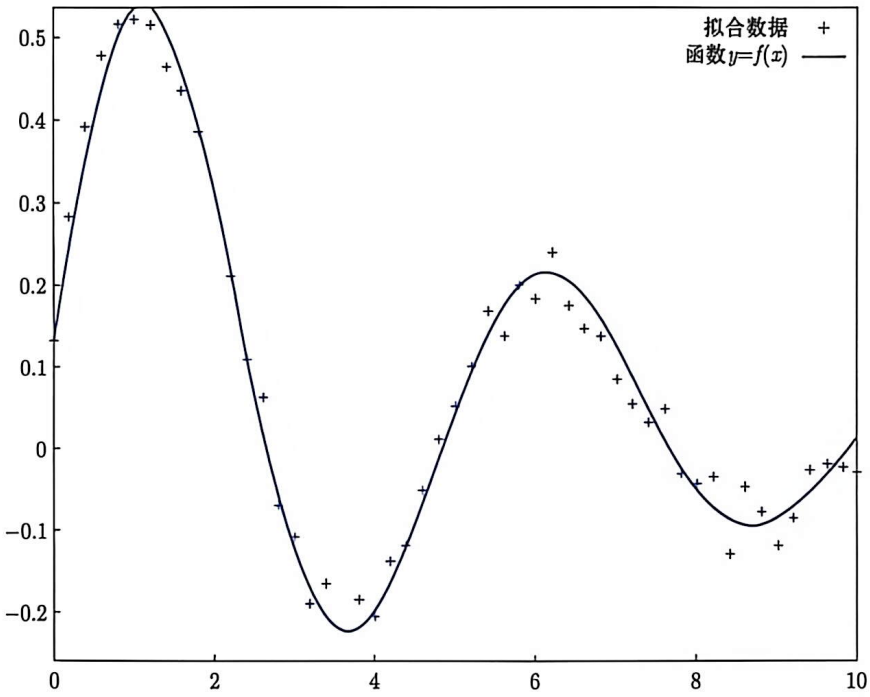


图 6.2 带有误差的拟合数据示意图

≫ y0 = 0.1235 ∗ exp(−0.2138 ∗ x) + 0.5467 ∗ exp(−0.1785 ∗ x).∗ sin(1.2574 ∗ x);

下面就这组数据进行 3 次、5 次、7 次和 9 次多项式拟合:

≫ p3 = polyfit (x, y, 3); p5 = polyfit(x, y, 5); p7 = polyfit(x, y, 7); p9 = polyfit (x, y, 9);

调用命令 polyval(*,*) 计算拟合多项式在各节点上的值，拟合效果见图 6.3.

≫ yp3 = polyval(p3,x);yp5 = polyval(p5,x);yp7 = polyval(p7,x);yp9 = polyval(p9,x);

≫ subplot(2,2,1);plot(x,y,'.',x,y0,'-',x,yp3,'-.');title("3 次多项式");

≫ subplot(2,2,2);plot(x,y,'.',x,y0,'-',x,yp5,'-.');title("5 次多项式");

≫ subplot(2,2,3);plot(x,y,'.',x,y0,'-',x,yp7,'-.');title("7 次多项式");

≫ subplot(2,2,4);plot(x,y,'.',x,y0,'-',x,yp9,'-.');title("9 次多项式");

扫描全能王 创建

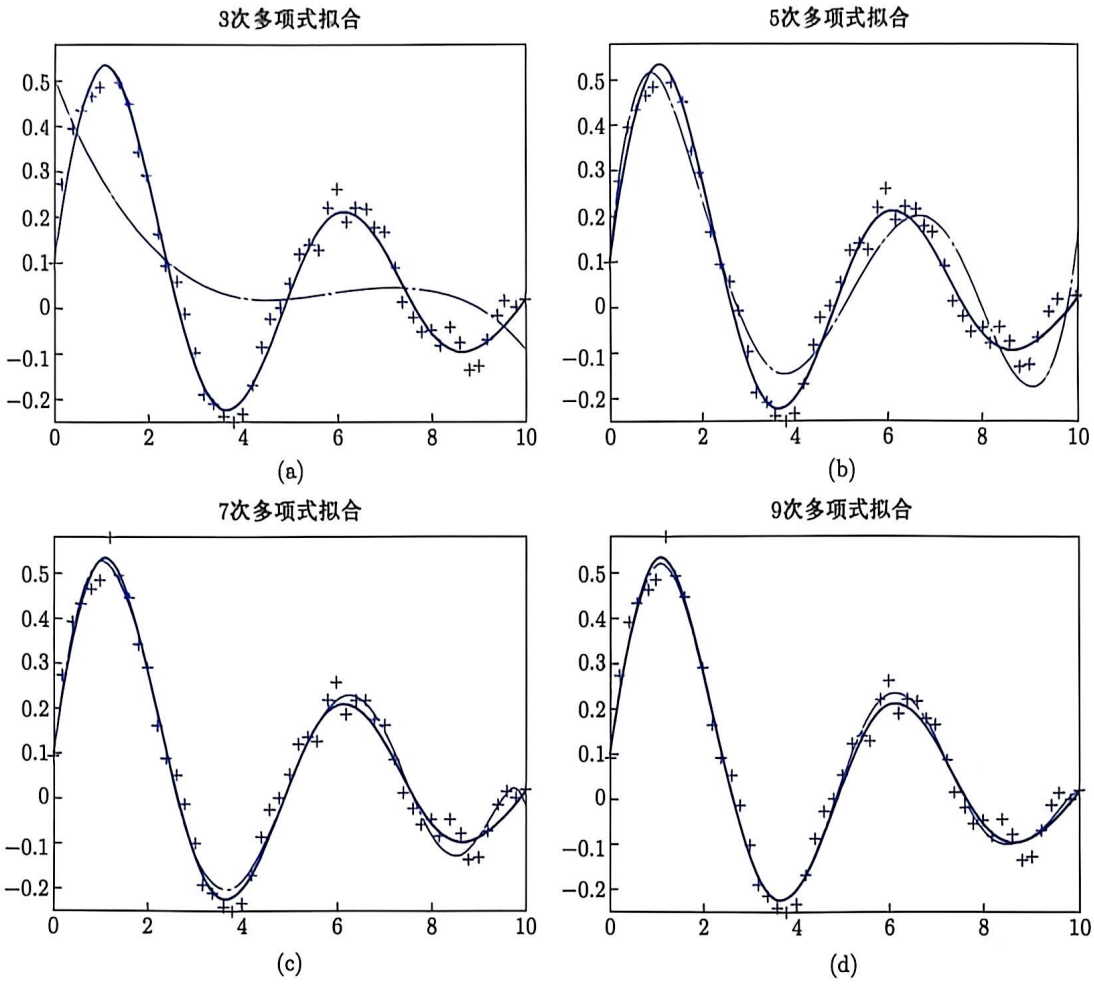


图 6.3　多项式拟合效果图

6.4.2　某些例子的实现

例 6.4若用数值软件计算, 直接利用左除命令, 对超定方程组求最小二乘解, 得到同样的结果. 具体过程如下:

$\gg$ x = [−0.9800　−0.7300　−0.4800　−0.2300　0.0200　0.2700　0.5200　0.7700]′;

$\gg$ y = [5.2389　3.2564　1.5427　0.9662　1.6298　2.4057　2.4004　2.2798]′;

$\gg$ A = [exp(x. ∗ x)　sin(5 ∗ x)　log(x + 1.5)]; A = A′ ∗ A;

$\gg$ abc = (A\y)′

abc = [0.6503　0.7854　−0.2378]

例 6.5用数值软件计算过程如下:

$\gg$ x = −1 : 0.25 : 1;

$\gg$ y = [−0.2209　0.3295　0.8826　1.4392　2.0003　2.5645
3.7601　4.2836];

$\gg$ p = polyfit(x, y, 2)

p = [0.03785108225108　2.26244666666667　2.00337316017316]

扫描全能王 创建

得到结果与例 6.2、例 6.5 结果一致.

6.4.3 回归问题

回归是监督学习的一个重要问题. 回归用于预测输入变量和输出变量之间的关系, 特别是当输入变量的值发生变化时, 输出变量的值随之发生的变化. 回归模型正是表示从输入变量映射到输出变量的函数. 回归问题的学习等价于函数拟合: 选择一条函数曲线使其很好地拟合已知数据且很好地预测未知数据.

如图 6.4所示, 回归问题分为学习和预测两个过程. 首先给定一个训练数据集

$$T=\{(x_1,y_1),(x_2,y_2),\cdots,(x_N,y_N)\}$$

这时 $x_i\in\mathbf{R}^n$ 是输入，$y_i\in\mathbf{R}$ 是对应的输出,$i=1,2,\cdots,N$. 学习系统基于训练数据构造一个模型, 即函数 $Y=f(X)$, 对于输入值 x_{N+1}, 预测系统根据学习的模型 $Y=f(X)$ 确定相应的输出 y_{N+1}.

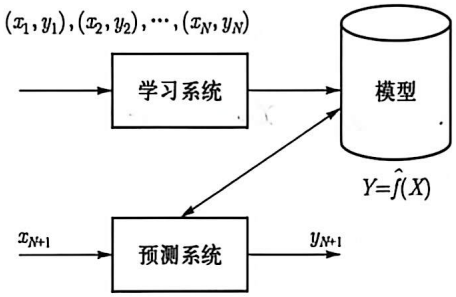


图 6.4 回归问题

回归问题按照输入变量的个数, 分为一元回归和多元回归, 按照输入和输出变量之间关系的类型即模型的类型, 分为线性回归和非线性回归. 回归学习最常用的损失函数是平方损失函数, 在此情况下, 回归问题可以由最小二乘法求解.

许多领域的任务都可以形式化为回归问题, 比如, 回归可以用于商务领域, 作为市场趋势预测、产品质量管理、客户满意度调查、投资风险分析的工具. 作为例子, 简单介绍股价预测问题. 假设知道某公司在过去不同时间点的市场上的股票价格 (比如股票平均价格), 以及在各个时间点之前可能影响该公司股价的信息 (比如该公司前周的营业额、利润). 目标是从过去的数据学习一个模型, 使它可以基于当前的信息预测该公司下个时间点的股票价格. 可以将这个问题作为回归问题解决. 具体地, 将影响股价的信息视为自变量 (输入的特征), 而将股价视为因变量 (输出的值). 将过去的数据作为训练数据就可以学习一个回归模型, 并对未来的股价进行预测. 可以看出这是一个困难的预测问题, 因为影响股价的因素非常多, 我们未必能判断哪些信息有用并能得到这些信息.

评注六

函数逼近是数学的一个经典分支, 在科学与工程计算中有着广泛的应用. 本章讨论了

扫描全能王 创建

离散数据的线性最小二乘拟合, 函数的最佳平方逼近的基本理论与基本方法, 这些问题实际上是求连续函数子空间上的极值. 由于最小二乘拟合和最佳平方逼近最后都归结为求解正则方程组, 而正则方程组往往是病态的, 这使得正交函数族有了特别重要的意义. 本章介绍了正交多项式的基本性质和构造方法. 正交多项式分离散型和连续型两种, 离散型正交多项式适用于最小二乘拟合, 连续型正交多项式适用于最佳平方逼近和数值积分中的高斯型求积公式. 正交多项式因自变量定义域和权函数的不同而不同, 使用时应注意.

本章知识要点　最小二乘法; 正交多项式; 函数的最佳平法逼近

算法历史及背后的数学家

习题六

1. 已知数据表如下:

x_i	1.36	1.49	1.73	1.81	1.95	2.16	2.28	2.48
y_i	14.094	15.069	16.844	17.378	18.435	19.949	20.963	22.495

试求其最小二乘一次拟合多项式.

2. 已知数据表如下:

x_i	1	3	4	5	6	7	8	9	10
y_i	10	5	4	2	1	1	2	3	4

试求其最小二乘二次拟合多项式.

3. 某平炉的生产记录见下表:

i	1	2	3	4	5
x_i	165	123	150	121	141
y_i	187	126	172	125	148

表中 i 为实验次数, x_i 为全部炉料熔化完毕时钢液的含碳量 (单位略), y_i 为熔毕至出钢所需的冶炼时间 (单位略), 求钢液含碳量与冶炼时间的函数关系.

4. 用最小二乘法求一形如 $y=a+bx^2$ (a,b 为常数) 的经验公式拟合下列数据:

x_i	19	25	31	38	44
y_i	19.0	32.3	49.0	73.3	97.8

5. 用最小二乘法求一形如 $y=ax+bx^3(a,b$ 为常数) 的经验公式拟合以下数据:

扫描全能王 创建

x_i	-3	-2	-1	0	1	2	3
y_i	-28.49	-8.99	-1.51	0.001	1.47	9.02	28.42

6. 用最小二乘法求一形如 $y = ae^{bx}$ (a,b为常数) 的经验公式拟合下列数据:

x_i	1	2	3	4	5	6	7	8
y_i	15.3	20.5	27.4	36.6	49.1	65.6	87.87	117.6

7. 某化学反应中, 测得生成物的质量浓度 y 与时间 t 的关系见下表 (单位略) :

t_i	1	2	3	4	6	8	10	12	14	16
y_i	4.00	6.41	8.01	8.79	9.53	9.86	10.33	10.42	10.53	10.61

试用最小二乘法求一形如 $y = \dfrac{t}{at+b}(a,b$为常数) 的经验公式拟合以上数据.

8. 用形如 $y = ae^{\frac{b}{2}}(a,b$为常数) 的经验公式拟合第 7 题中数据, 并与第 7 题的结果比较 (用残差平方和的大小描述拟合的好坏).

9. 设 $\{\varphi_k(x)\}(k = 0,1,2,\cdots)$ 是区间 $[0,1]$ 上权函数为 $\omega(x) = x$ 的最高项系数为 1 的正交多项式系, 其中 $\varphi_0(x) = 1$, 试求 $\int_0^1 x\varphi_k(x)\mathrm{d}x$ 和 $\varphi_2(x)$.

10. 证明函数系 $\{\varphi_0,\varphi_1,\cdots,\varphi_n\}$ 线性无关的充要条件是 Gram 行列式 $G(\varphi_0,.\varphi_1,\cdots,\varphi_n) \neq 0$.

11. 利用正交函数族, 求下列函数的最小二乘三次拟合多项式:

(1) $f(x) = \sin x, x_k = 0.2k, \omega_k = 1 \quad (k = 0,1,\cdots,5)$

(2) $f(x) = \ln(1+x), x_k = 0.2, \omega_k = 1 \quad (k = 0,1,\cdots,5)$

(3) $f(x) = \mathrm{e}^{-x}, x_k = 0.2k, \omega_k = \mathrm{e}^{0.2k} \quad (k = 0,1,\cdots,5)$

12. 求 $f(x) = \cos x, x \in \left[-\dfrac{\pi}{2},\dfrac{\pi}{2}\right]$ 在 $H = \mathrm{Span}\{1,x^2,x^4\}$ 中的最佳平方逼近多项式.

13. 求下列函数在指定区间上关于 $f(x)$ 的一次最佳平方逼近多项式:

(1) $f(x) = \sqrt{x}, [0,1]$;

(2) $f(x) = \mathrm{e}^x, [0,1]$;

(3) $f(x) = \sin\dfrac{\pi}{2}x, [0,1]$.

14. 对第 13 题中的函数和区间, 求二次最佳平方逼近多项式.

15. 利用 Legendre 多项式, 求 $f(x) = \sin x$ 在 $\left[-\dfrac{\pi}{2},\dfrac{\pi}{2}\right]$ 上的三次最佳平方逼近多项式 $P_3(x)$, 并计算均方误差 $\int_{-\frac{\pi}{2}}^{\frac{\pi}{2}}[\sin x - P_3(x)]^2\ \mathrm{d}x$.

扫描全能王 创建

数值实验六

1. 研究误差函数 $\operatorname{erf}(x)=\dfrac{2}{\sqrt{\pi}}\displaystyle\int_0^x \mathrm{e}^{-t^2}\,\mathrm{d}t$ 的拟合问题，先利用北太天元中的函数 erf( ) 产生 11 个数据点 $x_k=\dfrac{k-1}{10}, y_k=\operatorname{erf}(x_k)\,(k=1,2,\cdots,11)$.

(1) 试分别用最高次为 1 至 10 的多项式进行最小二乘拟合，比较各阶拟合多项式在数据点处与精确值 $\operatorname{erf}(x)$ 的误差，最大误差与拟合多项式的最高次数的关系如何?

(2) 由于函数 $\operatorname{erf}(x)$ 是奇函数，即 $\operatorname{erf}(-x)=-\operatorname{erf}(x)$，选择仅含有 x 的奇次幂项的多项式进行拟合，即

$$\operatorname{erf}(x)\approx a_1x+a_2x^3+\cdots+a_nx^{2n-1}\quad(n=1,2,\cdots,10;a_1,a_2,\cdots,a_n\text{为常数})$$

再分析误差与多项式最高次数之间的关系.

(3) 注意到函数 $\operatorname{erf}(x)$ 当 $x\to\infty$ 时，有 $\operatorname{erf}(x)\to 1$，而多项式拟合函数是无界的，因而多项式并不是合适的拟合函数. 令 $z=\dfrac{1}{1+x}$，选择拟合函数

$$\operatorname{erf}(x)\approx a_1+\mathrm{e}^{-x^2}\left(a_2+a_3z+a_4z^2+a_5z^3\right)(a_1,a_2,\cdots,a_5\text{为常数})$$

求出拟合函数，并分析误差情况.

2. 炼钢厂出钢时所用的盛钢水的钢包，在使用过程中由于钢液及炉渣对包衬耐火材料的侵蚀，使其容积不断增大. 经试验，钢包的容积与相应的使用次数的数据列表如下:

使用次数 x	容积 y	使用次数 x	容积 y
2	106.42	11	110.59
3	108.26	12	110.60
5	109.58	14	110.72
6	109.50	16	110.90
7	109.86	17	110.76
9	110.00	19	111.10
10	109.93	20	111.30

选用双曲线 $\dfrac{1}{y}=a+b\dfrac{1}{x}(a,b\text{为常数})$ 对数据进行拟合，用最小二乘法求出拟合函数，作出拟合曲线图.

3. 设有一组实测数据:

x_i	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
y_i	2.324 0	2.647 0	2.970 7	3.288 5	3.600 8	3.909 0	4.214 7	4.519 1	4.823 2	5.127 5

假设该组数据可能满足的函数为 $f(x)=ax+bx^2\mathrm{e}^{-cx}+d$，试求出最小二乘拟合函数的参数 a,b,c,d，并作图显示结果.

扫描全能王 创建

/第七章/

数值微分与数值积分

微积分学已经给出求函数导数及区间上积分的许多方法, 但这些方法对一些实际问题往往难以实行. 如实际问题常常只给出函数在一些离散点上的值, 而无解析表达式; 或虽有函数的解析表达式, 却因其原函数不能用初等函数表示, 或形式复杂, 难以计算. 因此需要建立利用离散点上函数的信息求函数导数及积分近似值的方法, 通常称为数值微分与数值积分法.

7.1 数 值 微 分

由导数的定义

$$f'(x)=\lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$$

容易想到, 当 h 充分小时, 可用差商近似导数, 这是最简单的数值微分公式.

7.1.1 差商型求导公式

利用差商求导数的近似公式通常有以下几种:

(1) 向前差商公式

$$f'(x)\approx\frac{f(x+h)-f(x)}{h} \tag{7.1}$$

(2) 向后差商公式

$$f'(x)\approx\frac{f(x)-f(x-h)}{h} \tag{7.2}$$

(3) 中心差商公式

$$f'(x)\approx\frac{f(x+h)-f(x-h)}{2h} \tag{7.3}$$

由 Taylor (泰勒) 公式很容易得到它们的余项公式

$$f'(x)-\frac{f(x+h)-f(x)}{h}=-\frac{f''(x+\theta_1 h)}{2}h=O(h)$$

$$f'(x)-\frac{f(x)-f(x-h)}{h}=\frac{f''(x-\theta_2 h)}{2}h=O(h) \tag{7.4}$$

扫描全能王 创建

$$f'(x)-\frac{f(x+h)-f(x-h)}{2h}=-\frac{f^{(3)}(x+\theta_1 h)+f^{(3)}(x-\theta_2 h)}{12}h^2=O(h^2)$$

其中 $0<\theta_1,\theta_2<1$. 由式 (7.4) 可以看出, 用差商近似导数, 其精确度与步长 h 有关, h 越小近似程度越高. 中心差商公式的余项与 h^2 同阶, 应比向前、向后差商公式的精确度高. 实际计算时, 如果 h 取得过小, 又会因有效数字损失而导致误差增大. 以 $f(x)=\mathrm{e}^x$ 为例的计算结果说明了上述事实 (见表 7.1).

表 7.1　计算结果示例

h	x	$y=\mathrm{e}^x$	向前差商 D_+	中心差商 D	$f'(1.15)-D_+$	$f'(1.15)-D$
10^{-2}	1.14	3.126 768 4				
	1.15	3.158 192 9	3.174 03	3.158 245	−0.015 837 1	−0.000 052 1
	1.16	3.189 933 3				
10^{-3}	1.149	3.155 036 3				
	1.15	3.158 192 9	3.159 776	3.158 2	−0.001 583 1	-7.1×10^{-6}
	1.151	3.161 352 7				
10^{-4}	1.149 9	3.157 877 1				
	1.15	3.158 192 9	3.158 38	3.158	−0.000 187 1	0.000 192 9
	1.150 1	3.158 508 7				
10^{-6}	1.149 999	3.158 189 7				
	1.15	3.158 192 9	3.2	3.2	−0.041 8	−0.041 8
	1.150 001	3.158 196 1				

例 7.1　设 $f(x)=\mathrm{e}^x$, 分别取步长 $h=10^{-2},10^{-3},10^{-4},10^{-6}$, 用向前差商公式和中心差商公式计算 $f'(1.15)$.

解　按式 (7.1) 和式 (7.3), 计算结果见表 7.1. 计算结果说明, 当步长由 10^{-2} 缩小到 10^{-4} 时, 误差在减小. 进而缩小到 10^{-6} 时, 由于有效数字的丢失, 误差反而增大. 当步长不太小时, 中心差商公式的精度优于向前差商公式.

从几何上看, 如图 7.1 所示, 向前、向后、中心差商公式分别是以弦 BC,AB,AC 的斜率近似曲线 $y=f(x)$ 在点 B 处的切线斜率. 一般地, AC 的斜率与点 B 处切线的斜率更接近.

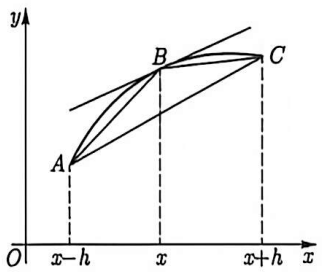


图 7.1

7.1.2　插值型求导公式

已知函数在一些离散点上的值时, 可用插值多项式近似函数, 因此可望用插值多项式的导数作为函数导数的近似值.

设已知函数 $f(x)$ 在 $[a,b]$ 内 $n+1$ 个节点 x_i $(i=0,1,\cdots,n)$ 处的函数值 $y_i=f(x_i)$ $(i=0,1,\cdots,n)$, 其插值多项式为 $\varphi_n(x)$, 用 $\varphi_n(x)$ 的导数近似函数导数, 即

$$f^{(k)}(x)\approx\varphi_n^{(k)}(x)\quad(k=1,2,\cdots)\tag{7.5}$$

扫描全能王 创建

式 (7.5) 称为插值型求导公式, 这是求数值微分的常用方法.

由定理 5.1, 只要 $f(x)$ 在 $[a,b]$ 内 $n+1$ 阶可导, 插值余项为

$$R_n(x)=f(x)-\varphi_n(x)=\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x) \tag{7.6}$$

将上式两端对 x 求导, 得

$$\begin{aligned} R_n'(x)&=f'(x)-\varphi_n'(x)=\frac{\mathrm{d}}{\mathrm{d}x}\left[\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x)\right]\\ &=\frac{\omega_{n+1}(x)}{(n+1)!}\frac{\mathrm{d}}{\mathrm{d}x}\left[f^{(n+1)}(\xi)\right]+\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}'(x) \end{aligned} \tag{7.7}$$

这就是 $k=1$ 时插值型求导公式 (7.5) 的截断误差.

对任意的 $x\in[a,b]$, 因为 ξ 未知, 故由式 (7.7) 很难估计误差, 但若取 $x=x_i$ $(i=0,1,\cdots,n)$, 则因 $\omega_{n+1}(x_i)=0$, 误差就容易估计. 由式 (7.7) 有

$$\begin{aligned} R_n'(x_i)&=f'(x_i)-\varphi_n'(x_i)\\ &=\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}'(x_i)\\ &=\frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{\substack{j=0\\ j\neq i}}^{n}(x_i-x_j)\quad (i=0,1,\cdots,n) \end{aligned} \tag{7.8}$$

因此, 插值型求导公式通常用于求节点处导数的近似值.

下面给出等距节点情况下, 求一阶导数的几个常用公式.

1. 两点公式 $(n=1)$: 过节点 $x_0,x_1=x_0+h$ 的 Lagrange 插值多项式为

$$L_1(x)=\frac{x-x_1}{-h}f(x_0)+\frac{x-x_0}{h}f(x_1)$$

故有

$$\begin{cases} f'(x_0)\approx L_1'(x_0)=\dfrac{f(x_1)-f(x_0)}{h}\\ f'(x_1)\approx L_1'(x_1)=\dfrac{f(x_1)-f(x_0)}{h} \end{cases} \tag{7.9}$$

截断误差为

$$\begin{cases} R_1'(x_0)=-\dfrac{h}{2}f''(\xi_0)\\ R_1'(x_1)=\dfrac{h}{2}f''(\xi_1) \end{cases} \quad (\xi_0,\xi_1\in(x_0,x_1)) \tag{7.10}$$

2. 三点公式: 过节点 $x_i=x_0+ih$ $(i=0,1,2)$, $f(x)$ 的 Lagrange 插值多项式为

$$L_2(x)=\frac{(x-x_1)(x-x_2)}{2h^2}f(x_0)+\frac{(x-x_0)(x-x_2)}{-h^2}f(x_1)+\frac{(x-x_0)(x-x_1)}{2h^2}f(x_2)$$

扫描全能王 创建

求导得

$$L_2'(x)=\frac{2x-x_1-x_2}{2h^2}f(x_0)-\frac{2x-x_0-x_2}{h^2}f(x_1)+\frac{2x-x_0-x_1}{2h^2}f(x_2)$$

分别代入 x_0,x_1,x_2 得三点公式

$$\begin{cases} f'(x_0)\approx\dfrac{1}{2h}\left[-3f(x_0)+4f(x_1)-f(x_2)\right] \\ f'(x_1)\approx\dfrac{1}{2h}\left[-f(x_0)+f(x_2)\right] \\ f'(x_2)\approx\dfrac{1}{2h}\left[f(x_0)-4f(x_1)+3f(x_2)\right] \end{cases} \tag{7.11}$$

截断误差为

$$\begin{cases} R_2'(x_0)=\dfrac{h^2}{3}f^{(3)}(\xi_0) \\ R_2'(x_1)=-\dfrac{h^2}{6}f^{(3)}(\xi_1),\quad \xi_i\in(x_0,x_2)\quad (i=0,1,2) \\ R_2'(x_2)=\dfrac{h^2}{3}f^{(3)}(\xi_2) \end{cases} \tag{7.12}$$

求二阶导数的三点公式为

$$f''(x_i)\approx L_2''(x_i)=\frac{1}{h^2}\left[f(x_0)-2f(x_1)+f(x_2)\right]\quad (i=0,1,2) \tag{7.13}$$

7.1.3　利用样条函数求数值微分

由 5.5 节的讨论, 当步长 $h\to 0$ 时, 三次样条函数 $S(x)$ 及其一、二阶导数分别一致收敛于被插值函数 $f(x)$ 及其一、二阶导数, 故用样条函数的导数近似函数导数

$$f^{(k)}(x)\approx S^{(k)}(x)\quad (k=1,2)$$

不仅可靠性好, 而且可以计算非节点处导数的近似值. 其截断误差为

$$f^{(k)}(x)-S^{(k)}(x)=O\left(h^{4-k}\right)$$

以三转角关系式为例

$$S(x)=\left(1+2\frac{x-x_i}{h_i}\right)\frac{(x-x_{i+1})^2}{h_i^2}f(x_i)+\left(1-2\frac{x-x_{i+1}}{h_i}\right)\frac{(x-x_i)^2}{h_i^2}f(x_{i+1})+$$

$$(x-x_i)\frac{(x-x_{i+1})^2}{h_i^2}m_i+(x-x_{i+1})\frac{(x-x_i)^2}{h_i^2}m_{i+1}\quad (x_i\leqslant x\leqslant x_{i+1})$$

对 x 求导得

$$f'(x)\approx S'(x)=\frac{6}{h_i^3}\left[(x-x_{i+1})^2+h_i(x-x_{i+1})\right]f(x_i)+$$

$$\frac{6}{h_i^3}\left[h_i(x-x_i)-(x-x_i)^2\right]f(x_{i+1})+$$

扫描全能王 创建

$$\frac{1}{h_i^2}\left[3\left(x-x_{i+1}\right)^2+2h_i\left(x-x_{i+1}\right)\right]m_i+$$
$$\frac{1}{h_i^2}\left[3\left(x-x_i\right)^2-2h_i\left(x-x_i\right)\right]m_{i+1}$$
$$\left(x_i\leqslant x\leqslant x_{i+1}\right) \tag{7.14}$$

特别地, $f'\left(x_i\right)\approx m_i\ (i=0,1,\cdots,n), m_i$ 由三转角方程求得.

$$f''(x)\approx S''(x)=\frac{6}{h_i^3}\left[2\left(x-x_{i+1}\right)+h_i\right]f\left(x_i\right)+\frac{6}{h_i^3}\left[h_i-2\left(x-x_i\right)\right]f\left(x_{i+1}\right)+$$
$$\frac{2}{h_i^2}\left[3\left(x-x_{i+1}\right)+h_i\right]m_i+\frac{2}{h_i^2}\left[3\left(x-x_i\right)-h_i\right]m_{i+1}$$
$$\left(x_i\leqslant x\leqslant x_{i+1}\right) \tag{7.15}$$

7.2 Newton-Cotes 求积公式

7.2.1 数值积分的基本思想

定积分的定义表明, 函数 $f(x)$ 在 $[a,b]$ 上的积分值是和式的极限

$$I(f)=\int_a^b f(x)\mathrm{d}x=\lim_{\lambda\to 0}\sum_{i=1}^{n}f\left(\xi_i\right)\Delta x_i$$

其中 $\lambda=\max\limits_{1\leqslant i\leqslant n}\Delta x_i$, 积分和式 $\sum\limits_{i=1}^{n}f\left(\xi_i\right)\Delta x_i$ 是一些点上函数值的线性组合. 构造数值积分方法的基本思想就是用被积函数在积分区间 $[a,b]$ 上的某些节点 $x_k\ (k=0,1,\cdots,n)$ 处的函数值的线性组合作为定积分的近似值, 即

$$I(f)=\int_a^b f(x)\mathrm{d}x\approx\sum_{k=0}^{n}A_k f\left(x_k\right) \tag{7.16}$$

其中 $\{A_k\}$ 称为求积系数, 它只与节点 $\{x_k\}$ 有关, 与被积函数 $f(x)$ 无关.

由节点 $\{x_k\}$ 和求积系数 $\{A_k\}$ 的不同取法, 可以得到不同的求积公式. 最直接自然的一种想法是用 $f(x)$ 在 $[a,b]$ 上的插值多项式 $\varphi_n(x)$ 代替 $f(x)$, 以 $\varphi_n(x)$ 在区间 $[a,b]$ 上的积分值作为所求积分 $I(f)$ 的近似值, 即

$$I(f)\approx\int_a^b\varphi_n(x)\mathrm{d}x$$

这样得到的求积公式称为插值型求积公式. 通常采用 Lagrange 插值.

设 $[a,b]$ 上有 $n+1$ 个互异节点 $x_0,x_1,\cdots,x_n, f(x)$ 的 n 次 Lagrange 插值多项式为

扫描全能王 创建

$$L_n(x)=\sum_{k=0}^{n}l_k(x)f(x_k)$$

其中 $l_k(x)=\prod\limits_{\substack{j=0\\j\neq k}}^{n}\dfrac{x-x_i}{x_k-x_j}$, 插值型求积公式为

$$I(f)\approx\int_a^b L_n(x)\mathrm{d}x=\sum_{k=0}^{n}\left(\int_a^b l_k(x)\mathrm{d}x\right)f(x_k) \tag{7.17}$$

显然, 求积系数 $A_k=\displaystyle\int_a^b l_k(x)\mathrm{d}x$ $(k=0,1,\cdots,n)$ 仅与节点 $\{x_k\}$ 有关, 与被积函数 $f(x)$ 的具体形式无关. 求积公式 (7.17) 的截断误差为

$$R_n(f)=\int_a^b f(x)\mathrm{d}x-\int_a^b L_n(x)\mathrm{d}x=\int_a^b\frac{f^{(n+1)}(\xi)}{(n+1)!}\omega_{n+1}(x)\mathrm{d}x \tag{7.18}$$

式 (7.18) 表明, 插值型求积公式的截断误差恰为插值余项在 $[a,b]$ 上的定积分值.

7.2.2　Newton-Cotes 求积公式

如果取等距节点 $\{x_k\}$, 则插值型求积公式更简便易算.

设 $x_k=a+kh$ $(k=0,1,\cdots,n), h=\dfrac{b-a}{n}$, 由式 (7.17)

$$A_k=\int_a^b l_k(x)\mathrm{d}x=\int_a^b\left(\prod_{\substack{j=0\\j\neq k}}^{n}\frac{x-x_j}{x_k-x_j}\right)\mathrm{d}x=\int_a^b\left[\prod_{\substack{j=0\\j\neq k}}^{n}\frac{x-a-jh}{(k-j)h}\right]\mathrm{d}x$$

令 $x=a+th$, 则有

$$A_k=\int_0^n\left[\prod_{\substack{j=0\\j\neq k}}^{n}\frac{(t-j)h}{(k-j)h}\right]h\ \mathrm{d}t=\frac{(-1)^{n-k}h}{k!(n-k)!}\int_0^n\prod_{\substack{j=0\\j\neq k}}^{n}(t-j)\mathrm{d}t$$

$$=\frac{(-1)^{n-k}(b-a)}{k!(n-k)!n}\int_0^n\prod_{\substack{j=0\\j\neq k}}^{n}(t-j)\mathrm{d}t$$

令

$$C_k^{(n)}=\frac{(-1)^{n-k}}{k!(n-k)!n}\int_0^n\prod_{\substack{j=0\\j\neq k}}^{n}(t-j)\mathrm{d}t \tag{7.19}$$

则 $A_k=(b-a)C_k^{(n)}$, 求积公式 (7.17) 可简化为

$$I(f)=\int_a^b f(x)\mathrm{d}x\approx(b-a)\sum_{k=0}^{n}C_k^{(n)}f(x_k) \tag{7.20}$$

扫描全能王 创建

式 (7.20) 称为 n 阶 Newton-Cotes (牛顿 – 科茨) 公式, 简记为 N-C 公式, $\{C_k^{(n)}\}$ 称为 Cotes 系数.

N-C 公式的截断误差为

$$R_n(f)=\int_a^b \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{j=0}^{n}(x-x_j)\,\mathrm{d}x=\frac{h^{n+2}}{(n+1)!}\int_0^n f^{(n+1)}(\xi)\prod_{j=0}^{n}(t-j)\mathrm{d}t \tag{7.21}$$

因为 $C_k^{(n)}$ 仅与等分区间数 n 有关, 与积分区间及被积函数均无关, 故可事先造好 Cotes 系数表 (见表 7.2), 这样使用 N-C 公式计算定积分近似值更加方便. 例如, $n=4$ 时, N-C 公式为

$$\begin{aligned} I(f) &= \int_a^b f(x)\mathrm{d}x \\ &\approx \frac{b-a}{90}\left[7f(a)+32f\left(a+\frac{b-a}{4}\right)+12f\left(\frac{a+b}{2}\right)+32f\left(a+\frac{3(b-a)}{4}\right)+7f(b)\right] \end{aligned} \tag{7.22}$$

式 (7.22) 是常用的求积公式, 称为 Cotes 公式.

表 7.2 Cotes 系数表

n	$C_k^{(n)}$						
1	$\frac{1}{2}$	$\frac{1}{2}$					
2	$\frac{1}{6}$	$\frac{2}{3}$	$\frac{1}{6}$				
3	$\frac{1}{8}$	$\frac{3}{8}$	$\frac{3}{8}$	$\frac{1}{8}$			
4	$\frac{7}{90}$	$\frac{16}{45}$	$\frac{2}{15}$	$\frac{16}{45}$	$\frac{7}{90}$		
5	$\frac{19}{288}$	$\frac{25}{96}$	$\frac{25}{144}$	$\frac{25}{144}$	$\frac{25}{96}$	$\frac{19}{288}$	
6	$\frac{41}{840}$	$\frac{9}{35}$	$\frac{9}{280}$	$\frac{34}{105}$	$\frac{9}{280}$	$\frac{9}{35}$	$\frac{41}{840}$

特别地, 当 $n=1$ 时, $C_0^{(1)}=C_1^{(1)}=\frac{1}{2}$, 因此有

$$I(f)=\int_a^b f(x)\mathrm{d}x\approx\frac{b-a}{2}[f(a)+f(b)] \tag{7.23}$$

如图 7.2 所示, 这是以过点 $(a,f(a)),(b,f(b))$ 的直线代替曲线 $y=f(x)$, 以梯形面积近似曲边梯形面积. 故式 (7.23) 又称为梯形公式.

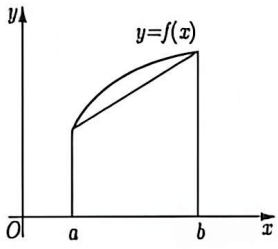


图 7.2

当 $n=2$ 时, N-C 公式为

$$I(f)=\int_a^b f(x)\mathrm{d}x\approx\frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right] \tag{7.24}$$

扫描全能王 创建

从几何上看, 这是以过曲线上三点 $(a,f(a))$, $\left(\dfrac{a+b}{2},f\left(\dfrac{a+b}{2}\right)\right)$, $(b,f(b))$ 的抛物线代替曲线 $y=f(x)$ 求曲边梯形面积的近似值, 因而此公式称为抛物线求积公式, 又称 Simpson(辛普森) 公式. 梯形公式与 Simpson 公式都是常用的求积公式.

7.2.3　误差估计

首先引入衡量数值积分公式近似程度的概念.

1. 求积公式的代数精确度

定义 7.1　*若当 $f(x)$ 为任意次数不高于 m 的多项式时, 求积公式*

$$\int_a^b f(x)\mathrm{d}x \approx \sum_{k=0}^{n} A_k f\left(x_k\right)$$

均精确成立, 而对某个 $m+1$ 次多项式, 公式不精确成立, 则称该求积公式具有 m 次代数精确度.

由定义容易看出, 求积公式的代数精确度越高, 使公式精确成立的多项式的次数越高, 从而使用这个公式越可靠. 因此, 代数精确度在一定程度上确实能反映求积公式的近似程度.

不难验证, 梯形公式具有一次代数精确度. 因为若 $f(x)$ 是一次多项式, 则

$$R_1(x)=f(x)-L_1(x)=\frac{f''(\xi)}{2}(x-a)(x-b)=0$$

从而有

$$\int_a^b f(x)\mathrm{d}x=\int_a^b L_1(x)\mathrm{d}x=\frac{b-a}{2}[f(a)+f(b)]$$

若取 $f(x)=x^2$, 则

$$\int_a^b f(x)\mathrm{d}x=\int_a^b x^2\,\mathrm{d}x=\frac{1}{3}\left(b^3-a^3\right)\neq\frac{b-a}{2}\left(a^2+b^2\right)$$

一般地, 由 n 次插值多项式导出的求积公式至少有 n 次代数精确度. 对 N-C 公式, 更有以下结论.

定理 7.1　*$2n$ 阶 Newton-Cotes 公式至少具有 $2n+1$ 次代数精确度.*

证明　设 $P_{2n+1}(x)=a_{2n+1}x^{2n+1}+a_{2n}x^{2n}+\cdots+a_1x+a_0$ 为 $2n+1$ 次多项式, 由式 (7.21)

$$\begin{aligned}R_{2n}\left(P_{2n+1}\right)&=\int_a^b\frac{P_{2n+1}^{(2n+1)}(\xi)}{(2n+1)!}\prod_{j=0}^{2n}(x-x_j)\,\mathrm{d}x\\&=a_{2n+1}\int_a^b\prod_{j=0}^{2n}(x-x_j)\,\mathrm{d}x\end{aligned}$$

令 $x=a+nh+th$ 代入上式, 得

扫描全能王 创建

$$
\begin{aligned}
R_{2n}\left(P_{2n+1}\right) &= a_{2n+1}h^{2n+2}\int_{-n}^{n}\prod_{j=0}^{2n}(t+n-j)\mathrm{d}t \\
&= a_{2n+1}h^{2n+2}\int_{-n}^{n}(t+n)(t+n-1)\cdots(t+1)t(t-1)\cdots(t-n)\mathrm{d}t \\
&= a_{2n+1}h^{2n+2}\int_{-n}^{n}t\left(t^2-1\right)\left(t^2-2^2\right)\cdots\left(t^2-n^2\right)\mathrm{d}t
\end{aligned}
$$

上式中被积函数是奇函数, 积分区间关于原点对称, 故积分值为零, 即

$$
R_{2n}\left(P_{2n+1}\right)=0
$$

所以 $2n$ 阶 N-C 公式至少有 $2n+1$ 次代数精度.

容易证明, 求积公式 (7.16) 具有 m 次代数精确度的充要条件是它对 $f(x)=1,x,\cdots,x^m$ 都能准确成立, 但对 $f(x)=x^{m+1}$, 式 (7.16) 不准确成立. 这一结论给出了判别一个求积公式的代数精确度的方法.

例 7.2　判别求积公式

$$
\int_{-1}^{1}f(x)\mathrm{d}x\approx\frac{1}{9}[5f(\sqrt{0.6})+8f(0)+5f(-\sqrt{0.6})] \tag{7.25}
$$

的代数精确度.

解　记

$$
\begin{aligned}
I(f) &= \int_{-1}^{1}f(x)\mathrm{d}x \\
\tilde{I}(f) &= \frac{1}{9}[5f(\sqrt{0.6})+8f(0)+5f(-\sqrt{0.6})]
\end{aligned}
$$

因为

$$
\begin{aligned}
&I(1)=\int_{-1}^{1}\mathrm{d}x=2, \qquad \tilde{I}(1)=\frac{1}{9}(5+8+5)=2 \\
&I(x)=\int_{-1}^{1}x\,\mathrm{d}x=0, \qquad \tilde{I}(x)=\frac{1}{9}[5\times\sqrt{0.6}+8\times0+5\times(-\sqrt{0.6})]=0 \\
&I\left(x^2\right)=\int_{-1}^{1}x^2\,\mathrm{d}x=\frac{2}{3}, \quad \tilde{I}\left(x^2\right)=\frac{1}{9}(5\times0.6+0+5\times0.6)=\frac{2}{3} \\
&I\left(x^3\right)=\int_{-1}^{1}x^3\,\mathrm{d}x=0, \quad \tilde{I}\left(x^3\right)=\frac{1}{9}\left[5\times(\sqrt{0.6})^3+0+5\times(-\sqrt{0.6})^3\right]=0 \\
&I\left(x^4\right)=\int_{-1}^{1}x^4\,\mathrm{d}x=\frac{2}{5}, \quad \tilde{I}\left(x^4\right)=\frac{1}{9}(5\times0.36+0+5\times0.36)=\frac{2}{5} \\
&I\left(x^5\right)=\int_{-1}^{1}x^5\,\mathrm{d}x=0, \quad \tilde{I}\left(x^5\right)=\frac{1}{9}\left[5\times(\sqrt{0.6})^5+0+5\times(-\sqrt{0.6})^5\right]=0 \\
&I\left(x^6\right)=\int_{-1}^{1}x^6\,\mathrm{d}x=\frac{2}{7}, \quad \tilde{I}\left(x^6\right)=\frac{1}{9}\left(5\times0.6^3+0+5\times0.6^3\right)=0.24\neq\frac{2}{7}
\end{aligned}
$$

扫描全能王 创建

所以求积公式 (7.25) 具有 5 次代数精确度.

利用余项公式及积分中值定理, 可以导出低阶 N-C 公式的误差估计.

2. 梯形公式和 Simpson 公式的误差估计

由式 (7.21), 梯形公式的截断误差为

$$R_1(f)=\int_a^b \frac{f''(\xi_x)}{2}(x-a)(x-b)\mathrm{d}x,\quad \xi_x\in(a,b)$$

因为 $(x-a)(x-b)$ 在区间 $[a,b]$ 内不变号, 如果 $f(x)$ 二阶连续可导, 则由反常积分中值定理, 存在 $\xi\in(a,b)$, 使得

$$\begin{aligned}R_1(f)&=\frac{f''(\xi)}{2}\int_a^b(x-a)(x-b)\mathrm{d}x \quad =\frac{f''(\xi)}{2}\left[\frac{x^3}{3}-(a+b)\frac{x^2}{2}+abx\right]_a^b\\&=-\frac{f''(\xi)}{12}(b-a)^3\end{aligned}$$

定理 7.2　*若 $f(x)\in C^{(2)}[a,b]$, 则梯形公式的误差为*

$$R_1(f)=\int_a^b f(x)\mathrm{d}x-\frac{b-a}{2}[f(a)+f(b)]=-\frac{(b-a)^3}{12}f''(\xi),\quad \xi\in(a,b)\tag{7.26}$$

借助于 $f(x)$ 的 Hermite 插值与定理 7.1, 可导出 Simpson 公式的误差估计.

定理 7.3　*若 $f(x)\in C^{(4)}[a,b]$, 则 Simpson 公式的误差为*

$$\begin{aligned}R_2(f)&=\int_a^b f(x)\mathrm{d}x-\frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right]\\&=-\frac{(b-a)^5}{2\,880}f^{(4)}(\xi)=-\frac{h^5}{90}f^{(4)}(\xi),\quad \xi\in(a,b)\end{aligned}\tag{7.27}$$

其中 $h=\dfrac{b-a}{2}$ 为步长.

证明　作 $f(x)$ 的三次插值多项式 $H_3(x)$, 使其满足

$$H_3(a)=f(a),\quad H_3(b)=f(b)$$

$$H_3\left(\frac{a+b}{2}\right)=f\left(\frac{a+b}{2}\right),\quad H_3'\left(\frac{a+b}{2}\right)=f'\left(\frac{a+b}{2}\right)$$

由式 (5.36), 插值余项为

$$R(x)=f(x)-H_3(x)=\frac{f^{(4)}(\xi_x)}{4!}(x-a)\left(x-\frac{a+b}{2}\right)^2(x-b),\quad \xi_x\in(a,b)$$

定理 7.1 表明, Simpson 公式至少具有 3 次代数精确度, 故有

扫描全能王 创建

$$\int_a^b H_3(x)\mathrm{d}x = \frac{b-a}{6}\left[H_3(a)+4H_3\left(\frac{a+b}{2}\right)+H_3(b)\right]$$

$$= \frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right]$$

于是

$$R_2(f) = \int_a^b f(x)\mathrm{d}x - \frac{b-a}{6}\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right]$$

$$= \int_a^b \left[f(x)-H_3(x)\right]\mathrm{d}x$$

$$= \int_a^b \frac{f^{(4)}(\xi_x)}{4!}(x-a)\left(x-\frac{a+b}{2}\right)^2(x-b)\mathrm{d}x$$

因为 $(x-a)\left(x-\dfrac{a+b}{2}\right)^2(x-b)$ 在 $[a,b]$ 内不变号, $f(x)\in C^{(4)}[a,b]$, 由反常积分中值定理得

$$R_2(f) = \frac{f^{(4)}(\xi)}{4!}\int_a^b (x-a)\left(x-\frac{a+b}{2}\right)^2(x-b)\mathrm{d}x$$

$$= -\frac{(b-a)^5}{2\ 880}f^{(4)}(\xi) = -\frac{h^5}{90}f^{(4)}(\xi), \quad \xi\in(a,b)$$

利用更复杂的运算可以导出 Cotes 公式的截断误差为

$$R_4(f) = -\frac{8}{945}h^7 f^{(6)}(\xi), \quad \xi\in(a,b) \tag{7.28}$$

其中 $h=\dfrac{b-a}{4}$.

例 7.3　分别用梯形公式与 Simpson 公式计算积分

$$I = \int_0^1 \mathrm{e}^x\ \mathrm{d}x$$

的近似值, 并估计截断误差.

解　由梯形公式 (7.23)

$$I \approx \frac{1}{2}\left(\mathrm{e}^0+\mathrm{e}^1\right) = 1.859\ 14$$

截断误差为

$$|R_1| = \frac{1}{12}|f''(\xi)| \leqslant \frac{\mathrm{e}}{12} = 0.226\ 52$$

由 Simpson 公式 (7.24)

$$I \approx \frac{1}{6}\left(\mathrm{e}^0+4\mathrm{e}^{\frac{1}{2}}+\mathrm{e}^1\right) = 1.718\ 86$$

截断误差为

扫描全能王 创建

$$|R_2| = \frac{1}{2\,880}\left|f^{(4)}(\xi)\right| \leqslant \frac{\mathrm{e}}{2\,880} \approx 0.000\,94$$

计算结果表明, 用 Simpson 公式计算的结果明显优于梯形公式的计算结果.

7.3　复化求积公式

前面导出的误差估计式表明, 用 N-C 公式计算积分近似值时, 步长越小, 截断误差越小. 但缩小步长等于增加节点数, 亦即提高插值多项式的次数, Runge 现象表明, 这样做并不一定能提高精度. 理论上已经证明, 当 $n \to \infty$ 时, N-C 公式所求得的近似值不一定收敛于积分的准确值, 而且随着 n 的增大, N-C 公式是不稳定的. 因此, 实际中不常用高阶 N-C 公式, 为提高计算精度, 可考虑对被积函数用分段低次多项式插值, 由此导出复化求积公式.

7.3.1　复化梯形公式

用分段线性插值函数近似被积函数, 等于把积分区间分成若干小区间, 在每个小区间上以梯形面积近似曲边梯形面积, 即用梯形公式求小区间上积分的近似值. 如图 7.3 所示, 这样求得的近似值显然比用梯形公式计算精度高. 定积分存在定理表明, 只要被积函数连续, 当小区间长度趋于零时, 小梯形面积之和趋于曲边梯形面积的准确值, 即定积分的准确值.

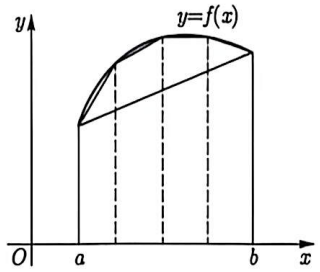


图 7.3

将积分区间 $[a,b]$ 分成 n 等份, 记 $h = \dfrac{b-a}{n}$, $x_k = a + kh$ $(k = 0,1,\cdots,n)$. 在每个小区间 $[x_k, x_{k+1}]$ $(k = 0,1,\cdots,n-1)$ 上用梯形公式并求和, 得

$$\int_a^b f(x)\mathrm{d}x = \sum_{k=0}^{n-1}\int_{x_k}^{x_{k+1}} f(x)\mathrm{d}x \approx \sum_{k=0}^{n-1}\frac{h}{2}\left[f\left(x_k\right) + f\left(x_{k+1}\right)\right]$$

整理得

$$\int_a^b f(x)\mathrm{d}x \approx \frac{h}{2}\left[f(a) + f(b) + 2\sum_{k=1}^{n-1} f\left(x_k\right)\right] = T_n \tag{7.29}$$

式 (7.29) 称为复化梯形公式.

如果 $f(x) \in C^{(2)}[a,b]$, 在小区间 $[x_k, x_{k+1}]$ 上, 梯形公式的截断误差为

$$\int_{x_k}^{x_{k+1}} f(x)\mathrm{d}x - \frac{h}{2}\left[f\left(x_k\right) + f\left(x_{k+1}\right)\right] = -\frac{h^3}{12}f''\left(\xi_k\right), \quad \xi_k \in \left(x_k, x_{k+1}\right)$$

因此

$$R_T(f) = \int_a^b f(x)\mathrm{d}x - T_n = -\frac{h^3}{12}\sum_{k=0}^{n-1} f''\left(\xi_k\right)$$

扫描全能王 创建

因为 $f''(x)$ 在 $[a,b]$ 连续, 由介值定理, 存在 $\xi \in (a,b)$, 使得

$$f''(\xi) = \frac{1}{n}\sum_{k=0}^{n-1} f''(\xi_k)$$

从而有

$$R_T(f) = \int_a^b f(x)\mathrm{d}x - T_n = -\frac{h^3}{12}nf''(\xi) = -\frac{b-a}{12}h^2 f''(\xi), \quad \xi \in (a,b) \tag{7.30}$$

这就是复化梯形公式的截断误差.

下面简单讨论复化梯形公式的数值稳定性. 设计算函数值 $f(x_k)$ 时产生误差为 δ_k $(k = 0,1,\cdots,n)$, 则用式 (7.29) 计算结果的误差为

$$\begin{aligned} |\delta| &= \frac{h}{2}\left|\delta_0 + \delta_n + 2\sum_{k=1}^{n-1}\delta_k\right| \\ &\leqslant nh \max_{0\leqslant k\leqslant n}\{|\delta_k|\} \\ &= (b-a)\max_{0\leqslant k\leqslant n}\{|\delta_k|\} \end{aligned}$$

因此, 无论 n 为多大, 复化梯形公式是数值稳定的.

7.3.2 复化 Simpson 公式

如果用分段二次插值函数近似被积函数, 即在小区间上用 Simpson 公式计算积分近似值, 就导出复化 Simpson 公式.

将区间 $[a,b]$ 分成 $n = 2m$ 等份, 分点为 $x_k = a + kh$ $(k = 0,1,\cdots,n), h = \dfrac{b-a}{n}$, 在每个小区间 $[x_{2k-2}, x_{2k}]$ 上, 用 Simpson 公式求积分, 则有

$$\begin{aligned} \int_{x_{2k-2}}^{x_{2k}} f(x)\mathrm{d}x &\approx \frac{x_{2k} - x_{2k-2}}{6}\left[f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})\right] \\ &= \frac{h}{3}\left[f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})\right] \end{aligned}$$

求和得

$$\int_a^b f(x)\mathrm{d}x = \sum_{k=1}^{m}\int_{x_{2k-2}}^{x_{2k}} f(x)\mathrm{d}x \approx \sum_{k=1}^{m}\frac{h}{3}\left[f(x_{2k-2}) + 4f(x_{2k-1}) + f(x_{2k})\right]$$

整理后得到

$$\int_a^b f(x)\mathrm{d}x \approx \frac{h}{3}\left[f(a) + f(b) + 2\sum_{k=1}^{m-1} f(x_{2k}) + 4\sum_{k=1}^{m} f(x_{2k-1})\right] \tag{7.31}$$

扫描全能王 创建

式 (7.31) 称为复化 Simpson 公式.

如果 $f(x) \in C^{(4)}[a,b]$, 由式 (7.27) 可得复化 Simpson 公式的截断误差为

$$R_s(f) = \int_a^b f(x)\mathrm{d}x - \frac{h}{3}\left[f(a) + f(b) + 2\sum_{k=1}^{m-1} f\left(x_{2k}\right) + 4\sum_{k=1}^{m} f\left(x_{2k-1}\right)\right]$$

$$= \sum_{k=1}^{m} -\frac{(2h)^5}{2\,880} f^{(4)}\left(\xi_k\right), \quad \xi_k \in [x_{2k-2}, x_{2k}]$$

因为 $f^{(4)}(x)$ 连续, 故存在 $\xi \in (a,b)$, 使得

$$f^{(4)}(\xi) = \frac{1}{m}\sum_{k=1}^{m} f^{(4)}\left(\xi_k\right)$$

代入上式得

$$R_s(f) = -\frac{(2h)^5}{2\,880} m f^{(4)}(\xi) = -\frac{b-a}{180} h^4 f^{(4)}(\xi), \quad \xi \in (a,b) \tag{7.32}$$

式 (7.32) 表明, 步长 h 越小, 截断误差越小. 与复化梯形公式的分析相类似, 可以证明, 当 $n = 2m \to \infty$ 时, 用复化 Simpson 公式所求得的近似值收敛于积分值, 而且算法具有数值稳定性.

例 7.4　根据下面的函数表, 计算积分 $I = \int_0^1 \frac{\sin x}{x}\,\mathrm{d}x$ 的近似值.

k	x_k	$f(x_k) = \frac{\sin x_k}{x_k}$	k	x_k	$f(x_k) = \frac{\sin x_k}{x_k}$
0	0	1.000 000	5	0.625	0.936 156
1	0.125	0.997 398	6	0.75	0.908 852
2	0.25	0.989 616	7	0.875	0.877 193
3	0.375	0.976 727	8	1	0.841 471
4	0.5	0.958 851			

解　(1) 由复化梯形公式, 有

$$I \approx \frac{1}{16}\left[f(0) + f(1) + 2\sum_{k=1}^{7} f\left(\frac{k}{8}\right)\right] = 0.945\,691$$

(2) 由复化 Simpson 公式, 有

$$I \approx \frac{1}{24}\left[f(0) + f(1) + 2\sum_{k=1}^{3} f\left(\frac{k}{4}\right) + 4\sum_{k=1}^{4} f\left(\frac{2k-1}{8}\right)\right] = 0.946\,084$$

与准确值 $I = 0.946\,083\,1\cdots$ 比较, 显然用复化 Simpson 公式计算精度较高. 事实上, 由误差公式 (7.30) 和式 (7.32) 有 $R_T(f) = O\left(h^2\right), R_s(f) = O\left(h^4\right)$, 故当 h 比较小时, 用复化 Simpson 公式计算误差较小.

扫描全能王 创建

由误差估计公式不仅可以计算所求近似值的误差, 反之, 亦可由给定的精度估计应取多大步长.

例 7.5 若用复化求积公式计算积分

$$I=\int_0^1 \mathrm{e}^{-x}\,\mathrm{d}x$$

的近似值, 要求计算结果有四位有效数字, n 应取多大?

解 因为当 $0\leqslant x\leqslant 1$ 时, 有

$$0.3<\mathrm{e}^{-1}\leqslant \mathrm{e}^{-x}\leqslant 1$$

于是

$$0.3<\int_0^1 \mathrm{e}^{-x}\,\mathrm{d}x<1$$

要求计算结果有四位有效数字, 即要求误差不超过 $\frac{1}{2}\times 10^{-4}$. 又因为

$$\left|f^{(k)}(x)\right|=\mathrm{e}^{-x}\leqslant 1,\quad x\in[0,1]$$

由式 (7.30)

$$|R_T|=\frac{1}{12}h^2\left|f''(\xi)\right|\leqslant\frac{h^2}{12}\leqslant\frac{1}{2}\times 10^{-4}$$

即

$$n^2\geqslant\frac{1}{6}\times 10^4$$

开方得

$$n\geqslant 40.8$$

因此若用复化梯形公式求积分, n 应等于 41 才能达到精度.

若用复化 Simpson 公式, 由式 (7.32)

$$|R_s|=\frac{h^4}{180}\left|f^{(4)}(\xi)\right|\leqslant\frac{h^4}{180}\leqslant\frac{1}{2}\times 10^{-4}$$

即得 $n\geqslant 3.2$. 故应取 $n=4$.

例 7.5 的计算结果表明, 为达到相同的精度, 用复化 Simpson 公式所需的计算量比复化梯形公式少, 这也说明了复化 Simpson 公式的精度较高.

7.3.3 逐次分半算法

如前所述, 复化求积公式的截断误差随着步长的缩小而减小, 而且由给定的精度可以预先确定步长. 但由于误差估计式中出现高阶导数, 使问题变得十分困难. 实际计算时, 通常是从某个步长 h 出发计算近似值, 然后在前次分割的基础上, 再将每个小区间等分, 即将积分区间逐次分半, 并以前后两次计算结果之差来估计误差, 直到求得满足精度的积分近似值. 这样既缩小了步长, 又保留了已有的节点, 减少了计算量. 这就是逐次分半法.

扫描全能王 创建

(一) 梯形公式的逐次分半算法

将区间 $[a,b]$ 分为 $n=2^m$ $(m=0,1,2,\cdots)$ 等份, 记 $h_m=\dfrac{b-a}{2^m}$, 记

$$T_{2^m}=\frac{h_m}{2}\left[f(a)+f(b)+2\sum_{k=1}^{2^m-1}f\left(a+kh_m\right)\right]\quad(m=0,1,2,\cdots)\tag{7.33}$$

称 $\{T_{2^m}\}$ 为梯形值序列. 因为区间逐次分半, 第 $m-1$ 次计算的节点是第 m 次计算时的偶数节点, 因而在计算 T_{2^m} 时只需计算新节点处的函数值. 由复化梯形公式 (7.29), 有

$$\begin{aligned}T_{2^m}&=\frac{h_m}{2}\left[f(a)+f(b)+2\sum_{k=1}^{2^m-1}f\left(a+kh_m\right)\right]\\&=\frac{h_m}{2}\left[f(a)+f(b)+2\sum_{k=1}^{2^{m-1}-1}f\left(a+2kh_m\right)+2\sum_{k=1}^{2^{m-1}}f\left(a+(2k-1)h_m\right)\right]\\&=\frac{h_m}{2}\left[f(a)+f(b)+2\sum_{k=1}^{2^{m-1}-1}f\left(a+kh_{m-1}\right)+2\sum_{k=1}^{2^{m-1}}f\left(a+(2k-1)h_m\right)\right]\end{aligned}$$

于是得到递推公式

$$T_{2^m}=\frac{1}{2}T_{2^{m-1}}+h_m\sum_{k=1}^{2^{m-1}}f\left(a+(2k-1)h_m\right)\quad(m=1,2,\cdots)\tag{7.34}$$

式 (7.34) 称为梯形公式的逐次分半计算公式.

复化梯形公式的误差估计式表明, 由式 (7.34) 所生成的序列 $\{T_{2^m}\}$ 收敛于积分值 $\displaystyle\int_a^b f(x)\mathrm{d}x$, 其误差为

$$\begin{aligned}R\left(f,T_{2^m}\right)&=\int_a^b f(x)\mathrm{d}x-T_{2^m}\\&=-\frac{b-a}{12}h_m^2f''\left(\xi_m\right),\quad\xi_m\in(a,b)\end{aligned}$$

同时有

$$R\left(f,T_{2^{m-1}}\right)=-\frac{b-a}{12}h_{m-1}^2f''\left(\xi_{m-1}\right),\quad\xi_{m-1}\in(a,b)$$

将以上两式相减得

$$\begin{aligned}T_{2^m}-T_{2^{m-1}}&=\frac{b-a}{12}\left[h_m^2f''\left(\xi_m\right)-h_{m-1}^2f''\left(\xi_{m-1}\right)\right]\\&=-\frac{b-a}{12}h_m^2\left[4f''\left(\xi_{m-1}\right)-f''\left(\xi_m\right)\right]\end{aligned}$$

由复化梯形公式误差估计可知, $f''\left(\xi_{m-1}\right)$ 与 $f''\left(\xi_m\right)$ 分别是 $f''(x)$ 在 $[a,b]$ 上 2^{m-1} 个点与 2^m 个点处的算术平均值 (每个小区间取一个点). 如果 $f(x)\in C^{(2)}[a,b]$, 则当 m 较大时,

扫描全能王 创建

有 $f''(\xi_{m-1}) \approx f''(\xi_m)$, 从而有

$$T_{2^m} - T_{2^{m-1}} \approx -3\frac{b-a}{12}h_m^2 f''(\xi_m) = 3R(f, T_{2^m})$$

即

$$|R(f, T_{2^m})| \approx \frac{1}{3}|T_{2^m} - T_{2^{m-1}}|$$

因此, 若事先给定误差限 ε, 则当 $|T_{2^m} - T_{2^{m-1}}| < 3\varepsilon$ 时就可停止计算, 并认为 T_{2m} 是满足精度要求的近似值. 以计算所得的近似值来估计误差称为事后误差估计.

算法 7.1 梯形公式的逐次分半算法

1. 输入 $a, b, f(x), \varepsilon$.
2. 置 $m = 1, h = \dfrac{b-a}{2}, T_0 = h[f(a) + f(b)]$.
3. 置 $F = 0$, 对 $k = 1, 2, \cdots, 2^{m-1}$,

$$F = F + f(a + (2k-1)h)$$

4. $T = \dfrac{1}{2}T_0 + hF$.
5. 若 $|T - T_0| < 3\varepsilon$, 则输出 $I \approx T$, 停机; 否则 $m+1 \Rightarrow m, \dfrac{h}{2} \Rightarrow h, T \Rightarrow T_0$, 转 3.

完全类似地, 可以导出

(二) Simpson 公式的逐次分半算法

将区间 $[a, b]$ 分成 2^m 等份, $h_m = \dfrac{b-a}{2^m}$, 记

$$S_{2^m} = \frac{h_m}{3}\left[f(a) + f(b) + 2\sum_{k=1}^{2^{m-1}-1} f(a + 2kh_m) + 4\sum_{k=1}^{2^{m-1}} f(a + (2k-1)h_m)\right] \tag{7.35}$$

按式 (7.35) 计算所得的序列 $\{S_{2^m}\}$ 称为 Simpson 序列. 当 $m \to \infty$ 时, $\{S_{2^m}\}$ 收敛于 $\int_a^b f(x)\mathrm{d}x$.

由复化 Simpson 公式的误差式 (7.32) 可得

$$\begin{aligned} R(f, S_{2^m}) &= \int_a^b f(x)\mathrm{d}x - S_{2^m} \\ &\approx \frac{1}{15}(S_{2^m} - S_{2^{m-1}}) \end{aligned}$$

因此, Simpson 公式的逐次分半算法以 $|S_{2^m} - S_{2^{m-1}}| < 15\varepsilon$ 为停步准则, 此时 $I \approx S_{2m}$ 即为满足精度要求的积分近似值.

扫描全能王 创建

算法 7.2　Simpson 公式的逐次分半算法

1. 输入 $a, b, f(x), \varepsilon$.
2. 置 $F_1 = f(a) + f(b), F_2 = f\left(\dfrac{a+b}{2}\right), S_0 = \dfrac{b-a}{6}(F_1 + 4F_2), m = 2, h = \dfrac{b-a}{4}$.
3. 置 $F_3 = 0$, 对 $k = 1, 2, \cdots, 2^{m-1}, F_3 = F_3 + f(a + (2k-1)h)$.
4. $S = \dfrac{h}{3}(F_1 + 2F_2 + 4F_3)$.
5. 若 $|S - S_0| < 15\varepsilon$, 则输出 $I \approx S$, 停机; 否则, 置 $m+1 \Rightarrow m, \dfrac{h}{2} \Rightarrow h, F_2 + F_3 \Rightarrow F_2, S \Rightarrow S_0$, 转 3 .

例 7.6　用 Simpson 公式的逐次分半算法按例 7.4 所给数据表计算积分 $I = \displaystyle\int_0^1 \frac{\sin x}{x}\,\mathrm{d}x$ 的近似值, 要求误差不超过 10^{-6}.

解　按式 (7.35) 计算

$$S_2 = \frac{1}{6}(1 + 0.841\ 471 + 4 \times 0.958\ 851) = 0.946\ 146$$

$$S_4 = \frac{1}{12}[1 + 0.841\ 471 + 2 \times 0.958\ 851 + 4 \times (0.989\ 616 + 0.908\ 852)] = 0.946\ 087$$

$$\frac{1}{15}|S_4 - S_2| = 3.93 \times 10^{-6} > 10^{-6}$$

$$S_8 = 0.946\ 084$$

$$\frac{1}{15}|S_8 - S_4| = \frac{3}{15} \times 10^{-6}$$

所以

$$I \approx S_8 = 0.946\ 084$$

在实际计算时, 常以 $|T_{2^m} - T_{2^{m-1}}| < \varepsilon$ 与 $|S_{2m} - S_{2m-1}| < \varepsilon$ 作为逐次分半算法的停步准则.

7.4　Romberg 求积公式

7.4.1　Richardson 外推法

式 (7.30) 表明, 复化梯形公式的截断误差为 $O(h^2)$, 因而梯形值序列 $\{T_{2^m}\}$ 收敛较慢. 事实上, 进一步分析可得出

$$R(f, T_n) = a_1 h^2 + a_2 h^4 + \cdots + a_k h^{2k} + \cdots \tag{7.36}$$

其中系数 $a_1, a_2, \cdots, a_k, \cdots$ 均与 h 无关, 式 (7.36) 称为 Euler-Maclaurin (欧拉 – 麦克劳林) 公式. 从此式出发, 利用加速收敛的技巧, 可以构造出快速收敛的近似值序列, Richardson

扫描全能王 创建

外推法就是这样一种方法.

设用某种数值方法求量 I 的近似值, 一般地, 近似值是步长 h 的函数, 记为 $I_1(h)$, 相应的误差为

$$I - I_1(h) = a_1 h^{p_1} + a_2 h^{p_2} + \cdots + a_k h^{p_k} + \cdots \tag{7.37}$$

其中 $0 < p_1 < p_2 < \cdots < p_k < \cdots$, a_i $(i = 1, 2, \cdots)$ 与 h 无关. 若用 αh 代替式 (7.37) 中的 h, 则得

$$\begin{aligned} I - I_1(\alpha h) &= a_1(\alpha h)^{p_1} + a_2(\alpha h)^{p_2} + \cdots + a_k(\alpha h)^{p_k} + \cdots \\ &= a_1\alpha^{p_1}h^{p_1} + a_2\alpha^{p_2}h^{p_2} + \cdots + a_k\alpha^{p_k}h^{p_k} + \cdots \end{aligned} \tag{7.38}$$

式 (7.38) 减去式 (7.37) 乘 α^{p_1}, 得

$$\begin{aligned} &I - I_1(\alpha h) - \alpha^{p_1}\left[I - I_1(h)\right] \\ =&a_2\left(\alpha^{p_2} - \alpha^{p_1}\right)h^{p_2} + a_3\left(\alpha^{p_3} - \alpha^{p_1}\right)h^{p_3} + \cdots + a_k\left(\alpha^{p_k} - \alpha^{p_1}\right)h^{p_k} + \cdots \end{aligned}$$

取 α 满足 $|\alpha| \neq 1$, 以 $1 - \alpha^{p_1}$ 除上式两端, 得

$$I - \frac{I_1(\alpha h) - \alpha^{p_1} I_1(h)}{1 - \alpha^{p_1}} = b_2 h^{p_2} + b_3 h^{p_3} + \cdots + b_k h^{p_k} + \cdots \tag{7.39}$$

其中 $b_i = a_i(\alpha^{p_i} - \alpha^{p_1})/(1 - \alpha^{p_1})$ $(i = 2, 3, \cdots)$ 仍与 h 无关. 令

$$I_2(h) = \frac{I_1(\alpha h) - \alpha^{p_1} I_1(h)}{1 - \alpha^{p_1}}$$

由式 (7.39), 以 $I_2(h)$ 作为 I 的近似值, 其误差至少为 $O\left(h^{p_2}\right)$, 因此 $I_2(h)$ 收敛于 I 的速度比 $I_1(h)$ 快. 不断重复以上做法, 可以得到一个函数序列

$$I_m(h) = \frac{I_{m-1}(\alpha h) - \alpha^{p_{m-1}} I_{m-1}(h)}{1 - \alpha^{p_{m-1}}} \quad (m = 2, 3, \cdots)$$

以 $I_m(h)$ 近似 I, 误差为 $I - I_m(h) = O\left(h^{p_m}\right)$. 随着 m 的增大, 收敛速度越来越快, 这就是 Richardson (理查逊) 外推法. 将此方法用于梯形值序列 $\{T_{2^k}\}$, 可以构造出一种收敛很快、计算简便的数值积分方法.

7.4.2 Romberg 求积公式

设 $I = \int_a^b f(x)\mathrm{d}x$, 记 2^k 等分区间 $[a, b]$ 用复化梯形公式所求得的近似值为 $T_{2^k} = T_0^{(k)} = I_1\left(\dfrac{b-a}{2^k}\right)$ $(k = 0, 1, 2, \cdots)$. 由 Euler-Maclaurin 公式

$$R\left(f, T_0^{(k)}\right) = a_1\left(\frac{b-a}{2^k}\right)^2 + a_2\left(\frac{b-a}{2^k}\right)^4 + \cdots + a_i\left(\frac{b-a}{2^k}\right)^{2i} + \cdots$$

取 $\alpha = \dfrac{1}{2}$, 由 Richardson 外推公式

扫描全能王 创建

$$I_2\left(\frac{b-a}{2^k}\right)=\frac{I_1\left(\frac{b-a}{2^{k+1}}\right)-\left(\frac{1}{2}\right)^2 I_1\left(\frac{b-a}{2^k}\right)}{1-\left(\frac{1}{2}\right)^2}=\frac{4T_0^{(k+1)}-T_0^{(k)}}{3}$$

记 $I_2\left(\frac{b-a}{2^k}\right)=T_1^{(k)}$, 则

$$T_1^{(k)}=\frac{4T_0^{(k+1)}-T_0^{(k)}}{3}\quad(k=0,1,2,\cdots)\tag{7.40}$$

且有

$$R\left(f,T_1^{(k)}\right)=O\left(\left(\frac{b-a}{2^k}\right)^4\right)$$

事实上, 容易验证, $\{T_1^{(k)}\}$ 就是 Simpson 值序列 $\{S_{2^k}\}$. 记 $h_k=\frac{b-a}{2^k}$, 则由复化梯形公式有

$$\begin{aligned}
&4T_0^{(k)}-T_0^{(k-1)}\\
=&2h_k\left[f(a)+f(b)+2\sum_{i=1}^{2^k-1}f\left(a+ih_k\right)\right]-\frac{h_{k-1}}{2}\left[f(a)+f(b)+2\sum_{i=1}^{2^{k-1}-1}f\left(a+ih_{k-1}\right)\right]\\
=&h_k\left[f(a)+f(b)+4\sum_{j=1}^{2^{k-1}}f\left(a+(2j-1)h_k\right)+2\sum_{j=1}^{2^{k-1}-1}f\left(a+2jh_k\right)\right]\\
=&3S_{2^k}
\end{aligned}$$

于是

$$S_{2^k}=\frac{4T_0^{(k)}-T_0^{(k-1)}}{3}=T_1^{(k-1)}$$

重复 Richardson 公式, 可得

$$\begin{aligned}
I_{m+1}\left(\frac{b-a}{2^k}\right)&=\frac{I_m\left(\frac{b-a}{2^{k+1}}\right)-\left(\frac{1}{2}\right)^{2m} I_m\left(\frac{b-a}{2^k}\right)}{1-\left(\frac{1}{2}\right)^{2m}}\\
&=\frac{4^m I_m\left(\frac{b-a}{2^{k+1}}\right)-I_m\left(\frac{b-a}{2^k}\right)}{4^m-1}
\end{aligned}$$

记 $T_m^{(k)}=I_{m+1}\left(\frac{b-a}{2^k}\right)$, 则上式可记为

$$T_m^{(k)}=\frac{4^m T_{m-1}^{(k+1)}-T_{m-1}^{(k)}}{4^m-1}\quad(m=1,2,\cdots;k=0,1,2,\cdots)\tag{7.41}$$

扫描全能王 创建

且 $R\left(f, T_m^{(k)}\right)=O\left(\left(\frac{b-a}{2^k}\right)^{2m+2}\right)$. 式 (7.41) 称为 Romberg (龙贝格) 求积公式, 其中 $T_0^{(k)}$ 由复化梯形公式求得. Romberg 公式的计算过程见表 7.3, 按行依次计算 $T_0^{(0)}$, $T_0^{(1)}$, $T_1^{(0)}$, $T_0^{(2)}$, $T_1^{(1)}$, $T_2^{(0)}$, $\cdots$.

表 7.3 Romberg 公式的计算过程

k	$T_0^{(k)}$	$T_1^{(k)}$	$T_2^{(k)}$	$T_3^{(k)}$	$\cdots$
0	$T_0^{(0)}$				
1	$T_0^{(1)}$	$T_1^{(0)}$			
2	$T_0^{(2)}$	$T_1^{(1)}$	$T_2^{(0)}$		
3	$T_0^{(3)}$	$T_1^{(2)}$	$T_2^{(1)}$	$T_3^{(0)}$	

表 7.3 中第一列是梯形序列值 $I-T_0^{(k)}=O\left(\left(\frac{b-a}{2^k}\right)^2\right)$. 以后各列则是对前一列用 Richardson 外推算法得到, 误差为 $I-T_m^{(k)}=O\left(\left(\frac{b-a}{2^k}\right)^{2m+2}\right)$, 因而, m 越大, 近似值序列 $\left\{T_m^{(k)}\right\}$ 的收敛速度越快.

按式 (7.34) 和式 (7.41) 计算, 以 $T_m^{(0)}$ $(m=0,1,2,\cdots)$ 作为积分近似值的算法称为 Romberg 方法, 其停步准则为 $\left|T_m^{(0)}-T_{m-1}^{(0)}\right|<\varepsilon$, 则 $T_m^{(0)}$ 即为满足精度的近似值.

算法 7.3 Romberg 方法

1. 输入 $a, b, f(x), \varepsilon$.
2. 置 $h=b-a, T_0^{(0)}=\frac{h}{2}[f(a)+f(b)], k=1$.
3. 计算

$$T_0^{(k)}=\frac{1}{2}\left[T_0^{(k-1)}+h\sum_{i=1}^{2^{k-1}} f\left(a+\left(i-\frac{1}{2}\right)h\right)\right]$$

对 $j=1,2,\cdots,k$,

$$T_j^{(k-j)}=\frac{4^j T_{j-1}^{(k-j+1)}-T_{j-1}^{(k-j)}}{4^j-1}$$

4. 若 $\left|T_k^{(0)}-T_{k-1}^{(0)}\right|<\varepsilon$, 则输出 $T_k^{(0)}\approx\int_a^b f(x)\mathrm{d}x$, 停机; 否则, $\frac{h}{2}\Rightarrow h, k+1\Rightarrow k$, 返回 3.

扫描全能王 创建

例 7.7　用 Romberg 方法计算积分 $I=\int_0^1 \frac{\sin x}{x}\,\mathrm{d}x$ 的近似值，要求误差不超过 $\frac{1}{2}\times 10^{-6}$.

解　根据例 7.4 中的函数表按式 (7.23)、式 (7.34) 和式 (7.41) 计算，得

$$T_0^{(0)}=\frac{1}{2}[f(0)+f(1)]=0.920\ 735\ 5$$

$$T_0^{(1)}=\frac{1}{2}T_0^{(0)}+\frac{1}{2}f(0.5)=0.939\ 793\ 2$$

$$T_1^{(0)}=\frac{4T_0^{(1)}-T_0^{(0)}}{3}=0.946\ 145\ 7$$

$$T_0^{(2)}=\frac{1}{2}T_0^{(1)}+\frac{1}{4}[f(0.25)+f(0.75)]=0.944\ 513\ 6$$

$$T_1^{(1)}=\frac{4T_0^{(2)}-T_0^{(1)}}{3}=0.946\ 087\ 0$$

$$T_2^{(0)}=\frac{16T_1^{(1)}-T_1^{(0)}}{15}=0.946\ 083\ 0$$

$$T_0^{(3)}=\frac{1}{2}T_0^{(2)}+\frac{1}{8}[f(0.125)+f(0.375)+f(0.625)+f(0.875)]=0.945\ 691\ 0$$

$$T_1^{(2)}=\frac{4T_0^{(3)}-T_0^{(2)}}{3}=0.946\ 083\ 5$$

$$T_2^{(1)}=\frac{16T_1^{(2)}-T_1^{(1)}}{15}=0.946\ 083\ 2$$

$$T_3^{(0)}=\frac{64T_2^{(1)}-T_2^{(0)}}{63}=0.946\ 083\ 2$$

因为

$$\left|T_3^{(0)}-T_2^{(0)}\right|=2\times 10^{-7}<\frac{1}{2}\times 10^{-6}$$

所以

$$I=\int_0^1 \frac{\sin x}{x}\,\mathrm{d}x\approx 0.946\ 083\ 2$$

事实上，由例 7.7 的计算过程可以看出，与准确值 $I=0.946\ 083\ 1\cdots$ 比较，$T_2^{(0)}$ 已达到精度要求，此时需要计算 4 个点的函数值. 要达到相同的精度，用复化 Simpson 公式需计算 8 个点的函数值，而用复化梯形公式则需计算 167 个点的函数值，所以 Romberg 方法优于复化梯形方法与复化 Simpson 方法.

对于一种确定的求积公式，我们可以增加节点个数，缩小步长 h，得到较精确的近似值；另一方面，在节点个数一定的前提下，不同求积公式的精确程度也是不同的. 理论分析与实

扫描全能王 创建

际计算均表明, 由 Romberg 公式得到的近似值的精确度高于复化求积公式得到的近似值, 复化 Simpson 公式的精确度又高于复化梯形公式. 由此得到启发, 在节点数固定的前提下, 可望通过适当地选取节点 $\{x_k\}$ 与求积系数 $\{A_k\}$, 提高数值积分方法的精确度.

7.5 Gauss 型求积公式

7.5.1 一般理论

(一) Gauss 型求积公式

Gauss 型求积公式的基本思想是: 在节点数 n 固定时, 适当地选取节点 $\{x_k\}$ 与求积系数 $\{A_k\}$, 使求积公式具有最高的代数精确度.

由定义 7.1, 求积公式

$$\int_a^b \rho(x) f(x) \mathrm{d}x \approx \sum_{k=1}^{n} A_k f\left(x_k\right) \tag{7.42}$$

其中 $\rho(x) \geqslant 0$ 为权函数, 具有 m 次代数精确度, 充要条件是对 $f(x)=x^l$ $(l=0,1,\cdots,m)$ 式 (7.42) 精确成立, 即

$$\int_a^b \rho(x) x^l \, \mathrm{d}x = \sum_{k=1}^{n} A_k x_k^l \quad (l=0,1,\cdots,m) \tag{7.43}$$

令

$$\mu_l = \int_a^b \rho(x) x^l \, \mathrm{d}x \quad (l=0,1,\cdots,m) \tag{7.44}$$

将式 (7.44) 代入式 (7.43), 得节点 $\{x_k\}$ 与求积系数 $\{A_k\}$ 应满足方程组

$$\begin{cases} A_1 + A_2 + \cdots + A_n = \mu_0 \\ A_1 x_1 + A_2 x_2 + \cdots + A_n x_n = \mu_1 \\ \cdots\cdots\cdots\cdots \\ A_1 x_1^m + A_2 x_2^m + \cdots + A_n x_n^m = \mu_m \end{cases} \tag{7.45}$$

式 (7.45) 含有 $2n$ 个未知数, $m+1$ 个方程. 可以证明, 当 $m+1 \leqslant 2n$ 时, 方程组 (7.45) 有解. 这表明 n 个节点的求积公式的代数精确度可达到 $2n-1$. 但若取 $2n$ 次多项式

$$p_{2n}(x) = (x-x_1)^2 (x-x_2)^2 \cdots (x-x_n)^2$$

显然, 对任意 n 个互异节点 $x_1, x_2, \cdots, x_n$, 都有 $P_{2n}\left(x_k\right)=0$ $(k=1,2,\cdots,n)$, 从而有 $\sum_{k=1}^{n} A_k P_{2n}\left(x_k\right)=0$. 然而

$$\int_a^b \rho(x) p_{2n}(x) \mathrm{d}x > 0$$

扫描全能王 创建

因此, n 个节点的求积公式的最高代数精确度为 $2n-1$.

定义 7.2　如果一组节点 $x_1, x_2, \cdots, x_n \in [a,b]$ 能使求积公式 (7.42), 即

$$\int_a^b \rho(x) f(x) \mathrm{d}x \approx \sum_{k=1}^{n} A_k f(x_k)$$

具有 $2n-1$ 次代数精确度, 则称这组节点 $\{x_k\}$ 为 Gauss 点, 公式 (7.42) 称为带权函数 $\rho(x)$ 的 Gauss 型求积公式.

例 7.8　试确定求积公式

$$\int_{-1}^{1} f(x) \mathrm{d}x \approx af(-\sqrt{0.6}) + bf(0) + cf(\sqrt{0.6})$$

中的待定参数 a, b 和 c, 使其代数精确度尽量高, 并指出公式具有几次代数精确度, 它是否是 Gauss 型求积公式.

解　记

$$I(f) = \int_a^b f(x) \mathrm{d}x, \quad \tilde{I}(f) = af(-\sqrt{0.6}) + bf(0) + cf(\sqrt{0.6})$$

因为

$$\begin{aligned}
I(1) &= \int_{-1}^{1} 1 \, \mathrm{d}x = 2, & \tilde{I}(1) &= a + b + c \\
I(x) &= \int_{-1}^{1} x \, \mathrm{d}x = 0, & \tilde{I}(1) &= -\sqrt{0.6}a + \sqrt{0.6}c \\
I\left(x^2\right) &= \int_{-1}^{1} x^2 \, \mathrm{d}x = \frac{2}{3}, & \tilde{I}\left(x^2\right) &= 0.6a + 0.6c
\end{aligned}$$

要使求积公式具有尽可能高的代数精确度, a, b, c 必须满足方程组

$$\begin{cases}
a + b + c = 2 \\
-\sqrt{0.6}a + \sqrt{0.6}c = 0 \\
0.6a + 0.6c = \dfrac{2}{3}
\end{cases}$$

求解得 $a = c = \dfrac{5}{9}, b = \dfrac{8}{9}$, 所以当 $a = c = \dfrac{5}{9}, b = \dfrac{8}{9}$ 时, 求积公式的代数精确度最高.

又因为

$$\begin{aligned}
I\left(x^3\right) &= \int_{-1}^{1} x^3 \, \mathrm{d}x = 0, & \tilde{I}\left(x^3\right) &= \frac{5}{9}\left[(-\sqrt{0.6})^3 + (\sqrt{0.6})^3\right] = 0 \\
I\left(x^4\right) &= \int_{-1}^{1} x^4 \, \mathrm{d}x = \frac{2}{5}, & \tilde{I}\left(x^4\right) &= \frac{5}{9}\left[(-\sqrt{0.6})^4 + (\sqrt{0.6})^4\right] = 0.4 \\
I\left(x^5\right) &= \int_{-1}^{1} x^5 \, \mathrm{d}x = 0, & \tilde{I}\left(x^5\right) &= \frac{5}{9}\left[(-\sqrt{0.6})^5 + (\sqrt{0.6})^5\right] = 0 \\
I\left(x^6\right) &= \int_{-1}^{1} x^6 \, \mathrm{d}x = \frac{2}{7}, & \tilde{I}\left(x^6\right) &= \frac{5}{9}\left[(-\sqrt{0.6})^6 + (\sqrt{0.6})^6\right] = \frac{2.16}{9} \neq \frac{2}{7}
\end{aligned}$$

扫描全能王 创建

所以此求积公式具有 5 次代数精确度, 它是 Gauss 型求积公式.

如何确定 $\{x_k\}$ 和 $\{A_k\}$, 使得式 (7.42) 具有 $2n-1$ 次代数精确度? 最直接的方法是解方程组(7.45). 但因 (7.45) 是非线性方程组, 求解十分困难. 一般利用正交多项式来求出 Gauss 点与求积系数.

(二) Gauss 点与正交多项式的关系

先举例说明.

例 7.9 确定 x_1, x_2, A_1, A_2, 使求积公式

$$\int_{-1}^{1} f(x)\mathrm{d}x \approx A_1 f(x_1) + A_2 f(x_2) \tag{7.46}$$

具有 3 次代数精确度.

解 取 $f(x)$ 为任意三次多项式, 利用多项式除法, $f(x)$ 一定能表示成如下形式

$$f(x) = (a_0 + a_1 x)(x - x_1)(x - x_2) + (b_0 + b_1 x)$$

如果求积公式 (7.46) 有 3 次代数精确度, 则

$$\begin{aligned}\int_{-1}^{1} f(x)\mathrm{d}x &= \int_{-1}^{1} (a_0 + a_1 x)(x - x_1)(x - x_2)\,\mathrm{d}x + \int_{-1}^{1} (b_0 + b_1 x)\,\mathrm{d}x \\ &= A_1 (b_0 + b_1 x_1) + A_2 (b_0 + b_1 x_2)\end{aligned}$$

并且,

$$\int_{-1}^{1} (b_0 + b_1 x)\,\mathrm{d}x = A_1 (b_0 + b_1 x_1) + A_2 (b_0 + b_1 x_2)$$

比较以上两式, 即得

$$\int_{-1}^{1} (a_0 + a_1 x)(x - x_1)(x - x_2)\,\mathrm{d}x = 0$$

对任意实数 a_0, a_1 都成立. 特别地, 取 $a_0 = 1, a_1 = 0$ 和 $a_0 = 0, a_1 = 1$, 得方程组

$$\begin{cases} \displaystyle\int_{-1}^{1} (x - x_1)(x - x_2)\,\mathrm{d}x = 0 \\ \displaystyle\int_{-1}^{1} x(x - x_1)(x - x_2)\,\mathrm{d}x = 0 \end{cases}$$

即

$$\begin{cases} \dfrac{2}{3} + 2x_1 x_2 = 0 \\ -\dfrac{2}{3}(x_1 + x_2) = 0 \end{cases}$$

由此解出 $x_1 = -x_2 = \dfrac{1}{\sqrt{3}}$, 这就是 Gauss 点. 再取两个特殊的 $f(x)$ (至多为三次多项式), 连同 Gauss 点代入式 (7.46), 可求出求积系数. 比如取 $f_1(x) \equiv 1, f_2(x) = x$ 代入式 (7.46), 令其精确成立, 即得

扫描全能王 创建

$$\begin{cases} \displaystyle\int_{-1}^{1} \mathrm{d}x = 2 = A_1 + A_2 \\ \displaystyle\int_{-1}^{1} x\,\mathrm{d}x = 0 = A_1\frac{1}{\sqrt{3}} - A_2\frac{1}{\sqrt{3}} \end{cases}$$

其解为 $A_1 = A_2 = 1$, 所以求积公式

$$\int_{-1}^{1} f(x)\mathrm{d}x \approx f\left(\frac{1}{\sqrt{3}}\right) + f\left(-\frac{1}{\sqrt{3}}\right)$$

具有 3 次代数精确度.

回顾以上求解过程, 我们首先用多项式除法将三次多项式表示成 $q(x)(x-x_1)(x-x_2)+r(x)$ 的形式, 其中 $q(x), r(x)$ 均为一次多项式. 利用 $r(x_1) = f(x_1), r(x_2) = f(x_2)$ 且求积公式对 $r(x)$ 精确成立, 导出 x_1, x_2 应满足的条件

$$\int_{-1}^{1} q(x)(x-x_1)(x-x_2)\,\mathrm{d}x = 0$$

令 $q(x)$ 为特殊多项式代入上式, 求出 Gauss 点 x_1, x_2. 然后再取 $f(x)$ 为特殊多项式, 如 $f(x) = 1, x$, 连同 Gauss 点代入求积公式, 就可解出求积系数 A_1, A_2. 上述求解过程可直接推广到一般情况.

用 n 次多项式 $\omega_n(x) = \prod_{k=1}^{n}(x-x_k)$ 除任意的次数不超过 $2n-1$ 的多项式 $p_{2n-1}(x)$, 得

$$p_{2n-1}(x) = q(x)\omega_n(x) + r(x) \tag{7.47}$$

其中 $q(x), r(x)$ 均为至多 $n-1$ 次多项式. 将式 (7.47) 代入求积公式 (7.42), 由求积公式对 $r(x)$ 精确成立, 得

$$\int_a^b \rho(x)q(x)\omega_n(x)\mathrm{d}x = 0 \tag{7.48}$$

对任意至多 $n-1$ 次多项式 $q(x)$ 均成立, 式 (7.48) 称为正交条件. 特别地, 取 $q(x) = 1, x, \cdots, x^{n-1}$, 分别代入式 (7.48), 得 n 元方程组, 其解即为 Gauss 点 $\{x_k\}$.

定理 7.4　*求积公式 (7.42) 的节点是 Gauss 点的充要条件是函数*

$$\omega_n(x) = \prod_{k=1}^{n}(x-x_k)$$

与任意次数不超过 $n-1$ 的多项式 $p(x)$ 在 $[a,b]$ 上关于权函数 $\rho(x)$ 正交, 即

$$\int_a^b \rho(x)p(x)\omega_n(x)\mathrm{d}x = 0$$

证明　必要性. 若 $x_1, x_2, \cdots, x_n$ 是 Gauss 点, 则求积公式

$$\int_a^b \rho(x)f(x)\mathrm{d}x \approx \sum_{k=1}^{n} A_k f(x_k)$$

扫描全能王 创建

具有 $2n-1$ 次代数精确度. 设 $p(x)$ 为至多 $n-1$ 次多项式, 那么 $p(x)\omega_n(x)$ 为至多 $2n-1$ 次多项式, 于是

$$\int_a^b \rho(x)p(x)\omega_n(x)\mathrm{d}x = \sum_{k=1}^{n} A_k p(x_k)\omega_n(x_k) = 0$$

即 $\omega_n(x)$ 与任意次数不超过 $n-1$ 的多项式 $p(x)$ 在区间 $[a,b]$ 上关于权函数 $\rho(x)$ 正交.

充分性. 设 $f(x)$ 为任意的次数不超过 $2n-1$ 的多项式, 用 $\omega_n(x)$ 除 $f(x)$ 得

$$f(x) = q(x)\omega_n(x) + r(x)$$

其中 $q(x), r(x)$ 均为至多 $n-1$ 次多项式, 且 $r(x_k) = f(x_k)$ $(k=1,2,\cdots,n)$. 取 $f_l(x) = x^l$ $(l=0,1,\cdots,n-1)$, 代入求积公式 (7.42), 并令其精确成立, 得 n 阶线性方程组

$$\begin{cases} A_1 + A_2 + \cdots + A_n = \mu_0 \\ A_1x_1 + A_2x_2 + \cdots + A_nx_n = \mu_1 \\ \qquad\qquad \cdots\cdots\cdots\cdots \\ A_1x_1^{n-1} + A_2x_2^{n-1} + \cdots + A_nx_n^{n-1} = \mu_{n-1} \end{cases} \tag{7.49}$$

其中 $\mu_l = \int_a^b \rho(x)x^l\,\mathrm{d}x$ $(l=0,1,\cdots,n-1)$. 其系数矩阵为 Vandermonde 矩阵, 故方程组(7.49) 有唯一解. 由于求积公式 (7.42) 是线性的, 以式 (7.49) 的解为求积系数, 式 (7.42) 对任意至多 $n-1$ 次多项式均精确成立. 特别地, 有

$$\int_a^b \rho(x)r(x)\mathrm{d}x = \sum_{k=1}^{n} A_k r(x_k)$$

从而有

$$\begin{aligned} \int_a^b \rho(x)f(x)\mathrm{d}x &= \int_a^b \rho(x)q(x)\omega_n(x)\mathrm{d}x + \int_a^b \rho(x)r(x)\mathrm{d}x \\ &= \int_a^b \rho(x)r(x)\mathrm{d}x = \sum_{k=1}^{n} A_k r(x_k) = \sum_{k=1}^{n} A_k f(x_k) \end{aligned}$$

所以, 求积公式 (7.42) 具有 $2n-1$ 次代数精确度, $\{x_k\}$ 为 Gauss 点.

由定理 7.4 及定理 6.3 可知, 区间 $[a,b]$ 上关于权函数 $\rho(x)$ 的正交多项式系中的 n 次多项式的根恰恰就是 Gauss 点. 因此, 求 Gauss 点等价于求 $[a,b]$ 上关于权函数 $\rho(x)$ 的 n 次正交多项式的 n 个实根. 如果取

$$p_k(x) = \frac{\omega_n(x)}{(x-x_k)\omega_n'(x_k)} = \prod_{\substack{j=1 \\ j\neq k}}^{n} \frac{x-x_i}{x_k-x_j} \quad (k=1,2,\cdots,n)$$

则 $p_k(x)$ 均为 $n-1$ 次多项式, 且

$$p_k(x_j) = \begin{cases} 1, & j=k \\ 0, & j\neq k \end{cases}$$

扫描全能王 创建

将 $p_k(x)$ 代入式 (7.42) 即得

$$\int_a^b \rho(x)\frac{\omega_n(x)}{(x-x_k)\,\omega_n'(x_k)}\mathrm{d}x = \sum_{l=1}^{n} A_l p_k(x_l) = A_k \tag{7.50}$$

式 (7.50) 即为计算 Gauss 型求积公式的求积系数的公式.

(三) Gauss 型求积公式的误差

Gauss 型求积公式的截断误差有以下估计.

定理 7.5　设 $f(x)$ 在 $[a,b]$ 上 $2n$ 阶连续可微, $\rho(x)\geqslant 0$, 则带权函数 $\rho(x)$ 的 Gauss 型求积公式的余项为

$$R(f)=\int_a^b \rho(x)f(x)\mathrm{d}x-\sum_{k=1}^{n}A_k f(x_k)=\frac{f^{(2n)}(\xi)}{(2n)!}\int_a^b \rho(x)\omega_n^2(x)\mathrm{d}x,\quad \xi\in(a,b) \tag{7.51}$$

证明　设在节点 $x_1,x_2,\cdots,x_n$ 上 $f(x)$ 的 $2n-1$ 次 Hermite 插值多项式为 $H(x)$, 由 Hermite 余项公式

$$f(x)-H(x)=\frac{f^{(2n)}(\xi_x)}{(2n)!}\omega_n^2(x),\quad \xi_x\in(a,b)$$

有

$$\begin{aligned}
R(f)&=\int_a^b \rho(x)f(x)\mathrm{d}x-\sum_{k=1}^{n}A_k f(x_k)\\
&=\int_a^b \rho(x)f(x)\mathrm{d}x-\sum_{k=1}^{n}A_k H(x_k)\\
&=\int_a^b \rho(x)f(x)\mathrm{d}x-\int_a^b \rho(x)H(x)\mathrm{d}x\\
&=\int_a^b \frac{f^{(2n)}(\xi_x)}{(2n)!}\rho(x)\omega_n^2(x)\mathrm{d}x
\end{aligned}$$

因为 $\rho(x)\omega_n^2(x)\geqslant 0$, 由反常积分中值定理, 存在 $\xi\in(a,b)$ 使得

$$R(f)=\frac{f^{(2n)}(\xi)}{(2n)!}\int_a^b \rho(x)\omega_n^2(x)\mathrm{d}x$$

进一步论证可以导出, 如果 $f(x)\in C[a,b]$, 当 $n\to\infty$ 时, Gauss 型求积公式收敛于积分准确值, 即

$$\int_a^b \rho(x)f(x)\mathrm{d}x=\lim_{n\to\infty}\sum_{k=1}^{n}A_k f(x_k)$$

证明过程可看参考文献 [9].

用 Gauss 型求积公式计算积分近似值时, 公式中的 Gauss 点与求积系数都是预先给出的, 一般精度较高. 舍入误差主要来源于函数值 $f(x_k)$ 的计算. 如果记 $f(x_k)$ 的近似值为 $f^*(x_k)$, 则

扫描全能王 创建

$$\begin{aligned}\left|\sum_{k=1}^{n} A_k f(x_k)-\sum_{k=1}^{n} A_k f^*(x_k)\right| &= \left|\sum_{k=1}^{n} A_k\left[f(x_k)-f^*(x_k)\right]\right| \\ &\leqslant \max_{1\leqslant k\leqslant n}\left|f(x_k)-f^*(x_k)\right| \sum_{k=1}^{n}\left|A_k\right|\end{aligned} \tag{7.52}$$

因为求积公式对 $f(x)\equiv 1$ 和 $f(x)=p_k^2(x)$ 精确成立, 故有

$$\int_a^b \rho(x)\mathrm{d}x=\sum_{k=1}^{n} A_k, \quad \int_a^b \rho(x)p_k^2(x)\mathrm{d}x=\sum_{l=1}^{n} A_l p_k^2(x_l)=A_k>0$$

代入式 (7.52) 得

$$\left|\sum_{k=1}^{n} A_k f(x_k)-\sum_{k=1}^{n} A_k f^*(x_k)\right| \leqslant \int_a^b \rho(x)\mathrm{d}x \max_{1\leqslant k\leqslant n}\left|f(x_k)-f^*(x_k)\right| \tag{7.53}$$

所以 Gauss 型求积公式是数值稳定的.

7.5.2 几种常用的 Gauss 型求积公式

对不同的权函数 $\rho(x)$, 选取不同的正交多项式系, 可以导出不同的 Gauss 型求积公式.

(一) Gauss-Legendre (高斯 – 勒让德) 求积公式

区间 $[-1,1]$ 上的 Legendre 多项式是关于权函数 $\rho(x)\equiv 1$ 的正交多项式系, 其表达式为

$$\begin{aligned}&P_0(x)=1\\ &P_n(x)=\frac{1}{2^n n!}\frac{\mathrm{d}^n}{\mathrm{d}x^n}\left[\left(x^2-1\right)^n\right] \quad (n=1,2,\cdots)\end{aligned} \tag{7.54}$$

由定理 7.4 可知, n 次 Legendre 多项式的零点 x_k $(k=1,2,\cdots,n)$ 是带权函数 $\rho(x)\equiv 1$ 的 Gauss 型求积公式的 Gauss 点, 求积系数为

$$A_k=\frac{2}{\left(1-x_k^2\right)\left[P_n'(x_k)\right]^2} \quad (k=1,2,\cdots,n) \tag{7.55}$$

按上述方法构造的 Gauss 型求积公式称为 Gauss-Legendre 求积公式.

例如, 当 $n=2$ 时, $P_2(x)=\dfrac{1}{2}\left(3x^2-1\right)$, 其零点为 $x=\pm\dfrac{1}{\sqrt{3}}$. 由式 (7.55), 有

$$A_k=\frac{2}{\left[1-\left(\pm\dfrac{1}{\sqrt{3}}\right)^2\right]\left(\pm\dfrac{3}{\sqrt{3}}\right)^2}=\frac{2}{\left(1-\dfrac{1}{3}\right)\times 3}=1 \quad (k=1,2)$$

所以两个节点的 Gauss-Legendre 公式为

$$\int_{-1}^{1} f(x)\mathrm{d}x\approx f\left(-\frac{1}{\sqrt{3}}\right)+f\left(\frac{1}{\sqrt{3}}\right)$$

表 7.4 给出了部分 Gauss-Legendre 求积公式的节点与求积系数值, 以便查用.

扫描全能王 创建

表 7.4　部分 Gauss-Legendre 求积公式的节点与求积系数值

n	x_k	A_k
1	0	2
2	±0.577 350 27	1
3	±0.774 596 67	0.555 555 56
	0	0.888 888 89
4	±0.861 136 31	0.347 854 85
	±0.339 981 04	0.652 145 15
5	±0.906 179 85	0.236 926 89
	±0.538 469 31	0.478 628 67
	0	0.568 888 89
6	±0.932 469 51	0.171 324 49
	±0.661 209 39	0.360 761 57
	±0.238 619 19	0.467 913 93

由式 (7.51) 及 n 次 Legendre 多项式 $P_n(x)$ 与 $\omega_n(x)$ 的关系, 容易得出 Gauss-Legendre 求积公式的截断误差为

$$\begin{aligned} R(f) &= \frac{f^{(2n)}(\xi)}{(2n)!}\int_{-1}^{1}\omega_n^2(x)\mathrm{d}x = \frac{f^{(2n)}(\xi)}{(2n)!}\frac{2^{2n}(n!)^4}{[(2n)!]^2}\int_{-1}^{1}P_n^2(x)\mathrm{d}x \\ &= \frac{2^{2n+1}(n!)^4}{(2n+1)[(2n)!]^3}f^{(2n)}(\xi), \quad \xi\in(-1,1) \end{aligned} \tag{7.56}$$

计算函数 $f(x)$ 在任意有限区间 $[a,b]$ 上的积分值的 Gauss-Legendre 公式可通过变量变换得到. 令 $x=\dfrac{a+b}{2}+\dfrac{b-a}{2}t$, 则

$$\begin{aligned} \int_a^b f(x)\mathrm{d}x &= \frac{b-a}{2}\int_{-1}^{1} f\left(\frac{a+b}{2}+\frac{b-a}{2}t\right)\mathrm{d}t \\ &= \frac{b-a}{2}\int_{-1}^{1}F(t)\mathrm{d}t \approx \frac{b-a}{2}\sum_{k=1}^{n}A_kF(t_k) \end{aligned} \tag{7.57}$$

其中 $\{t_k\}$ 为 n 次 Legendre 多项式 $P_n(t)$ 的零点, A_k 按式 (7.55) 计算.

例 7.10　求积分 $I=\displaystyle\int_0^{\frac{\pi}{2}}\sin t\ \mathrm{d}t$ 的近似值.

解　作变换 $t=\dfrac{\pi}{4}+\dfrac{\pi}{4}x$, 则

$$I=\int_0^{\frac{\pi}{2}}\sin t\ \mathrm{d}t=\frac{\pi}{4}\int_{-1}^{1}\sin\frac{\pi(x+1)}{4}\ \mathrm{d}x$$

若用 $n=2$ 的 Gauss-Legendre 求积公式计算, 由表 7.4,$x_1=-0.577\ 350\ 27$, $x_2=0.577\ 350\ 27$, $A_{1,2}=1$, 则

扫描全能王 创建

$$I \approx \frac{\pi}{4}\sin\left[\frac{\pi}{4}(1-0.577\ 350\ 27)\right]+\frac{\pi}{4}\sin\left(\frac{\pi}{4}\times 1.577\ 350\ 27\right)=0.998\ 472\ 5$$

其误差为

$$|R(f)|=\left|\frac{2^5\times(2!)^4}{5\times(4!)^3}f^{(4)}(\xi)\right|\leqslant\frac{2^9}{5\times 24^3}\times\left(\frac{\pi}{4}\right)^5=\frac{\pi^5}{10\times 24^3}\approx 0.002\ 2$$

若用 $n=3$ 的 Gauss-Legendre 求积公式, 查表 7.4 得

$$\begin{aligned}I &\approx 0.555\ 555\ 56f(-0.774\ 596\ 67)+0.888\ 888\ 89f(0)+0.555\ 555\ 56f(0.774\ 596\ 67)\\ &=0.555\ 555\ 6\left[\frac{\pi}{4}\sin\left[\frac{\pi}{4}\times(1-0.774\ 596\ 7)\right]+\frac{\pi}{4}\sin\left(\frac{\pi}{4}\times 1.774\ 596\ 7\right)\right]+\\ &\quad 0.888\ 888\ 9\times\frac{\pi}{4}\times\sin\frac{\pi}{4}\\ &=0.506\ 354\ 4+0.493\ 653\ 6\\ &=1.000\ 008\ 1\end{aligned}$$

其误差为

$$|R(f)|=\left|\frac{2^7\times(3!)^4}{7\times(6!)^3}f^{(6)}(\xi)\right|\leqslant\frac{2^7\times 6^4}{7\times 720^3}\times\left(\frac{\pi}{4}\right)^7\approx 1.17\times 10^{-5}$$

如果用三点的 Simpson 公式求积分近似值, 则有

$$I=\int_0^{\frac{\pi}{2}}\sin t\,\mathrm{d}t\approx\frac{\pi}{12}\left(\sin 0+4\sin\frac{\pi}{4}+\sin\frac{\pi}{2}\right)=\frac{\pi}{12}(2\sqrt{2}+1)\approx 1.002\ 279\ 9$$

I 的精确值为 1 . 由例 7.9 结果可见, 当被积函数充分光滑, 在节点数相同时, Gauss 型求积公式比 N-C 公式的精度高.

(二) Gauss-Chebyshev (高斯 − 切比雪夫) 求积公式

Chebyshev 多项式是 $[-1,1]$ 上关于权函数 $\rho(x)=\dfrac{1}{\sqrt{1-x^2}}$ 的正交多项式系, 其表达式为

$$T_n(x)=\cos(n\arccos x)\quad(n=0,1,2,\cdots)$$

n 次 Chebyshev 多项式 $T_n(x)$ 的零点为 $x_k=\cos\dfrac{2k-1}{2n}\pi$ $(k=1,2,\cdots,n)$. 以 x_k 为节点, 求积系数为 $A_k=\dfrac{\pi}{n}$ $(k=1,2,\cdots,n)$ 的带有权函数 $\rho(x)=\dfrac{1}{\sqrt{1-x^2}}$ 的 Gauss 型求积公式

$$\int_{-1}^{1}\frac{f(x)}{\sqrt{1-x^2}}\,\mathrm{d}x\approx\frac{\pi}{n}\sum_{k=1}^{n}f\left(\cos\left(\frac{2k-1}{2n}\pi\right)\right)\tag{7.58}$$

称为 Gauss-Chebyshev 求积公式. 其余项为

$$R(f)=\frac{\pi}{2^{2n-1}(2n)!}f^{(2n)}(\xi),\quad\xi\in(-1,1)\tag{7.59}$$

扫描全能王 创建

（三）Gauss-Laguerre (高斯 – 拉盖尔) 求积公式

Laguerre 多项式

$$L_n(x)=\mathrm{e}^x\frac{\mathrm{d}^n}{\mathrm{d}x^n}\left(\mathrm{e}^{-x}x^n\right)\quad(n=0,1,2,\cdots)$$

是区间 $[0,+\infty)$ 上关于权函数 $\rho(x)=\mathrm{e}^{-x}$ 的正交多项式系, 以 n 次 Laguerre 多项式 $L_n(x)$ 的零点 $\{x_k\}$ 为 Gauss 点导出的 Gauss 型求积公式

$$\int_0^{+\infty}\mathrm{e}^{-x}f(x)\mathrm{d}x\approx\sum_{k=1}^{n}A_kf\left(x_k\right)\tag{7.60}$$

称为 Gauss-Laguerre 求积公式. 求积系数的计算公式为

$$A_k=\frac{(n!)^2}{x_k\left[L_n'\left(x_k\right)\right]^2}\quad(k=1,2,\cdots,n)\tag{7.61}$$

求积公式 (7.60) 的截断误差为

$$R(f)=\frac{(n!)^2}{(2n)!}f^{(2n)}(\xi),\quad\xi\in(0,+\infty)\tag{7.62}$$

表 7.5 列出了部分 Gauss-Laguerre 求积公式的节点和求积系数值, 以供查用.

表 7.5　部分 Gauss-Laguerre 求积公式的节点和求积系数值

n	x_k	A_k	$A_k\mathrm{e}^{x_k}$
2	0.585 786 44	0.853 553 39	1.533 326 03
	3.414 213 56	0.146 446 61	4.450 957 34
3	0.415 774 56	0.711 093 01	1.077 692 86
	2.294 280 36	0.278 517 73	2.762 142 96
	6.289 945 08	0.010 389 26	5.601 094 63
4	0.322 547 69	0.603 154 10	0.832 739 12
	1.745 761 10	0.357 418 69	2.048 102 44
	4.536 620 30	0.038 887 91	3.631 146 31
	9.395 070 91	0.000 539 29	6.487 145 08

利用 Gauss-Laguerre 求积公式也可以求积分 $\int_0^{+\infty}g(x)\mathrm{d}x$ 的近似值. 令 $f(x)=\mathrm{e}^xg(x)$, 则

$$\int_0^{+\infty}g(x)\mathrm{d}x=\int_0^{+\infty}\mathrm{e}^{-x}f(x)\mathrm{d}x\approx\sum_{k=1}^{n}A_kf\left(x_k\right)=\sum_{k=1}^{n}A_k\mathrm{e}^{x_k}g\left(x_k\right)\tag{7.63}$$

而对积分 $\int_a^{+\infty}\varphi(t)\mathrm{d}t$, 只要作变换 $x=t-a$, 就有

$$\int_a^{+\infty}\varphi(t)\mathrm{d}t=\int_0^{+\infty}\varphi(x+a)\mathrm{d}x=\int_0^{+\infty}g(x)\mathrm{d}x$$

扫描全能王 创建

按式 (7.63) 即可计算其近似值.

(四) Gauss-Hermite (高斯 – 埃尔米特) 求积公式

求积公式

$$\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2} f(x)\mathrm{d}x \approx \sum_{k=1}^{n} A_k f(x_k) \tag{7.64}$$

称为 Gauss-Hermite 求积公式, 其中节点 x_k $(k=1,2,\cdots,n)$ 为 n 次 Hermite 多项式

$$H_n(x) = (-1)^n \mathrm{e}^{x^2} \frac{\mathrm{d}^n}{\mathrm{d}x^n}\left(\mathrm{e}^{-x^2}\right) \quad (n=0,1,2,\cdots) \tag{7.65}$$

的 n 个零点, 求积系数为

$$A_k = \frac{2^{n+1} n! \sqrt{\pi}}{\left[H_n'(x_k)\right]^2} \quad (k=1,2,\cdots,n) \tag{7.66}$$

因为 Hermite 多项式是 $(-\infty,+\infty)$ 上以 $\rho(x) = \mathrm{e}^{-x^2}$ 为权函数的正交多项式系, 故公式 (7.64) 为 Gauss 型求积公式. 其截断误差为

$$R(f) = \frac{n!\sqrt{\pi}}{2^n (2n)!} f^{(2n)}(\xi), \quad \xi \in (-\infty,+\infty) \tag{7.67}$$

部分 Gauss-Hermite 求积公式的节点与求积系数见表 7.6.

表 7.6 部分 Gauss-Hermite 求积公式的节点与求积系数

n	x_k	A_k	$A_k \mathrm{e}^{x_k^2}$
2	±0.707 106 78	0.886 226 93	1.461 141 18
3	0	1.181 635 90	1.181 635 90
	±1.224 744 87	0.295 408 98	1.323 931 18
4	±0.524 647 62	0.804 914 09	1.059 964 48
	±1.650 680 12	0.081 312 84	1.240 225 82
5	0	0.945 308 72	0.945 308 72
	±0.958 572 46	0.393 619 32	0.986 581 00
	±2.020 182 87	0.019 953 24	1.181 488 63

对积分 $\int_{-\infty}^{+\infty} g(x)\mathrm{d}x$, 令 $f(x) = \mathrm{e}^{x^2} g(x)$, 则

$$\int_{-\infty}^{+\infty} g(x)\mathrm{d}x = \int_{-\infty}^{+\infty} \mathrm{e}^{-x^2} f(x)\mathrm{d}x$$

故只要对 $\mathrm{e}^{x^2} g(x)$ 用 Gauss-Hermite 求积公式, 即可计算 $\int_{-\infty}^{+\infty} g(x)\mathrm{d}x$ 的近似值.

例 7.11 利用 $n=3$ 的 Gauss-Hermite 求积公式计算积分

$$I = \int_0^{+\infty} \mathrm{e}^{-x^3} \sqrt{1+x^2}\, \mathrm{d}x$$

扫描全能王 创建

解　因积分 $\int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x$ 收敛, $\mathrm{e}^{-x^2}\sqrt{1+x^2}$ 是偶函数, 故积分 $\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\mathrm{d}x$ 收敛, 且

$$\int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x = \frac{1}{2}\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x$$

查表 7.6 得

$$\begin{aligned}\int_{-\infty}^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x \approx & 1.181\ 635\ 90 \times \sqrt{1+0} + 0.295\ 408\ 98\times \\ & \left[\sqrt{1+(1.224\ 744\ 87)^2} + \sqrt{1+(-1.224\ 744\ 87)^2}\right] \\ = & 1.181\ 635\ 9 + 0.295\ 409\ 0 \times 3.162\ 277\ 7 \\ = & 2.115\ 801\ 2\end{aligned}$$

所以

$$I = \int_0^{+\infty} \mathrm{e}^{-x^2}\sqrt{1+x^2}\,\mathrm{d}x \approx 1.057\ 900\ 6$$

综上所述, 用 Gauss 型求积公式计算积分近似值的步骤为

(1) 把所求积分化成标准形式, 即通过变换将积分区间化成 $[-1,1]$, $[0,+\infty)$ 或 $(-\infty,+\infty)$, 根据被积函数形式, 确定用哪种求积公式;

(2) 选定 n, 查表得节点 $\{x_k\}$ 及求积系数 $\{A_k\}$;

(3) 按求积公式计算积分近似值.

Gauss 型求积公式不仅精度高, 而且可以计算反常积分, 瑕积分亦可化成无穷积分求近似值. 这是前几节所介绍的 N-C 公式, Romberg 方法等无法解决的. 但用 Gauss 型求积公式很难根据事先给定的精度及误差公式确定节点数 n, 并且当节点数 n 增加时, 节点 $\{x_k\}$ 与求积系数 $\{A_k\}$ 都会改变, 需重新查表按新公式计算, 前面计算过的结果不能继续使用, 因而增加了计算量.

7.6　振荡函数的积分

在工程实际中, 常常需要计算振荡函数的积分. 如果被积函数是一个变化激烈的振荡函数, 如

$$\int_a^b f(x)\cos\omega x\,\mathrm{d}x, \quad \int_a^b f(x)\sin\omega x\,\mathrm{d}x \tag{7.68}$$

其中 $f(x)$ 为非振荡函数, ω 为较大的正数. 利用前面给出的求积公式虽然也能计算这两个积分的近似值, 但为得到比较准确的近似值, 必须在每半个周期内都要取到几个节点. 否则, 计算效果可能很不理想. 例如, 用复化梯形公式和复化 Simpson 公式分别计算积分

扫描全能王 创建

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\ \mathrm{d}x$$

的近似值. 如果取 $n=10, h=\dfrac{\pi}{16}$, 用复化梯形公式和复化 Simpson 公式分别求得近似值为

$$T_{10}=-0.040\ 7,\quad S_{10}=-0.119\ 7$$

这与积分的准确值

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\ \mathrm{d}x=\frac{1}{20}=0.05$$

相差很大. 事实上, 如果用复化梯形公式计算此积分的近似值, 要求其误差不超过 0.001, 由误差公式

$$|R_n|=\frac{1}{12}\times\frac{5}{8}\pi h^2\times 20^2\sin\xi\leqslant\frac{125\pi}{6}h^2\leqslant 0.001$$

可以估计出 $h\leqslant 0.003\ 908\ 82$, 即至少应取 $n=503$ 个节点. 这样, 节点数比较多, 求积的工作量十分可观.

仔细分析误差公式可以发现, 用复化求积公式计算振荡积分, 之所以要取较多的节点, 原因在于被积函数求导时将出现 ω^2 或 ω^4, 当 ω 较大时, 误差就大. 解决此问题的一个方法是用 $f(x)$ 的插值函数 $\varphi(x)$ 代入式 (7.68), 以积分

$$\int_a^b\varphi(x)\cos\omega x\ \mathrm{d}x,\quad\int_a^b\varphi(x)\sin\omega x\ \mathrm{d}x \tag{7.69}$$

作为积分式 (7.68) 的近似. 特别地, 若用分段低次插值多项式 $\varphi(x)$ 近似 $f(x)$, 积分式 (7.69) 可以精确地求出.

例如, 将区间 $[a,b]$ 分为 n 等份, $h=\dfrac{b-a}{n}, x_i=a+ih\ (i=0,1,\cdots,n)$. 取 $\varphi(x)$ 为 $f(x)$ 的分段线性插值函数, 则有

$$\begin{aligned}
&\int_a^b\varphi(x)\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\int_{x_i}^{x_{i+1}}\varphi(x)\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\int_{x_i}^{x_{i+1}}\left\{f(x_i)+\frac{1}{h}\left[f(x_{i+1})-f(x_i)\right](x-x_i)\right\}\cos\omega x\ \mathrm{d}x\\
=&\sum_{i=0}^{n-1}\left\{\frac{1}{\omega}\left[f(x_i)-\frac{x_i}{h}\left(f(x_{i+1})-f(x_i)\right)\right](\sin\omega x_{i+1}-\sin\omega x_i)+\right.\\
&\left.\frac{1}{\omega h}\left[f(x_{i+1})-f(x_i)\right]\left(x\sin\omega x+\frac{1}{\omega}\cos\omega x\right)\Big|_{x_i}^{x_{i+1}}\right\}
\end{aligned}$$

扫描全能王 创建

$$=\sum_{i=0}^{n-1}\left\{\frac{1}{\omega}\left[f\left(x_{i+1}\right)\sin\omega x_{i+1}-f\left(x_i\right)\sin\omega x_i\right]-\frac{\sin\omega h}{\omega^2 h}\left[f\left(x_{i+1}\right)\sin\omega x_{i+1}-f\left(x_i\right)\sin\omega x_i\right]+\right.$$
$$\left.\frac{2\sin^2\frac{\omega h}{2}}{\omega^2 h}\left[f\left(x_{i+1}\right)\cos\omega x_{i+1}+f\left(x_i\right)\cos\omega x_i\right]\right\}$$
$$=\frac{1}{\omega}\left[f\left(x_n\right)\sin\omega x_n-f\left(x_0\right)\sin\omega x_0\right]-\frac{\sin\omega h}{\omega^2 h}\left[f\left(x_n\right)\sin\omega x_n-f\left(x_0\right)\sin\omega x_0\right]+$$
$$\frac{2}{\omega^2 h}\sin^2\frac{\omega h}{2}\left[f\left(x_n\right)\cos\omega x_n+f\left(x_0\right)\cos\omega x_0\right]+\frac{4}{\omega^2 h}\sin^2\frac{\omega h}{2}\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i$$

整理上式并令

$$\alpha=\frac{4}{\omega^2 h}\sin^2\frac{\omega h}{2},\quad \beta=\frac{2}{\omega^2 h}\sin\frac{\omega h}{2} \tag{7.70}$$

得

$$\begin{aligned}\int_a^b\varphi(x)\cos\omega x\,\mathrm{d}x=&f\left(x_0\right)\left[-\frac{1}{\omega}\sin\omega x_0+\beta\sin\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[\frac{1}{\omega}\sin\omega x_n-\beta\sin\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i\end{aligned} \tag{7.71}$$

于是得到求积公式

$$\begin{aligned}\int_a^b f(x)\cos\omega x\,\mathrm{d}x\approx&\int_a^b\varphi(x)\cos\omega x\,\mathrm{d}x\\=&f\left(x_0\right)\left[-\frac{1}{\omega}\sin\omega x_0+\beta\sin\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[\frac{1}{\omega}\sin\omega x_n-\beta\sin\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\cos\omega x_i\end{aligned} \tag{7.72}$$

完全类似地有

$$\begin{aligned}\int_a^b f(x)\sin\omega x\,\mathrm{d}x\approx&f\left(x_0\right)\left[\frac{1}{\omega}\cos\omega x_0-\beta\cos\omega\left(x_0+\frac{h}{2}\right)\right]+\\&f\left(x_n\right)\left[-\frac{1}{\omega}\cos\omega x_n+\beta\cos\omega\left(x_n-\frac{h}{2}\right)\right]+\alpha\sum_{i=1}^{n-1}f\left(x_i\right)\sin\omega x_i\end{aligned} \tag{7.73}$$

利用求积公式 (7.73) 计算积分

$$I=\int_0^{\frac{5}{8}\pi}\sin 20x\,\mathrm{d}x$$

取 $n=10,h=\dfrac{\pi}{16}$, 则有

$$\alpha=\frac{64}{400\pi}\sin^2\frac{5\pi}{8}\approx 0.043\ 471\ 1$$

扫描全能王 创建